

Lower Bounds and Explicit Constructions Through the Lens of Total Search Problems

Oliver Korten

Submitted in partial fulfillment of the
requirements for the degree of
Doctor of Philosophy
under the Executive Committee
of the Graduate School of Arts and Sciences

COLUMBIA UNIVERSITY

2026

© 2026

Oliver Korten

All Rights Reserved

Abstract

Lower Bounds and Explicit Constructions Through the Lens of Total Search Problems

Oliver Korten

[Author-written abstract goes here.]

Table of Contents

Acknowledgments	viii
Introduction and Background	1
Motivation: Circuit Lower Bounds, Explicit Constructions and the Probabilistic Method	1
The Search Problem Perspective	4
Range Avoidance	5
Our Results	5
Chapter 1: A Complexity Theory for Explicit Constructions	6
Chapter 2: Algorithmic Circuit Lower Bounds and the Structure of $\text{TF}\Sigma_2^P$	8
Chapters 3 and 4: Deterministic Algorithms for Explicit Construction Problems	10
Chapter 5: Connections to Static Data Structures, Random CSP Refutation, and More	15
Preliminaries	16
0.1 Basics	16
0.2 Complexity Classes, Search Problems, and Explicit Construction Problems	16
0.3 Range Avoidance	18
Chapter 1: Range Avoidance and the Complexity of Explicit Constructions	20
1.1 Explicit Constructions and Circuit Lower Bounds are Total Search Problems	20
1.2 Basic Upper Bounds on the Complexity of AVOID and Explicit Construction Problems	21

1.3	Reductions to Range Avoidance	23
1.3.1	Hard Functions, Random Strings, and Pseudorandom Generators	24
1.3.2	Strongly Explicit Extractors	30
1.3.3	Miscellaneous Reductions	33
1.4	Completeness	35
1.4.1	Complete Explicit Construction Problems	36
1.4.2	Undirected Remote Vertex	43
1.5	Direct Reductions to Hard Truth Tables	44
1.5.1	Rigidity	45
1.5.2	Space-Bounded Communication Complexity	48
1.5.3	Bit Probe Lower Bounds	49
1.6	A Brief Survey of Subsequent Developments	51
1.6.1	Restricted Variants of AVOID	51
1.6.2	Hardness of AVOID	52
1.6.3	The Algorithmic Method	55
	Chapter 2: Strong vs. Weak Range Avoidance and the Linear Ordering Principle	57
2.1	Introduction	57
2.1.1	Overview of Main Results	59
2.1.2	AC^0 Lower Bounds and Class Separations	63
2.1.3	Linear Ordering Principle	69
2.1.4	Open Problems	71
2.2	Preliminaries	73

2.2.1	Search Problems, Complexity Classes, and Basic Inclusions	73
2.2.2	Oracle Separations and the Decision Tree Model	78
2.3	Lower Bounds for Pigeonhole Principles	80
2.3.1	Pigeonhole Principle Basics	80
2.3.2	Depth 3 Lower Bound for Strong Avoid	84
2.3.3	BPP^{NP} Lower Bound for 1-1 Strong Avoid	93
2.4	LINEAR ORDERING PRINCIPLE	97
2.5	Proof of the Pigeonhole Switching Lemma	106
Chapter 3: Derandomization from Time-Space Tradeoffs		111
3.1	Introduction	111
3.1.1	Our Contributions	113
3.1.2	Relation to Prior Work	118
3.1.3	Known Time-Space tradeoffs:	119
3.1.4	Main Tool: the J-tree	120
3.2	Preliminaries	122
3.2.1	Basics	122
3.2.2	Machine Models	122
3.2.3	Search Problems	125
3.3	Pigeonhole Principles and Compression Schemes	125
3.3.1	Compressors and Decompressors	126
3.3.2	Particular Compression Schemes of Interest	128
3.4	J-Trees	132

3.5	Low-Space Simulations	139
3.5.1	Proofs of Main Theorems	140
3.5.2	Implications	148
3.6	BPP, TFNP, and the Weak Pigeonhole Principle	152
3.7	Succinct Representation of Primes	155
3.8	Alternate Proof of Nondeterministic Low-Space Simulation	156
Chapter 4: How to Construct Random Strings		161
4.1	Introduction	161
4.1.1	Our Results	165
4.1.2	Overview of Main Construction	170
4.1.3	Related Work	172
4.2	Preliminaries	173
4.2.1	Range Avoidance and Construction of Random Strings	175
4.3	PRGs for Uniform Classes with Near-Optimal Seed Length	177
4.3.1	Fooling $\text{TIME}^{\text{NP}}[T(n)]$ with $O(\log n)$ Seed Length	178
4.3.2	Recursive Generator Construction	179
4.3.3	Fooling Uniform Deterministic Time	183
4.3.4	Optimality of the Seed Length	185
4.4	Construction of Random Strings and Applications	186
4.4.1	Random String Construction Via PRG Concatenation	186
4.4.2	Explicit Constructions Under Plausible Hardness Assumptions	189
4.4.3	Hardness Condensation	192

4.4.4	Range Avoidance vs Uniform Range Avoidance	193
4.5	Barriers to Improvements	196
4.5.1	Multi-Reconstructive Extractors for Block Sources	196
4.5.2	No Relativizing Reduction from K^{poly} Randomness to Hard Truth Tables . .	203
4.6	Proof of Lemma 4.16	205
Chapter 5: Stronger Cell Probe Lower Bounds via Local PRGs		209
5.1	Introduction	209
5.1.1	Our Results	213
5.1.2	Subsequent Developments:	221
5.1.3	Our Techniques: Derandomizing Semirandom CSP Refutations	221
5.1.4	Open Problems	224
5.1.5	Prior Work on Data Structure Lower Bounds	226
5.2	Preliminaries	228
5.2.1	Basics	228
5.2.2	Generators and Data Structures	230
5.3	Cell Probe Lower Bounds via CSP-Pseudorandomness	232
5.3.1	From Locally-Computable Generators to XOR Schemes	232
5.3.2	CSP-Pseudorandomness via Limited Independence	239
5.3.3	Proof of Theorem 5.25	244
5.3.4	Lower Bounds for Concrete Problems	249
5.4	Beating the Limited Independence Bound	252
5.5	Algorithmic Implications of Our Lower Bounds	257

5.5.1	Range Avoidance	257
5.5.2	Linear Attacks on Local PRGs	260
5.6	Towards Stronger Lower Bounds	262
5.6.1	Consequences of Explicit Bit Probe Lower Bounds	263
5.6.2	Natural Bit Probe Lower Bounds Break NC^0 PRGs	264
5.6.3	Approaches to Strong Lower Bounds via Communication Complexity . . .	269
5.7	Kikuchi Method	276
	References	279

List of Figures

1.1	Extending a map $C : \{0, 1\}^n \rightarrow \{0, 1\}^{2^n}$ to a map $C^* : \{0, 1\}^n \rightarrow \{0, 1\}^{2^{k_n}}$. Dotted boxes indicate the number of bits along a wire.	38
1.2	A succinct circuit whose truth table is y , for any y in the range of C^* . Dotted boxes indicate the number of bits along a wire. Note that although x is shown as an input in this diagram, for any given y we fix a preimage x as constants/advice, and so the only true inputs to this circuit are i, j	39
1.3	A small circuit for a non-rigid truth table	47
2.1	Inclusion diagram of relevant classes and search problems. Solid arrows represent the inclusion of the class at the base of the arrow into the class at its tip. Dotted arrows indicate non-inclusion of the base class into the tip class in the decision tree model. The main linear axis of classes along the top are all included from left to right, indicated by the long solid arrow above them. Our major results are the separations marked with $\spadesuit$, $\clubsuit$ and the inclusion marked with $\diamond$. $\text{TFU}\Sigma_2^P$ essentially corresponds to $\text{FP}^{\Sigma_2^P \cap \Pi_2^P}$, see Lemma 2.23.	64
2.2	Example instance of FOREST TERMINATION with $M = 4$ and $N = 3$, with virtual root added at the top. Blue nodes are circles and red nodes are diamonds. We have sorted each layer left to right so that blue nodes come first, and within each color class the nodes are ordered left to right lexicographically. The number on each node represents its position in the ordering $\prec$. The nodes numbered 1 and 4 are the termination points of this instance.	103
3.1	Main Results. We use the shorthand $T = T(n)$	115
3.2	Relations between the total search problems associated with various weak pigeon-hole principles, and standard derandomization problems. Solid arrows represent deterministic polynomial time reductions, while dotted arrows represent NP-oracle reductions.	118
3.3	A J-tree of depth 3. Each arrow consists of n wires, where n is the code length of the decompressor D	134
3.4	“Forward phase” of $\text{Find}(y, 100, s, C, D)$ (where $k = 3$), during which the $z_1, \dots, z_k$ are computed.	137
3.5	“Backward phase” of $\text{Find}(y, 100, s, C, D)$ (where $k = 3$), during which the $v_1, \dots, v_k$ are computed.	137
3.6	Simulating a RAM memory with a J-tree. While the RAM memory on the left requires T bits to store explicitly, the virtual RAM on the right can be stored with $ \text{Mem} = W = T^{O(\epsilon)}$ bits.	142

Acknowledgments

[Optional acknowledgments go here.]

Introduction and Background

Motivation: Circuit Lower Bounds, Explicit Constructions and the Probabilistic Method

The central goal of complexity theory is to prove unconditional lower bounds on the resources needed to compute functions. Despite decades of intense study, we are still far from proving strong lower bounds for any functions of practical interest when the computational resource is sufficiently powerful, e.g. circuit size. We may roughly divide the current approaches to circuit lower bounds into two categories:

1. *Low-end*: Here, the plan is to start with some explicit function of practical interest, and prove lower bounds for this function against restricted classes of circuits, hopefully increasing the strength of the circuit class one can lower bound against over time.
2. *High-end*: Here, the plan is to start with an unnatural/contrived function which is provably hard for general circuits, and try to find more and more efficient algorithms for it in some incomparable computational model such as nondeterministic time or space. If we find a function f which requires large circuit complexity, and are able to “push it down” into a small complexity class like NP, PSPACE, or EXP, we will have succeeded in proving a lower bound for the *complete* problems for NP/PSPACE/EXP, which are natural and important.

The low-end approach has been a central focus since the 1980’s. Despite significant progress, it so far can only achieve lower bounds against highly restricted classes of circuits, in particular circuits which are heavily restricted in their depth [4, 53, 70, 142, 156, 69]. The high-end approach is a more recent development, having been brought to attention by the pioneering work of Williams on the *algorithmic method* [169, 170] (William’s method in fact utilizes aspects of both the high-end and low-end approaches).

A primary goal of this thesis will be to give a general algorithmic framework in which to cast this “high-end approach” to circuit lower bounds; in particular, we will cast this approach as the study of the complexity of a total search problem called Range Avoidance.

Our inability to prove strong circuit lower bounds is particularly frustrating in light of a classical theorem of Shannon from 1949:

Theorem (Shannon’s Bound [153]). *A random boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ requires circuit complexity $\geq 2^n/n$ with probability $\geq 1 - 2^{-2^{\Omega(n)}}$.*

This fact puts complexity theorists in a curious situation: we are in search of a kind of object which we know exists in overwhelming abundance, and yet we are unable to identify even one example of such an object. The familiar reader may recognize that this problem is actually not at all unique to complexity theory; mathematics abounds with nonconstructive arguments which seem to yield no information about the objects they prove to exist. Two years prior to Shannon’s seminal 1949 paper, Erdős used the same kind of nonconstructive argument to show the existence of n -vertex graphs with no clique or independent set of size $\gg \log n$ [46], introducing a style of proof that has since come to be known as the *probabilistic method*. In the ensuing century this method has become a standard and surprisingly versatile tool in theoretical computer science and combinatorics; see [7] for a comprehensive textbook on the subject.

In many cases, an application of the probabilistic method proving the existence of an interesting class of object has spawned a deep line of work dedicated to producing a concrete example of one such object. The project of replacing a nonconstructive argument by a constructive example is known generically as an *explicit construction problem*. In some fortunate cases, such as Ramsey graphs, error corrected codes, and expander graphs, decades of hard work eventually lead to optimal or near-optimal solutions. In many other cases no significant progress has been made to date. In light of Shannon’s aforementioned nonconstructive argument for the existence of exponentially hard boolean functions, we may also cast the central challenge in complexity theory as a special

case of this more general challenge of finding explicit constructions to match probabilistic arguments. A priori, there is no reason to believe that framing is at all useful. Indeed, it has become common wisdom that the probabilistic method is *inherently nonconstructive* as a proof method, and so in each case it is applied we must start from scratch in trying to match it constructively. Nonetheless we may ask:

(†) *Can we view Shannon’s probabilistic argument for the existence of hard boolean functions as a nontrivial first step in the direction of proving interesting circuit complexity lower bounds? More generally, is there any nontrivial information that can be **generically extracted** from the probabilistic method, beyond mere existence?*

The extent to which we can expect a positive answer to this question will depend on our motivations behind studying explicit construction problems. There are (at least) two common motivations we are aware of, the first being of an informal *philosophical* nature, and the second being of a formal *computational* nature. The philosophical motivation can be summarized by a maxim asserting that we do not fully understand a concept until we can produce an example. When we have some combinatorial property, the mere existence of objects satisfying that property does not give us any deeper information about *why* a certain object achieves the property. If we succeed in the quest of finding an explicit object, and have a mathematical proof that this object *in particular* has the relevant property, this proof will usually contain within it a furtherance of our general understanding of the concept in question.

This informal motivation utilizes a rather vague definition of the term “explicit.” In this framework, we would call an object, such as a Ramsey graph $G \subseteq \binom{[n]}{2}$, “explicit” provided we can present it by a comprehensible definition from which we can extract and compute various other properties of the object by direct mathematical argument. The class of *computational* motivations for studying explicit constructions centers itself on a more concrete definition of the notion of “explicitness:” in this context we will say that an object is explicit if it can be produced by a certain kind of algorithm. In the example of a Ramsey graph $G \subseteq \binom{[n]}{2}$, we might say that it is explicit if there is a $\text{poly}(n)$ time algorithm which outputs its adjacency matrix given n ; embedded in this

kind of definition is the necessity of reserving the notion of explicitness for *sequences of objects* rather than individuals.

TODO: below is where we want to go

1. want to say, first of all, on the formal/computational side of things, (1) we can try to give a positive answer to the key question by designing algorithms which can solve some general class of explicit construction problems whose definition contains all of those provable by the probabilistic method, and (2) if we succeed in getting a positive answer, this would have applications to circuit lower bounds
2. In the philosophical case, we won't expect a strong positive answer, i.e. there probably isn't some grand theory of combinatorics which lets us generically extract an interesting theory of some concept from a probabilistic existence proof. However, we still think that our investigations cast light on the philosophical stuff... and for example, we can hope to rank objects according to the strength of property, if there's some "universal" object which implies all other properties this is of informal interest....

The Search Problem Perspective

To get a positive answer to our key question ($\dagger$), we need the following ingredients:

1. Formally define a class of explicit construction problems, which contains all of the key examples of unsolved explicit construction problems studied in combinatorics and theoretical computer science.
2. Find some explicit construction procedure which can solve all problems in the aforementioned class.

We **TODO: start with 1**. A general definition of an explicit construction problem is as follows:

Definition. An explicit construction problem is defined by a set $\Pi \subseteq \{0, 1\}^*$, which we interpret as defining some combinatorial property of boolean strings, i.e. $x \in \Pi$ if and only if x has some

desired property. The associated computational problem is: given n , find some $x \in \{0, 1\}^n$ which has the property Π (meaning $x \in \{0, 1\}^n \cap \Pi$).

As per our previous discussion, it is reasonable to measure the complexity of algorithms solving an explicit construction problem in terms of the *length of the object produced*, i.e. n in the above case; we will say that an algorithm is efficient if it runs in time polynomial in n . It is standard in complexity theory to have the length of the input to a problem be the primary “complexity parameter,” and so for problems as above for which the input is simply a number n , we will formally define the input to the problem to be the unary input 1^n . We now make two observations about the kind of computational problem we have just defined.

1. There may be more than one possible solution for a given input 1^n , i.e. $|\Pi \cap \{0, 1\}^n| > 1$ for some n . This means the explicit construction problem is inherently a *search problem*, as opposed to a *decision/function problem*.
2. For all of the properties Π of interest, there will be *at least one solution* for every input 1^n , i.e. $\Pi \cap \{0, 1\}^n \neq \emptyset$, since explicit construction problems are only of interest once a nonconstructive proof of existence has been established. In complexity terminology, this makes the explicit construction problem a *total search problem*.

Range Avoidance

TODO: make sure to emphasize KKMP as the introducer of avoid...

1. define Range Avoidance, explain how it captures the probabilistic method, use the example of ramsey graphs from the Survey
2. (more?)

Our Results

TODO: maybe change section title, something to the effect of “contents”, “our results” sounds a bit to conference-paper-y...

1. give a complexity class for explicit constructions
2. an algorithmic framework in which to attack lower bound problems
3. give conditional algorithms that solve explicit construction problems (specialized hardness randomness; two sections can be explained in this way)
4. classify relativizing circuit lower bounds via black box $TF\Sigma_2$; give first separations here using tools from bounded depth circuit complexity
5. get algorithms for a special case of avoid, and using a shift in perspective, new lower bounds in a classical model of computation (cell probe)

Chapter 1: A Complexity Theory for Explicit Constructions

In the first chapter of this thesis, based on work first appearing in [101], we develop a complexity theory for explicit construction problems centered around the Range Avoidance problem. Our first batch of results in this chapter consists of a sequence of reductions showing that a wide range of important unsolved explicit construction problems can be reduced in polynomial time to AVOID. Our results cover the following explicit construction problems:

- Functions $f : \{0, 1\}^n \rightarrow \{0, 1\}$ with exponential circuit complexity (Theorem 1.9)
- Pseudorandom generators (Theorem 1.16)
- Strongly explicit two-source randomness extractors with 1 bit output for min-entropy $\log n + O(\log(1/\epsilon))$, and thus strongly explicit $O(\log n)$ -Ramsey graphs in both the bipartite and non-bipartite case (Theorem 1.23)
- Matrices with high rigidity over any finite field (Theorem 1.26)
- Strings of time-bounded Kolmogorov complexity $n - 1$ relative to any fixed polynomial time bound and any fixed Turing machine (Theorem 1.19)

- Communication problems outside of $\text{PSPACE}^{\text{CC}}$ (Theorem 1.41)
- Explicit data structure problems with extremely high bit-probe complexity (Theorem 1.44)

Together, these reductions give a very strong indication that AVOID is *at least as hard* as many of the core explicit construction problems of interest in the field. Our second main result is to show that some of the above explicit construction problems are actually *equivalent* in complexity to AVOID, meaning that a reduction also exists in the reverse direction, provided with increase the power of our reductions. The most important of these results is the following:

Theorem (Theorem 1.31). *For any $\epsilon > 0$, there is a polynomial time NP-oracle reduction from AVOID to the problem $2^{\epsilon n}$ -HARD of producing functions $f : \{0, 1\}^n \rightarrow \{0, 1\}$ requiring $2^{\epsilon n}$ -size boolean circuits.*

This can be interpreted as a kind of *completeness result*: if we take FP^{NP} reductions as our basic notion of reducibility, and “problems reducible to AVOID” as our complexity class for explicit constructions, then this theorem tells us that the construction of hard truth tables is a complete problem for this class; in other words it is the *hardest explicit construction problem*. As we shall explain in more detail **TODO: later? or before?**, for all of the explicit construction problems mentioned above, FP^{NP} algorithms are unknown and would constitute significant breakthroughs. Hence, building a complexity class around FP^{NP} reductions is still of great interest, although of course gives a weaker notion of equivalent than standard polynomial time reductions. **TODO: mention jerabek**

Theorem 1.31 is interesting for a few other reasons. First, using our previously mentioned equivalence between explicit construction algorithms and circuit lower bounds **TODO: reference**, we conclude that AVOID allows us to exactly characterize the truth of a certain circuit lower bound in terms of the solvability of a total search problem:

Theorem (Theorem 1.32). *The following are equivalent:*

1. $\text{E}^{\text{NP}} \not\subseteq_{i.o.} \text{size}[2^{o(n)}]$.

2. $\text{AVOID} \in \text{FP}^{\text{NP}}$

TODO: this realizes our first goal set out in begining: we have cast the problem of proving circuit lower bounds for E^{NP} , an important case of the “high-end,” precisely in terms of algorithms for a total search problem

Another important implication of Theorem 1.31 is that it yields a method for *amplifying* circuit lower bounds in the class E^{NP} . Recall that Theorem 1.31 gives a reduction from AVOID to $2^{\epsilon n}$ - HARD for any $\epsilon > 0$. On the other hand, in Theorem 1.9 we will show that the problem $2^n/n$ - HARD of producing functions of circuit complexity $\geq 2^n/n$ is reducible in polynomial time to AVOID . Taken together, this yields an FP^{NP} reduction from producing functions of near-maximum circuit complexity to producing functions of mildly exponential circuit complexity. Using again the equivalence between explicit constructions and circuit lower bounds for E^{NP} **TODO: ref** we obtain:

Theorem (Theorem 1.33). *The following are equivalent:*

1. $\text{E}^{\text{NP}} \not\subseteq_{i.o.} \text{size}[2^{o(n)}]$.
2. $\text{E}^{\text{NP}} \not\subseteq_{i.o.} \text{size}[2^n/n]$.

TODO: give some concluding remarks; mention that there’s been a ton of follow up work on Avoid validating this perspective... say that we’ll give a brief survey at the end of chapter 1

Chapter 2: Algorithmic Circuit Lower Bounds and the Structure of $\text{TF}\Sigma_2^{\text{P}}$

After the publication of [101] and the subsequent investigation by various authors into AVOID as an avenue through which to understand circuit lower bounds, an amazing breakthrough was achieved in a pair of works by Chen, Hirahara and Ren [29] and Li [114], who showed:

Theorem ([29, 114]). *The complexity classes $\text{S}_2^{\text{E}} \subseteq \text{ZPE}^{\text{NP}} \subseteq \Sigma_2^{\text{E}}$ contain languages with circuit complexity $\geq 2^n/n$ for all sufficiently large n .*

These results were obtained by giving new and surprising algorithms for AVOID. This is a direct realization of the goal we described above: make progress along the *high-end approach* to lower bounds, by casting lower bounds as an algorithmic problem and finding nontrivial **TODO: reductions/procedure**. The proof of these results utilized a technique used in the proof of Theorem 1.31 showing the FP^{NP} equivalence of $2^{\epsilon n}$ -HARD and AVOID.

In Chapter 2, based on joint work with Toniann Pitassi first appearing in [105], we embark on a more formal investigation into the structure of total search problems in the second level of the polynomial hierarchy ($\text{TF}\Sigma_2^{\text{P}}$), with the intent of illuminating the mechanisms behind the new circuit lower bounds in [29, 114]: how can these methods be extended, and what are their limitations?

TODO: make sure to mention KKMP here

A key property we identify which makes the main results of [29, 114] possible is a polynomial time reduction from AVOID to a $\text{TF}\Sigma_2^{\text{P}}$ search problem which has a *unique solution* on every input. We study a generalized version of AVOID called STRONG AVOID, first defined in [96], in which the input is a circuit $C : (\{0, 1\}^n \setminus \{0^n\}) \rightarrow \{0, 1\}^n$ and the goal is to find $y \in \{0, 1\}^n \setminus \text{range}(C)$. We show that in the relativized/black-box setting, STRONG AVOID is not polynomial time reducible to *any problem* with unique solutions. More strongly, we show that it is not solvable with polynomially many nonadaptive calls to a Σ_2^{P} decision oracle (it is easy to show that this property implies the previous). This result shows that the new circuit lower bounds in [29, 114] crucially rely on a certain *generosity* in Shannon’s counting argument proving the existence of hard boolean functions: if the counting argument *merely* showed that the total number of boolean functions exceeded the number of small circuits by at least 1, this would not in general be sufficient to obtain the kinds of algorithms needed for the main circuit lower bounds in [30, 114]. After this, we exhibit a different natural problem (a problem whose complexity lies strictly in between that of AVOID and STRONG AVOID), which has the property that (1) it *is* reducible to a problem with unique solutions, however (2) in the black-box setting, it *does not* have FZPP^{NP} or even FBPP^{NP} algorithms. This result shows that **TODO: of some structural interest**.

In the last part of Chapter 2, we strengthen the algorithmic upper bound for AVOID given by

Chen, Hirahara, Ren, and Li in [30, 114], by showing that AVOID has a polynomial time reduction to a natural search problem LINEAR ORDERING PRINCIPLE which has the following two properties:

1. Every input to LINEAR ORDERING PRINCIPLE has a unique solution.
2. There is a $FZPP^{NP}$ algorithm which solves LINEAR ORDERING PRINCIPLE.

The definition of LINEAR ORDERING PRINCIPLE is simple and natural: given a circuit $R : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ which purports to define the comparison relation of some total order on $\{0, 1\}^n$ (i.e. $R(x, y) = 1 \leftrightarrow x < y$), find either (1) a witness that R fails to define a total order, or (2) the least element in the order it defines ¹. TODO: fix footnote leakingTODO: was studied by krajicek

TODO: mention Volkavich paper

Chapters 3 and 4: Deterministic Algorithms for Explicit Construction Problems

Chapters 3 and 4 are dedicated to the following question:

Under what assumptions can we obtain deterministic polynomial time algorithms for important explicit construction problems?

Obtaining deterministic algorithms for problems solvable with randomness is known as *derandomization* and is a central subject of study in complexity theory TODO: cite something... arora barak?. The classical hardness/randomness paradigm of Nisan and Wigderson (developed also in earlier work of [173]) tells us that in many cases, we can achieve derandomization for very broad classes of problems if we assume strong (but widely-believed) *computational hardness assumptions*. The most famous and striking instantiation of this equivalence is due to Impagliazzo and Wigderson who proved:

Theorem ([83]). *If $E \notin_{i.o.} \text{size}[2^{o(n)}]$ then $BPP = P$.*

¹As stated the problem may have multiple valid solutions, but it is easily shown equivalent to a problem with unique solutions.

In Chapter 3, based on work originally appearing in [103], we examine a special case of the Range Avoidance problem called LOSSY CODE which seems to be of a markedly lower complexity: it lies inside TFNP rather than the larger class $\text{TF}\Sigma_2^P$. LOSSY CODE is defined as follows: we are given a pair of circuits $C : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$, $D : \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, and must find $x \in \{0, 1\}^n$ such that $D(C(x)) \neq x$. We think of C as defining a “compression algorithm” which compresses every n bit string into an $n - 1$ bit string, and D as a decompression procedure which attempts to invert C ; the LOSSY CODE problem is to find a string on which this compression/decompression procedure fails to recover the original string. Observe that, if we treat D as an instance of AVOID, then any AVOID solution on D must be a LOSSY CODE solution for (C, D) , hence this problem is at most as difficult as AVOID.

In the original publication of this work, the primary motivation for studying LOSSY CODE was mostly exploratory: we did not know of any concrete explicit construction problem which directly reduced to it, but it seemed to be an important special case of AVOID which lay in an interesting intersection of complexity classes: TFNP and FZPP. We also provided two examples of explicit construction problems which are weakly related to LOSSY CODE, namely the construction of large primes and the construction of hard truth tables. The connections of these two problems to LOSSY CODE are the following: in each case there is an NP search problem famously not known to be NP complete (the integer factoring and circuit minimization problems respectively), so that if this problem collapses to FP then the explicit construction problem (large primes and hard truth tables respectively) reduces to LOSSY CODE. In subsequent work of [38], a much stronger motivation for studying LOSSY CODE was given: they show that any problem solvable in *catalytic logspace* could be reduced in polynomial time to LOSSY CODE. The class catalytic logspace has long been known to be contained in ZPP, but has resisted unconditional derandomization into P.

Since LOSSY CODE is solvable by random sampling (like AVOID), and solutions are checkable in polynomial time (unlike AVOID), we can directly apply the aforementioned theorem of Impagliazzo-Wigderson to show:

Corollary. *If $E \notin_{i.o.} \text{size}[2^{o(n)}]$ then LOSSY CODE is solvable in deterministic polynomial time.*

Our main result in Chapter 3 is that we can achieve the same conclusion under a type of hardness assumption which is categorically different than the above, and more generally which is different from any hardness assumption used in prior work on derandomization that we are aware of. In the classical hardness-randomness paradigm, to obtain deterministic polynomial time derandomization, we need to make a hardness assumption which posits hardness against a complexity class *at least as strong as* randomized polynomial time [83, 85, 31]. Note that the Impagliazzo-Wigderson theorem requires hardness against *non-uniform* algorithms, which is even stronger. In contrast, we give a conditional derandomization for LOSSY CODE under a hardness assumption against *deterministic, uniform* time-bounded algorithms, provided we restrict their space. Specifically, one of our main results is:

Theorem. *Assume that for some $\epsilon > 0$ and some time-constructible time bound $T(n) = 2^{\Theta(n)}$, there is a language L such that:*

1. *L is decidable in time $T(n)$ on a Turing machine with random-access memory.*
2. *Any one-tape Turing machine running in time $T(n)^{1+\epsilon}$ and space $T(n)^\epsilon$ fails to correctly decide L on more than finitely many input lengths.*

Then any explicit construction problem which is polynomial time reducible to LOSSY CODE is solvable in deterministic polynomial time.

TODO: talk about the rest of this work

In Chapter 4, based on joint work with Rahul Santhanam originally appearing in [107], we move our attention back to the Range Avoidance problem and the plethora of important explicit construction problems which reduce to it. Unlike the case of LOSSY CODE, the Impagliazzo-Wigderson theorem or any of its subsequent relatives (e.g. [85, 100, 152, 160]) do not indicate any natural hardness assumption under which we can solve AVOID in polynomial time; they merely give conditions under which it can be solved in deterministic polynomial time *with an NP-oracle*. This is due to the fact that the complexity of verifying AVOID solutions is a coNP problem rather than a P problem. Work of Ilango, Li and Williams [80] (and followup work in [34,

79, 146]) has shown that this is no accident: under widely-believed cryptographic/complexity-theoretic assumptions, there is no polynomial time algorithm for AVOID in the worst case; in particular, no reasonable “hardness assumption” should imply such a strong derandomization. However, as observed already in [80], their worst-case hardness results for AVOID do not imply the hardness of any of the important explicit construction problems that reduce to it. A key distinction is that explicit construction problems have *unary input* (given 1^n , output a string of length n with some property). For this reason, if we have an explicit construction problem which reduces in polynomial time to AVOID, then there is some *uniform instance sequence* $(C_n)_{n \in \mathbb{N}}$ so that C_n is an AVOID instance of size $\text{poly}(n)$, there is a uniform $\text{poly}(n)$ -time algorithm which produces C_n given 1^n , and any algorithm solving AVOID *only on this sequence of instances* can be transformed into an algorithm solving our explicit construction problem. The hardness results for AVOID in [81, 34, 79, 146] are not able to show hardness for such a uniform instance sequence. Hence, it remains a possibility that we can solve AVOID on all uniform instance sequences in polynomial time, without contradicting any known hardness assumptions.

In Chapter 4 we achieve precisely this, under one additional stipulation: the AVOID instances in question have sufficiently superlinear *stretch*, e.g. of the form $C_n : \{0, 1\}^n \rightarrow \{0, 1\}^{n \log n}$ or $C_n : \{0, 1\}^n \rightarrow \{0, 1\}^{n \log \log n}$. The hardness assumptions underlying the correctness of our algorithm are of the following form: there is some language decidable in time $T(n)$, which cannot be solved in time $T(n)^\epsilon$ with $2^{\epsilon n}$ bits of advice for some positive constant ϵ . Such assumptions are well-studied and considered standard in the literature in the case $T(n) = 2^{\Theta(n)}$; the key distinction is that we need to make such assumptions in the setting $T(n) \gg 2^n$, e.g. $T(n) = 2^{2^n}$ or $2^{2^{2^n}}$. The precise details of our hardness assumptions will determine the runtime of our algorithms; under sufficiently strong assumptions we achieve polynomial time algorithms, while in other cases our algorithms will have larger (but still strongly subexponential runtimes), e.g. quasipolynomial or $2^{\text{quasipoly}(\log n)}$. Despite the exotic runtimes involved, we believe our hardness assumptions are natural and highly plausible, and yield the first complexity theoretic evidence *of any kind* that a wide class of explicit construction problems can be solved by efficient deterministic algorithms.

Some key examples of such results in the case of matrix rigidity and explicit circuit lower bounds are:

Theorem (See Theorems 4.30 and 4.31).

1. *Assuming that there is a language in $\text{TIME}[2^{2^n}]$ that is not computable in $\text{TIME}^{\text{NP}}[2^{\epsilon 2^n}]/2^{\epsilon n}$ (even infinitely often), then for some constant C there is (1) a quasipolynomial time algorithm which produces a matrix $M \in \mathbb{F}_2^{n \times n}$ which requires changing $\geq \frac{n^2}{\log^C n}$ entries to drop its rank below $\frac{n}{\log^C n}$, and (2) a language in EXP that requires Boolean circuits of size $\frac{2^n}{n^C}$ for all n .*
2. *Under a suitably strengthened hardness assumption for triply-exponential time bounds, there is a $\text{poly}(n)$ time algorithm which produces a matrix $M \in \mathbb{F}_2^{n \times n}$ which is Valiant-Rigid whenever n is a power of 2.*

At a high level, our approach is the following. An easy argument (first explained in [144]) shows that to achieve deterministic algorithms for uniform range avoidance which stretch n bits to $\gg n \log n$ bits, it suffices to give a polynomial time algorithm which can produce n -bit strings with polynomial time-bounded Kolmogorov complexity $\frac{n}{\log n}$. Another easy argument shows that to achieve this, it suffices to have a deterministic algorithm which can produce a list of $\log n$ many n -bit strings, each of which has time-bounded Kolmogorov complexity very close to n . Given such a list we can simply concatenate all of its constituent strings to achieve our target. So our task is reduced to the problem of constructing very short lists of strings, with the guarantee that one of them must have very large time-bounded kolmogorov complexity. We accomplish this by designing a complexity-theoretic PRG which *fools the test of having high Kolmogorov complexity* and whose seed length is $\leq \log \log n$; enumerating all of the seeds of this PRG yeilds our desired list. In general such a short seed length is not possible if the tests we are trying to fool have advice, but in our case we only wish to fool a *single, uniformly defined* test. Using this critical fact, we are able to recursively apply classical hardness-based pseudorandom generators [129, 83, 160] on a sequence of exponentially shrinking input lengths to iteratively reduce the seed length.

Chapter 5: Connections to Static Data Structures, Random CSP Refutation, and More

In the final chapter of our thesis, we describe a web of connections between the following three topics:

1. Algorithms for NC^0 -AVOID, which is the special case of AVOID in which the instance $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ has every output bit depending on only $O(1)$ many inputs.
2. Static data structure lower bounds in the bit probe model.
3. Cryptographic pseudorandom generators in NC^0 , and the closely related subject of semi-random CSP refutation.

Preliminaries

We will delay most of the technical definitions we need until the chapters in which they are used, but cover here some basic definitions and conventions which will be used throughout this thesis.

0.1 Basics

We review some standard mathematical notation which will be used throughout. For a natural number N use $[N] = \{1, \dots, N\}$, and for a finite set V and integer k we use $\binom{V}{k}$ to denote the set of subsets of V of size k , and $\{0, 1\}^n, \{0, 1\}^{\leq n}$ to denote the set of binary strings of length n and length $\leq n$ respectively. To every finite set X we implicitly associate the uniform distribution X , so that $x \sim X$ denotes a uniformly random sample from X , and $\mathbb{E}_{x \in X} f(x), \mathbb{E}_{x \sim X} f(x)$ are both used to denote the value $|X|^{-1} \sum_{x \in X} f(x)$ for some real-valued function f . By convention all logarithms are base 2 unless otherwise specified. Our asymptotic notation is standard (with the exception of Chapter 4 where we will introduce some specialized notation for very fast-growing functions). We use $\text{poly}(n)$ to denote an unspecified function of n , $f(n)$, which is bounded by $n^{O(1)}$.

0.2 Complexity Classes, Search Problems, and Explicit Construction Problems

We assume familiarity with the standard complexity classes such as $P, NP, E = \text{Time}[2^{O(n)}], \text{EXP} = \text{Time}[2^{\text{poly}(n)}]$, etc, which we always write in sans-serif font. For background on complexity classes and other standard notions in complexity theory, the reader may consult [11]. For a general decisional complexity class C (i.e. a set of languages) we use $\text{i.o.-}C$ to denote the set of languages L so that there exists $L' \in C$ such that for infinitely many n , L and L' contain the same n -bit strings. We say that $C \not\subseteq_{\text{i.o.}} C'$, or that “ C is separated from C' almost everywhere,” if there

exists $L \in \mathcal{C} \setminus \text{i.o.-}\mathcal{C}'$.

We proceed to define *search problems*, a primary subject of this thesis:

Definition 0.1. A search problem is a relation $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$; given an “instance” $x \in \{0, 1\}^*$, we say that $y \in \{0, 1\}^*$ is a “solution” to x if $(x, y) \in R$. We say that a function $f : \{0, 1\}^* \rightarrow \{0, 1\}^* \cup \{\perp\}$ solves the search problem R if $(x, f(x)) \in R$ whenever $f(x) \neq \perp$, and $(x, y) \in R \rightarrow f(x) \neq \perp$. A search problem R is polynomially bounded if there is a polynomial p so that for all $x \in \{0, 1\}^n$, either there exists $y \in \{0, 1\}^{\leq p(n)}$ so that $(x, y) \in R$, or else there does not exist $y \in \{0, 1\}^*$ so that $(x, y) \in R$. Finally, a search problem is said to be total if for all $x \in \{0, 1\}^*$, there exists $y \in \{0, 1\}^*$ so that $(x, y) \in R$.

We will only concern ourselves in this thesis with search problems that are polynomially bounded, and primarily focus on *total* search problems. The following are the most important complexity classes for total search problems we will consider:

Definition 0.2. We use FP to denote the set of search problems R solvable by a polynomial time algorithm; more formally, there is a function f solving R , such that $f(x)$ is computable from x in $\text{poly}(|x|)$ time. For a language L , we use FP^L (resp. $\text{FP}_{\parallel}^L$) to denote those search problems solvable in polynomial time with oracle access to L (resp., in polynomial time with nonadaptive calls to an L oracle). If $\mathcal{C}$ is a class with a polynomial time many-one complete problem, we write $\text{FP}^{\mathcal{C}}$ ($\text{FP}_{\parallel}^{\mathcal{C}}$) to denote FP^L (resp. $\text{FP}_{\parallel}^L$) for some complete problem L (the definition is invariant to the choice of complete problem).

For $1 \leq k \in \mathbb{N}$, $\text{TF}\Sigma_k^{\text{P}}$ denotes the class of polynomially bounded, total search problems R such that $R \in \Pi_{k-1}^{\text{P}}$ (where $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ is considered as a language $R \subseteq \{0, 1\}^*$ via any standard pairing function). We will be particularly interested in the cases $k = 1, 2$, which form the classes TFNP and $\text{TF}\Sigma_2^{\text{P}}$.

Next, we define the notion of reducibility amongst search problems that will be used in this thesis:

Definition 0.3. Let R_1, R_2 be polynomially bounded search problems, and C a complexity class with a polynomial time complete problem. We say that R_1 is FP^C (resp. $\text{FP}_{\parallel}^C$) many-one reducible to R_2 if there is an FP function f and a FP^C function (resp. $\text{FP}_{\parallel}^C$ function) g so that the following holds: for any $x, y \in \{0, 1\}^*$, if $(f(x), y) \in R_2$ then $(x, g(x, y)) \in R_1$.

Finally, we recall our formal definition of an explicit construction problem:

Definition 0.4. An explicit construction problem is defined by a language $\Pi \subseteq \{0, 1\}^*$, which we interpret as defining some combinatorial property of boolean strings. The associated search problem is $EC_{\Pi} = \{(x, y) \mid y \in \Pi, |y| = |x|\}$; equivalently, given 1^n , find some $y \in \{0, 1\}^n$ which has the property Π .

We observe that explicit construction problems are special cases of *sparse/unary* search problems: these are search problems with defining relations of the form $\{(x, y) \mid (|x|, y) \in R\}$ for some $R \subseteq \mathbb{N} \times \{0, 1\}^*$. Such search problems have only *one instance* of every length, and hence can always be solved by a nonuniform algorithm which hardcodes the answer as advice.

0.3 Range Avoidance

We recall here the definition of Range Avoidance, which is also abbreviated “AVOID:”

Definition 0.5 (Range Avoidance/ AVOID). *Range Avoidance is a search problem in which the instance is a boolean circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, $m > n$, and the solutions are any $y \in \{0, 1\}^m$ such that $y \notin \text{range}(C)$. If $\mathcal{C}$ is a subclass of the family of all multi-output boolean circuits, we denote $\mathcal{C}$ -AVOID the search problem in which we restrict attention to inputs C which are in the circuit class $\mathcal{C}$.*

We make a note about the history of the name “Range Avoidance.” When the above search problem was originally introduced in [96], it went under the name “EMPTY;” to be more precise, EMPTY was used to refer to a more general (harder) search problem, of which Range Avoidance is a special case and referred to as “1-EMPTY.” In the followup work [102] this name persists, although

the prefix 1 is dropped. Finally, in [144], the name Range Avoidance was coined, and since then it has become the accepted terminology, which we will use throughout this thesis. Finally, we note that in Chapter 2, we will investigate the harder variant of AVOID first studied in [96] under the name “EMPTY;” in that chapter, we refer to the harder variant as “Strong AVOID” and the standard AVOID problem as “Weak AVOID.”

We also make note of a complexity class definition which appears throughout [97, 102]:

Definition 0.6. *APEPP is the class of all polynomially bounded search problems which are polynomial time many-one reducible to Range Avoidance.*

This definition is in line with the standard practice in total search problem complexity (and complexity more generally) of grouping search problems into completeness classes characterized by some key problem. Unlike “EMPTY,” the term APEPP has not been replaced in subsequent work, however it has mostly fallen out of use. This is primarily due to the fact that there are two distinct notions of reduction that can be used to reduce problems to Range Avoidance (polynomial time and FP^{NP} respectively), both of which are interesting and seemingly distinct in their power. Rather than making two separate definitions based on the type of reduction used, we will simply refer to problems as being “FP-equivalent” or “ FP^{NP} -equivalent” to Range Avoidance (in the first case, we will also say polynomial-time equivalent).

Chapter 1: Range Avoidance and the Complexity of Explicit Constructions

In this chapter, based on work first appearing in [101], **TODO: something**

TODO: at the end, give an overview of primary results here **TODO:** include the stuff from original info which gives some background on the various explicit construction problems such as rigidity. Say something brief like: many of these problems are famous and have received intense scrutiny, we give a very brief overview of some of the work here.

1.1 Explicit Constructions and Circuit Lower Bounds are Total Search Problems

We briefly remind the reader of the main takeaway of **TODO: part something** in the introduction: we may formalize explicit construction problems, and in particular circuit lower bounds, as *total search problems*. Applying this perspective to circuit lower bounds, we can in several cases draw an exact equivalent between the *truth of a circuit lower bound* and the *existence of an algorithm for some search problem*.

Recall that an explicit construction problem is formally defined in terms of some “property” (= language) $\Pi \subseteq \{0, 1\}^n$; the goal is to output $x \in \{0, 1\}^n \cap \Pi$ given 1^n as input. In some cases (such as in the next paragraph), we will define an explicit construction only for a restricted set of input lengths $n \in \mathcal{N} \subseteq \mathbb{N}$, e.g. $\mathcal{N} = \{2^k \mid k \in \mathbb{N}\}$. In such cases, in the formalism of search problems we may take $\Pi \cap \{0, 1\}^n = \{0, 1\}^n$ for other values of n . By this convention, we see that there is an algorithm for Π if and only if there is an algorithm which gets the correct answer on all $n \in \mathcal{N}$ ¹.

We now recall **TODO: put this also in that intro section where we introduce search problem framing** the search problem corresponding to circuit lower bounds:

Definition 1.1. Let $s : \mathbb{N} \rightarrow \mathbb{N}$ be some “size bound.” Define the search problem $s(n)$ -HARD as

¹Provided $\mathbb{1}\{n \in \mathcal{N}\}$ is computable in $\text{poly}(n)$ time which will hold in all relevant cases.

follows: given 1^{2^n} , output the truth table of a boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ such that f is not computed by any circuit of size at most $s(n)$.

TODO: when putting this part in the intro see commented out part below

The connection between algorithms for this search problem, and circuit lower bounds for exponential time classes (spelled out in Section **TODO: sec** of the introduction) is then:

Observation 1.2. *The following are equivalent for any $s : \mathbb{N} \rightarrow \mathbb{N}$:*

1. *There is a language in E (resp. E^{NP}) which requires circuits of size $\geq s(n)$ for all n .*
2. *There is a polynomial time algorithm (resp. with an NP oracle) for the following explicit construction problem: given 1^{2^n} , output the truth table of a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ requiring circuits of size $\geq s(n)$.*

The same observation holds if we replace “circuits” by any restricted nonuniform model of computation, e.g. boolean formula, branching programs, ACC circuits of depth d , e.t.c.

1.2 Basic Upper Bounds on the Complexity of AVOID and Explicit Construction Problems

We start with some basic complexity upper bounds on the Range Avoidance problem and on explicit construction problems. For starters, since verifying a candidate solution for AVOID amounts to checking whether $y \notin \text{range}(C)$ for a boolean circuit C , we have:

Observation 1.3. *AVOID is a search problem in $TF\Sigma_2^P$.*

Since, by the weak pigeonhole principle, the codomain of $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, $m > n$ is at least twice as large as the domain of C , we may attempt to solve AVOID by repeatedly sampling random strings $y \sim \{0, 1\}^m$ in the codomain and testing whether $y \in \text{range}(C)$. This test is decidable with one call to an NP oracle, hence:

Observation 1.4. *AVOID is a search problem in $FZPP^{NP}$: there is a randomized algorithm which outputs a valid AVOID solution on every input C , uses an NP oracle, and has expected run time $\text{poly}(|C|)$.*

In fact, we may say more: solving AVOID reduces to the (promise) problem of hitting *dense* coNP sets:

Definition 1.5. Let coNP-HITTING be the following problem: given a boolean circuit $T : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, with the promise that

$$\Pr_{x \sim \{0,1\}^n} [\text{for all } y \in \{0, 1\}^m, T(x, y) = 1] \geq \frac{1}{2}$$

find some $x \in \{0, 1\}^n$ such that for all y , $T(x, y) = 1$.

The argument for the containment of AVOID in ZPP^{NP} in fact yields:

Observation 1.6. There is a polynomial time reduction from AVOID to coNP-HITTING.

In the next section we will show that wide a variety of important explicit construction problems reduce to AVOID; hence by the previous observation all of these problems reduce to coNP-HITTING. However, this same fact can be concluded without passing through the intermediate problem AVOID, and was observed long before the study of Range Avoidance by Santhanam [150]. For nearly all of the explicit construction problems of interest to theoretical computer scientists, it is easy to show that (1) verifying that an object has the desired property is a coNP problem, and (2) a random object has the desired property. Indeed, (2) is the means by which the object in question is shown to exist in the first place. These two properties together directly yield a reduction from the explicit construction problem to coNP-HITTING.

The classical hardness/randomness paradigm tells us that reducing a search problem to coNP-HITTING gives a *conditional* derandomization result for that problem. We can apply this conditional derandomization to AVOID and to all of the explicit construction problems reducible to it: assuming a widely-believed circuit lower bound holds, all of these problems can be solved in *deterministic* polynomial time with an NP oracle.

Theorem 1.7 ([129, 84, 100]). *If there is a language in E^{NP} (resp. E) which requires NP-oracle circuits of size $2^{\Omega(n)}$ for sufficiently large n , then there is an FP^{NP} (resp. $FP_{\parallel}^{NP}$) algorithm for*

coNP-HITTING, and hence for AVOID and all of the explicit construction problems that reduce to it.

In Section 1.4, we will strengthen the above theorem (in the case of E^{NP} lower bounds vs. FP^{NP} algorithms): assuming E^{NP} requires boolean circuits (without any oracle gates) of $2^{\Omega(n)}$ size, there is an FP^{NP} algorithm for AVOID. In fact, we will show that such an algorithm for AVOID is *exactly equivalent* to the circuit lower bound for E^{NP} .

Both of these conditional derandomization results for AVOID merely allow us to drop the randomness from an NP oracle algorithm, without removing the necessity of the NP oracle itself. Indeed, in the initial publication of the work in this chapter it was posed as the primary open problem whether there was any reasonable assumption which implied a polynomial time deterministic algorithm for AVOID. Two answers to this question emerged subsequently. On the one hand, a line of work initiated by Ilango, Li and Williams [80, 34, 146, 79] has shown that AVOID is likely *not solvable* in polynomial time in the worst case. These results will be discussed further in the closing section of this chapter. On the other hand, in Chapter 4 we will show that despite these negative results, there are reasonable hardness assumptions which would imply polynomial time algorithms for the AVOID instances which arise from many of our reductions in the next section. As a result, under these hardness assumptions we get conditional polynomial time algorithms to solve various important (and currently unsolved) explicit construction problems, such as the construction of highly rigid matrices.

1.3 Reductions to Range Avoidance

In this section, we show that a wide-reaching sample of the most important and well-studied explicit construction problems in theoretical computer science can be reduced in polynomial time to AVOID. We interpret these results as giving overwhelming evidence that AVOID is the right problem with which to characterize the complexity of explicit construction problems. More specifically, the results here show that AVOID is at least as hard as most of the explicit construction problems we care about; in the following section we will show that it is also *at most as hard* as some of these

same explicit construction problems (although the notion of “as hard” will involve a weaker form of reduction in the next section then we are able to use here). Together, these facts indicate that *AVOID* *precisely* captures the complexity of explicit construction problems.

We start with an essential and basic ingredient used in most of our reductions: efficient and information theoretically-optimal encodings of low weight binary strings:

Lemma 1.8. [62] *For any $k \leq n$, there exists a map $\Phi : \{0, 1\}^{\lceil \log \binom{n}{k} \rceil} \rightarrow \{0, 1\}^n$ computable in $\text{poly}(n)$ time such that every n -bit string of weight k is in the range of Φ .*

We typically use this in combination with the following standard bounds which hold for all $n \in \mathbb{N}$ and $\epsilon \in (0, 1)$:

1. $\log \binom{n}{(\frac{1}{2} - \epsilon)n} \leq n - \epsilon^2 n$
2. $\log \binom{n}{\epsilon n} \leq H(\epsilon)n$

Here $H(\cdot)$ denotes the binary entropy function. If we want to encode a string of weight *at most* $(\frac{1}{2} - \epsilon)n$ (resp. ϵn) it will be sufficient for our purposes to take a union bound over the possible values $k \leq (\frac{1}{2} - \epsilon)n$ of the weight. Since $\log \binom{n}{k}$ is increasing in k up to $k = \lfloor \frac{n}{2} \rfloor$ this will incur at most an additive $\lceil \log n \rceil$ bit increase in the encoding length over the above bounds, e.g. it will allow us to encode a string of weight at most $(\frac{1}{2} - \epsilon)n$ using $\leq \lceil n - \epsilon^2 n \rceil + \lceil \log n \rceil$ bits.

1.3.1 Hard Functions, Random Strings, and Pseudorandom Generators

The first and most important reduction we cover is the construction of truth tables of boolean functions which require exponential circuit complexity. Recall the definition of $s(n)$ -HARD: given 1^{2^n} , output the truth table of a boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ such that f is not computed by any circuit of size at most $s(n)$. Shannon famously showed that, for $s(n) \leq \frac{2^n}{n}$, this problem is *total* for n sufficiently large. We show here that for $s(n) \leq \frac{2^n}{2n}$, this problem reduces in polynomial time to *AVOID*. Recall that the input to this problem is 1^{2^n} , which has length 2^n , so when we say our reduction runs in “polynomial time” we mean $\text{poly}(2^n)$ time.

Theorem 1.9. $\frac{2^n}{2^n}$ -HARD reduces in polynomial time to AVOID.

Proof. This proof directly follows Shannon’s aforementioned classical argument. We construct an instance of AVOID in the form of a circuit C which maps an encoding of a circuit to its corresponding truth table. C interprets its input as a circuit on n bits (using an encoding of circuits to be described below), evaluates it on every possible input to generate a 2^n bit truth table, and outputs the truth table.

We now describe the encoding of circuits used by C , which will guarantee that any length- 2^n truth table f which has a circuit of size $\frac{2^n}{2^n}$ is in the range of C . Given a circuit of size $s \geq n$ on n inputs computing f , for each of its s gates we can use 2 bits to encode whether it is an $\wedge$, $\vee$, or $\neg$ gate, and an additional $2 \log(s + n) \leq 2 \log s + O(1)$ bits to specify its inputs. We can then use an additional $\log s$ bits to specify which gate is the terminal output gate. Overall this requires $2s \log s + O(s)$ bits. It is clear that from such an encoding, C can efficiently decode the represented circuit and evaluate it on all possible input values. For $s \leq \frac{2^n}{2^n}$, we have that $2s \log s + O(1) < 2^n$ for n sufficiently large. So C is indeed a valid instance of AVOID, and any string outside the range of C is a solution to $\frac{2^n}{2^n}$ -HARD. It is also clear from the above description that C can be constructed in $\text{poly}(2^n)$ time.

□

Following the original publication of this work, it was shown in [29] that the above argument could be sharpened using a more involved circuit encoding procedure:

Theorem 1.10. [29] $\frac{2^n}{n}$ -HARD reduces in polynomial time to AVOID.

A related problem was studied in [97], referred to there as “COMPLEXITY.” Rather than an explicit construction problem with sparse input, in this problem you are given as input one truth table $f : \{0, 1\}^n \rightarrow \{0, 1\}$, and asked to produce another truth table g such that g requires large circuits, even with access to f -oracle gates. More formally:

Definition 1.11. The problem COMPLEXITY is defined as follows: given an $f : \{0, 1\}^n \rightarrow \{0, 1\}$, find another $g : \{0, 1\}^n \rightarrow \{0, 1\}$ such that g requires f -oracle circuits of size $\frac{2^n}{n^2}$.

In [97] the following is shown:

Theorem 1.12. *COMPLEXITY reduces in polynomial time to AVOID.*

Finally, we discuss another important generalization of Theorem 1.9 due to subsequent work Ren, Santhanam and Wang [144]. An important contribution of [144] was to emphasize the importance of “parameterizing” AVOID instances in two key ways. For an AVOID instance $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, we may classify it according to (1) the *stretch* of the instance, which denotes the quantitative relationship between m and n , and (2) the *circuit complexity* C , e.g. whether it is a constant depth boolean formula, an unbounded depth formula, or a fully general boolean circuit. When reducing the construction of hard truth tables to AVOID, there is a natural tradeoff between the quantitative strength of the lower bound and stretch of AVOID instance: if we wish to produce $f : \{0, 1\}^n \rightarrow \{0, 1\}$ hard for circuits of size s , we can reduce to an AVOID instance which stretches $\text{poly}(s(n))$ bits to 2^n bits. More interestingly, [144] show that if we wish to produce explicit truth tables hard for some class of circuits C (e.g. formulas, branching programs, circuits), we can reduce this problem to an AVOID instance C where each output bit of C is in the *same class* C . This will hold (using the same proof as in Theorem 1.9) provided that the “truth table generator” mapping circuits in the class C to their truth tables is computable in the same class C . This is sometimes referred to as a circuit class having the “universal property,” and many important classes of restricted circuits are known to have it (see [144] for a further discussion). As a consequence we have:

Theorem 1.13 ([144]). *Let C be a circuit class with the universal property, for example $C \in \{AC^0, ACC^0, TC^0, NC^1, L\}$. Then producing $f : \{0, 1\}^n \rightarrow \{0, 1\}$ requiring C -circuits of size $s(n)$ is polynomial time reducible to solve a C -AVOID instance stretching $\text{poly}(s(n))$ bits to 2^n bits.*

As a corollary, if AVOID for C -circuits is solvable in polynomial time (resp. with an NP oracle) then E (resp. E^{NP}) requires C -circuits of size $2^{\Omega(n)}$.

The second part (“as a corollary”) follows from Observation 1.2 connecting algorithms for hard truth table construction and circuit lower bounds.

Next, we turn our attention to the explicit construction of pseudorandom generators.

Definition 1.14. *We will say that a sequence $R = (x_1, \dots, x_m)$ of n -bit strings is a pseudorandom generator if, for all n -input circuits of size n :*

$$|\Pr_{i \sim m} [C(x_i) = 1] - \Pr_{x \sim \{0,1\}^n} [C(x) = 1]| \leq 1/n$$

Standard applications of the probabilistic method show that such pseudorandom generators exist of size polynomial in n . In particular, for some polynomial $p(n)$ we may define the following explicit construction problem:

Definition 1.15. *$p(n)$ -PRG is the following search problem: given 1^n , output a pseudorandom generator $R = (x_1, \dots, x_m)$, $x_i \in \{0, 1\}^n$, $m = p(n)$.*

A polynomial time algorithm for $p(n)$ PRG would suffice to derandomize BPP [129]. We will show here that the explicit construction of a PRG of size n^6 is reducible to AVOID. As noted in **TODO: somewhere in the intro?**, the results of Impagliazzo and Wigderson [84] imply that PRG reduces directly to $2^{\epsilon n}$ -HARD for any constant $\epsilon > 0$, so a reduction of PRG construction to AVOID follows from Theorem 1.9. However, we provide here a much simpler direct proof that PRG construction reduces to AVOID, relying only on Yao's next bit predictor lemma, and neither the nearly disjoint subsets construction of Nisan and Wigderson [129] nor the rather involved worst-case to average-case reductions of Impagliazzo and Wigderson [84]. Together with our completeness result in Section 1.4, this gives an alternative, self-contained proof that worst-case-hard truth tables can be used to construct pseudorandom generators (although it yields a weaker result, as our derandomization will require an NP oracle).

The basic idea of the proof is as follows: if R fails to be a pseudorandom generator for circuits of size n , then applying Yao's lemma there exists a next bit predictor circuit P of size $O(n)$ which can predict the i^{th} bit of most strings in R from the previous $i - 1$ bits. This will allow us to compress R by omitting the i^{th} bits of all the strings in R and replacing them with the predictor P and some advice bits which will correct the errors made by P .

Theorem 1.16. n^6 -PRG reduces in polynomial time to AVOID.

Proof. We start by describing the AVOID instance C . C interprets its input as representing a list $R^- = (z^1, \dots, z^{n^6})$ of n^6 strings of length $n - 1$, a circuit P of size cn (for a fixed universal constant c to be defined later) with $n - 1$ input bits and one output bit, a $\log n$ -bit index $j \in [n]$, and a string S encoding an n^6 -bit string with weight at most $\frac{n^6}{2} - n^4$. Given these, C feeds each $n - 1$ -bit string $z^i \in R^-$ through P , then inserts the output as an extra bit at the j^{th} position of z^i to obtain an n -bit string x^i . In the final step, C decodes the n^6 -bit string represented by S , and flips the j^{th} bit of x^i if and only if the i^{th} position of S has a 1. C then outputs $R = (x^1, \dots, x^{n^6})$.

It is clear that C can be implemented as a circuit of size polynomial in n , and further that C can be constructed in polynomial time given 1^n . We now show there are fewer inputs than outputs. Note that the input size is equal to the number of bits needed to specify R^-, P, j, S , which is $n^7 - n^6 + \tilde{O}(n) + \log n + \text{bits}(S)$, where $\text{bits}(S)$ is the number of bits needed to specify S . Since S is an n^6 -bit string with weight at most $\frac{n^6}{2} - n^4 = n^6(\frac{1}{2} - \frac{1}{n^2})$, we can apply Lemma 1.8 with $\epsilon = \frac{1}{n^2}$ to give an encoding for S using $n^6(1 - \frac{1}{n^4}) + \log(n^6) = n^6 - n^2 + 6 \log n$ bits. Thus the overall number of bits of input is at most $n^7 - n^6 + \tilde{O}(n) + \log n + n^6 - n^2 + 6 \log n = n^7 + \tilde{O}(n) + 7 \log n - n^2$ which is strictly less than the n^7 bits of output (for sufficiently large n). So this is indeed a polynomial time reduction to a valid instance of AVOID. It remains to show that any string outside the range of C is a pseudorandom generator.

Let $R = (x^1, \dots, x^{n^6})$ be a sequence of n -bit strings which fails to be a pseudorandom generator; we aim to show that R lies in the range of C . By definition, there exists a circuit D of size n such that:

$$|\Pr_{i \sim [n^6]} [D(x^i) = 1] - \Pr_{x \sim \{0,1\}^n} [D(x) = 1]| > 1/n$$

For $x \in \{0,1\}^n$ and $j \in [n]$, let x_{-j} be the $n - 1$ -bit string obtained by deleting the j^{th} bit of x . By Yao's next bit predictor lemma [173] [161], the previous inequality implies the existence of an index $j \in [n]$ and a circuit $P : \{0,1\}^{n-1} \rightarrow \{0,1\}$ of size cn for some fixed universal constant c ,

such that

$$\Pr_{i \sim [n^6]} [P(x_{-j}) = x_j] > \frac{1}{2} + \frac{1}{n^2}$$

Since R has size n^6 , this implies that P correctly guesses the j^{th} bit of x^i from the other $n-1$ bits for at least $n^6(\frac{1}{2} + \frac{1}{n^2}) = \frac{n^6}{2} + n^4$ of indices $i \leq n^6$. So if we let S be the n^6 -bit string with a 1 at the indices i where P guesses the wrong value of x_j^i , we see that the weight of S is at most $\frac{n^6}{2} - n^4$ as required. Taking $R^- = (x_{-j}^1, \dots, x_{-j}^{n^6})$, we have that from R^-, P, j, S , our AVOID instance will successfully reconstruct R . Thus, any string outside the range of C is a pseudorandom generator. $\square$

Combining this with the prior discussion, we have:

Corollary 1.17. *Every language in BPP is polynomial time many-one reducible to AVOID.*

Finally, we turn to the construction of strings of high time-bounded Kolmogorov complexity:

Definition 1.18. *Let U be any fixed Turing machine, and let $T : \mathbb{N} \rightarrow \mathbb{N}$ be a time bound. For a string x , $K_U^T(x)$ denotes the length of the shortest string y such that U outputs x on input y in $\leq T(|x|)$ steps.*

For a Turing machine U , time bound T , and “length” parameter $\ell : \mathbb{N} \rightarrow \mathbb{N}$ we define the following explicit construction problem (K_U^T, ℓ) -RANDOM: given 1^n , output an n bit string x such that $K_U^T(x) \geq \ell(n)$. In the special case $\ell(n) = n - 1$ we omit the parameter ℓ .

Theorem 1.19. *For any fixed Turing machine U and fixed polynomial time bound T , K_U^T -RANDOM reduces in polynomial time to AVOID.*

Proof. We construct a circuit C with $n - 1$ input bits and n output bits as follows. If the input is of the form 0^*1y (0^* denotes an arbitrary length string of zeros), C simulates U on y for $t(n)$ steps and outputs the result, padded/truncated to length n if it is not already. Otherwise, C outputs 0^n . It is clear that C can be produced in polynomial time given 1^n for any fixed U, t . Further, for any n -bit string x with $K_U^t(x) \leq n - 2$, there is some y of length $k \leq n - 2$ such that U outputs x on input y in $t(n)$ steps, and so C will output x on input $0^{n-k-2}1y$. So any string x outside the range of C must have $K_U^t(x) \geq n - 1$. $\square$

1.3.2 Strongly Explicit Extractors

Here we describe a technique which allows us to reduce to AVOID the construction of circuits computing functions $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ with certain strong pseudorandom properties. The reductions here run in time $\text{poly}(n)$, and produce a circuit for f of size $\text{poly}(n)$; this is in contrast to the reduction in Theorem 1.9 which produce n -bit boolean functions in time $2^{O(n)}$ given as truth tables.

The reductions here hinge on a simple high-level construction involving k -wise independent generators. To explain the method in sufficient generality we require the following definition:

Definition 1.20. Let $\mathcal{A} \subseteq \binom{\{0,1\}^n}{k}$, $\mathcal{B} \subseteq (\{0, 1\}^n)^k$ be given. We say that $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a $(\mathcal{A}, \mathcal{B})$ extractor if whenever $(x_1, \dots, x_k) \in \mathcal{A}$, $(f(x_1), \dots, f(x_k)) \notin \mathcal{B}$.

The idea behind the definition is as follows. We think of $\mathcal{A} \subseteq \{S \subseteq \{0, 1\}^n, |S| = k\}$ as defining a class of *structured* subsets of k inputs, with $|\mathcal{A}| \ll \binom{2^n}{k}$. For any fixed set of k inputs $(x_1, \dots, x_k)$ and a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$, we may think of $(f(x_1), \dots, f(x_k)) \in \mathcal{B}$ as some unlikely event, i.e. $|\mathcal{B}| \ll 2^{nk}$. We then have that f is an $(\mathcal{A}, \mathcal{B})$ extractor iff $f(S)$ avoids the event $f(S) \in \mathcal{B}$ whenever $S \in \mathcal{A}$. If $\Pr[(y_1, \dots, y_k) \in \mathcal{B}]$ is very small for a random sequence $y_1, \dots, y_k \sim \{0, 1\}^n$, then for any fixed $S = \{x_1, \dots, x_k\} \in \mathcal{A}$ of size k , if we choose f uniformly at random we are very likely to have $f(S) = \{f(x_1), \dots, f(x_k)\} \notin \mathcal{B}$. If $|\mathcal{A}|$ is also sufficiently small than we can union bound over $S \in \mathcal{A}$ and argue that a random f will be an $(\mathcal{A}, \mathcal{B})$ extractor with high probability.

This existence argument can be improved in the following sense: say that instead of choosing the function $f \sim \{0, 1\}^n \rightarrow \{0, 1\}^n$ uniformly at random, we sampled it from a k -wise independent distribution $\mathcal{F}$: for every fixed $\{x_1, \dots, x_k\} \in \{0, 1\}^n$ distinct, $(f(x_1), \dots, f(x_k))$ is distributed uniformly on $(\{0, 1\}^n)^k$ when $f \sim \mathcal{F}$. Then in this case we have $f(S) \notin \mathcal{B}$ with the same high probability over $f \sim \mathcal{F}$, and the rest of the argument goes through as before. Thus we can argue for the existence of an $(\mathcal{A}, \mathcal{B})$ extractor inside of any k -wise independent function family $\mathcal{F}$.

The following reduction formalizes this using the standard construction of k -wise indepen-

dent families by univariate polynomials over a finite field. The reduction works provided that the “smallness” of $\mathcal{A}, \mathcal{B}$ can also be exhibited by reductions to Range Avoidance, i.e. we can cover these sets by the range of some explicit length-expanding functions $A : \{0, 1\}^\ell \rightarrow \binom{\{0, 1\}^n}{k}$, $B : \{0, 1\}^r \rightarrow \{0, 1\}^{nk}$. The technique here is inspired by a construction of Razborov [141] showing that strongly explicit Ramsey graphs exist in $\text{AC}^0[2]$.

Theorem 1.21. *Let $A : \{0, 1\}^\ell \rightarrow \binom{\{0, 1\}^n}{k}$, $B : \{0, 1\}^r \rightarrow \{0, 1\}^{nk}$ be given as circuits. Provided $\ell + r < nk$, the following problem is reducible in polynomial time to Range Avoidance: output the description of a polynomial size circuit $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ which is an $(\text{range}(A), \text{range}(B))$ extractor. Moreover:*

1. *The circuit f produced by the reduction lies in $\text{AC}^0[2]$.*
2. *If $A, B \in \text{NC}^2$ the Range Avoidance instance produced by the reduction also lies in NC^2 .*

Proof. We construct an instance of Range Avoidance $C : \{0, 1\}^{\ell+r} \rightarrow \{0, 1\}^{nk}$. Elements of $\{0, 1\}^{nk}$ are interpreted as vectors $\bar{\alpha} = (\alpha_1, \dots, \alpha_k) \in \mathbb{F}_{2^n}$, and to each $\bar{\alpha}$ we associate the function $f_{\bar{\alpha}} : \{0, 1\}^n \rightarrow \{0, 1\}^n$ given by:

$$f_{\bar{\alpha}}(x) = \sum_{i=1}^k \alpha_i x^{i-1}$$

where $\{0, 1\}^n$ and $\mathbb{F}_{2^n}$ are associated in some standard way. We will construct C so that $f_{\bar{\alpha}}$ fails to be a $(\text{range}(A), \text{range}(B))$ extractor, then $\bar{\alpha} \in \text{range}(C)$ which will give the theorem. The fact that $f_{\bar{\alpha}}$ lies in $\text{AC}^0[2]$ for any choice of $\bar{\alpha}$ follows from results of [71] on the complexity of arithmetic over $\mathbb{F}_{2^n}$.

C is constructed as follows. We interpret the input as encoding (a, b) with $a \in \{0, 1\}^\ell$ and $b \in \{0, 1\}^r$. Let $(x_1, \dots, x_k) = A(a)$ and let $(v_1, \dots, v_k) = B(b)$. We then compute the unique degree $k - 1$ polynomial $p \in \mathbb{F}_{2^n}[x]$ such that $p(x_i) = v_i$ for all i ; this can be accomplished in polynomial time using Gaussian elimination on the corresponding Vandermonde matrix (by assumption $A(a)$ outputs a list of k distinct strings). Finally C outputs the coefficients $(\alpha_1, \dots, \alpha_k)$ of p . Clearly we have $\bar{\alpha} \in \text{range}(C)$ whenever $f_{\bar{\alpha}}$ fails to be an $(\text{range}(A), \text{range}(B))$ extractor.

The complexity of C is upper bounded by the complexity of computing A, B , together with that of Gaussian elimination over $\mathbb{F}_{2^n}$ which lies in NC^2 . $\square$

The above Lemma can be used to reduce the construction of various “structured seed extractors” with near-optimal parameters to Range Avoidance. An important case of a structured seed extractor is a 2-source extractor:

Definition 1.22. *We say that a function $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ is a (k, ϵ) 2-source extractor if the following holds: for any two sets $X, Y \subseteq \{0, 1\}^n$ of size 2^k , $|\Pr_{x \sim X, y \sim Y}[f(xy) = 1] - \frac{1}{2}| \leq \epsilon$.*

To see that this is a special case of our general $(\mathcal{A}, \mathcal{B})$ -extractor paradigm, consider our two-source extractor f to be a function $f : \{0, 1\}^{2n} \rightarrow \{0, 1\}$, for which we only care about its first output bit. The family of simple sets $\mathcal{A}$ is the set of “combinatorial rectangles:” sets of the form $L \times R$ for some $L, R \subseteq \{0, 1\}^n$, $|L| = |R| = 2^k$. Such a combinatorial rectangle has size 2^{2k} but can be encoded using $2^{k+1}n$ bits, which yields the appropriate circuit A for the reduction. The “bad event” $\mathcal{B}$ is that the first bit of the function f is biased towards 1 or 0 over the rectangle; here we can use the sparse string encodings from Lemma 1.8 to form the circuit B . More formally we have:

Theorem 1.23. *There is a $\text{poly}(n, \epsilon^{-1})$ -time reduction from strongly explicit constructing $(\log n + 2 \log(1/\epsilon) + 2, \epsilon)$ 2-source extractors to AVOID. The circuits output by the reduction lie in AC_2^0 .*

Proof. Let $m = 2^k$. We require circuits A, B of the following form. A takes $2mn$ bits of input and outputs subsets of $\{0, 1\}^{2n}$ of size m^2 ; the range of A contains all sets of the form $\{(\ell, r) \mid \ell \in L, r \in R\}$ for some $|L| = |R| = m$, $L, R \subseteq \{0, 1\}^n$. B takes as input strings of length $m^2(2n - 1) + m^2 - \epsilon^2 m^2 + O(\log m)$ and outputs elements of $(\{0, 1\}^{2n})^{m^2}$; its range contains all sequences $x^1, \dots, x^{m^2} \in \{0, 1\}^{2n}$ such that the bits $x_1^1, \dots, x_1^{m^2}$ have bias $> \epsilon$ (the fraction which equal 1 differs from $\frac{1}{2}$ by more than ϵ). A may be constructed in the obvious way (it takes the description of L, R and outputs all cross-concatenations). B may be constructed using Lemma 1.8. We may now apply Theorem 1.21 to get a reduction to AVOID, provided $2mn + m^2(2n - 1) + m^2 - \epsilon^2 m^2 + O(\log m) < 2nm^2$; for n sufficiently large this is satisfied provided $m \geq \frac{4n}{\epsilon^2}$, in other words $k \geq \log n + 2 \log(1/\epsilon) + 2$. $\square$

Recent advances have yielded 2-source extractors with parameters very close to optimal, but still not meeting the bounds attainable here [113, 27]. Other kinds of structured-seed extractors studied in the literature include extractors for varieties and for affine spaces, see [42, 28]; similar reductions exist for constructions of each of these using Theorem 1.21.

1.3.3 Miscellaneous Reductions

We discuss here a few more important explicit construction problems which can be shown reducible to Range Avoidance. The first is the problem of *matrix rigidity*, famously introduced by Valiant [162]:

Definition 1.24. [162] We say that $n \times n$ matrix M over $\mathbb{F}_q$ is (r, s) rigid if for any matrix $S \in \mathbb{F}_q^{n \times n}$ with at most s non-zero entries, $M + S$ has rank greater than r .

Definition 1.25. For any $q : \mathbb{N} \rightarrow \mathbb{N}$ such that $q(n)$ is a prime power for all n , and let $r, s : \mathbb{N} \rightarrow \mathbb{N}$ be given. Then $(r(n), s(n), q)$ -RIGID is the following search problem: given 1^n , output an $n \times n$ matrix M over $\mathbb{F}_{q(n)}$ which is $(r(n), s(n))$ rigid.

Theorem 1.26. For any constant $\epsilon > 0$ so that $H(\epsilon) + 3\epsilon < 1$, and any efficiently computable $q(n)$ satisfying the above, $(\epsilon n, \epsilon n^2, q(n))$ -RIGID reduces in polynomial time to AVOID.

In particular $\epsilon = \frac{1}{8}$ suffices.

Proof. Let M be any $n \times n$ matrix over $\mathbb{F}_q$ which is not (r, s) rigid. So there exists an $n \times r$ matrix L , an $r \times n$ matrix R , and an $n \times n$ matrix S with at most s non-zero entries, such that $M = LR + S$. It is clear that from the descriptions of L, R, S we can efficiently compute M .

L and R can each be described explicitly using $nr \log q$ bits. For S , we encode it by specifying an n^2 -bit string T of weight s denoting the entries of S which are nonzero, together with an $s \log q$ -bit string C giving the correct values of the nonzero entries. Using this scheme we may assume s is exactly equal to ϵn^2 since if it is smaller we can add some arbitrary indices and put their unchanged values in C . Applying the encoding scheme in Lemma 1.8 for T , overall the number of bits in this

encoding is at most $\log \binom{n^2}{s} + (2nr + s) \log q$. Setting $r = \epsilon n$ and $s = \epsilon n^2$ this is at most:

$$\begin{aligned} \log \binom{n^2}{\epsilon n^2} + (3\epsilon n^2) \log q &\leq \\ H(\epsilon)n^2 + (3\epsilon n^2) \log q &\leq \\ (H(\epsilon) + 3\epsilon)n^2 \log q & \end{aligned}$$

By our choice of ϵ this is bounded above by $(1 - \Omega(1))n^2 \log q$ bits, whereas an arbitrary matrix in $\mathbb{F}_q^{n \times n}$ requires $n^2 \log q$ bits. $\square$

In Section 1.5 of this chapter, we will show that 2 problems, namely the construction of communication matrices outside $\text{PSPACE}^{\text{cc}}$ and data structure problems with near maximal bit-probe complexity, can be reduced directly to the construction of truth tables of circuit complexity $\frac{2^n}{n}$ (or indeed, complexity $s(n)$ for various smaller exponential bounds $s(n)$). Using Theorem 1.9 this yields a reduction of each of these explicit construction problems to AVOID; we will leave the formal definition of these problems until Section 1.5. Finally, we include one more important problem which was subsequently shown to reduce to AVOID in [65]:

Definition 1.27 (GV Codes). *For $r, p \in (0, 1)$, we say that a matrix $G \in \mathbb{F}_2^{\lceil rn \rceil \times n}$ is the generator matrix of an (r, p) linear code if it has full row rank and every nonzero codeword generated by G (i.e. in the span of the rows of G) has hamming weight at least pn . We define the search problem (r, p) -CODE as: given 1^n , output a matrix $G \in \mathbb{F}_2^{\lceil rn \rceil \times n}$ which is the generator matrix of an (r, p) linear code.*

By the classical GV bound, for every pair of fixed constants r, p with $r < 1 - H(p)$, the problem (r, p) -CODE is total for n sufficiently large (recall that H is the binary entropy function). It is a central open problem in the field of coding theory to find codes meeting this bound. In [65] it is shown that the explicit construction of such codes reduces to AVOID:

Theorem 1.28 ([65]). *For any $r, p \in (0, 1)$ with $r < 1 - H(p)$, (r, p) -CODE reduces in polynomial time to AVOID.*

1.4 Completeness

In this section, we exhibit several problems which are *equivalent* to AVOID under suitable reductions. The first two complete problems are explicit construction problems investigated in the previous section: construction of strings of high time-bounded Kolmogorov complexity, and construction of hard truth tables. At a conceptual level we view these completeness results as important in at least two respects:

1. The results in the previous section merely show that AVOID is *at least as hard* as many of the central explicit construction problems studied in the literature. These completeness results indicate that AVOID is also *at most as hard* as some of these problems. This indicates that AVOID precisely meets our goal **TODO: state this goal in the intro** of defining the *right complexity class for explicit constructions*.
2. These results show that solving either of these two complete explicit construction problems would necessitate solving *all of the explicit construction problems in the last section*; in other words, hard truth tables and Kolmogorov-random strings are *the hardest explicit construction problems*. This is especially interesting in the case of hard truth tables; the construction of these objects (equivalently, the quest for circuit lower bounds) is a central goal of complexity theory, and we show here that meeting this goal would simultaneously unlock new explicit constructions of a wide-reaching class of combinatorial objects.

In addition to these conceptual takeaways, our completeness result for hard truth tables is of a special technical interest, since it implies a suprising hardness amplification theorem for the class E^{NP} , and its proof yields a construction of a *hardness extractor*.

Finally, in the last subsection we show the completeness of another $TF\Sigma_2^P$ problem “UNDIRECTED REMOTE VERTEX” which was shown in [98] to reduce in polynomial time to AVOID. We show here that a reduction exists in the other direction as well. Unlike the previous results, this problem is not an explicit construction problem.

1.4.1 Complete Explicit Construction Problems

We start with the following simple completeness result, which uses a more restricted form of reduction than the subsequent reductions: we only need to make *nonadaptive* calls to an NP oracle for this reduction, whereas the subsequent reductions will require adaptive calls to an NP oracle.

Theorem 1.29. *Let $\epsilon > 0$ be any fixed constant, $T(n) = n^{1+\epsilon}$, $\ell(n) = n - n^\epsilon$. Let U be any fixed universal Turing machine. Then (K_U^T, ℓ) -RANDOM is equivalent to AVOID under many-one $\text{FP}_{\parallel}^{\text{NP}}$ reductions.*

Proof. We have already shown a polynomial time many-one reduction from (K_U^T, ℓ) to AVOID in the previous section; we now show the reverse direction. Let $C : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ be an avoid instance and let s be the circuit size of C . For some parameter k to be chosen later, consider the circuit $C^{\otimes k} : \{0, 1\}^{kn} \rightarrow \{0, 1\}^{kn+k}$ which is obtained by concatenating k copies of C . Let $m = kn + k$. Any string in the range of $C^{\otimes k}$ has $T := O(m + ks)$ -time bounded Kolmogorov complexity at most $s + m + O(\log m)$: we simply supply the description of C , together with the length kn input and a $O(\log m)$ size program description of the following program: given a length m string and a description of C , break the m -bit string into k contiguous chunks of length n , applies C to each chunk, and outputs the result. Setting $k \geq (s + n)^{\frac{2}{\epsilon}}$ we have that $n, s \leq k^{\epsilon/2} \leq m^{\epsilon/2}$, therefore $T \leq m^{1+\epsilon}$ and $kn \leq m - m^\epsilon$. Now, if $y \in \{0, 1\}^m$ is a string with T -time bounded Kolmogorov complexity $> kn$, then, after dividing y into k blocks of $n + 1$ bits, we obtain a list of k strings, one of which must be outside the range of C . Using k parallel calls to an NP oracle we may determine which of these are indeed AVOID solutions for C and output the lexicographically first. □

For the remaining two completeness results we will require the following reduction which allows us to increase the *stretch* of AVOID instances using an NP oracle.

Lemma 1.30. *For any $m(n) \leq \text{poly}(n)$, there is a FP^{NP} many-one reduction from AVOID to itself, which maps instances $C : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ to instances $C' : \{0, 1\}^n \rightarrow \{0, 1\}^{m(n)}$.*

Proof. Let $C : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ be given. For a positive integer i , we define $C^i : \{0, 1\}^n \rightarrow \{0, 1\}^{n+i}$ inductively as follows. For the base case, C^1 is simply defined as C . For $i > 1$, we first compute C^{i-1} on the input x to obtain a string x' of length $n + i - 1$, and then compute C on the first n bits of x' and concatenate the output with the remaining bits of x' . Since C replaces the first n bits with $n + 1$ bits, if C^{i-1} has output length $n + i - 1$ then C^i has output length $n + i$. We claim that for any positive i , if we are given an $n + i$ -bit string outside the range of C^i , we can find an n -bit string outside the range of C in $i \cdot \text{poly}(|C|)$ time using an NP oracle. In particular, we prove by induction on i that this can be accomplished with i calls to an NP search solver. For $i = 1$ this is trivially the case, since $C^1 = C$. Now say $i > 1$. Let $y = xz$ be outside the range of C^i for some $x \in \{0, 1\}^{n+1}$, $z \in \{0, 1\}^{i-1}$. We use the NP search solver to find a preimage of x or report that none exist. If x has no preimage we have solved AVOID for C and are finished. Otherwise, we are given some preimage $x' \in \{0, 1\}^n$ for x under C . So then $x'z$ must be outside the range of C^{i-1} , since if it were not then $xz = y$ would have a preimage under C^i . By induction we may use this string $x'z$, and $i - 1$ additional NP search oracle calls, to find an AVOID solution for C . Setting $i = m(n)$ we get the stated result. $\square$

We now proceed to the main completeness result, that AVOID reduces to the construction of truth tables with $2^{\Omega(n)}$ circuit complexity. As mentioned before **TODO: mention in intro**, the core of this theorem was originally proven by Jeřábek [91], and the main construction underlying the reduction dates back further to the work of Goldreich, Goldwasser, and Micali [60] and Paris, Wilkie and Woods [135]. Jeřábek's result is phrased in the language of proof complexity, stating that the theorem asserting the existence of hard Boolean functions is *equivalent* to the empty pigeonhole principle in a particular theory of Bounded Arithmetic. We demonstrate below that when translated to the language of search problems and explicit constructions, his proof yields a FP^{NP} reduction from AVOID to the problem of constructing a hard truth tables.

Theorem 1.31. *Let $\epsilon > 0$ be any constant. Then AVOID is FP^{NP} many-one reducible to $2^{\epsilon n}$ -HARD.*

Proof. By Lemma 1.30 we know that AVOID is FP^{NP} many-one reducible to the case of in-

stances stretching n bits to $2n$ bits. Now, let $C : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ be such an instance and let $k = 2\lceil \log |C| \rceil \lceil \frac{1}{\epsilon} \rceil$. Consider the following map $C^* : \{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$, defined informally as follows: given a string $x \in \{0, 1\}^n$, apply C once to get two n -bit strings, then apply C to both of those n -bit strings to get four, and continue k times until we have 2^k many n -bit strings, or equivalently a $2^k n$ -bit string. This process is illustrated in Figure 1.1 below:

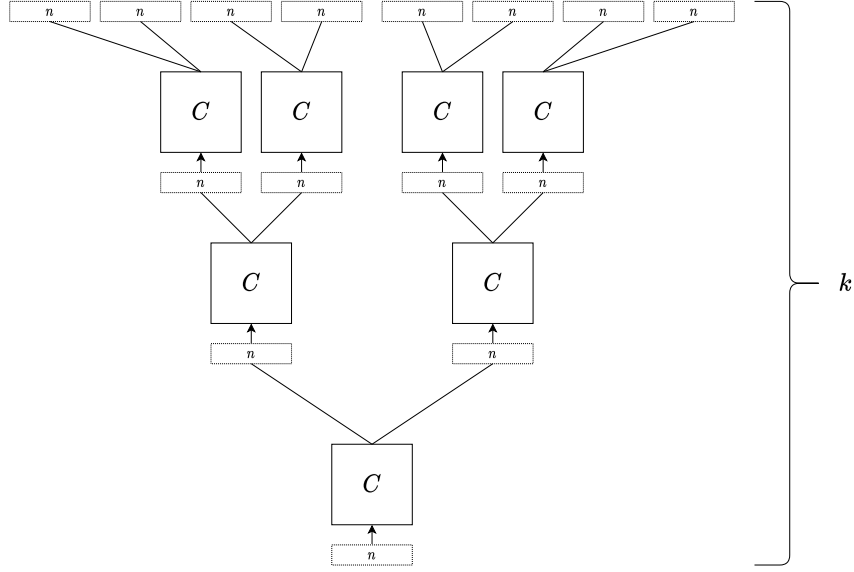


Figure 1.1: Extending a map $C : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ to a map $C^* : \{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$. Dotted boxes indicate the number of bits along a wire.

To define this function more formally, first we define the following maps $L, R : \{0, 1\}^{2n} \rightarrow \{0, 1\}^n$, where L takes a $2n$ -bit string and outputs the first n bits, and R takes a $2n$ -bit string and outputs the last n bits. Given a nonempty sequence $\sigma_1, \dots, \sigma_t \in \{L, R\}^*$, let $C^\sigma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be the function $\sigma_t \circ C \circ \dots \circ \sigma_1 \circ C$. Now, given a binary string, we can associate it with such a sequence by associating 0 with L and 1 with R , and so we will abuse notation and write C^x for a binary string x as shorthand for C^σ where σ is the sequence of L, R associated with the binary string x . We are now ready to formally define our function C^* . As C^* is a map $\{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$, we can think of the output as being clumped into 2^k blocks, each containing n bits. Given this terminology, C^* behaves as follows: on input x , the i^{th} block of the output of C^* will be $C^{\|i\|}(x)$, where $\|i\|$ denotes the standard representation of i as a k -bit binary string.

From here, the proof proceeds in two steps. First, we show that, by setting $m = n2^k =$

$\text{poly}(|C|)$, any solution to $2^{\epsilon n}$ -HARD on input 1^m will be a string that is not in the range of C^* . Second, we will show that given a string outside the range of C^* , we can find a string outside the range of C using only a polynomial number of calls to an NP-oracle.

To carry out the first of these steps, we will show that any string in the range of C^* , when interpreted as a truth table of length $m = n2^k$ on $\lceil \log n \rceil + k$ variables, can be computed by a circuit of size $O(|C|k)$. Since, by construction of k , we have that $m \geq |C|^{\frac{2}{\epsilon}}$, a solution to $2^{\epsilon n}$ -HARD on input 1^m will be a truth table of length m not computable by a circuit of size $m^\epsilon \geq |C|^2$, and thus a circuit of size $O(|C|k) = O(|C| \log |C|)$ would be a contradiction for all input lengths greater than some absolute constant. We construct such a circuit for any string in the range of C^* as follows: let y be a $2^k n$ -bit string such that for some $x \in \{0, 1\}^n$, $C^*(x) = y$. The circuit computing y will have x written as advice/constants, and will feed x through k copies of the circuit C in series. We will split the $\lceil \log n \rceil + k$ input variables into a block of k variables we call i , and a block of $\lceil \log n \rceil$ variables we call j . We then use i to determine whether to apply L or R to the output of one of the copies of C before feeding it into the next, to get some resulting string x^i , and then we use j to index into the j^{th} position of x^i , to get $y_{i,j}$. A diagram of this circuit is shown in Figure 1.2:

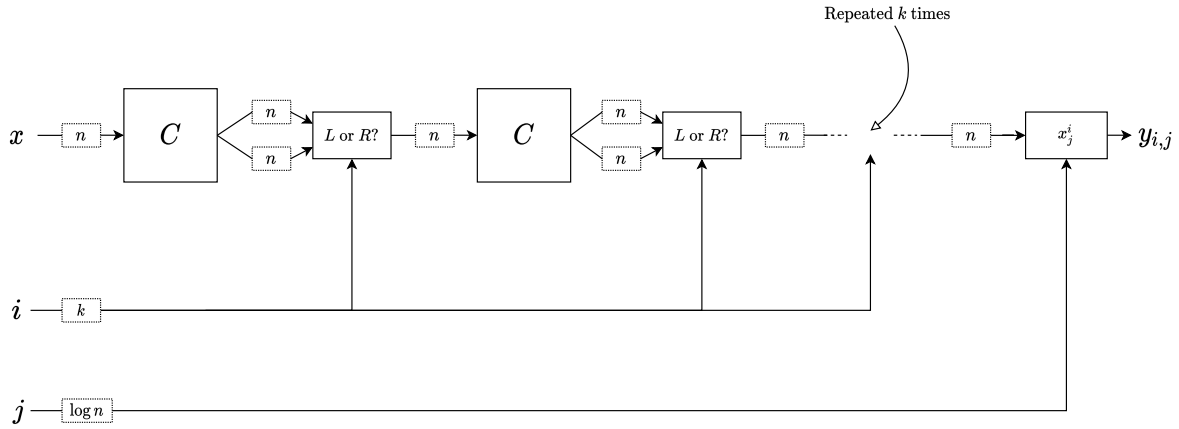


Figure 1.2: A succinct circuit whose truth table is y , for any y in the range of C^* . Dotted boxes indicate the number of bits along a wire. Note that although x is shown as an input in this diagram, for any given y we fix a preimage x as constants/advice, and so the only true inputs to this circuit are i, j .

To see that this circuit has size $O(|C|k)$, note that the subcircuits computing either L or R

depending on a bit of i can be computed easily with $O(n)$ gates over the basis $\{\wedge, \vee, \neg\}$ (this is essentially a multiplexer), and also that the final subcircuit indexing into an n -bit string can be computed with $O(n)$ $\{\wedge, \vee, \neg\}$ gates as well. Since $|C| \geq n$, and this circuit contains only k copies of C and the aforementioned subcircuits, plus the constants describing the string x of length n , this circuit has $O(|C|k)$ size as claimed.

Thus, we now know that any solution to $2^{\epsilon n}$ -HARD on input 1^m will not be in the range of C^* , and by assumption $2^{\epsilon n}$ -HARD is total for sufficiently large input lengths so such a solution exists. It remains only to show that we can use a string outside the range of C^* , together with a NP oracle, to find a string outside the range of C . We proceed exactly as in the proof of Lemma 1.30. Let y be any string outside the range of C^* . Refer to Figure 1.1 which gives a diagram of a circuit computing C^* ; at a layer $i \in [k]$ of this circuit, we have 2^i blocks of n bits feeding into 2^i copies of C , and these copies of C then output 2^{i+1} blocks of n bits at the next layer. So working back from the output layer k , we can test if any consecutive $2n$ -bit block of y is outside of the range of C . If none of them are, then we find a preimage for all blocks, interpret this as the output of the previous layer, and continue our search from there. We follow this process all the way back to the input layer or until we find an AVOID solution for C . If we never find an AVOID solution for C , then this process will terminate at the input layer with a string x such that $C^*(x) = y$, which is impossible by assumption, so at some point we must indeed find a string outside the range of C . Checking whether a particular string lies in the range of C , and finding a preimage if it is, is a single instance of an NP search problem of input length $|C|$ and can be solved with $\text{poly}(|C|)$ calls to an NP (decision) oracle. We perform this test at most $2^k = \text{poly}(|C|)$ times (once for every copy of C in the diagram in Figure 1.1), so overall this process can be accomplished in polynomial time with an NP oracle. $\square$

Combining this result with Observation 1.2, we are able to obtain an *exact algorithmic characterization* of the possibility of proving $2^{\Omega(n)}$ circuit lower bounds for E^{NP} :

Theorem 1.32. *The following are equivalent:*

1. $E^{NP} \not\subseteq_{i.o.} \text{size}[2^{o(n)}]$.

2. $AVOID \in FP^{NP}$

Proof. By Observation 1.2, (1) holds if and only if there is a FP^{NP} algorithm for $2^{\epsilon n}$ -HARD for some fixed constant $\epsilon > 0$. Combining Theorems 1.9 and 1.31, $2^{\epsilon n}$ -HARD and AVOID are FP^{NP} equivalent, hence $2^{\epsilon n}$ -HARD $\in FP^{NP}$ if and only if $AVOID \in FP^{NP}$. $\square$

The astute reader may observe that, chaining together the reduction of $2^n/n$ -HARD to AVOID from Theorems 1.9 and 1.10 with the reduction from AVOID to $2^{\epsilon n}$ -HARD in Theorem 1.31, we obtain a reduction from $2^n/n$ -HARD to $2^{\epsilon n}$ -HARD; in other words, there is an FP^{NP} algorithm which takes truth tables with mildly exponential circuit complexity, and transforms them into truth tables with near maximal circuit complexity. As an immediate consequence we obtain the following unusual *hardness amplification* theorem for E^{NP} :

Theorem 1.33. *The following are equivalent:*

1. $E^{NP} \not\subseteq_{i.o.} \text{size}[2^{o(n)}]$.

2. $E^{NP} \not\subseteq_{i.o.} \text{size}[2^n/n]$.

The possibility of such an equivalence was previously considered in [20]. In this work, Buresh-Oppenheim and Santhanam define a notion of “hardness extraction” which is essentially the exact object constructed in our reduction from $2^n/n$ -HARD to $2^{\epsilon n}$ -HARD. Informally, a hardness extractor is a procedure which takes a truth table of length N and circuit complexity s , and produces a truth table with nearly maximum circuit complexity relative to its size, whose length is as close to s as possible. The proof of Theorem 1.31 can in fact be viewed as a construction of a near-optimal hardness extractor using an NP oracle. In particular our procedure is able to extract approximately the square root of the input’s hardness. In the following we will refer to an arbitrary string $x \in \{0, 1\}^N$ as a “truth table” and define the circuit complexity of x to be the circuit complexity of

the function $f^{(x)} : \{0, 1\}^{\lceil \log N \rceil} \rightarrow \{0, 1\}$ given by

$$f^{(x)}(z) = \begin{cases} x_i & \text{if } z = \text{binary}(i) \text{ for some } i \leq N \\ 0 & \text{otherwise} \end{cases}$$

Theorem 1.34. *There is an FP^{NP} algorithm which, given a truth table $x \in \{0, 1\}^M$ and circuit complexity $\geq s$, outputs a truth table y of length $N = \Omega(\sqrt{\frac{s}{\log M}})$ and circuit complexity $\Omega(\frac{N}{\log N})$.*

Proof. The proof follows from a more careful analysis of Theorem 1.31; we give a sketch here. Adapting the proof of Theorem 1.31, for any N we can efficiently construct a circuit C with N outputs and $\lfloor \frac{N}{2} \rfloor$ inputs, such that every N -bit string outside its range requires circuits of size $\delta(\frac{N}{\log N})$ for some fixed $\delta > 0$. In particular, it is clear from the proof of Theorem 1.9 that such a C can be constructed of circuit size $O(N^2)$. Now, for any choice of N, M let $k = k(N, M)$ to be the minimum integer such that $2^k \frac{N}{2} \geq M$. Following the argument in the proof of Theorem 1.31, we can construct a map $C^* : \{0, 1\}^{\lfloor \frac{N}{2} \rfloor} \rightarrow \{0, 1\}^{2^k \lfloor \frac{N}{2} \rfloor}$ such that any element of its range has circuit complexity $O(|C|k) = O(N^2k)$, and such that given a string outside the range of C^* , we can find a string outside the range of C using 2^k calls to an NP search oracle on instances of size $\text{poly}(N)$. Now, we set N as a function of M by $N = \epsilon \sqrt{\frac{s}{\log M}}$ for ϵ sufficiently small. We get a value of $k = k(N, M) \leq \log M$. This in turn means that a circuit of size $O(N^2k) = O(\epsilon^2 s)$ whose truth table is $x0^{\lfloor \frac{N}{2} \rfloor 2^k - M}$ would contradict the fact that x has hardness at least s (for sufficiently small choice of ϵ). Thus, $x0^{\lfloor \frac{N}{2} \rfloor 2^k - M}$ must lie outside the range of C^* , so using x and our NP oracle we can find an N -bit string outside the range of C . This process requires at most $2^k \leq M$ calls to an NP-search oracle on instances of size $\text{poly}(N) \leq \text{poly}(M)$, so overall this reduction runs in FP^{NP} □

A natural goal would be to improve this procedure to extract $(\frac{s}{\log M})^{\frac{1}{2} + \epsilon}$ or ideally $\Omega(\frac{s}{\log M})$ bits of hardness. The only obstacle here is improving the $O(N^2)$ upper bound on the circuit complexity of C , the circuit which takes descriptions of $\log N$ -input circuits of size $\approx N$ and outputs their truth tables. However, Williams observes in [168] (see footnote 7 of that paper) that an $N^{2-\epsilon}$ upper

bound on the circuit size of C would imply a (nonuniform) breakthrough for 3SUM, so improving this extractor in its current form appears difficult.

Finally, we explain here a **TODO: observation made in the intro**, that our results yield a new and completely elementary proof of a weakened version of the Impagliazzo-Wigderson Theorem that a $2^{\Omega(n)}$ circuit lower bound for E suffices to fully derandomize BPP:

Theorem 1.35. *If $E^{\text{NP}} \not\subseteq_{i.o.} \text{size}[2^{o(n)}]$ then $\text{BPP} \subseteq \text{P}^{\text{NP}}$.*

Proof. By Theorem 1.32, $E^{\text{NP}} \not\subseteq_{i.o.} \text{size}[2^{o(n)}]$ implies a FP^{NP} algorithm for AVOID. By Corollary 1.17, every language in BPP is many-one reducible to AVOID, hence $\text{BPP} \subseteq \text{P}^{\text{NP}}$. $\square$

1.4.2 Undirected Remote Vertex

Finally, we turn our attention to a problem originally defined in [96], which captures the principle: every graph with bounded degree and many vertices must have large diameter.

Definition 1.36 (REMOTE VERTEX). *REMOTE VERTEX is the following problem. The input is a circuit $N : \{0, 1\}^n \rightarrow (\{0, 1\}^n)^m$, a parameter $\ell \in \mathbb{N}$, and some $v \in \{0, 1\}^n$, such that $n > \ell \lceil \log m \rceil$. The interpretation is that N defines the outgoing neighborhood function of a directed graph on vertex set $\{0, 1\}^n$, where $N(u)$ lists the set of $\leq m$ neighbors of u (possibly with repetition). A solution is some $u \in \{0, 1\}^n$ such that there is no directed path from v to u of length at most ℓ .*

In the UNDIRECTED REMOTE VERTEX problem, the input is the same, but we allow as a solution any witness that N fails to represent a undirected graph. In particular, we allow as a solution any pair $u, w \in \{0, 1\}^n$ such that $u \in N(w)$ but $w \notin N(u)$.

Theorem 1.37. *UNDIRECTED REMOTE VERTEX is equivalent to AVOID under many-one FP^{NP} reductions.*

Proof. The reduction of UNDIRECTED REMOTE VERTEX to AVOID is easy and does not require an NP oracle. Say that the UNDIRECTED REMOTE VERTEX instance has 2^n vertices, degree m ,

path length ℓ , and start node $v \in \{0, 1\}^n$. We simply take our avoid instance $C : \{0, 1\}^{\ell \lceil \log m \rceil} \rightarrow \{0, 1\}^n$ to be the map which takes as input a description of a walk of length ℓ starting at v , and outputs the node that it reaches. Note that this reduction works just as well for the harder REMOTE VERTEX problem.

We now prove the other direction. By Lemma 1.30, we may assume the AVOID instance is of the form $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, $m \geq 2n \log n$ (this is the only place we will use an NP oracle in our reduction). Consider a graph G on vertex set $V = V_0 \times V_1$, with $V_0 = \mathbb{F}_2^n, V_1 = \mathbb{F}_2^m$. A vertex (x, y) has neighbors $(x + e_i, y)$ for each standard basis vector $e_i \in \mathbb{F}_2^n$, as well as the neighbor $(x, y + C(x))$. This defines an undirected graph with degree at most $n + 1$ and 2^{n+m} vertices. For $\ell = 2n + 2$, the instance $(G, 0^{n+m}, \ell)$ defines a valid instance of of UNDIRECTED REMOTE VERTEX provided $m + n > \ell \lceil \log(n + 1) \rceil$ which holds for our choice of parameters provided n is sufficiently large. Notice that our instance defines an undirected graph by construction, so the only relevant solutions are vertices at distance $> \ell$ from 0^{n+m} . We claim that if (x, y) is a valid undirected remote vertex solution, then $y \notin \text{range}(C)$; equivalently, every (x, y) with $y \in \text{range}(C)$ is reachable from the origin in at most $2n + 1$ steps. Say that $y = C(z)$. In n steps we may traverse from the start vertex $(0^n, 0^m)$ to $(z, 0^m)$ using bit flips, then in one step we may traverse from $(z, 0^m)$ to $(z, C(z) + 0^m) = (z, y)$, and finally in n more steps we may traverse from (z, y) to (x, y) using bit flips again. $\square$

1.5 Direct Reductions to Hard Truth Tables

Ideally we could extend the main completeness result of the previous Section (Theorem 1.31) to work with polynomial time reductions, as opposed to FP^{NP} reductions. However, the NP oracle seems highly necessary for the proof techniques used above. Indeed, subsequent to the initial publication of this work, [81] gave strong cryptographic evidence that the completeness result in Theorem 1.31 requires an NP oracle, a result which we will discuss further in the closing section of this chapter. Despite this obstacle, we show that there are some important explicit construction problems, which are not *obviously* about circuit complexity, which nonetheless can be reduced to

the problem of finding truth tables of hard functions via polynomial time reductions.

A simple way to phrase the following results is that any truth table with sufficiently large circuit complexity will necessarily satisfy a variety of other pseudorandom properties for which no explicit constructions are known, including: rigidity over $\mathbb{F}_2$, high space-bounded communication complexity, and high bit-probe complexity. To show this, we demonstrate that the failure of a string x to possess any of these properties implies a smaller than worst case circuit for x .

1.5.1 Rigidity

We begin with the case of rigidity. We recall the definition of the search problem $(r(N), s(N), 2)$ -RIGID, wherein given 1^N we are asked to produce an $N \times N$ matrix over $\mathbb{F}_2$ which requires changing $\geq s(N)$ entries to reduce its rank to $r(N)$. Any matrix achieving $(\epsilon N, \epsilon N^2, 2)$ rigidity for an absolute constant $\epsilon > 0$ (or even $\epsilon \gg \frac{1}{\log \log N}$) would be sufficient to carry out Valiant's program [162].

Theorem 1.38. *For any sufficiently small $\delta > 0$, there exists some $\epsilon > 0$ such that $(\epsilon N, \epsilon N^2, 2)$ -RIGID reduces in polynomial time to $\delta \frac{2^n}{n}$ -HARD.*

Proof. To prove this, it suffices to show that for any $N \times N$ matrix M which is not $(\epsilon N, \epsilon N^2)$ -rigid, we can construct a Boolean circuit with $f(\epsilon)O(\frac{N^2}{\log N})$ gates which decides the value of $M_{i,j}$ given the $2\lceil \log N \rceil$ -bit input (i, j) , for some function f which approaches zero as ϵ approaches zero. This then implies that for any fixed δ , an N^2 -bit truth table requiring circuits of size $\frac{\delta N^2}{\log N}$ must be $(\epsilon N, \epsilon N^2)$ -rigid for some $\epsilon > 0$ which is a function only of δ (and otherwise determined by f and the constants hidden in the $O(\cdot)$ term).

Say M is not $(\epsilon N, \epsilon N^2)$ -rigid. So there exists an $N \times \epsilon N$ matrix L , an $\epsilon N \times N$ matrix R , and an $N \times N$ matrix S with at most ϵN^2 nonzero entries, such that $LR \oplus S = M$. We will construct a circuit allowing us to efficiently index M which uses these matrices L, R, S as advice. To encode L and R , we can utilize the well-known theorem of Shannon that any truth table of length N can be computed by a circuit of size $O(\frac{N}{\log N})$ [153]. Thus, L can be specified as a list of ϵN circuits, each of size $O(\frac{N}{\log N})$, where the j^{th} circuit L_j represents the j^{th} column, and $L_j(i)$ computes $L_{i,j}$. The

same can then be done for R (indexing columns instead). To encode S , we employ a refinement of Shannon's result due to Lupanov [62], which tells us that for sufficiently large N , any truth table of length N with at most ϵN nonzero entries can be computed by circuit of size

$$\frac{\log \binom{N}{\epsilon N}}{\log \log \binom{N}{\epsilon N}} + o\left(\frac{N}{\log N}\right) \leq H(\epsilon)O\left(\frac{N}{\log N}\right)$$

Where H denotes the binary entropy function. Thus S can be computed by a circuit of size $H(\epsilon)O(\frac{N^2}{\log N})$.

It remains to show that the additional circuitry we need to compute $M_{i,j}$ given i, j and the encodings of L, R, S does not increase things too much. By definition, we have that:

$$M_{i,j} = \langle row_i(L), col_j(R) \rangle \oplus S_{i,j}$$

where the dot product is taken over $\mathbb{F}_2$. Now consider the circuit diagram shown in Figure 1.3:

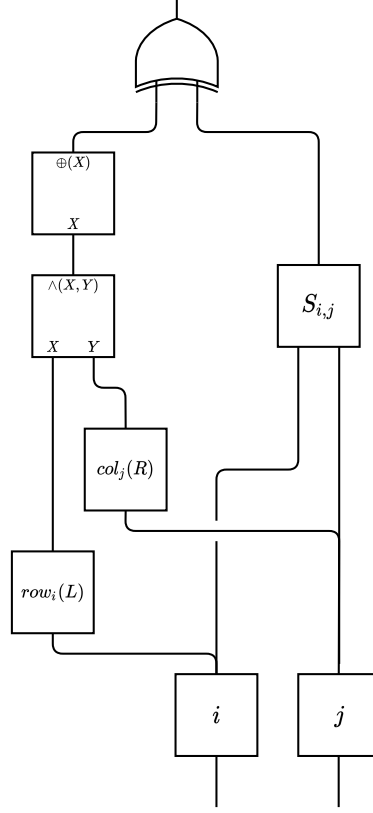


Figure 1.3: A small circuit for a non-rigid truth table

The subcircuit computing $\text{row}_i(L)$ is defined as follows: as described above, L represents a $N \times \epsilon N$ matrix, specified as a list of ϵN circuits of size $O(\frac{N}{\log N})$, each computing a column of L . $\text{row}_i(L)$ feeds i into each of these circuits in parallel to get an ϵN -bit string giving value of the i^{th} row of L . $\text{col}_j(R)$ is defined analogously. The circuit $S_{i,j}$ simply computes the $(i, j)^{\text{th}}$ entry of S given i, j . Finally, the subcircuit $\oplus(X)$ computes the parity of its input string, the subcircuit $\wedge(X, Y)$ computes the bit-wise AND of two equal length strings, and the terminal gate computes the two-bit parity function. Given the previous equation relating the $(i, j)^{\text{th}}$ index of M to the $(i, j)^{\text{th}}$ rows/columns/indices of L, R, S , it is straightforward to see that this circuit performs the necessary calculation. It is clear that the $\oplus(X)$ and $\wedge(X, Y)$ can be implemented with a number of gates linear in their input size, which in this case is ϵN . From the analysis above, each of $\text{row}_i(L)$ and $\text{col}_j(R)$ can be implemented using $\epsilon O(\frac{N^2}{\log N})$ gates, and $S_{i,j}$ can be implemented using $H(\epsilon)O(\frac{N^2}{\log N})$ gates. So overall, this circuit has size $(H(\epsilon) + \epsilon)O(\frac{N^2}{\log N})$. Since $H(\epsilon) + \epsilon$ approaches

zero for decreasing ϵ , this implies that for any fixed $\delta > 0$, an N^2 -bit truth table requiring circuits of size $\frac{\delta N^2}{\log N}$ must be $(\epsilon N, \epsilon N^2)$ rigid for some $\epsilon > 0$ which is a function only of δ (and which is otherwise determined by the constants hidden in the $O(\cdot)$ expressions above). $\square$

We thus conclude that if $\mathbf{E}$ contains a language of circuit complexity $\Omega(\frac{2^n}{n})$, then there is a polynomial time construction of $(\Omega(n), \Omega(n^2))$ -rigid matrices over $\mathbb{F}_2$.

1.5.2 Space-Bounded Communication Complexity

The class $PSPACE^{CC}$ was defined originally in [13] as a generalization of the class PH^{CC} to an unbounded alternation of quantifiers. We will not give this original definition, but rather a simplification due to [157].

Definition 1.39. *Let $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$. We say f has a space- s protocol if there is a deterministic protocol deciding f of the following form. Alice receives $x \in \{0, 1\}^n$, and Bob receives $y \in \{0, 1\}^n$. There is an s -bit shared memory, and Alice and Bob alternate turns writing to this memory. On a player's turn, they can modify the contents of the s -bit shared memory as a function only of its previous contents and their private input x or y , or decide to halt and output some $z \in \{0, 1\}$ (this decision is also a function only of their input and the shared memory's previous state). This protocol is valid if $f(xy) = z$ for all x, y .*

By a result of Song [157], we have that if $f \in PSPACE^{CC}$ then f has a $\text{poly}(\log n)$ space protocol (Song uses a slightly different model where Alice and Bob have private s -bit tapes, but our model is at least as strong up to a doubling in space since sharing the tape only increases their ability to communicate). Due to a basic counting argument, most functions f require $\Omega(n)$ space, and any such function must lie outside of $PSPACE^{CC}$. However, it has been a long standing open problem to give an explicit construction a communication matrix outside of even PH^{CC} [157]. We thus define the following search problem:

Definition 1.40. δ -SPACE is the following search problem. Given 1^N , where $N = 2^n$, output a communication matrix $\{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ such that f requires space- δn communication

protocols.

Theorem 1.41. *For any $\delta < \epsilon < \frac{1}{2}$, δ -SPACE reduces in polynomial time to $2^{(\frac{1}{2}+\epsilon)n}$ -HARD.*

Proof. The proof follows the exact same strategy as above, constructing a smaller-than-worst-case circuit for any matrix with a δn -space protocol. We will omit the detailed construction of our circuit since it is very similar to that of the above proof, but give here a clear sketch of its essential structure. Let $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$, and let $N = 2^n$. We can represent a space s protocol for f as $2s + 4$ different $N \times 2^s$ binary matrices, with each of the first $2s$ matrices telling us how one of the two players will modify a certain cell of the shared memory as a function of its previous state and their input, and the last four matrices determining whether a certain player will halt or continue and the value they will output if they halt, again as a function of their input and the shared memory contents. For $s = \epsilon \log N$ (with $\epsilon < 1$), we can then take these matrices as advice to our circuit, for a total of $O(\epsilon N^{1+\epsilon} \log N)$ bits of advice. We can then add circuitry to simulate $2^{s+1} = 2N^\epsilon$ steps of this protocol on a given input $x, y \in \{0, 1\}^n \times \{0, 1\}^n$, indexing into the advice matrices in order to determine what to do next using the same indexing constructions we had for the proof of Theorem 1.38. Any matrix solvable by a space s protocol will be solved by a protocol that halts after 2^{s+1} steps (otherwise it will loop forever), so simulating for 2^{s+1} steps suffices. Since each of the indexing operations can be implemented using a number of gates linear in $\epsilon N^{1+\epsilon} \log N$ using the constructions from the proof of Theorem 1.38, our overall circuit for f will have size $O(2N^\epsilon \epsilon N^{1+\epsilon} \log N) = O(N^{1+2\epsilon} \log N)$. Thus, for any $\epsilon < \frac{1}{2}$, a truth table of length N^2 requiring circuits of size $N^{1+2\epsilon}$ will require $\delta \log N$ -space protocols for any $\delta < \epsilon$. $\square$

1.5.3 Bit Probe Lower Bounds

In [44], Elias and Flower defined a broad model for studying the space/query complexity of data structure problems, known as the “bit-probe model.”

Definition 1.42. *Given two sets D, Q , a function $f : D \times Q \rightarrow \{0, 1\}$ and an integer b , the bit-probe complexity of f for space b , denoted $BC_b(f)$, is the minimum over all encodings $G : D \rightarrow \{0, 1\}^b$*

of the number of bits of $G(x)$ that need to be probed in order to determine $f(x, y)$ for the worst case $x \in D, y \in Q$, given access to y .

In this general definition of a data structure problem, we think of D as the set of all possible pieces of “data” we might wish to encode in our data structure, b as the number of bits we can use to encode a piece of data, Q as the set of queries we wish to answer about an encoded piece of data, and f as telling us the correct answers to all data/query pairs. $BC_b(f)$ then tells us the minimum number of probes required by any space- b data structure in order to answer every query correctly for every possible piece of data.

This model was investigated further by Miltersen [122], who showed, using a simple counting argument, that most problems require an infeasible amount of space/probes in this model, but pointed out that no explicit data structure problem is known to be infeasible in this sense. We thus define the following explicit construction problem:

Definition 1.43. δ -PROBE is the following search problem: given 1^N where $N = 2^n$ output a truth table $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ such that $BC_{2^{\delta n}}(f) > \delta n$.

Theorem 1.44. For any $\delta < 2\epsilon < 1$, δ -PROBE reduces in polynomial time to $2^{(\frac{1}{2}+\epsilon)n}$ -HARD.

Proof. Say f is a function $f : D \times Q \rightarrow \{0, 1\}$ such that $BC_b(f) \leq k$. So in particular there is an encoding $G : D \rightarrow \{0, 1\}^b$ such for any $x \in D, y \in Q$, given access to y we can determine $f(x, y)$ using at most k probes to $G(x)$. Let $H : Q \rightarrow [b]^k$ be the function which, given y , tells us which positions of $G(x)$ to query. Finally, let $\phi : \{0, 1\}^k \times Q \rightarrow \{0, 1\}$ be the function which determines $f(x, y)$ given the results of the probes and the value of y . We will now give a compact representation for f , and then use it to construct a circuit computing f .

Let R be a $|D| \times b$ binary matrix whose rows are indexed by elements of D , where $R_{i,j}$ gives the value of the j^{th} bit of $G(i)$. Let S be a $|Q| \times k[\log b]$ matrix whose rows are indexed by elements of Q , such that the i^{th} row of S is $H(i) \in [b]^k$. Finally, let Z be a $2^k \times |Q|$ matrix such that $Z_{i,j} = \phi(i, j)$.

Now, given $x \in D$, $y \in Q$, we can use S, R, Z to determine $f(x, y)$ as follows. First find the y^{th} row of S to get $H(y)$, which is a list of k indices in $[b]$. Next, find the x^{th} row of R , which is the b -bit string $G(x)$. Then, probe the indices specified by $H(y)$ to get some k -bit string w , and finally output $Z_{w,y}$, which is precisely $\phi(w, y) = f(x, y)$.

As in the previous proofs, it can easily be shown that these indexing operations can be accomplished with circuits of size linear in S, R, Z . Therefore, f has a circuit of size $O(|S| + |R| + |Z|) = O(|Q|k \log b + |D|b + 2^k|Q|)$. So for any function $D \times Q \rightarrow \{0, 1\}^n$ requiring circuits of size $\omega(|Q|k \log b + |D|b + 2^k|Q|)$, we must have $BC_b(f) > k$.

In particular, if we take $D = Q = \{0, 1\}^n$, and for any fixed $\epsilon < 1$ we take $b = 2^{\epsilon n}$, $k = \epsilon n$, we get that for any function $f : \{0, 1\}^{2n} \rightarrow \{0, 1\}$ requiring circuits of size $\omega(2^{(1+\epsilon)n})$, it must be the case that $BC_{2^{\epsilon n}}(f) > \epsilon n$, since

$$|Q|k \log b + |D|b + 2^k|Q| = 2^n \epsilon^2 n^2 + 2^n 2^{\epsilon n} + 2^{\epsilon n} 2^n = O(2^{(1+\epsilon)n})$$

So if we take $\delta < \epsilon$, any truth table of length N^2 with hardness $N^{1+\epsilon}$, or in other words any solution to $2^{(\frac{1}{2} + \frac{\epsilon}{2})n}$ -HARD on input 1^{N^2} , must satisfy $BC_{2^{\delta n}}(f) > \delta n$. $\square$

1.6 A Brief Survey of Subsequent Developments

Since the publication of the main content of this chapter in [101], a significant body of work on Range Avoidance has emerged [144, 66, 103, 33, 81, 29, 114, 34, 55, 37, 105, 165, 107, 49, 106, 108, 79, 57, 74, 75, 67, 54, 78, 146, 56]. We discuss here some of the key developments in the area by other researchers which will not otherwise be covered in the later chapters of this thesis.

1.6.1 Restricted Variants of AVOID

As mentioned briefly in Section 1.3, the works of [144, 65] brought attention to the study of structured instances of AVOID in which the input $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is of a special form. Recall from Section 1.3 that [144] proved that if we can solve such instances where C comes

from a particular circuit class, then we can prove exponential lower bounds against that same class for a language in E^{NP} . Ren, Santhanam and Wang [144] then paired this with the following remarkable result, which is obtained from the randomized encodings method of [10] from low-depth cryptography:

Theorem 1.45 ([144]). *There is a polynomial time reduction from AVOID on instances $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ specified by a sequence of m boolean formula computing each output bit of the mapping, to instances $C' : \{0, 1\}^{n'} \rightarrow \{0, 1\}^{m'}$ in which C' is an NC_4^0 circuit: each output bit depends on only 4 input bits.*

This implies that even obtaining a FP^{NP} algorithm for NC_4^0 avoid would already yield breakthrough circuit lower bounds. The instances C' produced by this reduction have $m' = n' + o(n')$, hence it remains a possibility that NC^0 instances with much larger stretch, e.g. $m' = (n')^{1+\epsilon}$, could be solved unconditionally without leading to any breakthrough. This possibility was investigated further in [65], who showed that if $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is NC_k^0 and $m \gg n^{k-1}$, an FP^{NP} algorithm is unconditionally possible. This result was significantly improved in [106], the work forming the basis of the final chapter of this thesis, where among other things we will show that a polynomial time algorithm (no NP oracle required) exists once $m \gg n^{k/2}$. In addition to giving the first nontrivial algorithm for NC^0 AVOID, [65] also showed that various important explicit construction problems reduce to AVOID instances computed by boolean formula, including Rigid Matrix and GV-Code constructions. Applying Theorem 1.45 above, this implies that each of these problems can be reduced in turn to NC_4^0 AVOID.

1.6.2 Hardness of AVOID

In the initial publication of the work in this chapter [101], the following was posed as a central open problem: is there any convincing evidence for/against $AVOID \in FP$, or $AVOID \in FP/Poly$? The question of $AVOID \in FP$ is strongly motivated by the results in Section 1.3: evidence for $AVOID \in FP$ would yield evidence that a variety of important explicit construction problems can be solved efficiently. The question of $AVOID \in FP/Poly$ is separately motivated by the following

problem: can the completeness of hard truth tables from Theorem 1.31 be extended to polynomial time reductions? More generally, does AVOID reduce in polynomial time to any explicit construction problem? If such a reduction existed this would place AVOID in FP/poly, since any explicit construction problem can be solved in P/poly (we simply hard code one solution for each input length).

It was shown in [81] that there is indeed strong evidence that AVOID is hard. In particular, a sufficiently strong cryptographic assumption implies $\text{AVOID} \notin \text{FP/poly}$. This assumption, known as *indistinguishability obfuscation* or $\mathcal{IO}$ for short, posits the existence of a universal compiler which can take any program (circuit) C and produce an equivalent one $\mathcal{IO}(C)$ which has the same functionality as the original, but which hides all implementation details: for any pair C, C' with the same functionality, $\mathcal{IO}(C)$ and $\mathcal{IO}(C')$ should look the same to a computationally-bounded observer. This concept was originally introduced in [15] (where a more formal definition can be found) and has received considerable attention in the years since. In [81], it is shown that assuming the existence of $\mathcal{IO}$ we can obtain conditional hardness for AVOID:

Theorem 1.46 ([81]). *Assume that there exists $\mathcal{IO}$ scheme secure against subexponential time nonuniform algorithms and that $\text{NP} \not\subseteq \text{coNP/poly}$. Then $\text{AVOID} \notin \text{FP/poly}$.*

For a considerable span of time after the original definition of $\mathcal{IO}$ in [15] there was little public consensus on whether or not secure $\mathcal{IO}$ was a plausible assumption. This changed after a recent breakthrough of Jain, Lin and Sahai [87], who demonstrated how to construct $\mathcal{IO}$ from a few concrete cryptographic hardness assumptions which have each withstood years of attack and are considered highly plausible. As a result, the existence of $\mathcal{IO}$ is now considered by many as “likely true” and hence the results of [81] give a rather convincing argument that AVOID is not solvable by polynomial size circuits. A subsequent work of [34] obtains a similar kind of cryptographic hardness for AVOID, which moreover applies to C-AVOID for very simple classes of circuits C . In particular, assuming the hardness of certain nonstandard variants of LWE and LPN, [34] obtain hardness for $\text{TC}^0\text{-AVOID}$, and hence for $\text{NC}_4^0\text{-AVOID}$ by Theorem 1.45. The cryptographic assumptions used in [34] are introduced in the paper itself and have an unclear

relationship with the more standard variants of LWE and LPN studied widely in the literature.

Following these initial works, a different and much simpler kind of hardness reduction was discovered independently in two works [79, 146]. Both of these results relied on a hardness assumption first considered by Rudich [148], which posits the existence of a “demi-bit generator:”

Definition 1.47 (Demi-Bit Generator). *A demi-bit generator is a circuit $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$, so that no NP/poly algorithm can certify $y \notin \text{range}(G)$ for more than $o(1)$ fraction of strings $y \in \{0, 1\}^{n+1}$.*

In [79, 146], it is shown that the existence of demi-bit generators implies the hardness of AVOID. In both proofs, if we assume that G is of a low complexity (e.g. NC^1), the derived hard instances of AVOID will have similarly low complexity. In addition, [79] shows that under a standard strengthening of Rudich’s assumption (a strengthening which was already a primary focus of Rudich’s original paper on demi bits), even the problem COMPLEXITY (defined in Section 1.3) is not solvable in polynomial time ².

An interesting feature of all of the above hardness results shown so far for AVOID is that they crucially use *nondeterminism* in their reductions; hence they rely on assumptions which at the very least imply $\text{NP} \neq \text{coNP}$, and more generally which posit some kind of hardness against nondeterministic algorithms. However, all of these results make up for the increased strength in assumption by yielding a stronger hardness conclusion: they imply that AVOID is hard for FNP algorithms as well. It remained an open question whether a simpler deterministic reduction could be used to show hardness; indeed, it was posed as an open problem in [104] whether AVOID might be NP hard under Turing reductions. This possibility was ruled out in a very elegant recent work [57], who showed that, assuming the polynomial time hierarchy is infinite, there is no such reduction. Specifically, they showed:

Theorem 1.48 ([57]). *Any decision problem which has a randomized turing reduction to AVOID lies in $\text{AM} \cap \text{coAM}$.*

²This hardness result is for a slightly modified definition of the problem in which the input length of the oracle function f is twice that of the hard function g .

This theorem is proven via a new algorithmic upper bound for AVOID; the authors show that there is a constant round interactive protocol, wherein an all powerful prover can convince a polynomial time verifier that a particular string $y \in \{0, 1\}^{n+1}$ is an AVOID solution for $C : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ with confidence $1 - \epsilon$, in time $\text{poly}(|C|, \epsilon^{-1})$. This is in contrast to the traditional random sampling procedure, which leads to one of the following situations:

1. A randomized polynomial time algorithm can obtain a solution with confidence $\frac{1}{2}$, but seemingly no better.
2. A randomized NP-oracle algorithm can obtain a solution with confidence $1 - \epsilon$ in time $\text{poly}(|C|, \epsilon^{-1})$, but this requires using the NP oracle to generate negative certificates $y \notin \text{range}(C)$.

The protocol of [57] in some sense interpolates between the above two settings. It is worth noting that, unlike the AVOID upper bounds proven in [29, 114, 105] which will be discussed at length in the next chapter, the upper bound for AVOID proven in [57] does not seem to yield any novel circuit lower bounds. The distinction is that, in order for an algorithmic upper bound for AVOID to yield a circuit lower bound, it must have the property that it outputs (with high probability) a *fixed solution* $y_C \notin \text{range}(C)$ determined uniquely by the instance C . The interactive protocol of [57] allows a prover to boost the confidence in correctness of an AVOID solution, but the randomness in the protocol creates entropy in the solution y that the verifier outputs. Indeed, when the prover follows the protocol honestly the verifier will output a solution distributed almost uniformly on the space of all solutions. The distinction between upper bounds for search problems which do/don't produce solutions uniquely determined by the input is discussed at great length in the introduction of the next chapter.

1.6.3 The Algorithmic Method

As we saw in Section 1.3.1, algorithms for C-AVOID imply C-circuit lower bounds for exponential time classes [144], however for classes C below P/poly the reverse implication is not

known. It is then natural to ask: for circuit classes C such that we have unconditional C lower bounds for languages in $\text{EXP}/\text{NEXP}/\text{EXP}^{\text{NP}}$, can we strengthen these lower bounds to achieve algorithms for $C\text{-AVOID}$? Two important special cases are $C = \text{AC}^0$, where lower bounds for very simple functions (parity) have been known for decades [53, 4], and $C = \text{ACC}^0$, where lower bounds inside NEXP were more recently obtained by Williams [170].

Williams' breakthrough result was established by a general method laid out in his preceding work [168] and denoted "The Algorithmic Method." The main theorem of [168] says that for any "sufficiently nice" class of circuits C , any algorithm for testing the satisfiability of C -circuits which has sufficiently nontrivial advantage over brute-force search implies that NEXP does not have polynomial size C circuits. Williams' lower bound for ACC^0 is then obtained by designing the necessary satisfiability algorithms for ACC^0 . In a series of works [144, 33], Williams' algorithmic method is adapted to the context of AVOID , where results of the following form are given: if a certain computational task associated with C -circuits has a slightly nontrivial algorithm, then $C\text{-AVOID}$ (with a suitable stretch parameter) collapses into polynomial time (perhaps with an NP oracle). The main technical ingredient in both of these papers is the design of highly specialized PCPs. We will not cover them in detail, but mention only a major result of the second paper, which achieves the goal of extending the best known lower bounds techniques for ACC^0 to achieve matching $\text{ACC}^0\text{-AVOID}$ algorithms:

Theorem 1.49 ([33]). *$\text{ACC}^0\text{-AVOID}$ with stretch $n \mapsto 2^{\log^{\omega(1)} n}$ is solvable in FP^{NP} . More specifically, for each $d, r \in \mathbb{N}$ there exists $c \in \mathbb{N}$ and a FP^{NP} algorithm $\mathcal{A}$ which solves AVOID on instances $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ in which each output is computed by a depth d $\text{poly}(m)$ size circuit over the basis $\{\vee, \wedge, \text{MOD}_r\}$, provided $m \geq 2^{\log^c n}$.*

It should be noted that this theorem is "optimal" in terms of the stretch in the sense that it recovers the best known quantitative lower bounds against ACC^0 : any AVOID algorithm of the above kind immediately implies that E^{NP} requires $2^{n^{\Omega(1)}}$ size ACC^0 circuits by Theorem 1.13, and obtaining an FP^{NP} algorithm in a stronger stretch regime would imply a better lower bound than is currently known.

Chapter 2: Strong vs. Weak Range Avoidance and the Linear Ordering

Principle

2.1 Introduction

As laid out in the introduction of this thesis, a primary motivation for the work in [101] covered in the previous chapter, and all of the subsequent followup works by other authors studying AVOID, is to make progress along the “high-end” approach circuit lower bounds. In particular, rather than viewing $E^{NP} \notin \text{size}[2^{o(n)}]$ as a *lower bound bound*, we instead study the equivalent *upper bound problem* of giving FP^{NP} algorithms for AVOID. This **TODO: noun, avenue?** was recently realized in spectacular fashion by a pair of breakthrough works of Chen, Hirahara, Ren [29] and Li [114], who gave for the first time in decades a new answer to the following question:

What is the smallest uniform complexity class which contains a language of exponential circuit complexity?

The classical answer to this question was first given by Kannan [94], who showed that the class Δ_2^E ($2^{O(n)}$ time with a Σ_2^P oracle) suffices; in the decades since this had remained the best such result¹. Chen, Hirahara, Ren and Li showed that in fact the classes $S_2^P \subseteq ZPE^{NP} \subseteq \Sigma_2^E$ all contain a function of circuit complexity $2^n/n$, which is within a $(1 \pm o(1))$ factor of the maximum possible circuit complexity of a boolean function. The results are established by giving a new algorithm for AVOID which runs in the class FS_2^P , the functional variant of the decision class S_2^P .

The authors of [29, 114] made progress on a classical lower bound problem by discovering a new kind of algorithm for the total search problem Range Avoidance, a search problem in the class

¹ Δ_2^E was the smallest known complexity class with an unconditional *exponential* circuit lower bound. If the goal is to prove merely superpolynomial lower bounds, lower bounds for smaller classes can be established using Karp-Lipton theorems [95], an approach pioneered in Kannan’s original paper [94]. This method can at best show “sub-half-exponential” lower bounds, and only for infinitely many input lengths.

$\text{TF}\Sigma_2^P$. Since the introduction of $\text{TF}\Sigma_2^P$ in [96], there has been no follow up work developing a structural classification of the problems therein, despite the considerable attention that has been devoted to Range Avoidance and explicit construction problems more generally. The work of [29, 114] indicates that some problems in $\text{TF}\Sigma_2^P$ admit highly nontrivial algorithms, algorithms which have consequences in seemingly unrelated areas of complexity. Do such algorithms exist for all search problems in $\text{TF}\Sigma_2^P$? What more can we say about the relation of AVOID to other problems in this class? The purpose of this chapter, based on joint work with Toniann Pitassi originally appearing in [105], is to address these two questions in particular, and more broadly to further the systematic study of $\text{TF}\Sigma_2^P$ beyond its introduction in [96], with an emphasis both on *inclusions* and *black box separations*. Prior to this work, no non-trivial separations between problems in $\text{TF}\Sigma_2^P$ were known.

In contrast to $\text{TF}\Sigma_2^P$, TFNP has received a thorough investigation over the past three decades and its structure is rather well understood in comparison. A major distinction we reveal between $\text{TF}\Sigma_2^P$ and TFNP is that the former has a plethora of *resource-constrained* subclasses, many of which can be separated from one another by explicit and natural search problems. By “resource-constrained subclass” we mean a class of search problems characterized by the existence of some resource-constrained algorithm which can find a solution. One example we will see is the class $\text{psFZPP}^{\text{NP}}$, the class of problems where for every input x , there is a canonical solution y_x which is output with high probability by some polynomial time NP-oracle algorithm. Another class is $\text{FP}^{\Sigma_2^{\text{NP}} \cap \Pi_2^P}$, consisting of those problems which can be solved in polynomial time with oracle access to a language in $\Sigma_2^P \cap \Pi_2^P$. We will introduce and study several other such subclasses, and exhibit four natural search problems which exhibit separations (in the decision tree model) between them. This situation is in stark contrast to the known structure of TFNP : while FNP does contain some intermediate resource-constrained classes, such as psFZPP and $\text{FP}^{\text{NP} \cap \text{coNP}}$ (these are roughly analogous to $\text{psFZPP}^{\text{NP}}$ and $\text{FP}^{\Sigma_2^{\text{NP}} \cap \Pi_2^P}$ mentioned above), it is known that in the decision tree model all of these classes collapse to FP . As a result, in the decision tree model the only resource-based distinctions amongst the standard TFNP problems is the distinction between FP

and the rest of TFNP.

2.1.1 Overview of Main Results

A defining feature of search problems which distinguishes them from decision problems is that a given instance may have many different solutions. Indeed, if a search problem has a unique solution on every instance, then it may be equivalently phrased as a decision problem: given (x, i) , output the i^{th} bit of the unique solution for x . For many search problems of interest it is not clear how to reduce them to a decision problem of the same complexity. For example given total search problem R we may define the decision problem LexFirst_R , where given an instance (x, i) we must output the i^{th} bit of the lexicographically first solution to x . While LexFirst_R is clearly at least as hard as R , in many cases it will be much harder and a reduction does not seem to exist in the opposite direction.

Say that $\mathcal{A}$ is a deterministic algorithm solving some search problem R . Then $\mathcal{A}$ naturally associates to each instance x a *canonical solution* $y_x := \mathcal{A}(x)$ which it outputs. In particular we can think of $\mathcal{A}$ as *defining a second search problem* R' with $(x, y) \in R'$ iff $\mathcal{A}(x) = y$; if $\mathcal{A}$ solves the original search problem R , then R is reducible to R' since $(x, y) \in R' \rightarrow (x, y) \in R$. Now, if the algorithm $\mathcal{A}$ lies in some restricted complexity class, then this places the same complexity upper bound derived search problem R' . The point of this is that when we have some nontrivial deterministic algorithm solving a search problem R , we may think of it as giving a reduction from R to a search problem with *unique solutions*.

For most of the classical problems in the class TFNP it is believed that there is *no decision problem* which captures their complexity precisely, a conjecture supported by black box separations [17]. More generally it is believed that for most of the standard problems $R \in \text{TFNP}$, any unique-solution problem R' which we can reduce R to must lie *outside of* TFNP^2 . The new results in [114, 29] reveal that the situation for AVOID is different: because their Range Avoidance algorithms are deterministic and are upper bounded inside $\text{TF}\Sigma_2^P$, they imply that Range Avoidance is

²This is equivalent to saying that we believe these standard problems, e.g. PPAD or PPP, not to lie in $\text{FP}^{\text{NP} \cap \text{coNP}}$.

reducible to a problem in $\text{TF}\Sigma_2^P$ with unique solutions. To make this discussion more formal we need to define the class $\text{TF}\Sigma_2^P$ and its *unique solution* subclass $\text{TFU}\Sigma_2^P$:

Definition 2.1 ($\text{TF}\Sigma_2^P$ and $\text{TFU}\Sigma_2^P$). *A polynomially-bounded³ search problem R lies in $\text{TF}\Sigma_2^P$ if it is a total search problem, and there exists a coNP verifier V so that $(x, y) \in R$ if and only if $V(x, y) = 1$. We say $R \in \text{TFU}\Sigma_2^P$ if moreover every instance has a unique solution.*

In this terminology Li’s result implies the following: Range Avoidance is polynomial-time reducible to a problem in $\text{TFU}\Sigma_2^P$. His result is in fact a significant strengthening of this, but for now we focus on this particular consequence, which seems quite unintuitive on the surface: given an instance $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ of Range Avoidance, there are at least 2^n distinct solutions, and it is not clear how to narrow down to any particular solution which is “more special” than the others. Our first contribution is to clarify this result in the following way: we introduce a *natural* $\text{TF}\Sigma_2^P$ search problem, whose containment in $\text{TFU}\Sigma_2^P$ is obvious from the definition, and then show that Range Avoidance reduces to this problem:

Definition 2.2 (LINEAR ORDERING PRINCIPLE (LOP)). *Given $\prec: \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ specified by a boolean circuit, find a witness that $\prec$ does not define a total ordering on $\{0, 1\}^n$, or else find the minimal element in the ordering it defines. A witness that $\prec$ does not define a total order consist of $x, y, z \in \{0, 1\}^n$ such that one of the following holds: (a) $x \prec x$; (b) $x \neq y$, $x \not\prec y$, and $y \not\prec x$; (c) $x \prec y \prec z$ and $x \not\prec z$.*

This problem was originally defined in the context of Bounded Arithmetic in [35]. While this problem does not literally have unique solutions as stated, it has a property which we show is morally equivalent. Observe that one type of solution in this problem is easier to verify than the other: a witness that $\prec$ fails to define a linear order can be verified in polynomial time, whereas a candidate minimum element can only be verified in coNP. Moreover, if an instance has no easily verifiable solutions, then it has a unique solution. This follows from the fact that every linear order has a unique minimal element. We summarize this by saying that LOP has *essentially*

³The “polynomially bounded” condition just means we are restricting attention to search problems whose solutions have polynomially bounded length.

unique solutions; a very easy argument (Section 2.2, Lemma 2.25) shows that any problem with essentially unique solutions is reducible to a problem in $\text{TFU}\Sigma_2^P$. We then prove:

Theorem 2.3. *Range Avoidance is polynomial time reducible to LOP.*

We believe this result goes a long way in explaining the new upper bounds for Range Avoidance. In particular our reduction isolates the two key steps in Li’s algorithm which allow us to single out a special *low complexity canonical solution* for Range Avoidance: the first step prepares a special subset of solutions using a certain tree-like iteration construction, and the second step singles out a fixed canonical solution among these by defining a certain *total ordering* on these special solutions. Recall our comment under Definition 2.1 that Li’s result is in fact stronger than a reduction from Range Avoidance to a problem in $\text{TFU}\Sigma_2^P$: more generally he shows that Range Avoidance lies in the complexity class FS_2^P (we will review the definition of this class in Section 2.2). Our result subsumes this upper bound as well, since another very direct argument shows that LOP lies in FS_2^P . **TODO: rtalk about the followup work by Volkavic**

At this point we have seen that there are search problems in $\text{TF}\Sigma_2^P$ which seem on the surface to have no distinguished solutions, but which nonetheless can be reduced to problems in $\text{TFU}\Sigma_2^P$ by some highly non-obvious means. This naturally points to the following question: are all problems in $\text{TF}\Sigma_2^P$ reducible to $\text{TFU}\Sigma_2^P$? We give a negative answer in the decision tree model. Our separation is exhibited by the following relative of Range Avoidance which was introduced originally in [96] under the name EMPTY:

Definition 2.4 (STRONG AVOID). *Given $f : \{0, 1\}^n \setminus \{0^n\} \rightarrow \{0, 1\}^n$, find $y \in \{0, 1\}^n \setminus \text{range}(f)$.*

From now on we will refer to Range Avoidance as “WEAK AVOID” to distinguish it from STRONG AVOID. Our black box separation will be significantly stronger than just showing that STRONG AVOID is not reducible to a problem in $\text{TFU}\Sigma_2^P$; we will show more generally that STRONG AVOID cannot be solved by making non-adaptive queries to any language in Σ_2^P , which is equivalent to proving size lower bounds for depth-3 AC_0 circuits solving STRONG AVOID:

Theorem 2.5. *In the decision tree model, STRONG AVOID is not in $\text{FP}_{\parallel}^{\Sigma_2^P}$. More specifically, let $C : \{0, 1\}^{(N-1)\log N} \rightarrow \{0, 1\}^{N^\epsilon}$ be a depth-3 circuit of size 2^{N^ϵ} and let $D : \{0, 1\}^{N^\epsilon} \rightarrow \{0, 1\}^{\log N}$ be an arbitrary postprocessing function, where $N = 2^n = |\{0, 1\}^n|$. Then provided ϵ is sufficiently small, $D \circ C$ cannot solve STRONG AVOID: there must be some input $f : [N-1] \rightarrow [N]$ so that $D(C(f))$ fails to find a $y \notin \text{range}(f)$.*

This immediately implies non-reducibility to $\text{TFU}\Sigma_2^P$, since any problem $R \in \text{TFU}\Sigma_2^P$ with unique solutions can be solved with non-adaptive queries to the language $\{(x, i) \mid (x, y) \in R \rightarrow y_i = 1\}$ which lies in Σ_2^P . The connection of these kinds of separations to depth 3 AC^0 lower bounds was spelled out in a recent paper of Vyas and Williams [165] for the case of WEAK AVOID: their work established that nontrivial upper bounds for WEAK AVOID are equivalent to certain depth 3 circuits solving the so-called “Missing String” problem: given an explicit list of 2^{n-1} n -bit strings, output a string not in the list. The input size here is $\approx N = 2^n$, and the question is whether a depth 3 circuit exists of size polynomial or quasipolynomial in N . Note that the Missing String problem is simply the black-box variant of WEAK AVOID. Li’s result showed that quasipolynomial size depth-3 circuits for this problem actually *do exist*, solving the original question of Vyas and Williams in the positive. We show that if the problem is modified so that the list of strings has length $N-1$ rather than $\frac{N}{2}$, then depth-3 circuits require exponential size to solve this problem. We note that our lower bound holds against a stronger class of circuits than what was originally considered by Williams and Vyas: in their model the depth 3 circuit is of the form $C : \{0, 1\}^{(N-1)\log N} \rightarrow \{0, 1\}^{\log N}$ and must output the exact solution to AVOID. Here we allow C to output an arbitrary string in $\{0, 1\}^{N^\epsilon}$, which can then be postprocessed arbitrarily to construct a solution to AVOID.

By a simple reduction, we also obtain quasipolynomial depth-3 circuit size lower bounds even for *moderately weak* AVOID instances with domain $[N]$ and codomain $[N + N/\log^{O(1)} N]$. This result gives a complete characterization of the degree of “Weakness” necessary to obtain quasipolynomial size depth 3 circuits: if the codomain has size $N + N/\log^{O(1)} N$ then circuits of quasipolynomial size suffice, and if its size is $N + N/\log^{\omega(1)} N$ then they do not; see Lemma 2.45 for details.

Our last main result exhibits a more fine-grained separation amongst the subclasses of $\text{TF}\Sigma_2^P$.

Above we have highlighted one important distinction, between the problems which are reducible to $\text{TFU}\Sigma_2^P$ and those which aren't. However both of the problems we've seen so far which reduce to $\text{TFU}\Sigma_2^P$ also have an additional property: they are solvable by a polynomial time randomized algorithm using a NP oracle. For WEAK AVOID this follows from its definition, while for LOP it follows from its containment in the class FS_2^P . We show that this is not possible for all problems reducible to $\text{TFU}\Sigma_2^P$. Our separation is exhibited by the following natural search problem:

Definition 2.6 (STRONG 1-1 AVOID). *Given $f : \{0, 1\}^n \setminus \{0^n\} \rightarrow \{0, 1\}^n$, find a pair $x \neq y$ in $\{0, 1\}^n \setminus \{0^n\}$ such that $f(x) = f(y)$, or else $y \in \{0, 1\}^n \setminus \text{range}(f)$,*

Observe that STRONG 1-1 AVOID enjoys the same property as LOP of having *essentially unique solutions*: it is easy to verify the collision solutions $f(x) = f(y) \wedge x \neq y$, and any instance with no collision solutions has a unique solution. This follows from the fact that any injective function $f : [N] \rightarrow [N + 1]$ misses exactly one point in its codomain. Hence, like WEAK AVOID and LOP, the problem STRONG 1-1 AVOID is reducible to $\text{TFU}\Sigma_2^P$. However we prove the following lower bound:

Theorem 2.7. *In the decision tree model, STRONG 1-1 AVOID is not solvable in FBPP^{NP} .*

Aside from revealing further the structure of $\text{TF}\Sigma_2^P$, this result yeilds a new separation for *decision classes* which was not previously known:

Theorem 2.8. *In the decision tree model, $\Sigma_2^P \cap \Pi_2^P \not\subseteq \text{BPP}^{\text{NP}}$. In particular $\Sigma_2^P \cap \Pi_2^P \not\subseteq S_2^P$.*

In the next two subsections we will describe these main results in some more technical detail. A diagram of the structure of $\text{TF}\Sigma_2^P$ and our main results is given in Figure 2.1; some classes in this diagram will not be defined until Section 2.2.1.

2.1.2 AC^0 Lower Bounds and Class Separations

Our two main separations show that STRONG AVOID has no non-trivial upper bound inside of $\text{TF}\Sigma_2^P$ (Theorem 2.5), and that STRONG 1-1 AVOID has no randomized NP-oracle algorithm

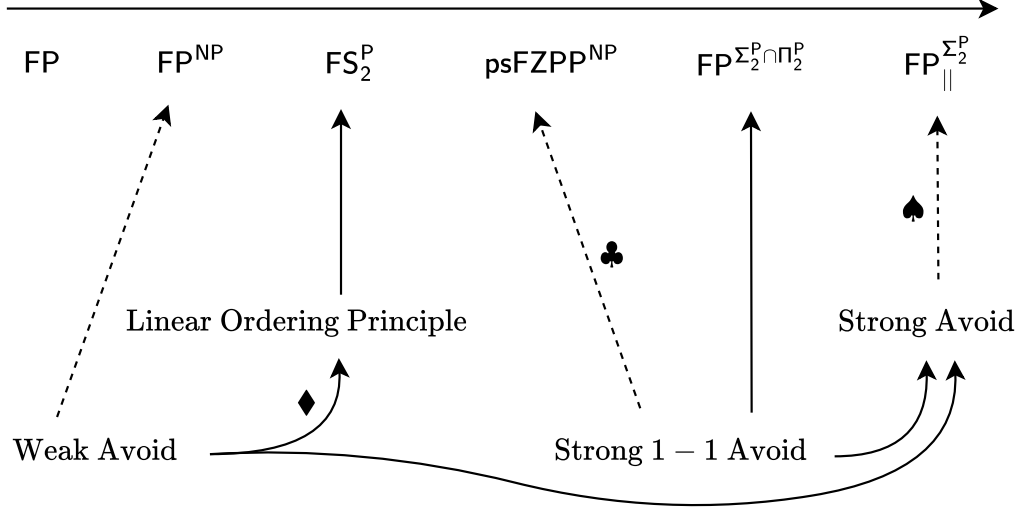


Figure 2.1: Inclusion diagram of relevant classes and search problems. Solid arrows represent the inclusion of the class at the base of the arrow into the class at its tip. Dotted arrows indicate non-inclusion of the base class into the tip class in the decision tree model. The main linear axis of classes along the top are all included from left to right, indicated by the long solid arrow above them. Our major results are the separations marked with ♠, ♣ and the inclusion marked with ♦. $\text{TFU}\Sigma_2^P$ essentially corresponds to $\text{FP}^{\Sigma_2^P \cap \Pi_2^P}$, see Lemma 2.23.

(Theorem 2.41). The first lower bound is the more involved of the two, and requires proving a novel depth 3 AC^0 circuit lower bound for a total search problem, which appears to be the first circuit lower bound of this kind.

Theorem 2.5 yields a very fine-grained separation for the STRONG AVOID problem. It is easy to construct a depth 4 circuit $C : \{0, 1\}^{(N-1)\lceil \log N \rceil} \rightarrow \{0, 1\}^{\lceil \log N \rceil}$ solving STRONG AVOID, where moreover the bottom fan-in is only $O(\log N)$ [165]. More strongly, it is possible to construct a depth $O(\log N)$ decision tree, which at each step queries a depth 3, $\text{poly}(N)$ size circuit on the input f , and at each leaf outputs a correct solution $y \notin \text{range}(f)$. Our lower bound can be interpreted as saying that if such a decision tree is forced to be non-adaptive, then either the circuits it queries at each step must grow to exponential size, or else the number of queries must grow to $N^{\Omega(1)}$. This also contrasts the situation with WEAK AVOID, where as mentioned above, Li's construction gives depth-3 AC_0 circuits of size $N^{O(\log N)}$ for solving WEAK AVOID.

We believe our lower bound is of independent interest in circuit complexity. In particular, we give a very precise depth-3 lower bound for a *total search problem*. It is of course possible to

construct contrived examples of total search problems which are hard for AC^0 circuits, for example “given $x \in \{0, 1\}^n$ output $b \in \{0, 1\}$ such that $\text{Parity}(x) = b$,” in such examples we can derive hardness of the search problem *by reduction from a decision problem*. In contrast, the lower bound we show here *cannot be established by reduction from any decision problem*. This follows from the fact that any decision problem which is reducible to a $TF\Sigma_2^P$ search problem lies in $\Sigma_2^P \cap \Pi_2^P$. In the decision tree model, this means that any language which can be solved by a small depth decision tree querying instances of STRONG AVOID has both Σ_2^P and Π_2^P circuits. However, the lower bound we are trying to show for STRONG AVOID rules out the existence of any $FP^{\Sigma_2^P \cap \Pi_2^P}$ algorithm for STRONG AVOID. Phrased more succinctly, our lower bound establishes that STRONG AVOID is harder than *any* decision problem which can be reduced to it, which by definition means we cannot establish the lower bound itself by reduction from a decision problem. In light of this, to prove Theorem 2.5 we must develop new AC^0 lower bound techniques which are specially tailored to total search problems.

Our second main lower bound (Theorem 2.7) places STRONG 1-1 AVOID outside of $FBPP^{NP}$. Recall that STRONG 1-1 AVOID has the property of having *essentially unique solutions* and is thus reducible to a problem in $TFU\Sigma_2^P$. We will see (Lemma 2.23) that this means it is reducible to a *decision problem* in $\Sigma_2^P \cap \Pi_2^P$. Combining this with the above lower bound we obtain Theorem 2.8, which separates the decision tree class $\Sigma_2^P \cap \Pi_2^P$ from BPP^{NP} and in particular from S_2^P . This is in contrast to the situation for $NP \cap coNP$, which is known to collapse to P in the decision tree model. This improves a previous result of Fortnow and Yamakami [52] who showed that $\Sigma_2^P \cap \Pi_2^P \not\subseteq P^{NP}$ in the decision tree model.

2.1.2.1 Lower Bound Methods

A main technical ingredient in both lower bounds is a new Switching Lemma (Lemma 2.40) specialized for AVOID. Switching lemmas have been used for both circuit lower bounds for computing functions, and in proof complexity to prove lower bounds on the size of proofs of hard tautologies (e.g., [16, 139, 19]). Between the two, our argument bears a stronger resemblance to

the second, however there are some key conceptual differences. The basic idea behind all switching lemmas is to show that under a random restriction ρ (from a suitable distribution) a low-width DNF is likely to be represented by a low-depth decision tree. Since a low-depth decision tree representation for a function f implies that both f as well as its negation can be represented by low-width DNFs, this in turn allows us to collapse an AND of low-width DNFs into a single low-width DNF, thus reducing the circuit depth by 1.

A major difference between various switching lemmas is the choice of distribution over restrictions, and the way in which the decision tree *represents* a DNF. In the original Switching Lemmas used to prove AC_0 lower bounds for parity, the restrictions are simply uniformly random partial restrictions, and the notion of represents is with respect to *every* input. That is, the decision tree computes the same function as the DNF.

In the case of switching lemmas used in proof complexity to prove AC^0 -Frege lower bounds for the $[N] \rightarrow [N + 1]$ pigeonhole principle, we think of the input as specifying a purported 1-1 function from $[N]$ to $[N + 1]$ (which cannot actually exist if N is finite). The chosen distribution over restrictions are partial 1-1 matchings from $[N]$ to $[N + 1]$, and a low-depth “PHP decision tree” in this context can make queries to a pigeon or to a hole at each vertex, and every path in the tree corresponds to a partial matching. Since, in reality, no 1-1 function from $[N]$ to $[N + 1]$ exists, these trees do not represent the original DNF in any standard way. However, if the input variables instead corresponded to a total 1-1 assignment from $[N]$ to $[N]$ (which do exist), then we can apply the same PHP Switching Lemma to prove that under a random partial 1-1 restriction, a low-width DNF is likely to convert to a low-depth PHP decision tree, which now represents the DNF in the sense that it agrees with the DNF on all input assignments that correspond to 1-1 mappings from $[N]$ to $[N]$.

In the case of AVOID, we have to modify the way of constructing a decision tree associated with a DNF so that the decision tree represents the original DNF in the sense that they are truth-functionally equivalent with respect to all 1-1 input functions from $[N]$ to $[M]$, where now M is strictly larger than N . To achieve this, we modify the notion of pigeonhole decision trees as

follows. As in the original PHP Switching Lemma in proof complexity, each node of our decision tree will query either a pigeon or a hole. When a pigeon is queried at a node, we allow edges for all possible holes that it could be mapped to. But when a hole is queried, now we have to allow for the possibility that this hole is unmapped: in addition to allowing edges for each pigeon that could map to this hole, we allow an extra edge corresponding to the case where nothing maps to this hole. With this modification, our pigeonhole decision trees will represent the original DNF with respect to all 1-1 inputs from $[N]$ to $[M]$.

Another crucial distinction is how we use the Switching Lemma to reduce the depth of the circuit by one. In the proof complexity setting for the pigeonhole principle lower bounds, we think of N as infinite, and therefore with respect to 1-1 inputs, a DNF can be written as a low-width *matching disjunction*, where each term in the disjunction corresponds to a partial 1-1 function (or matching) from $[N]$ to $[N]$. After applying the PHP Switching Lemma and a union bound, each matching disjunction f (under ρ) becomes a low-depth “matching decision tree”, enabling a reduction in the overall circuit depth by one. To summarize, in the classical PHP Switching Lemma, the underlying depth-2 subcircuits are always low-width matching disjunctions, both before and after each application of the PHP Switching Lemma. In our case, the underlying depth-2 subcircuits are not of the same type before and after applying our Pigeonhole Switching Lemma; a consequence is that our Switching Lemma *cannot be applied twice*. This is not a defect of our method, but rather a necessary feature of any technique here, since our search problem *can be solved* by circuits of one higher depth. More specifically, we show that initially the bottom depth-2 subcircuits of C can be expressed as low-width matching disjunctions. But after applying the Pigeonhole Switching Lemma, and subsequent depth reduction, the new depth-2 subcircuits become *hole disjunctions* which are a generalization of matching disjunctions, where now each term in the DNF can specify not only a partial 1-1 matching, but also a subset of holes that are unmapped. With these appropriate modifications, our proof of the switching lemma is similar to previous proofs.

Equipped with the Pigeonhole Switching Lemma, we can give the high level view of both proofs. We start with Theorem 2.5. To prove Theorem 2.5, we would like to restrict attention

to the class of 1-1 input functions from $[N]$ to $[N + 1]$. However if we truly restrict ourselves to 1-1 functions, then a lower bound is not possible by Lemma 2.26: for these inputs, there is a unique solution, and therefore a polynomial-size depth-3 circuits can easily check whether the unique solution has its i^{th} bit equal to 1 or to 0 and hence solve the problem unconditionally. To circumvent this barrier, we will prove a strengthening of Theorem 2.5, by giving a lower bound for AVOID on input functions $f : [N] \rightarrow [M]$, where M is larger than $N + 1$. By enlarging the range of f , we can focus our attention of f 's that are 1-1 since now for every input, there are at least $M - N$ distinct solutions. Note that this implies the lower bound stated above (with $M = N + 1$), since there is a direct reduction from AVOID on instances $[N] \rightarrow [M]$ to instances $[N] \rightarrow [N + 1]$: we simply map every element of $[M] \setminus [N + 1]$ to the element $N + 1$. Observe that this reduction does not preserve injectivity.

Now assume there exists small-size s depth-3 circuit C computing STRONG AVOID on 1-1 functions from $[N]$ to $[M]$, $M \gg N$. We can first apply a standard argument (the Width Reduction Lemma 2.44) so that we can assume that the bottom-level fanin of C is at most $O(\log s)$. After this step, we can assume that C is a size- s , depth-3 circuit, where the bottom depth-2 subcircuits are low-width matching disjunctions. Next we apply our Pigeonhole Switching Lemma (as discussed above) which will guarantee that there exists a matching restriction ρ such that under ρ , all depth-2 matching disjunctions in C will convert to low-depth pigeonhole decision trees. This will allow us to reduce the overall circuit depth by 1, and afterwards each output bit of C will be computed by a low-width *hole disjunction*. As discussed above, a hole disjunction is a type of DNF that generalizes matching disjunctions: each term t in the hole disjunction can be viewed as partial information about the input f . The partial information consists of two parts: (i) first, t_1 specifies a small partial matching, pairing up some pigeons in $[N]$ to some holes in $[N + 1]$; (ii) secondly, t_2 specifies a small set of holes (disjoint from the holes mentioned in t_1 that are not in the range of f). It remains to prove a lower bound for circuits C for solving AVOID, where each output bit is specified by a low-width hole disjunction. This is also accomplished using a kind of restriction, but rather than choosing it at random we apply a careful deterministic process involving a novel

covering argument. This step is somewhat reminiscent of early proofs of the Switching Lemma (e.g., [53, 4]).

The proof of Theorem 2.7 again uses the Pigeonhole Switching Lemma, together with a direct argument. We want to prove depth lower bounds for FBPP^{NP} , which informally are randomized decision trees of small height which, instead of querying variables, are allowed to query the value of an arbitrary low-width DNF over the inputs. By Yao's minimax principle, it suffices to prove that any low-depth P^{NP} decision tree cannot solve AVOID with probability $2/3$, with respect to the uniform distribution of 1-1 functions. We think of this distribution in the following way: first sample a uniform *partial* 1-1 assignment ρ , then sample a uniform extension of ρ to a total assignment. Applying our Pigeonhole Switching Lemma and a union bound, we can argue that with high probability over the first choice of ρ , all of the NP queries in our P^{NP} decision tree T can be simplified to small depth pigeonhole decision trees, which overall allows T to be replaced by a low depth pigeonhole decision tree. It then remains only to argue that a pigeonhole decision tree of low depth cannot solve 1-1 STRONG AVOID with non-trivial probability on a uniform extension f of ρ , which can be accomplished with a direct argument.

2.1.3 Linear Ordering Principle

We now discuss in more detail our results on the newly defined $\text{LINEAR ORDERING PRINCIPLE}$ problem, abbreviated LOP . Recall that our main result here is Theorem 2.3, which says that WEAK AVOID is polynomial time reducible to LOP . The proof follows much of the high level structure of Li's result placing WEAK AVOID in FS_2^{P} , with some key modifications. Roughly speaking, Li's proof shows that given an instance of $\text{AVOID } f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$, we can define a comparison relation $\sqsubset$ on $\{0, 1\}^{\text{poly}(n)}$, so that for some unique distinguished element $\pi^* \in \{0, 1\}^{\text{poly}(n)}$ we have $\pi^* \sqsubset \pi$ for all $\pi \neq \pi^*$, and π^* contains a solution to the original AVOID instance. In our case we need to define a similar comparison relation, which in addition *globally* acts as a total order on $\{0, 1\}^{\text{poly}(n)}$. To explain the argument more clearly we split the reduction into two parts. We first introduce an intermediate search problem called $\text{FOREST TERMINATION}$

and reduce WEAK AVOID to this problem, then we reduce FOREST TERMINATION to LOP. We note that our proof, as well as Li's, also bears a strong resemblance to the work of [135] who gave the first proof of the weak pigeonhole principle in the bounded arithmetic theory T_2 .

Our subsequent results show some appealing structural properties of the complexity class defined by reducibility to LOP. We start by proving closure under a broad class of reductions:

Theorem 2.9. *Any search problem which has a polynomial time P^{NP} Turing reduction to LOP also has a polynomial time many-one reduction to LOP.*

Combining this closure property with the fact that LOP has essentially unique solutions, we are able to conclude that LOP is *equivalent* in complexity to a decision problem. In particular we can define a decisional complexity class L_2^P for which LOP is the “complete problem” (despite being a search problem and not a language). We start by presenting a machine-based definition of L_2^P :

Definition 2.10. *A language L is in the complexity class L_2^P if there is a polynomial time relation $R : (\{0, 1\}^*)^3 \rightarrow \{0, 1\}$ and a polynomial p , so that for all x , $R(x, \cdot, \cdot)$ defines a total order on $\{0, 1\}^{p(n)}$ whose minimal element a has $a_1 = L(x)$.*

We then have the following equivalent characterizations:

Theorem 2.11. *The following are equivalent for a language L :*

1. $L \in L_2^P$
2. L is polynomial time many-one reducible to LOP.
3. L is P^{NP} -Turing reducible to LOP.

Conversely, the search problem LOP is polynomial time truth table reducible⁴ to a language in L_2^P .

We mentioned in passing before that LOP is easily shown to lie in the class FS_2^P , the functional analogue of the decision class S_2^P (this will be shown in Lemma 2.27. The same reason shows that

⁴This is essentially the most restrictive reduction possible when reducing a search problem to a language.

$L_2^P \subseteq S_2^P$. For those unfamiliar with the somewhat unconventional class S_2^P , this is a complexity class introduced independently by Russell-Sundaram and Canetti [149, 24]. Their goal was to identify the smallest class in the polynomial hierarchy which is sufficient to capture randomized algorithms, in particular BPP and MA. Beyond this purpose the class rarely appears, and so far no natural problems have been exhibited which lie in S_2^P and not one of its more traditional subclasses (such as BPP or NP). In addition it seems that S_2^P does not have a complete problem, due to its definition involving a promise. We have identified here a subclass L_2^P of S_2^P , which is characterized exactly by a simple and natural total search problem, and which nonetheless maintains the interesting properties that motivated the original definition of S_2^P :

Theorem 2.12.

1. $P^{NP} \subseteq L_2^P$ and $BPP \subseteq MA \subseteq L_2^P$
2. L_2^{E5} contains a language of circuit complexity $2^n/n$.

In each case, the result stated for L_2^P was previously known to hold for S_2^P and is now shown to be inherited by the more natural subclass L_2^P . The only interesting property that is known of S_2^P which we were unable to prove for L_2^P is the Karp-Lipton theorem; we discuss this further Sections 2.1.4 and 2.4.

2.1.4 Open Problems

TODO: mention Volkavich stuff here, and Noah's paper We conclude our introduction with a few interesting problems which remain open. The first is rather broad:

Problem 2.13. *Show any additional inclusions or black-box separations which are not implied by the arrows in Figure 2.1.*

We specifically highlight the following:

Problem 2.14. *Is there an FP^{NP} Turing reduction from LOP to WEAK AVOID?*

⁵ L_2^E is the exponential-time analogue of L_2^P , where we replace “polynomial time” with $2^{O(n)}$ time” in its definition

Our interest in this problem is the following observation:

Observation 2.15. *Say that LOP is FP^{NP} Turing reducible to WEAK AVOID. Then there is a particular language L of circuit complexity $\geq 2^n/n$, and a deterministic NP oracle algorithm $\mathcal{A}$ running in time $2^{O(n)}$, such that given oracle access to any language L' of circuit complexity $2^{\Omega(n)}$, $\mathcal{A}^{L'}$ computes L .*

This follows by composing the reduction of WEAK AVOID to LOP, which produces a unique solution, with the (purported) second reduction from LOP back to WEAK AVOID. Such a consequence would be rather surprising and interesting purely from the perspective of circuit complexity.

Next we highlight the problem of better clarifying the relationship between L_2^{P} and S_2^{P} :

Problem 2.16. *Does $\text{L}_2^{\text{P}} = \text{S}_2^{\text{P}}$? Can they be separated in the decision tree model? Does L_2^{P} satisfy a Karp-Lipton theorem?*

For this problem, we would say L_2^{P} “satisfies a Karp-Lipton theorem” if one could unconditionally prove the implication “ $\text{NP} \in \text{P/poly} \rightarrow \text{PH} = \text{L}_2^{\text{P}}$.” A notable property of S_2^{P} is that it is the smallest complexity class for which this statement is known to hold.

The next problem we highlight is in the realm of depth 3 circuits:

Problem 2.17. *Does WEAK AVOID have depth 3 circuits of polynomial size?*

Recall that Li’s upper bound is only quasipolynomial, of size around $N^{\log N}$. This problem seems intimately connected to the long-standing open question in proof complexity of whether the Weak Pigeonhole Principle has polynomial size bounded depth Frege proofs; a *quasipolynomial* upper bound was shown by Paris Wilkie and Woods [135] using a very similar technique to Li’s, and [118] give a different bounded-depth Frege upper bound of lower depth, but still quasipolynomial size. Despite the strong aesthetic similarities we do not know a formal connection in either direction between these problems.

Lastly, the search problems discussed here have other connections to bounded arithmetic. In particular, the LOP principle has been investigated in several papers, defined explicitly for the

first time in [35] and studied mostly within the context of characterizing the strength of Jerebek’s bounded arithmetic theory of approximate counting relative to weaker theories, and also as a new avenue for approaching the longstanding problem of separating Buss’ T_2 hierarchy by sentences of fixed complexity. Buss, Kolodziejczyk and Thapen [22] observe that the LOP principle is provable in both T_2^2 and in APC_2 , and ask whether or not LOP is provable in the weaker theory $T_2^1 + sWPHP$, where $sWPHP$ is the surjective weak pigeonhole principle, and corresponds to the search problem WEAK AVOID. Atserias and Thapen [12] resolve this question, proving that in the relativized setting, $sWPHP$ does not prove the LOP principle over T_2^1 . In fact they prove a stronger result, that $sWPHP$ cannot prove the HOP principle over T_2^1 , where HOP is a Σ_1^b version of LOP. It seems possible that the techniques here could be used to give a negative answer to Problem 2.14. A relatively unexplored area that is likely to be fruitful is to discover more relationships between natural search problems lying in the second level of the polynomial hierarchy (and higher) and corresponding systems of bounded arithmetic.

2.2 Preliminaries

2.2.1 Search Problems, Complexity Classes, and Basic Inclusions

We define here a variety of subclasses inside of $TF\Sigma_2^P$, classified according to the computational resources necessary to solve a search problem. We then prove some of the more basic results relating these classes to each other and to the four main search problems of interest in this work.

Definition 2.18 (FP^{NP} , $FP^{\Sigma_2^P \cap \Pi_2^P}$, $FP_{||}^{\Sigma_2^P}$). *Let R be a search problem and C a class of decision problems. We say $R \in FP^C$ if there is a language $L \in C$ and a polynomial time algorithm making queries to L which “solves R ,” given x it outputs y such that $(x, y) \in R$. We say $R \in FP_{||}^C$ if there is such an algorithm, which moreover makes its queries nonadaptively: given an input x it computes in polynomial time a list of queries $z_1, \dots, z_m$, uses its L -oracle to test in unit time the membership of each z_i in L , and then uses the oracle responses to output an answer in polynomial time.*

We include for reference the following class considered in [114, 29]:

Definition 2.19 ($\text{svF}\Sigma_2^{\text{P}}$). $\text{svF}\Sigma_2^{\text{P}}$ is the class of search problems having “singled-valued” $\text{F}\Sigma_2^{\text{P}}$ algorithms. We say R lies in this class if there is a choice of canonical solution $\{(x, y_x) \in R \mid x\}$ for each input x , a second relation $R' \in \text{TF}\Sigma_2^{\text{P}}$, and a polynomial time function f so that whenever $(x, z) \in R'$, $f(z) = y_x$.

In Lemma 2.22 we relate this class to the others we have defined here. Beyond this we will not need to reference this class further: every time we prove an upper bound for a search problem it will be in a class lower than $\text{svF}\Sigma_2^{\text{P}}$, and our separation for STRONG AVOID will hold even against the larger class $\text{FP}_{\parallel}^{\Sigma_2^{\text{P}}} \supseteq \text{svF}\Sigma_2^{\text{P}}$. We next review the randomized classes:

Definition 2.20 ($(\text{ps})\text{FZPP}^{\text{NP}}$, $(\text{ps})\text{FBPP}^{\text{NP}}$). A relation R is in FBPP^{NP} if there exists a randomized polynomial time algorithm with access to a SAT oracle which, given x , outputs y such that $(x, y) \in R$ with probability $\geq 2/3$. If the algorithm always outputs a valid answer or $\perp$ and answers $\perp$ with probability $< \frac{1}{3}$ this places R in the subclass FZPP^{NP} .

For a search problem R where each x may have many solutions y , it is possible that a randomized algorithm outputs different correct answers on the same input x as a function of its random coin tosses. If for each x there exists a canonical y_x with $(x, y_x) \in R$ and some randomized algorithm computes $x \mapsto y_x$ with high probability, we say that algorithm is pseudodeterministic; we use the prefix ps- to denote the pseudodeterministic analogue of a randomized class.

The last standard complexity class we examine is the functional analogue of S_2^{P} , defined as follows:

Definition 2.21 (FS_2^{P}). A search problem R is in FS_2^{P} if there exists polynomial time algorithm V taking three inputs, so that for all x there exists y_x with $(x, y_x) \in R$ so that:

1. There exists π_1 such that for all π_2 , $V(x, \pi_1, \pi_2) = y_x$.
2. There exists π_2 such that for all π_1 , $V(x, \pi_1, \pi_2) = y_x$.

Search Problems vs. Function Problems: As discussed in the introduction, some kinds of algorithms for search problems have the property that they associate to each input a *fixed solution* which the algorithm produces on that input. With the exception of the non-pseudodeterministic randomized classes FBPP^{NP} and FBPP^{NP}, each of the classes we have just defined describes algorithms of this sort. These classes are arranged nicely in the following hierarchy, as indicated in Figure 2.1:

Lemma 2.22. $FP \subseteq FP^{NP} \subseteq FS_2^P \subseteq psFZPP^{NP} \subseteq FP^{\Sigma_2^P \cap \Pi_2^P} \subseteq svFS_2^P \subseteq FP_{||}^{\Sigma_2^P}$

Proof. All inclusions follow directly from the definition, with the exceptions of $FP^{NP} \subseteq FS_2^P$ which is due to Russell-Sundaram [149] and $FS_2^P \subseteq psZPP^{NP}$ which is due to Cai [23]. $\square$

Recall the class $TFU\Sigma_2^P$, referenced heavily in the introduction, consisting of those $TF\Sigma_2^P$ search problems with unique solutions. This class is directly associated to $FP^{\Sigma_2^P \cap \Pi_2^P}$ in this hierarchy:

Lemma 2.23. *The following are equivalent for any $R \in TF\Sigma_2^P$:*

1. *R is polynomial time reducible to a problem in $TFU\Sigma_2^P$.*
2. *$R \in FP^{\Sigma_2^P \cap \Pi_2^P}$.*

Proof. Say R is polynomial time reducible to $R' \in TFU\Sigma_2^P$. So there are polynomial time functions so that for all x , if $(f(x), y) \in R'$ then $(x, g(x, y)) \in R$. Consider the language L defined as follows: $(x, i) \in L$ iff for the unique y with $(f(x), y) \in R'$, we have $y_i = 1$. Then $L \in \Sigma_2^P \cap \Pi_2^P$ and g yeilds a reduction from R to L .

In the other direction note that any language $L \in \Sigma_2^P \cap \Pi_2^P$ defines a search problem in $TFU\Sigma_2^P$: given x find $b \in \{0, 1\}$ so that $L(x) = b$. The result follows directly from this fact. $\square$

Recall from the introduction that the problems LOP and STRONG 1-1 AVOID do not quite have unique solutions, but come very close. We define this property of having *essentially unique solutions* as follows:

Definition 2.24 (Essentially Unique Solutions). *We say that a total search problem $R \in \text{TF}\Sigma_2^P$ has “essentially unique solutions” if there are verifiers V_1, V_2 such that:*

1. V_1 is testable in polynomial time, while V_2 is testable in coNP.
2. For all x , either there exists y so that $V_1(x, y) = 1$ and $(x, y) \in R$, or else there exists a unique y such that $V_2(x, y) = 1$ and $(x, y) \in R$.

We then have:

Lemma 2.25. *If R has essentially unique solutions then it is polynomial time reducible to a search problem $R' \in \text{TFU}\Sigma_2^P$ which actually has unique solutions. By Lemma 2.23 this is equivalent to the statement $R \in \text{FP}^{\Sigma_2^P \cap \Pi_2^P}$.*

Proof. Let V_1, V_2 witness that R has essentially unique solutions. Consider the search problem R' : given x , output either the lexicographically first y such that $V_1(x, y) = 1$, or else the unique y such that $V_2(x, y) = 1$. By the definition of V_1, V_2 we see that R' is a total search problem with unique solutions. Clearly R is polynomial time reducible to R' . We need to show that $R' \in \text{F}\Sigma_2^P$. For a fixed x , say that there exists y with $V_1(x, y) = 1$, and let y_0 be the lexicographically first such y . Then we can confirm that $(x, y_0) \in R'$ in coNP by confirming $V_1(x, y_0) = 1$ and that for all $y' < y_0$ we have $V_1(x, y') = 0$. On the other hand say that there is no y with $V_1(x, y) = 1$, and let y^* be the unique element with $V_2(x, y^*) = 1$. Then we can confirm that $(x, y^*) \in R'$ in coNP by checking that for all y we have $V_1(x, y) = 0$, and using V_2 to confirm $V_2(x, y^*) = 1$ (recall that V_2 is in coNP by definition). $\square$

We now state formally the claim made in the introduction that LOP and STRONG 1-1 AVOID have essentially unique solutions:

Lemma 2.26. *LINEAR ORDERING PRINCIPLE and STRONG 1-1 AVOID have essentially unique solutions; hence both problems lie in $\text{FP}^{\Sigma_2^P \cap \Pi_2^P}$.*

Proof. Any injective function $f : [N] \rightarrow [N + 1]$ leaves a unique point in $[N + 1]$ out of its range. Every total linear order on a finite set has a unique minimal element. $\square$

We next prove formally that LOP is contained in the class FS_2^P , which follows quite directly from the definitions:

Lemma 2.27. *LINEAR ORDERING PRINCIPLE is in FS_2^P .*

Proof. Let $\prec: \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ be an instance of LINEAR ORDERING PRINCIPLE. We construct the FS_2^P solver V for $\prec$ as follows. Let $X = \{0, 1\}^{3n+1}$ be partitioned so that the first 2^n elements A are identified with $\{0, 1\}^n$, and the remaining $> 2^{3n}$ elements B are identified with potential witnesses that $\prec$ fails to define a linear order. Given $\pi_1, \pi_2 \in X$, $V(\prec, \pi_1, \pi_2)$ behaves as follows:

1. If $\pi_i \in B$ codes a witness that $\prec$ does not define a linear order for some $i \in \{1, 2\}$, we output π_i ; if both do then we output the lexicographically first between them.
2. Say $\pi_i \in B$ and $\pi_{i'} \in A$ for $\{i, i'\} = \{1, 2\}$, and π_i is not a witness that $\prec$ fails to define a linear order. In this case we output $\pi_{i'}$.
3. If both $\pi_1, \pi_2 \in A$, we think of them as representing elements a_1, a_2 of $\{0, 1\}^n$ and compare them according to $\prec$. If $a_1 \prec a_2$ we output π_1 , otherwise we output π_2 .

First say that $\prec$ is not a total order. Among all witnesses to this let $\pi \in B$ be the lexicographically first. Then for all $\pi' \in X$, we have $V(\prec, \pi, \pi') = V(\prec, \pi', \pi) = \pi$. On the other hand say $\prec$ is a total order and let $\pi \in A$ correspond to its unique minimal element. Then again for all $\pi' \in X$, we have $V(\prec, \pi, \pi') = V(\prec, \pi', \pi) = \pi$. Thus in all cases V gives an FS_2^P algorithm solving the LINEAR ORDERING PRINCIPLE problem on input $\prec$. $\square$

Finally, amongst the 4 search problems studied here we have the two following obvious inclusions which we haven't mentioned yet:

Observation 2.28. *WEAK AVOID and STRONG 1-1 AVOID are polynomial time reducible to STRONG AVOID.*

2.2.2 Oracle Separations and the Decision Tree Model

All of the upper bounds and inclusions we show in this paper are unconditional and hold relative to every oracle. Since showing any unconditional separations amongst the classes we have identified would imply $P \neq NP$, we can only hope to establish separations in a restricted model. As is standard, our restricted model will correspond to the “decision tree model” of complexity classes, which can be framed either in terms of oracles and Turing machines, or more directly in terms of decision trees and bounded depth circuits, c.f. [39]. We choose here the latter terminology.

All of our search problems are defined in terms of a function or relation specified by a boolean circuit. Take for example the WEAK AVOID problem, whose instance is a boolean circuit computing $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$. Clearly this problem remains total if f were not represented by a small circuit, but instead was an arbitrary function $f : [2^n] \rightarrow [2^{n+1}]$. In the decision tree model, the relevant search problem has as its input an arbitrary function $f : [N] \rightarrow [2N]$ where $N = 2^n$. For example, the decision tree variant of the simplest complexity class FP then corresponds to algorithms which can access f only by querying its on $\text{poly}(n) = \text{poly}(\log N)$ different inputs $x \in [N]$. If we imagine $f : [N] \rightarrow [2N]$ is specified by an assignment $\alpha : \{0, 1\}^{[2^n] \times [n+1]}$, where $\alpha_{x,i} = f(x)_i$, then this is equivalent to allowing the algorithm to query $\text{poly}(n) = \text{poly}(\log N)$ variables of the assignment α .

Decision tree analogues of other complexity classes can be defined similarly. It is a standard result that a decision tree separation implies a separation of the associated Turing machine classes relative to an oracle; more specifically it is equivalent to a separation relative to a so-called “generic oracle” [18].

Most of the definitions in our decision tree models will be standard, e.g. bounded depth formulae and DNFs. One decision tree model we study whose definition and notation is less standardized is a FP^{NP} decision tree:

Definition 2.29 (FP^{NP} Decision Trees). *Let $f : \{0, 1\}^n \rightarrow A$ be a function where A is some set. A FP^{NP} decision tree T computing f is defined by a binary tree, with each internal node labeled*

by a DNF formula on the variables $\{x_i \mid i \leq n\}$ and each leaf labeled by a value $y \in A$. On an input $x \in \{0, 1\}^n$, we traverse T starting at the root. At each internal node associated to a DNF D , we test if $D(x) = 1$; if so we proceed to the right child of the current node, otherwise we proceed to the left. When we reach a leaf we output the value associated to it; T computes f if the value reached is $f(x)$. We say that T has complexity $\leq r$ if its depth as a tree is at most r , and each DNF associated to its nodes has width at most r .

We next define the decision tree variant of $\text{FP}_{\parallel}^{\Sigma_2^P}$ which is the subject of our main lower bound Theorem 2.5:

Definition 2.30. Let $R \subseteq \{0, 1\}^N \times [M]$ black-box (relativized) search problem. We say that R is in $\text{FP}_{\parallel}^{\Sigma_2^P}$ if there exist Σ_2^P circuits $\Phi_1, \dots, \Phi_k$ with $k \leq \text{poly log}(N)$ and an arbitrary post-processing function $S : \{0, 1\}^k \rightarrow [M]$ so that for all inputs $f \in \{0, 1\}^N$, $(f, S(\Phi_1(f), \dots, \Phi_k(f))) \in R$. A Σ_2^P circuit is a $2^{\text{poly log } N}$ -size depth 3 circuit with bottom fanout $\leq \text{poly log } N$.

In our case the input will be some $f \in \{0, 1\}^{N \log N}$ representing a function $f : [N] \rightarrow [N + 1]$, and the relevant search problem is to output some $y \in [N + 1]$ outside its range. The fact that this captures the relativized version of $\text{FP}_{\parallel}^{\Sigma_2^P}$ is based on the well known equivalence between relativized levels of the polynomial hierarchy and quasipolynomial size bounded depth circuits [53]; in particular the Σ_2^P circuits $\Phi_1, \dots, \Phi_k$ above correspond to a sequence of non-adaptive Σ_2^P oracle queries.

Note on bottom fanin: The depth 3 circuit model which captures $\text{FP}_{\parallel}^{\Sigma_2^P}$ has the additional restriction that the bottom level of each depth 3 circuit has fanin $\text{poly log } N$. Our lower bound will apply also to the stronger model in which the depth 3 circuits are only constrained in their size and not their bottom fanin.

Finally, we make note of one separation indicated in Figure 2.1 which was shown prior to our work:

Lemma (Wilson and Vyas-Williams[171][165]). *In the decision tree model, WEAK AVOID is not in FP^{NP} .*

2.3 Lower Bounds for Pigeonhole Principles

2.3.1 Pigeonhole Principle Basics

We will be concerned here with search problems where the input is a function $f : [N] \rightarrow [M]$ with $M > N$, and the goal is to find an empty pigeonhole of f .

Definition 2.31. We use $[M]^{[N]}$ to refer to the set of all functions $f : [N] \rightarrow [M]$. We define a set of propositional variables, referred to as “bit variables”, given by

$$\text{BITS}_{N,M} := \{f_{x,i} \mid x \in [N], i \in [\lceil \log M \rceil]\}$$

We associate each truth assignment $\alpha : \text{BITS}_{N,M} \rightarrow \{0, 1\}$ with the function $f_\alpha : [N] \rightarrow [M]$, where where the value α assigns to $f_{x,i}$ indicates the i^{th} bit of $f_\alpha(x)$ in binary; if M is not a power of two then we think of all strings in $\{0, 1\}^{\lceil \log M \rceil}$ of binary value exceeding M as being redundant representations of the element M . Similarly we associate every function $f : [N] \rightarrow [M]$ with the assignment $\alpha_f : \text{BITS}_{N,M} \rightarrow \{0, 1\}$ using the same correspondence.

For a conjunction of literals (term) τ over the variables $\text{BITS}_{N,M}$, we define its pigeon-width to be the size of the set $\{x \in [N] \mid f_{x,i} \text{ or } \neg f_{x,i} \text{ occurs in } \tau \text{ for some } i\}$.

From now on we will often not refer explicitly to assignments $\alpha : \text{BITS}_{N,M} \rightarrow \{0, 1\}$, only to functions $f : [N] \rightarrow [M]$. The relevance of this definition is that, when defining circuits/computational devices whose input is a function $f : [M] \rightarrow [N]$, we must specify how the device is able to access/read the input f : typically this will correspond to the ability to read the bits $\text{BITS}_{N,M}$.

For the vast majority of this section we will restrict our attention to functions $f : [N] \rightarrow [M]$ which are 1-1 (injective): consequently when we speak about the evaluation of a formula which takes f as an input, we will typically only care about the behavior of that formula on this special class of inputs. We use the following notation to express this:

Definition 2.32. Let $\mathcal{F}_{N,M} \subset [M]^{[N]}$ denote the set of all 1-1 functions $[N] \rightarrow [M]$. For two

predicates $F, G : [M]^{[N]} \rightarrow \{0, 1\}$, we say that F is equivalent to G (with respect to all 1-1 functions), denoted by $F \equiv G$, if $F(f) = G(f)$ for all $f \in \mathcal{F}_{N,M}$.

We now introduce notation for describing partial information about an input f . A natural such unit of partial information is a partial assignment $\rho : \text{BITS}_{N,M} \rightarrow \{0, 1, \star\}$, however for our purposes it will be useful to define additional types of partial information as well:

Definition 2.33 (Matchings and Hole Restrictions). *A partial matching π is a partially defined 1-1 function from $[N]$ to $[M]$. We use $\text{dom}(\pi), \text{range}(\pi)$ to refer to the domain and range of π , and $\text{nodes}(\pi) = \text{dom}(\pi) \sqcup \text{range}(\pi)$. More generally a “hole restriction” $\tau = \langle \pi, E \rangle$ consists of a partial matching π and a set $E \subseteq [M]$ satisfying $\text{range}(\pi) \cap E = \emptyset$. Let $\text{nodes}(\tau) = \text{nodes}(\pi) \cup E$. We use $|\tau|$ to refer to the value $|\pi| + |E|$. We think of the hole restriction $\langle \pi, E \rangle$ as describing the following partial information about a function $f : [N] \rightarrow [M]$: “ $f(x) = \pi(x)$ for all $x \in \text{dom}(\pi)$, and $y \notin \text{range}(f)$ for all $y \in E$.” For a total assignment $f \in \mathcal{F}_{N,M}$, we say f is compatible with $\langle \pi, E \rangle$, written $f \parallel \langle \pi, E \rangle$, if the above statement holds. For two hole restrictions $\tau_1 = \langle \pi_1, E_1 \rangle, \tau_2 = \langle \pi_2, E_2 \rangle$, we say they are consistent, written $\tau_1 \parallel \tau_2$, if there is a total assignment f which is compatible with both. We say τ_1 extends τ_2 , written $\tau_1 \supseteq \tau_2$, if $\pi_1 \supseteq \pi_2$ and $E_1 \supseteq E_2$. We will think of a partial matching π as a special kind of hole restriction of the form $\langle \pi, \emptyset \rangle$, and a total 1-1 assignment f as a special hole restriction of the form $\langle f, \overline{\text{range}(f)} \rangle$, where $\overline{\text{range}(f)} = [M] \setminus \text{range}(f)$. In this way we will use the above terminology to define relations between matchings, hole restrictions and total 1-1 assignments, e.g. $f \parallel \pi$ and $\pi \subseteq \tau$.*

The main subject of this section is circuits $C : \{0, 1\}^{|\text{BITS}_{N,M}|} \rightarrow \{0, 1\}^{\lceil \log M \rceil}$ which solve AVOID, in the sense that $C(f) \notin \text{range}(f)$ for all f , where $C(f)$ denotes feeding C the representation of f as an assignment to its bit variables. We thus need some notation for basic kinds of circuits computing a function of f , which will be hole and matching disjunctions:

Definition 2.34 (Matching and Hole Disjunctions). *A hole disjunction $\phi = \vee_i \tau_i$ over $[M]^{[N]}$ is defined by a collection of hole restrictions $[N] \rightarrow [M]$, which we refer to as the “terms” of ϕ . We say $\phi(f) = 1$, or “ f satisfies ϕ ,” if $f \parallel \tau_i$ for some term τ_i of ϕ . In this way each hole disjunction is*

associated with a boolean function $\mathcal{F}_{N,M} \rightarrow \{0, 1\}$. If $|\tau_i| \leq w$ for all i we say that ϕ has width w . We call ϕ a matching disjunction if all τ_i are partial matchings.

Recall that we are primarily concerned here with the values a formula takes on 1-1 assignments $f \in \mathcal{F}_{N,M}$. In such a setting we can simplify any circuit so that the bottom two logical layers are a set of matching disjunctions:

Observation 2.35. *Let D be a DNF formula on the bit variables $\text{BITS}_{N,M}$. Then there is a matching disjunction ϕ so that $D \equiv \phi$. Moreover if D has pigeon-width w and size s , then ϕ will have width at most w and size at most sM^w .*

Proof. Say that $D = \vee_i \wedge_j \ell_{i,j}$ is a DNF where each $\ell_{i,j}$ is a literal on $\text{BITS}_{N,M}$. For each term $t_i = \wedge_j \ell_{i,j}$ we may replace it by the matching disjunction:

$$\theta_i = \bigvee_{\pi \in \text{match}(t_i)} \pi$$

where $\text{match}(t_i)$ is the set of minimal partial matchings π so that for each literal $\ell_{i,j} = (\neg)^\xi f_{x,b}$ in t_i (with $\xi \in \{0, 1\}$), there is an edge $(x, y) \in \pi$ with $y_b = \neg\xi$. Clearly we have that $\theta_i(f) = t_i(f)$ for all 1-1 assignments f , and thus we may replace D with the matching disjunction $\vee_i \theta_i$ without affecting its behavior on 1-1 assignments. Recall that the pigeon-width of a term over $\text{BITS}_{N,M}$ is the total number of pigeons $x \in [N]$ mentioned in the term; clearly the width of θ_i is at most the pigeon width of t_i since it only mentions the $\leq w$ pigeons relevant to t_i ; for this same reason we may bound its size (number of terms) by M^w , the number of mappings from these $\leq w$ pigeons onto the M available holes. $\square$

Definition 2.36. *Let $\phi = \tau_1 \vee \dots \vee \tau_s$ be a hole disjunction with $\tau_i = \langle \pi_i, E_i \rangle$ and let $\kappa = \langle \sigma, U \rangle$ be a hole restriction. We define ϕ restricted by τ , denoted by $\phi \upharpoonright \kappa$, as follows: (i) First, any term $\tau_i \in \phi$ that is inconsistent with κ is set to 0 by κ and thus these terms disappear from $\phi \upharpoonright \kappa$ (we also say they are “killed” by κ). If all terms are set to 0, then $\phi \upharpoonright \kappa = 0$. (ii) Otherwise for any term τ_i consistent with κ , we replace τ_i with $\tau_i \upharpoonright \kappa := \langle \pi_i \setminus \sigma, E_i \setminus U \rangle$. If $\tau_i \upharpoonright \kappa = \langle \emptyset, \emptyset \rangle$ this means κ already satisfies this term; if this happens for any of the τ_i then we set $\phi \upharpoonright \kappa = 1$.*

The last basic computational model acting on inputs $f \in \mathcal{F}_{N,M}$ we consider is a “pigeonhole decision tree”:

Definition 2.37. *A pigeonhole decision tree T over $\mathcal{F}_{N,M}$ is defined by a rooted tree with fanout $\leq M$, with leaves labeled by values from some finite set $\mathcal{Z}$. Each internal node $v \in T$ is labelled by either a “pigeon query” or a “hole query”. Pigeon nodes (those that make a pigeon query) are labeled by a query $q(v) \in [N]$ and have $\leq M$ outgoing edges each labeled by a distinct element $(q(v), y) \in [N] \times [M]$. Hole nodes are labeled by a query $q(v) \in [M]$ and have $\leq N + 1$ outgoing edges, which are either labelled by a distinct element $(x, q(v)) \in [N] \times [M]$ or by $q(v) \in [M]$. (The label $q(v)$ corresponds to nothing mapping to pigeon $q(v)$.) In this way, we can associate each node $v \in T$ to some $\langle \pi, E \rangle$ with $\pi \subseteq [N] \times [M]$, $E \subseteq [M]$, consisting of the labels of all edges on the path from the root to v in T . We then require that this pair $\langle \pi, E \rangle$ associated to v is a hole disjunction.*

T is said to be a complete pigeonhole decision tree if for every $f \in \mathcal{F}_{N,M}$ there is a (unique) root to leaf path in T that is consistent with f . In the case that T is complete we may associate T with a function $T : \mathcal{F}_{N,M} \rightarrow \mathcal{Z}$, where $T(f)$ is the value at the leaf of T consistent with f . The depth of T is the length of the longest root to leaf path. We will sometimes define pigeonhole decision trees without having specific associated leaf values in mind; in this case we refer to the tree as “unlabeled.”

Note: The queries in a pigeonhole decision tree *do not* correspond directly to queries of the underlying bit variables $\text{BITS}_{N,M}$. While it is possible to query $\log M$ of the variables $\text{BITS}_{N,M}$ to determine $f(x)$ for some $x \in [N]$, if we want to determine the preimage of $y \in [M]$ under f (or determine it has no preimage), we would need to query $\approx N \log M$ variables of $\text{BITS}_{N,M}$ to make this determination directly. Instead, a pigeonhole decision tree corresponds more directly to a special kind of P^{NP} decision tree on the bit variables.

We next observe that if T is a complete depth d pigeonhole decision tree with binary leaf values, then we may represent both T and $\neg T$ by a width d hole disjunction:

Observation 2.38. *Let $F : \mathcal{F}_{N,M} \rightarrow \{0,1\}$ be some predicate. If $F \equiv T$ for some complete pigeonhole decision tree T of depth d , then there exist hole disjunctions ϕ_1, ϕ_2 of width $\leq d$ so that $F \equiv \phi_1$ and $\neg F \equiv \phi_2$.*

We will need the following variant of the Switching Lemma, which says that for any low-width matching disjunction ϕ , if we sample a random partial matching ρ , then with high probability $\phi \upharpoonright \rho$ will have a low-depth, complete pigeonhole decision tree. We defer the proof of the Switching Lemma to Section 2.5.

Definition 2.39 (Distribution of Partial Restrictions). *Let $\mathcal{M}_K^{N,M}$ be the set of all partial matchings $[N] \rightarrow [M]$ with exactly $N - K$ edges. We use $\rho \sim \mathcal{M}_K^{N,M}$ to denote a sample from the uniform distribution on this set. When N, M are clear from context we write $\mathcal{M}_K$ as shorthand for $\mathcal{M}_K^{N,M}$.*

Lemma 2.40 (Pigeonhole Switching Lemma). *Let $M, N, d \in \mathbb{N}$. Let ϕ be a width- w matching disjunction over $[M]^{[N]}$. If $M - N \leq K \leq \frac{N}{4}$ and N, K, d, w sufficiently large, then*

$$\Pr_{\rho \sim \mathcal{M}_K} [\phi \upharpoonright \rho \text{ has no depth } \leq d \text{ pigeonhole decision tree}] \leq \exp\left(d(\log wK^5 - \log N^{1/2} + O(1))\right)$$

2.3.2 Depth 3 Lower Bound for Strong Avoid

In this section we prove the following circuit lower bound, which is a restatement of Theorem 2.5.

Theorem 2.5 (Restated). *There is some absolute constant $\epsilon > 0$ so that the following holds. Let $\Phi_1, \dots, \Phi_k$ be depth 3, size $\leq s$ unbounded fanout formulas over the bit variables $\text{BITS}_{N,N+1}$. Provided $k \leq N^\epsilon$ and $s \leq 2^{N^\epsilon}$, there exists a string $z \in \{0,1\}^k$ so that for all $y \in [N+1]$, there exists an assignment $f : [N] \rightarrow [N+1]$ such that:*

1. $\Phi_i(f) = z_i$ for all i
2. $y \in \text{range}(f)$

In particular it is not possible to determine an empty pigeonhole of f by reading the values of $\Phi_1(f), \dots, \Phi_k(f)$.

Overview of Proof. As mentioned in Section 2.1.2, we want to prove Theorem 2.5 by restricting attention to the class of inputs that correspond to 1-1 functions, and in order to do this we will need to prove a strengthening of Theorem 2.5, by proving the lower bound for AVOID on input functions $f : [N] \rightarrow [M]$, where M is larger than $N + 1$. As discussed in Section 2.1.2, this implies the lower bound stated above (with $M = N + 1$).

Thus we assume for sake of contradiction that there exist depth-3 AND-of-OR-of-AND circuits $\Phi_1, \dots, \Phi_k$ of total size at most s that solves AVOID on all one-to-one instances $f : [N_0] \rightarrow [M_0]$, where $M_0 = N_0 + N_0^\epsilon$, for some $0 < \epsilon < 1$.

1. The first step is to argue that there exists a matching restriction ρ such that after applying ρ , all depth-2 subcircuits (which are DNFs) have pigeon-width at most $w \approx \log s$. This is accomplished in the Width Reduction Lemma (2.44) by a standard Chernoff argument and a union bound. Composing this with Observation 2.35, we can then convert all of the depth two subcircuits to *matching disjunctions* of the same width w , without affecting their behavior on 1-1 assignments. Overall, we are left with depth-3 circuits $\Phi_1, \dots, \Phi_k$ which are ANDs of matching disjunctions of width at most $w = c \log s$ for some constant $c > 0$, and there size is at most $sM^{c \log s}$. These circuits still solve AVOID but now with respect to 1-1 functions on the reduced domain and range, $[N_1], [M_1]$, where $N_1 = \Omega(N_0)$, and $M_1 \approx N_1 + N_1^\epsilon$.
2. At this point each Φ_i is an AND of low-width matching disjunctions, i.e. we have $\Phi_i = \bigwedge_j \phi_{i,j}$ where the $\phi_{i,j}$ are width w matching disjunctions. The next step is to apply our Pigeonhole Switching Lemma (2.40) which will tell us that there exists a matching restriction ρ such that for all i, j , $\phi_{i,j} \upharpoonright \rho$ simplifies to a depth d pigeonhole decision tree. We will be able to choose the parameters in our switching lemma so that after the second restriction, there are N_2 remaining pigeons and $M_2 = 2N_2$ remaining holes, with $d^2 k^2 < N_2$. By

Observation 2.38, this allows us to rewrite $\neg\phi_{i,j}$ as a width= d hole disjunction, and thus $\neg\Phi_i$ as width d hole disjunction. The simplified circuits $\neg\Phi_1, \dots, \neg\Phi_k$ still solve AVOID, but now with respect to 1-1 functions on the reduced domain and range leftover after this restriction, i.e. on instances $[N_2] \rightarrow [2N_2]$.

3. The final step is to prove that if $\phi_1, \dots, \phi_k$ are width- d hole disjunctions that solve AVOID with respect to all 1-1 functions from $[N_2]$ to $[2N_2]$, then we must have $d^2 k^2 > N_2$. This is accomplished by Lemma 2.41. The proof is a novel argument based on coverings. Recall that our goal is to give some sequence of values $z_1, \dots, z_k \in \{0, 1\}$, so that for all $y_0 \in [2N_2]$ we can find a 1-1 instance $f : [N_2] \rightarrow [2N_2]$ so that $\phi_1(f) = z_1, \dots, \phi_k(f) = z_k$, but y_0 is not an AVOID solution for f . This indicates that the values $\phi_1(f), \dots, \phi_k(f)$ are insufficient to determine an AVOID solution. To construct $z_1, \dots, z_k$, we start by repeatedly searching for some ϕ_i whose underlying terms have a *small hitting set*, which is a small set of pigeons and holes so that every term of ϕ_i mentions at least one of them. If some ϕ_i is found, we apply a partial restriction ρ to the hitting set variables, which reduces the width of ϕ_i by 1. Since there are k ϕ_i 's, and each has width at most d , after dk iterations, each ϕ_i either has been set to a constant, or is promised to have no small hitting set. We set $z_i = 1$ or $z_i = 0$ for all the ϕ_i which have been forced to a constant by our restriction ρ in this process, and set $z_i = 1$ for all the unkilld ϕ_i . Then to finish the proof, we need to show that for any y_0 we can find an instance $f \parallel \rho$ so that $y_0 \in \text{range}(f)$, and $\phi_i(f) = 1$ for every ϕ_i which was unkilld in the first step. Let $y_0 \in [2N_2]$ be given. Starting with the partial assignment ρ , we extend it so some unmapped pigeon x_0 goes to y_0 , and then we iterate over each unkilld ϕ_i and try extend ρ to satisfy one of its terms greedily. The correctness will follow from the fact that none of the unkilld ϕ_i have a small hitting set.

We now formalize the above, which relies on the three Lemmas mentioned in the proof overview. Two of these Lemmas (Lemmas 2.44 and 2.41) will then be proven, while the more involved Lemma 2.40 is postponed to Appendix 2.5.

Proof of Theorem 2.5. Fix some ϵ, δ to be specified later. Let N_0 be sufficiently large, and let $M_0 = N_0 + N_0^\epsilon$. Assume towards a contradiction that $\Phi_1, \dots, \Phi_k$ are depth-3, size $s = 2^{N_0^{\epsilon/2}}$ circuits so that for all assignments $f : [N_0] \rightarrow [M_0]$, we may determine an empty pigeonhole for f by reading the values $\Phi_1(f), \dots, \Phi_k(f)$, and that $k \leq N_0^{\epsilon/5}$. Without loss of generality we may assume:

$$\Phi_i = \bigwedge_j D_{i,j}$$

where $D_{i,j}$ is a DNF over the variables BITS_{N_0, M_0} . Obviously if $\Phi_1(f), \dots, \Phi_k(f)$ could determine an empty pigeonhole on all assignments f then it would do so on all 1-1 assignments; we will derive a contradiction already from this fact.

Applying the width reduction lemma (Lemma 2.44) with $w = c \log s$ for a sufficiently large constant $c > 1$, and a union bound over i, j , we can find a partial matching ρ_0 leaving N_1 pigeons unmapped, where $N_1 = \delta N_0$, such that for all i, j , $D_{i,j} \upharpoonright \rho_0$ has pigeon-width w . We then convert each $D_{i,j}$ to matching disjunction $\phi_{i,j}$ using Observation 2.35; the width of the matching disjunctions is at most w , and their number is at most $s' := s \cdot M_0^{c \log s} \leq 2^{N_0^\epsilon}$. At this point each circuit Φ_i has been reduced to a conjunction of width w matching disjunctions of size s' . The reduced circuits still solve AVOID with respect to all 1-1 functions f with domain $N_1 = \delta N_0$ and range $M_1 = N_1 + N_0^\epsilon$.

Next we apply our Switching Lemma (Lemma 2.40) with the following choice of parameters:

$$N := N_1$$

$$M := M_1 = N_1 + N_0^\epsilon$$

$$K := N_0^\epsilon$$

$$w = c \log s = c N_0^\epsilon$$

$$d := K^{1/5} = N_0^{\epsilon/5}$$

Choosing ϵ sufficiently small, it follows that for all i, j :

$$\Pr_{\rho \sim \mathcal{M}_K} [\mathcal{T}_{\phi_{i,j} \upharpoonright \rho} \text{ has depth } \geq d] < \frac{1}{ks'}$$

Thus by a union bound over i, j , we conclude that there exists a partial matching ρ so that for each i, j there is a pigeonhole decision tree $T_{i,j}$ of depth d such that $\phi_{i,j} \equiv_{\rho} T_{i,j}$.

Applying Observation 2.38 we can find hole disjunctions $\psi_{i,j}$ of width d so that $\neg \phi_{i,j} \equiv_{\rho} \psi_{i,j}$, and therefore $\neg \Phi_i \equiv_{\rho} \bigvee_j \psi_{i,j}$ which is a width d hole disjunction. By the conditions on the original $\Phi_1, \dots, \Phi_k$, for all 1-1 assignments f extending ρ we must be able to determine an empty pigeonhole of f by reading the values $\neg \Phi_1(f), \dots, \neg \Phi_k(f)$.

After applying ρ , we are left with a reduced domain and range $[N_2]$ and $[M_2]$, where $N_2 = K = N_0^{\epsilon}$, and $M_2 = M_1 - (N_1 - K) = 2K = 2N_2$. Therefore on the smaller input size $[2N_2]^{[N_2]}$, we have found a sequence of k width- d hole disjunctions such that on all 1-1 assignments $g : [N_2] \rightarrow [2N_2]$, we can determine an empty pigeonhole of g by reading the values of these hole disjunctions applied to g . At this point we reach a contradiction with Theorem 2.41, since by our choice of parameters we have $4k^2d^2 < N_2$ is satisfied (recall that $k \leq N_0^{\epsilon/5}$ by assumption).

□

Equipped with the Pigeonhole Switching Lemma (Lemma 2.40, proof in Section 2.5), it is left to prove Lemmas 2.44 and 2.41. We will first prove Lemma 2.41 followed by a proof of Lemma 2.44.

Lemma 2.41. *Let $\phi_1, \dots, \phi_k$ be hole disjunctions of width w over $[2N]^{[N]}$. Then provided $N > 4k^2w^2$, there exists a string $z \in \{0, 1\}^k$ so that for all $y \in [2N]$, there exists a 1-1 function $f : [N] \rightarrow [2N]$ such that:*

1. $\phi_i(f) = z_i$ for all i
2. $y \in \text{range}(f)$

In particular it is not possible to determine an empty pigeonhole of f from $\phi_1(f), \dots, \phi_k(f)$ for all 1-1 assignments f .

To prove this we need the following definition:

Definition 2.42 (Hitting Sets). *If ϕ is a hole disjunction and $A \subseteq [N] \sqcup [M]$, we say that A is a hitting set for ϕ if $A \cap \text{nodes}(\tau) \neq \emptyset$ for all terms $\tau \in \phi$.*

We can observe the following property of hitting sets:

Observation 2.43. *Let $\pi : [N] \rightarrow [M]$ be a partial matching, ϕ a nonempty hole disjunction, and $A \subseteq [N] \sqcup [M]$ a non-empty hitting set for $\phi \upharpoonright \pi$. Say that π' is an extension of π such that $A \subseteq \text{nodes}(\pi')$. Then for each term $\tau \in \phi \upharpoonright \pi$, τ has strictly smaller width in $\phi \upharpoonright \pi'$ than it originally did in $\phi \upharpoonright \pi$.*

We can now prove the main claim:

Proof. To prove this we will construct partial matching $\pi : [N] \rightarrow [2N]$, a set $I \subseteq [k]$, and values $\{z_i \in \{0, 1\} \mid i \in I\}$ so that:

1. For all total 1-1 assignments f extending π , $\phi_i(f) = z_i$ for all $i \in I$.
2. For all $y_0 \in [2N]$, there exists a total 1-1 assignment f extending π so that $\phi_i(f) = 1$ for all $i \notin I$, and $y_0 \in \text{range}(f)$.

We will initialize $\pi = \emptyset$ and expand it in stages by the following procedure:

1. If there exists a nonempty $A \subseteq [N] \sqcup [2N]$, $|A| \leq 2kw + 2$ and some $i \in [k]$ so that A is a hitting set for $\phi_i \upharpoonright \pi$, then extend π to a 1-1 map π' so that $A \subseteq \text{nodes}(\pi')$. If none exist then halt the procedure and output the current π .
2. Set $\pi := \pi'$ and return to the previous step.

We claim that we will always be able to extend π in the appropriate way until no further hitting sets can be found, and that at the end we will have $|\pi| \leq kw(2kw + 2) = 2k^2w^2 + 2kw$. To see this,

observe that if ϕ is a hole disjunction of width w and $\text{nodes}(\pi)$ is a hitting set for ϕ , then every term in ϕ will either be killed in $\phi \upharpoonright \pi$ or else have its width decreased by 1 by Observation 2.43. Therefore the above procedure can only repeat kw times before all the ϕ_i have been killed. In each step at most $2kw + 2$ edges need to be added to π in order to cover A , so the total size of π at any step is at most $kw(2kw + 2) = 2k^2w^2 + 2kw$, and therefore there are always enough available pigeons/holes to extend π in the appropriate way at each step since $N > k^2w^2$.

Now we can choose $I \subseteq [k]$, which will consist of those indices i so that $\phi_i \upharpoonright \pi$ is forced to a constant (i.e. one of its terms is already satisfied by π in which case it is forced to one, or else all of its terms are killed and it is forced to 0). For $i \in I$ we denote by $z_i \in \{0, 1\}$ the value it is forced to. It remains to show that for any given $y_0 \in [2N]$, we can find a total 1-1 assignment f extending π so that $y_0 \in \text{range}(f)$ and $\phi_i(f) = 1$ for all $i \notin I$. To do this we will construct a hole restriction $\langle \rho, U \rangle$, where ρ is another partial matching disjoint consistent with π , and $U \subseteq [2N]$ satisfying:

1. $y_0 \in \text{range}(\pi \cup \rho)$
2. $|\rho| \leq kw, |U| \leq kw$
3. $U \cap \text{range}(\pi \cup \rho) = \emptyset$
4. For each $i \in [k] \setminus I$ there is a term $\langle \sigma, E \rangle \in \phi_i$, so that for all $x \in \text{dom}(\sigma)$ we have $\pi \cup \rho(x) = \sigma(x)$, and $E \subseteq U$.

If we can accomplish this then the proof is complete. We simply extend $\pi \cup \rho$ to a total 1-1 assignment f which leaves U out of its range. By the assumption $N > 4k^2w^2$ such a total assignment exists; here we are using the fact that the total number of holes $2N$ is larger than the size of U plus the number of holes filled thus far by $\pi \cup \rho$. By construction we then have $\phi_i(f) = 1$ for all $i \notin I$ and $y \in \text{range}(f)$.

Observe that by construction of π , for each $i \notin I$ we have that $\phi_i \upharpoonright \pi$ has no hitting set of size $\leq 2kw + 2$. We will construct ρ, U in stages. Initially we check if $y_0 \in \text{range}(\pi)$, if not we initialize $\rho = \{(x_0, y_0)\}$ where x_0 is an arbitrary element unmapped in π , otherwise we initialize $\rho = \emptyset$. In

addition we initialize $U = \emptyset$. Now we go through each $i \in [k] \setminus I$ in order and do the following. We search for some term $\tau = \langle \sigma, E \rangle \in \phi_i \upharpoonright (\pi \cup \rho)$ so that $\text{nodes}(\tau) \cap \text{nodes}(\langle \rho, U \rangle) = \emptyset$. If found, then we add the nodes in E to U and the edges in σ to ρ . We claim that it is always possible to find such a term while maintaining that $\pi \cup \rho$ is 1-1 and $U \cap \text{range}(\pi \cup \rho) = \emptyset$. In particular say that we have gotten to some step $i \in [k] \setminus I$ where this is not possible. Let $A = \text{nodes}(\langle \rho, U \rangle)$. Observe that by construction $|A| \leq 2kw + 2$. Recall that by construction of π we have that A cannot be a hitting set for $\phi_i \upharpoonright \pi$. By definition of a hitting set this implies the existence of the required term $\tau \in \phi_i$. $\square$

Next, we prove the Width Reduction Lemma:

Lemma 2.44 (Width Reduction). *Let $\epsilon > 0$ be a sufficiently small constant. Let ϕ be a DNF over $\text{BITS}_{N,M}$ with s terms. Say a partial matching ρ is sampled as follows: sample $A \subseteq [N]$ by including each element independently with probability $1 - \epsilon$; now choose a uniform 1-1 function $A \rightarrow [M]$.*

$$\Pr_{\rho}[D \upharpoonright \rho \text{ has pigeon width} \geq w] < \exp(\log s - \Omega(w))$$

Proof. Let $\phi = \lambda_1, \dots, \lambda_s$. We will ignore each term whose pigeon width is already $< w$; its pigeon width cannot increase under the restriction. Let $\lambda \in \phi$ be a term of width $\geq w$. We will bound the probability that ρ does not kill the term λ and then take a union bound over the s terms. Without loss of generality we may assume λ is the literal $f_{x,1}$; if it were of the form $f_{x,i}$ or $\neg f_{x,i}$ for some $i > 1$ the argument is completely symmetric. Let $B \subset M$ consist of those $y \in M$ whose first

bit (in binary) is 1.

$$\begin{aligned}
\Pr_{\rho}[\lambda \text{ survives } \rho] &= \sum_{U \subseteq \text{dom}(\lambda)} \Pr[\text{dom}(\lambda) \cap \text{dom}(\rho) = U] \cdot \left(\frac{|B|}{M}\right)^{|U|} \\
&\leq \sum_{t=1}^w \Pr[|\text{dom}(\lambda) \cap \text{dom}(\rho)| = t] \cdot 2^{-t} \\
&\leq 2^{-w/2} + \Pr[|\text{dom}(\lambda) \cap \text{dom}(\rho)| < \frac{w}{2}] \\
&\leq 2^{-w/2} + \Pr_{\xi_1, \dots, \xi_w \sim \text{Bern}(1-\epsilon)} \left[\sum_i \xi_i \leq \frac{w}{2} \right] \leq \exp(-\Omega(w))
\end{aligned}$$

Where the last inequality follows from Chernoff's bound and $\text{Bern}(1 - \epsilon)$ denotes the Bernoulli distribution with expected value $1 - \epsilon$. Thus by a union bound

$$\Pr_{\rho}[\phi \upharpoonright \rho \text{ has width } \geq w] \leq \exp(\log s - \Omega(w))$$

□

Lower Bounds for Moderately Weak Avoid: Above we showed that depth-3 circuits cannot solve the STRONG AVOID problem unless their size is exponential. In contrast, Li's construction gives *quasipolynomial* size depth 3 circuits which solve the WEAK AVOID problem, i.e. AVOID on instances $[N] \rightarrow [2N]$ when the codomain is at least twice as large as the domain. By a standard iteration construction (the same argument which lets us reduce $[N] \rightarrow [2N]$ AVOID to $[N] \rightarrow [N^2]$ AVOID as in [101]), for every constant d we can obtain quasipolynomial size depth 3 circuits solving AVOID on instances $[N] \rightarrow [N + N/\log^d N]$. We show here that this is essentially optimal: for every constant c there exists another constant d so that $[N] \rightarrow [N + N/\log^d N]$ AVOID cannot be solved by depth 3 circuits of size $2^{\log^c N}$. The proof is a simple reduction to Theorem 2.5 which is originally due to Razborov [140].

Lemma 2.45. *For every $c \in \mathbb{N}$ there exists $d \in \mathbb{N}$ so that the following holds. Let $\Phi_1, \dots, \Phi_k$ be depth 3 circuits of size s such that for every assignment $f : [N] \rightarrow [N + N/\log^d N]$, an AVOID solution can be determined from the values $\Phi_1(f), \dots, \Phi_k(f)$. Then either $k \geq \log^c N$ or*

$$s \geq 2^{\log^c N}.$$

Proof. Let $c \in \mathbb{N}$ be fixed. We will choose $M = \log^d N$ for an appropriate choice of d which will be specified later and which will depend only on c . We then reduce the AVOID problem on instances $[M] \rightarrow [M+1]$ to instances $[N] \rightarrow [N + N/\log^d N]$ as follows: given $g : [M] \rightarrow [M+1]$, let $g^{\otimes \ell} : [M\ell] \rightarrow [(M+1)\ell]$ be the map which sends each block $[1, M], \dots, [M(\ell-1), M\ell]$ to its corresponding block in $[1, M+1], \dots, [(M+1)(\ell-1), (M+1)\ell]$ according to g . Setting $\ell = \frac{N}{M}$ we see that g is a map $[N] \rightarrow [N + \ell]$, where $\ell = N/M = N/\log^d N$.

Obviously given g we can generate $g^{\otimes \ell}$ without any computational overhead by simply substituting variables. In addition, any AVOID solution to $g^{\otimes \ell}$ uniquely determines an AVOID solution to g , obtained by simply forgetting the block number and outputting the position within the relevant block. Thus if the circuits $\Phi_1(f), \dots, \Phi_k(f)$ could solve AVOID on inputs $[N] \rightarrow [N + N/\log^d N]$ then they can also solve AVOID on inputs $[M] \rightarrow [M+1]$. By Theorem 2.5 there is some absolute constant ϵ , such that for this to be possible we must have $k \geq M^\epsilon$ and $s \geq 2^{M^\epsilon}$. Setting $d > \frac{c}{\epsilon}$ the theorem follows. $\square$

2.3.3 BPP^{NP} Lower Bound for 1-1 Strong Avoid

We restate the main result to be proved:

Theorem 2.7 *In the decision tree model, STRONG 1-1 AVOID is not solvable in FBPP^{NP}.*

Recalling the definition of BPP^{NP} (in the decision tree model), if 1-1 STRONG AVOID has BPP^{NP} decision trees of complexity w , then there is a distribution $\mathcal{T}$ over P^{NP} decision trees of complexity r , so that for all $f : [N] \rightarrow [N+1]$ $\mathcal{T}$ outputs a 1-1 STRONG AVOID Solution with probability $\geq 2/3$. By Yao's minimax principle, this implies the existence of a fixed tree $T \in \text{support}(\mathcal{T})$, so that $T(f)$ succeeds in finding an empty pigeonhole of f with probability $\geq 2/3$ when f is sampled uniformly from the space of 1-1 functions $\mathcal{F}_{N,N+1}$. It thus suffices to rule out the existence of such a deterministic tree T . We will use here the notation $\mathcal{F}_N := \mathcal{F}_{N,N+1}$, and $f \sim \mathcal{F}_N$ to denote a uniform sample from this set. Thus Theorem 2.7 follows from the following Theorem:

Theorem 2.46. *There is an absolute constant $\epsilon > 0$ so the following holds: if T is a P^{NP} decision tree of complexity N^ϵ over the bit variables $\text{BITS}_{N,N+1}$ and leaves labeled by elements of $[N + 1]$, then:*

$$\Pr_{f \sim \mathcal{F}_N} [T(f) \notin \text{range}(f)] \leq N^{-\Omega(1)}$$

The high level idea of the proof is as follows. Observe that a random $f \sim \mathcal{F}_{N,M}$ can be sampled by first choosing a uniform partial matching ρ of a certain size, and then choosing a uniform completion of ρ to a total assignment. Applying our Switching Lemma, we can argue that with high probability, after we sample ρ we can replace all of the NP queries in our P^{NP} decision tree T with small depth pigeonhole decision trees, which overall allows T to be replaced by a low depth pigeonhole decision tree. It then remains only to argue that a pigeonhole decision tree of low depth cannot solve 1-1 STRONG AVOID with non-trivial probability on a uniform extension f of ρ , which can be accomplished with a direct argument.

Proof of Theorem 2.46. By definition of a P^{NP} decision tree, T is a binary tree of depth $r = N^\epsilon$ with each internal node branching on the value of some DNF of width r over the bit variables $\text{BITS}_{N,N+1}$. Let $D_1, \dots, D_s$ be the set of all DNF associated to the nodes of T ; so $s \leq 2^r$. As in the proof of Theorem 2.5 we may apply Observation 2.35 and replace all D_i by width $r \log N$ matching disjunctions ϕ_i since we only care about their behavior on assignments in $\mathcal{F}_N$. Fix some parameter K to be specified later. We will consider sampling $f \sim \mathcal{F}_N$ in the following way: first choose a uniform partial matching ρ with $N - K$ edges, then a uniform extension $f \supseteq \rho$ of ρ to a total 1-1 assignment. We define $\mathcal{B}$ as the event that $\phi_i \upharpoonright \rho$ does not have a pigeonhole decision tree of depth $\leq r$ for some $i \leq s$. Observe in the case $\rho \notin \mathcal{B}$, we may construct a pigeonhole decision tree $\mathcal{T}$ of depth r^2 so that $\mathcal{T} \equiv_\rho T$; we simply simulate the original computation of T , and each time a DNF D_i was originally queried, we instead simulate the depth $\leq r$ pigeonhole decision which represents ϕ_i . Let $\mathbb{P}_d(J)$ be the maximum, over all pigeonhole decision trees Q of depth d on $g : [J] \rightarrow [J + 1]$, of $\Pr_{g \sim \mathcal{F}_J} [Q(g) \notin \text{range}(g)]$. Then we have:

$$\begin{aligned}
\Pr_{f \sim \mathcal{F}_N} [T(f) \notin \text{range}(f)] &= \Pr_{\rho, f \supseteq \rho} [T(f) \notin \text{range}(f)] \\
&\leq \Pr_{\rho} [\rho \in \mathcal{B}] + \max_{\rho \notin \mathcal{B}} \Pr_{f \supseteq \rho} [T(f) \notin \text{range}(f)] \\
&\leq \Pr_{\rho} [\rho \in \mathcal{B}] + \mathbb{P}_{r^2}(K) \\
&\leq 2^r \max_i \Pr_{\rho} [\phi_i \upharpoonright \rho \text{ has no pigeonhole decision tree of depth } \leq r] + \mathbb{P}_{r^2}(K)
\end{aligned}$$

We will prove in the next lemma that $\mathbb{P}_{r^2}(K) \leq \frac{r^2}{K+1}$. Setting ϵ sufficiently small and $K = r^3$, we can apply Lemma 2.40 and complete the proof. $\square$

It remains to prove the bound on $\mathbb{P}_{r^2}(K)$. At the cost of increasing the depth of the tree by 1, we assume that a pigeonhole decision tree always queries a hole before outputting it as a solution.

Lemma 2.47. $\mathbb{P}_d(N) = \frac{d}{N+1}$

Proof. Let T be a tree witnessing $\mathbb{P}_d(N)$.

Say that T first queries a hole $y \in N$, so that

$$T(f) = \begin{cases} T_1(f) & \text{if } f(1) = y \\ \dots & \\ T_N(f) & \text{if } f(N) = y \\ y & \text{if } y \notin \text{range}(f) \end{cases}$$

then we have:

$$\begin{aligned}
\mathbb{P}_d(N) &= \Pr_f [T(f) \notin \text{range}(f)] \\
&= \Pr_f [y \notin \text{range}(f)] + \sum_x \Pr_f [f(x) = y] \cdot \Pr_f [T_x(f) \notin \text{range}(f) \mid f(x) = y] \\
&\leq \frac{1}{N+1} + \frac{N}{N+1} \mathbb{P}_{d-1}(N-1)
\end{aligned}$$

Here we are using that fact that a uniform f conditioned on $f(x) = y$ is equivalent, up to relabeling, to a uniform member of $\mathcal{F}_{N-1}$, and the labeling of pigeons and holes has no effect on the value of $\mathbb{P}_{d-1}(N-1)$. By the same reasoning if T first queries a pigeon x then we have the simpler bound:

$$\Pr_f[T(f) \notin \text{range}(f)] = \sum_y \Pr_f[f(x) = y] \cdot \Pr_f[T_y(f) \notin \text{range}(f) \mid f(x) = y] \leq \mathbb{P}_{d-1}(N-1)$$

so overall

$$\mathbb{P}_d(N) \leq \max\{\mathbb{P}_{d-1}(N-1), \frac{1}{N+1} + \frac{N}{N+1}\mathbb{P}_{d-1}(N-1)\}$$

Therefore by induction on d we conclude that the optimal pigeonhole decision tree achieving $\mathbb{P}_d(N)$ only queries holes, and without loss of generality queries them in order $1, \dots, d$ (in the base case we use the assumption that T must query a hole before using it as an answer), then outputs the index of the first empty hole that was queried (if any). Therefore we have:

$$\mathbb{P}_d(N) = \Pr_f[\{1, \dots, d\} \not\subseteq \text{range}(f)] = \frac{d}{N+1}$$

□

Next we show that Theorem 2.8, separating the decision class $\Sigma_2^P \cap \Pi_2^P$ from BPP^{NP} , follows as a corollary of Theorem 2.7.

Proof of Theorem 2.8. Lemma 2.26 says that STRONG 1-1 AVOID is solvable in $\text{FP}^{\Sigma_2^P \cap \Pi_2^P}$. This result holds relative to every oracle (and thus in the decision tree model). Hence relative to every oracle, if $\Sigma_2^P \cap \Pi_2^P \subseteq \text{BPP}^{\text{NP}}$ then STRONG 1-1 AVOID collapses into BPP^{NP} . The result then follows from Theorem 2.7. □

2.4 LINEAR ORDERING PRINCIPLE

In this section we investigate the complexity of the LINEAR ORDERING PRINCIPLE. We restate the definition in a slightly different terminology:

Definition (LINEAR ORDERING PRINCIPLE, Restatement of 2.2). *The input to LOP is a binary relation $\prec$ on $[N] \times [N]$ specified by a circuit. The following solutions are sought:*

1. *A witness that $\prec$ does not define a total order on $[N]$ (trivial solution)*
2. *An element $a_0 \in [N]$ so that $a_0 \prec a$ for all $a \neq a_0 \in [N]$ (nontrivial solution)*

We say an instance $\prec$ is nontrivial if it has no trivial solutions. The trivial solutions are enumerated formally as follows:

Antireflexivity Violation: x such that $x \prec x$

Transitivity Violation: x, y, z such that $x \prec y \prec z$ and $x \not\prec z$

Totality Violation: $x \neq y$ such that $x \not\prec y$ and $y \not\prec x$

Note that in the introduction we defined $\prec$ to be a relation on $\{0, 1\}^n \times \{0, 1\}^n$. We state it in this more general form here, since the space we define a linear ordering on in our main reduction will not have size being a power of two. It is straightforward to pad an instance of LOP which is defined on some subset of $\{0, 1\}^n$ to the whole space without affecting the value of the solution.

Recall from Section 2.2.1 that LOP has “essentially unique solutions:” the trivial solutions are checkable in polynomial time, and any instance without trivial solutions has a unique nontrivial solution. Our primary interest in this problem stems from the following reduction (stated in the Introduction):

Theorem 2.3 WEAK AVOID is polynomial-time many-one reducible to LOP.

We will describe the reduction in two parts, using an intermediate search problem called FOREST TERMINATION:

Definition 2.48 (FOREST TERMINATION). *The input consists of:*

1. A function $\text{Pred} : [M] \times [N] \times [N] \rightarrow [M]$

2. A function $\text{Col} : [M] \times [N] \rightarrow \{0, 1\}$

specified by boolean circuits. We think of the input as representing a layered rooted forest F of depth N with nodes partitioned into sets $L_1, \dots, L_N$, each of size M , with nodes in L_1 being roots and all other nodes in $\{L_j\}_{j>1}$ having a unique parent in L_{j-1} . We say that the function Pred “validly represents a forest” if the following holds: for each $u \in [M]$, $k \leq j \leq i \in [N]$, we have that $\text{Pred}(\text{Pred}(u, i, j), j, k) = \text{Pred}(u, i, k)$. If this holds then we may think of Pred as representing some forest F in the following strong sense: for any node $u \in L_i$ with $i > 1$, the nodes $\text{Pred}(u, i, 1), \dots, \text{Pred}(u, i, i)$ form the unique path from L_1 to u in F . Finally for each $u \in F$ with think of $\text{Col}(u, i)$ as coloring the node $u \in L_i$ red or blue.

We say that $u \in F \cup \{\perp\}$ is a termination point of F if the following holds:

1. $u = \{\perp\}$ and L_1 contains no blue node.

2. u is a node in F , the path from L_1 to u in F uses only blue nodes, and u does not have a blue child in F .

Now the search problem is: given Pred, Col , find a witness that Pred does not validly represent a forest, or else find a termination point in the forest it represents.

The totality of this problem follows by starting at a blue node in L_1 (if one exists) and finding a maximal blue path through its descendants in F . We start by giving a reduction from WEAK AVOID to FOREST TERMINATION. This is the aspect of the proof which is very similar to arguments by Li and by Paris Wilkie and Woods [114, 135].

Lemma 2.49. WEAK AVOID with $2n$ stretch is polynomial-time reducible to FOREST TERMINATION.

Proof. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ be given; we denote by $f_0, f_1 : \{0, 1\}^n \rightarrow \{0, 1\}^n$ the functions obtained by restricting to the first and last n bits of output respectively. Let $\mathcal{S}_n$ denote the set of all binary strings of length at most n , including the empty string ϵ . Naturally $\mathcal{S}_n$ is associated with the nodes of a full binary tree of depth n ; however since the FOREST TERMINATION instance we construct involves a forest whose structure and depth differ from that of $\mathcal{S}_n$, we will use the terminology of binary strings to refer to elements of $\mathcal{S}_n$ in order to avoid confusion. We say $s \sqsubseteq s'$ if s is a prefix of s' , and $s \sqsubset s'$ if it is a proper prefix. We use $s \cdot s'$ for concatenation. If $s \sqsubseteq s'$, with $s' = s \cdot p$, we use $s' - s$ to denote p . We define the “preorder” $<$ on $\mathcal{S}_n$ recursively with respect to prefixes: for a prefix p and two distinct extensions $p \cdot s, p \cdot s'$ with $s < s'$ (with the relative order of s, s' defined recursively), we put $p \cdot s < p \cdot s' < p$. In the view of $\mathcal{S}_n$ as a depth n perfect binary tree, this corresponds to the recursive subtree ordering “left subtree $<$ right subtree $<$ root.” Note that this is a total ordering with ϵ being the greatest element and 0^n the least. Finally, for an element $s \in \mathcal{S}_n$, we say that s' is a “left outlet” of s if either $s' = s$ or $s' = p0$ for some $p \sqsubseteq s$. We use $\text{LO}(s)$ to denote its set of left outlets; note that $|\text{LO}(s)| \leq n$. Observe that for any $s' \leq s$, there exists a unique $p \in \text{LO}(s)$ such that $p \sqsubseteq s'$.

For some $s \in \mathcal{S}_n$ and some value $x \in \{0, 1\}^n$, define $f_s(x)$ recursively as follows: if $s = \epsilon$ then $f_s(v) = v$. If $s = bs'$ for some bit b and substring s' , $f_s = f_{s'}(f_b(x))$.

We define a “transcript” as an element $\pi \in (\{0, 1\}^n)^n$. We say the transcript π is valid for some $s \in \mathcal{S}_n$ if $\pi_i = 0^n$ for all $i > |\text{LO}(s)|$. In this way the valid transcripts for s are in one-to-one correspondence with functions $\text{LO}(s) \rightarrow \{0, 1\}^n$. Thus for a valid transcript π for s and some $p \in \text{LO}(s)$, we use $\pi(p) \in \{0, 1\}^n$ to denote the value the transcript associates to p .

For a transcript π , and strings $s' < s \in \mathcal{S}_n$, we define the transcript $\pi' = \text{Pred}(\pi, s, s')$ as follows. We first check if π is valid for s ; if not we set π' to be some fixed canonical choice of invalid transcript for s' . Otherwise, for each $p \in \text{LO}(s')$, we set $\pi'(p) = f_{p-q}(\pi(q))$ where $q \in \text{LO}(s)$ is the unique left outlet of s which is an ancestor of p .

We now construct an instance of FOREST TERMINATION with $[N] = [2^{n+1} - 1] = \mathcal{S}_n$ and $M = [2^{n^2}] = (\{0, 1\}^n)^n$. $[N]$ is associated naturally to $\mathcal{S}_n$, so that each layer corresponds to an

element of $\mathcal{S}_n$ and the layers are ordered according to $<$, i.e. L_1 is associated to $0^n \in \mathcal{S}_n$ and L_N to $\epsilon \in \mathcal{S}_n$. $[M]$ is associated to the set of possible transcripts, so that L_s will be the set of all possible transcripts for the string s (some valid and others invalid). We use Pred defined above to give the Pred function in the instance of FOREST TERMINATION. It should be noted that the depth of the forest defined here is $N = 2^{n+1} - 1$, which is much larger than the depth of tree naturally associated to $\mathcal{S}_n$ (whose depth is n); this is why we have used the notation of binary strings rather than nodes in a tree to refer to elements of $\mathcal{S}_n$. It remains to define the node coloring function Col .

If $(\pi, s) \in [M] \times [N]$ are such that π is not a valid transcript for s , then $\text{Col}(\pi, s)$ is red; if π is a valid transcript for s and $|s| < n$ then it is colored blue ($|s|$ denotes its length as a binary string). Otherwise if π is a valid transcript for s and $|s| = n$, we check that $\pi(s) = \neg f_s(s)$, where $\neg$ denotes the string obtained by flipping all bits. If so we color it blue, otherwise we color it red. Note that since $|s| = n$, $f(s)$ is well defined, and thus so is $f_s(s)$.

It follows by construction that Pred validly defines a forest in the sense of Definition 2.48. It remains to show that given any termination point we may determine a solution to the original instance of Range Avoidance. We first claim that there exists a blue node in the first layer L_{0^n} , so that the solution to FOREST TERMINATION cannot be $\perp$. Observe that $\text{LO}(0^n) = \{0^n\}$, thus the transcript π which sets $\pi(0^n) = \neg f_{0^n}(0^n)$ will be blue. Now, say that $\pi \in L_s$ is a termination node. We claim that L_s cannot be the last layer, i.e. $s \neq \epsilon$. Say towards a contradiction that π is a termination node in layer L_ϵ . So π is a valid transcript for ϵ , which stores a single value $x = \pi(\epsilon)$. By the assumption π is a termination point all of its predecessors are blue. In particular if we view x as a length- n element of $\mathcal{S}_n$, then we have that $\text{Pred}(\pi, \epsilon, x)$ is blue, which by construction means that $f_x(x) = \neg f_x(x)$, which is a direct contradiction.

Now say that π is a termination node in layer L_s where $s \neq \epsilon$; let s' be the successor of s in the ordering $<$. First say that $|s'| = n$; in this case (π, s) cannot be a termination point, since we may construct a blue transcript π' for s' such that $\text{Pred}(\pi', s', s) = \pi$ by simply appending the value $\neg f_{u'}(u')$ as $\pi'(u')$ to the original transcript π for s . Otherwise we have $s = s'1$. Let $p = s'0$; so p is a left-outlet of s , and therefore π stores some values $x_0 = \pi(p), x_1 = \pi(s)$. We claim that

x_0x_1 must be a solution to the AVOID instance f . Say this is not true and there is some $x \in \{0, 1\}^n$ such that $f(x) = x_0x_1$; i.e. $f_0(x) = x_0$ and $f_1(x) = x_1$. Then we can generate a blue transcript π' for s' as follows. Observe that $\text{LO}(s') = \text{LO}(s) \setminus \{p\} \cup \{s'\}$. Thus if we remove p from the domain of π and set $\pi'(s') = x$ to obtain π' , then π' will be a blue transcript for s' and we will have $\text{Pred}(\pi', s', s) = \pi$, which means (π, s) cannot be a termination point. To see this, note that in the computation of $\text{Pred}(\pi', s', s)$, the only modification made to π' to generate π will be to compute $z_0 = f_0(\pi'(s'))$, $z_1 = f_1(\pi'(s'))$, and add z_0 as the value π associates to p and z_1 as the value it associates to s . By construction $z_0 = x_0$ and $z_1 = x_1$ so we end up with the original transcript π .

□

Next we reduce FOREST TERMINATION to LOP, which is simpler:

Lemma 2.50. FOREST TERMINATION is polynomial-time many-one reducible to LOP.

We will prove shortly that any search problem which is P^{NP} -reducible to LOP is also polynomial-time many-one reducible to LOP (Theorem 2.9); therefore it suffices to give a P^{NP} reduction.

Proof. Let Pred, Col an instance of FOREST TERMINATION. We start by using an NP oracle to search for a witness that Pred fails to validly represent a forest; if found we output this as our solution, otherwise we know that the instance indeed represents a forest F on the nodes $L_1 \sqcup \dots \sqcup L_N$. For conceptual simplicity, we modify F into a tree T by adding a virtual blue root node $\perp$ whose children consist of all nodes in L_1 . We use $<$ for the lexicographical ordering on each L_i . We now define a new ordering $\prec$ on the nodes of T . Its definition is given recursively with respect to subtrees of T as follows: say that u is a node with blue-rooted subtrees $B_1 < \dots < B_K$ and red-rooted subtrees $R_1 < \dots < R_{K'}$ ordered lexicographically by their respective roots. Define $\prec$ recursively inside each B_i, R_j ; now between them use the ordering $B_1 \prec \dots \prec B_K \prec \{u\} \prec R_1 \prec \dots \prec R_{K'}$. By induction on the depth of T we see that $\prec$ is a total ordering. The relation $\prec$ will be our instance of LOP output by the reduction; it remains to show that $\prec$ is efficiently computable and that its minimal element yields a FOREST TERMINATION solution.

We first show how to compute $\prec$. Given $u_0 \neq u_1 \in T$:

1. If u_0 is an ancestor of u_1 then let v be the child of u_0 from which u_0 can be reached. If v is blue then set $u_1 \prec u_0$, else if it is red set $u_0 \prec u_1$.
2. Otherwise let v be the least common ancestor of u_0 and u_1 , and let w_0, w_1 be the children of w from which u_0 and u_1 can be reached respectively. We define the relative ordering of w_0, w_1 in $\prec$, which is then inherited by the pair u_0, u_1 : if w_b is blue and w_{-b} is red for some $b \in \{0, 1\}$, then set $w_b \prec w_{-b}$. Otherwise use the lexicographical order $<$ to order the pair w_0, w_1 .

The only nontrivial step here is to compute a least common ancestor of u_1, u_2 , which can be accomplished with binary search. Say u_1, u_2 lie in layers $L_i, L_{i'}$ respectively, with $i' > i$. The using Pred we find the ancestor of u_2 in L_i ; call it $\hat{u}_2$. If $\hat{u}_2 = u_1$ then u_1 is the ancestor of u_2 . Otherwise let $j = \lceil i/2 \rceil$, and compute the ancestors v_1, v_2 of $u_1, \hat{u}_2$ in layer L_j . If they are distinct then we repeat the same procedure starting from v_1, v_2 . Otherwise if $v_1 = v_2$ we prune all the layers $L_1, \dots, L_{j-1}$ from consideration and repeat the procedure from $u_1, \hat{u}_2$, taking L_j to be the root layer with $v_1 = v_2$ as its root, and renumbering the layers accordingly. Overall the number of steps required is $O(\log N)$.

Finally it remains to show that any solution to LOP on the instance $\prec$ yields a solution to the given Tree Termination instance, i.e. if $u \in \{\perp\} \sqcup L_1 \sqcup \dots \sqcup L_N$ is minimal in the ordering $\prec$ then it must correspond to a termination point. Say that u is not a termination point, then either:

1. There is an edge (v, w) in the path from the root to u so that w is colored red. In this case we must have $v \prec u$.
2. u has a blue child v . In this case we have $v \prec u$.

In each case we see that u cannot be the minimal element. An example of a FOREST TERMINATION instance and its corresponding order $\prec$ is given in Figure 2.2. □

We now observe some special properties held by the search problem LOP, the first of which is its strong closure properties under a wide class of reductions:

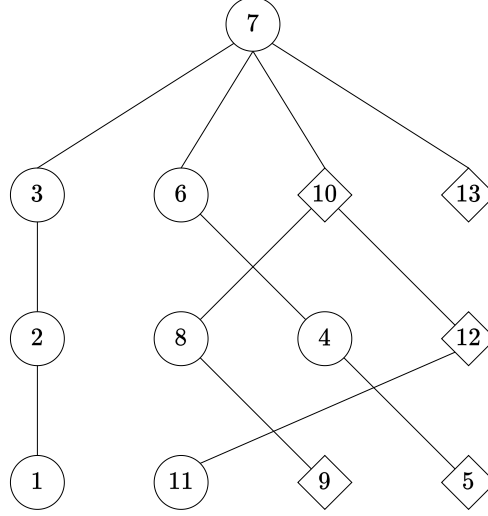


Figure 2.2: Example instance of FOREST TERMINATION with $M = 4$ and $N = 3$, with virtual root added at the top. Blue nodes are circles and red nodes are diamonds. We have sorted each layer left to right so that blue nodes come first, and within each color class the nodes are ordered left to right lexicographically. The number on each node represents its position in the ordering $\prec$. The nodes numbered 1 and 4 are the termination points of this instance.

Theorem (Theorem 2.9). *Let R be any search problem which has a polynomial-time, NP-oracle Turing reduction to LOP. Then R is polynomial-time many-one reducible to LOP.*

Proof. Let A be a polynomial time P^{NP} Turing reduction from some search problem R to LOP. Let x be an instance of the search problem of length n . We first claim that we can modify A so that:

1. $A(x)$ only calls the LOP oracle on instances $\prec$ which are nontrivial, i.e. they define a true total order.
2. $A(x)$ does not use its NP oracle.

To achieve the first condition, before each call to the LOP oracle on an instance $\prec$, we first use the NP oracle to check if $\prec$ defines a total order, and to find a violation if not. If a violation is found we no longer need to use the LOP oracle on this instance; otherwise we know that $\prec$ is a nontrivial instance and we use the oracle to solve it. Now to achieve the second condition, if ϕ is an instance of SAT with m variables, we define the ordering $\prec_\phi$ on $\{0, 1\}^m$ by: $y \prec_\phi z$ if

$\phi(y) = 1 \wedge \phi(z) = 0$, otherwise $\prec_\phi$ orders them lexicographically. Clearly $\prec_\phi$ is a nontrivial LOP instance, and the minimal element of $\prec_\phi$ will tell us if ϕ is satisfiable.

At this point we have a polynomial time reduction A which makes adaptive oracle calls to LOP on nontrivial instances, each of which has a unique solution. We next modify $A(x)$ to make exactly m calls on all computation paths, and have each call of the form $\prec: \{0, 1\}^m \times \{0, 1\}^m \rightarrow \{0, 1\}$ for some fixed value $m = \text{poly}(n)$. We can accomplish this easily by padding the algorithm with dummy queries and padding any instance of LOP to a larger bit-length. We now define a new instance $\prec_x: \{0, 1\}^{m^2} \times \{0, 1\}^{m^2} \rightarrow \{0, 1\}$ as follows. An element of $\{0, 1\}^{m^2}$ is given by a sequence $(u_1, \dots, u_m)$ with $u_i \in \{0, 1\}^m$. To compare two elements $\bar{u} = (u_1, \dots, u_m), \bar{v} = (v_1, \dots, v_m)$, we find the first index $i \in [m]$ so that $u_i \neq v_i$. We simulate $A(x)$ through the first $i - 1$ queries, plugging in $u_j = v_j$ as the answer to the j^{th} LOP oracle query. Once we get to the i^{th} query, we look at the instance $\prec_i$ of LOP defining the next query, and compare u_i, v_i using $\prec_i$; we then order $\bar{u}, \bar{v}$ accordingly, e.g. $\bar{u} \prec_x \bar{v}$ if $u_i \prec_i v_i$. It follows by construction that $\prec_x$ is a total order, and its least element corresponds to the unique sequence of correct oracle responses in the computation $A(x)$. Therefore by the correctness of A we may read off in polynomial time from this sequence the solution to the search problem R on input x . $\square$

At this point we can prove Theorem 2.3 that WEAK AVOID is polynomial time many-one reducible to LOP:

Proof of Theorem 2.3. By [101] there is a P^{NP} reduction from the general WEAK AVOID problem to the special case when the stretch is $2n$. Composing this with the reductions in Lemmas 2.49 and 2.50 we obtain a P^{NP} reduction from WEAK AVOID to LOP. Applying Theorem 2.9 this in turn yields a polynomial-time many-one reduction. $\square$

Recall that LOP enjoys the special property of having essentially unique solutions. Together with its closure under P^{NP} reductions, this endows LOP with the unusual property of being equivalent to a decision problem. We define now a decisional complexity class L_2^P , possessing a few equivalent definitions, whose complete problem is equivalent to the search problem LOP:

Definition (Reminder of Definition 2.10). A language L is in the complexity class L_2^P if there is a polynomial time relation $R : (\{0, 1\}^*)^3 \rightarrow \{0, 1\}$ and a polynomial p , so that for all x , $R(x, \cdot, \cdot)$ defines a total order on $\{0, 1\}^{p(n)}$ whose minimal element a has $a_1 = L(x)$.

Recalling Lemma 2.27, we have the following upper bound for this class:

Observation 2.51. $L_2^P \subseteq S_2^P$.

At first sight L_2^P seems to crucially involve a promise, namely that $\prec_x$ defines a total order for all x , just like the promise defining the class S_2^P . However it turns out that this promise can be eliminated and LOP can be given a purely syntactic characterization. This is summarized in Theorem 2.11 which we restate here:

Theorem 2.11 *The following are equivalent for a language L :*

1. $L \in L_2^P$
2. L is P^{NP} -Turing reducible to LOP.
3. L is polynomial time many-one reducible to LOP.

Conversely, the search problem LOP is polynomial time truth table reducible to a language in L_2^P .

Proof. If $L \in L_2^P$ then by definition it is many-one reducible to LOP which gives (1) $\rightarrow$ (2). Theorem 2.9 gives (2) $\rightarrow$ (3); inspecting the proof, for each x we may define $\prec_x$ so that the value $L(x)$ is the first bit of the LOP solution to $\prec_x$, which actually gives (2) $\rightarrow$ (1). $\square$

Finally we prove Theorem 2.12 from the introduction, which tells us that L_2^P satisfies two of the three most interesting properties of the larger class S_2^P :

Theorem 2.12

1. $P^{NP} \subseteq L_2^P$ and $BPP \subseteq MA \subseteq L_2^P$
2. L_2^{E6} contains a language of circuit complexity $2^n/n$.

⁶ L_2^E is the exponential-time analogue of L_2^P , where we replace “polynomial time” with $2^{O(n)}$ time” in its definition

Proof. The inclusion $P^{NP} \subseteq L_2^P$ follows directly from the closure of L_2^P under P^{NP} reductions. For the second inclusion, we know that every language in MA is P^{NP} -reducible to the construction of a $O(\log n)$ -seed length PRG for $O(n)$ -size circuits [129]. The explicit construction of such PRGs is in turn reducible to WEAK AVOID [101], and WEAK AVOID is reducible to LOP by Theorem 2.3. Applying Theorem 2.9 the result follows.

For the second part, the proof is identical to the case of S_2^E shown in [114] given our reduction from WEAK AVOID to LOP. Given $x \in \{0, 1\}^n$ we may produce in $2^{O(n)}$ time an instance of WEAK AVOID $C_n : \{0, 1\}^\ell \rightarrow \{0, 1\}^{2^n}$ so that any solution is a truth table of a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ of circuit complexity $2^n/n$. By Theorem 2.3 we may then construct in $2^{O(n)}$ time a nontrivial instance of LOP $\prec$ so that from the unique minimal element of $\prec$ we may read off a uniquely defined WEAK AVOID solution f_n for C_n . We then accept/reject x based on $f_n(x)$. Using the closure properties of L_2^E the language $\{f_n\}_{n \in \mathbb{N}}$ thus defined lies in L_2^E . $\square$

The only interesting property of S_2^P which we cannot prove is inherited by its subclass L_2^P is the Karp-Lipton theorem:

Theorem ([23], credited to Sengupta). *If $NP \in P/\text{poly}$ then $PH = S_2^P$.*

The proof of this result does not seem to generalize to L_2^P .

2.5 Proof of the Pigeonhole Switching Lemma

As in the standard proof of the switching Lemma, we need to define a procedure for representing a matching disjunction by a *canonical* complete pigeonhole decision tree. We start with some preliminary definitions.

Definition 2.52 (Complete Pigeonhole Trees on a subset A). *For a partial matching π and a hole restriction τ , we say that τ covers π if $\text{nodes}(\tau) \supseteq \text{nodes}(\pi)$. If $\tau = \langle \sigma, E \rangle$, we say that τ is a minimal cover of π if it fails to be a cover after removing any edge from σ or any element from E .*

For every tuple (A, N, M) where $A \subseteq [N] \sqcup [M]$, we define a canonical complete pigeonhole decision tree, denoted by $C_A(N, M)$, as follows. This tree is unlabelled, and is defined

with respect to some fixed ordering of the elements of A . So let's assume the ordering $a_1, \dots, a_r$ of A . We build $C_A(N, M)$ in r steps:

1. In step 1, if $a_1 \in [N]$, we query pigeon a_1 at the root, with all possible outedges: (a_i, y) , $y \in [M]$. Otherwise if $a_1 \in [M]$, we query hole a_1 with all possible outedges: (x, a_i) , $x \in [N] \sqcup \{\perp\}$.
2. In step $i > 1$, for each leaf node l in T , let $\tau_l = \langle \sigma_l, E_l \rangle$ denote the hole restriction associated with the path to l . If a_i is a pigeon or hole in $\text{nodes}(\tau_l)$ (that is, if a_i has already been "determined" by the hole restriction thus far), then we do nothing. Otherwise, we query a_i at the leaf l , with all possible outedges that are consistent with τ_l .

Observe that the leaves of $C_A(N, M)$ are in one-to-one correspondence with the set of all hole restrictions that minimally cover A , and thus $C_A(N, M)$ is a complete pigeonhole decision tree.

Definition 2.53 (Canonical Pigeonhole Trees). Let $\phi = \pi_1 \vee \dots \vee \pi_s$ be a matching disjunction over $[N] \sqcup [M]$ where we have fixed this ordering on the terms. The canonical pigeonhole tree for ϕ , $\mathcal{T}_\phi(N, M)$, is defined recursively below. When N, M is clear from context, we will abbreviate $\mathcal{T}_\phi(N, M)$ by $\mathcal{T}_\phi$.

1. If $s = 0$, then ϕ is a constant function. If $\phi = 1$ then $\mathcal{T}_\phi(N, M)$ consists of a single node labelled by 1, and similarly if ϕ is 0, then the decision tree is a single node labelled by 0.
2. Otherwise, let π_1 be the first term in ϕ according to the ordering of terms in ϕ . First, we create the complete pigeonhole tree $C_{\text{nodes}(\pi_1)}(N, M)$. For each leaf l in the tree constructed so far, let $\tau_l = \langle \sigma_l, E_l \rangle$ be the hole restriction associated with the path to l . Then for each leaf l :
 - (a) Let $P \subseteq [N]$ be the subset of pigeons that have not been matched by π_l , and let $H \subseteq [M]$ be the subset of holes not matched by π_l and not in E_l . P and H define the pigeons and holes that are still undetermined after applying τ_l . If the size of P is N' and the

size of H is M' , then we can rename the vertices so that we identify P with $[N']$ and H with $[M']$.

(b) Recursively construct $\mathcal{T}_{\phi \upharpoonright \tau_l}(N', M')$. and attach a copy of this subtree to the leaf l .

It follows from the definition that $\mathcal{T}_\phi \equiv \phi$. Next, we restate the Pigeonhole Switching Lemma.

Lemma 2.40 *Let $M, N, d \in \mathbb{N}$. Let ϕ be a width- w matching disjunction over $[M]^{[N]}$. If $M - N \leq K \leq \frac{N}{4}$ and N, K, d, w sufficiently large, then:*

$$\Pr_{\rho \sim \mathcal{M}_K} [\mathcal{T}_{\phi \upharpoonright \rho} \text{ has depth } \geq d] \leq \exp\left(d(\log wK^5 - \log N^{1/2} + O(1))\right)$$

Proof. Let $\phi = \lambda_1 \vee \dots \vee \lambda_s$ be a matching disjunction of width w . Let $\mathcal{B}$ be the set of partial matchings ρ with $N - K$ edges, such that $\mathcal{T}_{\phi \upharpoonright \rho}$ has depth $\geq d$. If $\rho \in \mathcal{B}$, there exists a total assignment $f \supseteq \rho$ yielding a path P of length d from the root in $\mathcal{T}_{\phi \upharpoonright \rho}$. We define a sequence of hole restrictions $\tau_1, \dots, \tau_\ell$ and a sequence of partial matchings $\sigma_1, \dots, \sigma_\ell$, and set $S \subseteq [\ell] \times [w]$ as follows. Define $\tau_0 = \emptyset$. Once τ_i has been defined, let a_{i+1} be the index of the first term in ϕ so that $\lambda_{a_{i+1}} \upharpoonright \rho \cup \tau_1 \cup \dots \cup \tau_i$ is unkilld. Then let τ_{i+1} be the minimal map $\tau_{i+1} \subseteq f$ which covers $\lambda_{a_{i+1}} \upharpoonright \rho \cup \tau_1 \cup \dots \cup \tau_i$, and let $\sigma_i = \lambda_{a_{i+1}} \upharpoonright \rho \cup \tau_1 \cup \dots \cup \tau_i$. Finally let $S \subseteq [\ell] \times [w]$ contain each index $(i, j) \in [\ell] \times [w]$ such that the j^{th} edge of λ_i occurs in σ_i . We claim that such a sequence can be constructed so that at the end $\ell \leq d$, and if we take $\sigma = \cup_i \sigma_i$, then σ is a partial matching with $\text{nodes}(\sigma) \cap \text{nodes}(\rho) = \emptyset$ and $|\sigma| = t$ for some $\frac{d}{2} \leq t \leq dw$. This follows from the definition of $\mathcal{T}_{\phi \upharpoonright \rho}$ and the fact that f achieves a path of length $\geq d$ in this decision tree.

Thus, $\rho \in \mathcal{M}_K$, while $\rho \cup \sigma \in \mathcal{M}_{K-t}$ for some $\frac{d}{2} \leq t \leq dw$. We will define a map

$$\text{Dec} : \bigcup_{t \geq \frac{d}{2}} \mathcal{M}_{K-t} \times \Gamma_t \times \Delta_t \rightarrow \mathcal{M}_K$$

for some finite sets $\{\Delta_t\}_{t \leq N}$ so that whenever $\rho \in \mathcal{B}$ and σ, S are constructed from ρ as above,

there exists some “advice” $\delta \in \Delta_t$ so that $\text{Dec}(\rho \cup \sigma, S, \delta) = \rho$. This will imply that

$$\Pr_{\rho \sim \mathcal{M}_K} [\rho \in \mathcal{B}] \leq \frac{1}{|\mathcal{M}_K|} \sum_{t \geq \frac{d}{2}} |\mathcal{M}_{K-t}| |\Gamma_t| |\Delta_t|$$

which will yield the theorem provided we can choose the sets Γ_t, Δ_t sufficiently small.

Let $\rho \in \mathcal{B}$, τ_i, a_i, σ_i, S be as above. Define $\gamma_i := \rho \cup \tau_1 \cup \dots \cup \tau_i \cup \sigma_{i+1} \cup \dots \cup \sigma_\ell$, where $\gamma_0 = \rho \cup \sigma$. Let $\sigma = \cup_i \sigma_i$ and $t = |\sigma_i|$. First, we claim that if we are given $\tau_1, \dots, \tau_{i-1}, \sigma_1, \dots, \sigma_{i-1}$, and γ_{i-1} then we may decode from these the index a_i . To do this, we simply search for the first satisfied term in $\phi \upharpoonright \gamma_{i-1}$. Next, observe that if we have knowledge of a_i , then using S we may reconstruct σ_i . Once we have $\gamma_{i-1}, \sigma_i, \tau_i$, we may construct γ_i directly, by removing σ_i in γ_{i-1} and replacing it with τ_i . So it remains to show how to specify τ using the extra information Δ_t , so that we may recover τ_i from σ_i . Observe that $\text{nodes}(\tau)$ is contained in $\text{nodes}(\sigma) \cup ([N] \sqcup [M] \setminus \text{nodes}(\rho \cup \sigma))$. Therefore if we construct a map $\delta : [K] \rightarrow [K + L]$, where $M - N = L$ and $|\delta| \leq 2d$ which specifies, for each $u \in \text{nodes}(\sigma)$ the preimage/image of u in τ (or the lack of preimage if u is marked empty), then from σ_i we may reconstruct τ_i by searching for the images/preimages of the nodes in $[K] \sqcup [K + L]$ corresponding to $\text{nodes}(\sigma_i)$. Thus for every t we can take $\Delta_t = \Delta$ to be the set of all hole restrictions $[K] \rightarrow [K + L]$ of size at most d , so that $|\Delta| \leq \binom{K}{2d} \binom{K+L}{2d} (2d)!$. Finally it remains to show that the sets S lie in Γ_t for some suitably small set Γ_t . Observe that the set $S \subseteq [\ell] \times [w]$ constructed in this argument has the following form: $|S \cap \{i\} \times [w]| \geq 1$ for all $i \in \ell$, and $|S| \leq t$. There are at most $(2w)^t$ sets of this form (ranging over all possible values of ℓ), thus we can take $|\Gamma_t| = (2w)^t$.

It remains to estimate the value

$$\begin{aligned} \Pr_{\rho \sim \mathcal{M}_K} [\rho \in \mathcal{B}] &\leq \frac{1}{|\mathcal{M}_K|} \sum_{t \geq \frac{d}{2}} |\mathcal{M}_{K-t}| |\Gamma_t| |\Delta_t| \\ &= \sum_{t \geq \frac{d}{2}} \frac{|\mathcal{M}_{K-d}|}{|\mathcal{M}_K|} |\Gamma_t| |\Delta_t| \end{aligned}$$

Observe that for each $\kappa \in \mathcal{M}_{K-t}$ there are at least $\binom{N-K+t}{t}$ elements $\alpha \in \mathcal{M}_K$ such that $\alpha \subseteq \kappa$,

and on the other hand for any $\alpha \in \mathcal{M}_K$ there are at most $\binom{K}{t} \binom{K+L}{t} t!$ elements $\kappa \in \mathcal{M}_{K-t}$ so that $\alpha \subseteq \kappa$. Thus

$$\frac{|\mathcal{M}_{K-t}|}{|\mathcal{M}_K|} \leq \frac{\binom{K}{t} \binom{K+L}{t} t!}{\binom{N-K+t}{t}}$$

So overall we have:

$$\begin{aligned} \Pr_{\rho \sim \mathcal{M}_K} [\rho \in \mathcal{B}] &\leq \sum_{t \geq \frac{d}{2}} \frac{\binom{K}{t} \binom{K+L}{t} t!}{\binom{N-K+t}{t}} |\Gamma_t| |\Delta_t| \\ &\leq |\Delta| \sum_{t \geq \frac{d}{2}} \frac{\binom{K}{t} \binom{2K}{t} t!}{\binom{N-K+t}{t}} (2w)^t \\ &\leq \binom{K}{2d} \binom{K+L}{2d} (2d)! \sum_{t \geq \frac{d}{2}} \frac{\binom{K}{t} \binom{2K}{t} t!}{\binom{N-K+t}{t}} (2w)^t \\ &\leq \left(\frac{2K}{\sqrt{2d}} \right)^{4d} \sum_{t \geq \frac{d}{2}} \left(\frac{8wK^2}{N} \right)^t \end{aligned}$$

Where we apply the assumptions $K \geq L$ and $K \leq \frac{N}{4}$. We may safely assume $\frac{wK^2}{N} = o(1)$, otherwise the conclusion of the theorem holds trivially. Thus by geometric decay of terms in the above sum we have, for some absolute $C \in \mathbb{N}$, the bounds:

$$\begin{aligned} \Pr_{\rho \sim \mathcal{M}_K} [\rho \in \mathcal{B}] &\leq C \left(\frac{2K}{\sqrt{2d}} \right)^{4d} \left(\frac{8wK^2}{N} \right)^{\frac{d}{2}} \\ &\leq (2K)^{4d} \left(\frac{8wK^2}{N} \right)^{\frac{d}{2}} \end{aligned}$$

provided K, d are sufficiently large with respect to C . So overall

$$\begin{aligned} \Pr_{\rho \sim \mathcal{M}_K} [\rho \in \mathcal{B}] &\leq \exp(4d \log(2K) + \frac{d}{2} \log(8wK^2) - \frac{d}{2} \log N) \\ &\leq \exp(d(\log(16K^4) + \log(8wK) + \log N^{1/2})) = \exp\left(d(\log wK^5 - \log N^{1/2} + O(1))\right) \end{aligned}$$

□

Chapter 3: Derandomization from Time-Space Tradeoffs

3.1 Introduction

In this chapter as well as the next, we will give a deterministic polynomial time algorithm for solving a certain class of explicit construction problems. In both chapters, the correctness of our algorithm will hinge on a certain complexity-theoretic *hardness assumption*, placing these results within the rich landscape of *hardness v.s. randomness* tradeoffs which have become ubiquitous in complexity theory since their introduction in [173] and subsequent generalized treatment in [129, 84]. However, the method here (as well as in the next chapter) will exploit particular properties of the class of explicit construction problems we consider in order to achieve a derandomization under assumptions which are categorically different than those required to instantiate the general hardness v.s. randomness paradigm. The results in this chapter are based on work originally appearing in [103].

We motivate the main algorithm in this chapter by a certain *counter-factual* thought experiment. When an explicit construction problem is defined, we have a nonconstructive proof that it is total; our starting point is to imagine a world in which some explicit construction problem is *not solvable in polynomial time*, i.e. the nonconstructivity of said proof is *inherent*. When a particular theorem appears to be inherently non-constructive, in that no polynomial time algorithm can witness the solutions it guarantees, one perspective we can take is that this theorem is “effectively false” in a certain context, despite being unconditionally true in general. This is essentially the outlook presented by Yao in his seminal paper introducing the basis of theoretical cryptography [173]. Here, Yao focuses on the central tenets of Shannon’s information theory. He argues that although Shannon’s theorems are of course provably correct, in many scenarios it appears computationally infeasible to witness their truth. For example, an output of some function $f: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$

under the uniform distribution on $\{0, 1\}^n$ has entropy n , and thus by a result of Shannon can be encoded on average using n bits. However for an observer who sees only the outputs of this distribution, there seems to be no efficient way to generate such a code without the ability to efficiently invert f . If we posit that there are some specific functions f for which it is in fact impossible to efficiently realize Shannon’s theorem in this sense, one might venture to say that Shannon’s theorem *effectively fails* for f in the computational realm, perhaps allowing us to carry out tasks which would, in the absence of computational constraints, be deemed impossible. Indeed it is widely conjectured that there are efficiently computable functions f of this form, and this conjecture underpins the security of many cryptographic protocols. Similar situations are abundant in the field of cryptography, where the computational infeasibility of *witnessing* impossibility theorems from information theory allows us to effectively bypass them.

With this perspective in mind, we turn our attention to the following kind of explicit construction problem. Suppose that there is a pair of efficiently computable functions $\mathcal{G} = \{g_n: \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}\}_{n \in \mathbb{N}}$, $\mathcal{F} = \{f_n: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n\}_{n \in \mathbb{N}}$, and our explicit construction problem is: given 1^n , find some n -bit string x such that $f_n(g_n(x)) \neq x$ in $\text{poly}(n)$. Note that this does not quite capture explicit construction problems reducible to AVOID, since rather than a mere length-extending map g_n we are also given its supposed “inverse” f_n . Now, if this explicit construction problem is not efficiently solvable, then we have found a pair of efficiently computable functions $\mathcal{F}, \mathcal{G}$, such that no polynomial time algorithm is able to construct an n -bit string x such that $f_n(g_n(x)) \neq x$ in $\text{poly}(n)$ time. In this hypothetical world where $\mathcal{F}, \mathcal{G}$ are an “effective counterexample” to the weak pigeonhole principle, we have access to an efficient compression scheme which allows us to encode any n -bit string using $n - 1$ bits. What improbable feats can be accomplished in such a world?

In this work we give one answer to this question: if the weak pigeonhole principle *effectively fails* in the above sense, we can utilize this failure to construct an efficient data structure which allows us to simulate RAM computations in low space and near-linear time on a 1-tape Turing machine. Our simulation proceeds by bootstrapping our compression scheme that saves 1 bit, into

a *dynamic* compression scheme that can store an m -bit memory in m^ϵ bits, such that each cell of memory can be read or updated in m^ϵ time. Stated in contrapositive, if we assume mild time-space tradeoffs for simulating RAM machines on a 1-tape Turing machine, we obtain a generic deterministic algorithm to witness the weak pigeonhole principle for any pair of maps $\mathcal{F}, \mathcal{G}$.

Aside from the aforementioned conditional derandomization, another lasting contribution of the work in [103] covered in this chapter is the introduction of the LOSSY CODE problem in TFNP TODO: jerabek studied it too so maybe this is not accurate, which has come to recent attention in connection to the study of *catalytic space* TODO: explain more.

TODO: also mention the R-LOSSY thing...

3.1.1 Our Contributions

3.1.1.1 Derandomization from Time-Space Tradeoffs

Let $\mathcal{C}, \mathcal{D} = \{C_n : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}\}_{n \in \mathbb{N}}, \{D_n : \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n\}_{n \in \mathbb{N}}$. We call such a pair a “uniform compression scheme,” where $\mathcal{C}$ is the “compressor” which encodes n -bit “messages” by $n - 1$ -bit “codewords,” and $\mathcal{D}$ is the “decompressor” which maps $n - 1$ -bit codewords to the messages they represent¹. We use the term “uniform” since in the relevant cases, $\mathcal{C}$ and $\mathcal{D}$ will each be described by a single Turing machine which computes C_n (resp. D_n) for all n . We will say that a string $x \in \{0, 1\}^n$ is “incompressible” for such a scheme if $D_n(C_n(x)) \neq x$, i.e. if $\mathcal{D}$ cannot be used to recover the message x from the codeword assigned to it by $\mathcal{C}$. In this chapter, we study the complexity of the explicit construction problem of generating incompressible strings for uniform compression schemes. This task can be viewed naturally as a derandomization problem, since a randomly chosen n -bit string will be incompressible with high probability with respect to any fixed compression scheme.

Our results then show how to derive efficient derandomized algorithms for generating incompressible strings, assuming certain *uniform* lower bounds. The lower bounds we consider are time-space tradeoffs for simulating RAM by 1-tape machines. Roughly, they posit that there are

¹The formal definition given in Section 3.3 is slightly more general, but we will focus on this special case for now.

problems solvable in time T on a RAM machine which cannot be solved on a 1-tape machine using $T^{0.01}$ space and $T^{1.01}$ time. Depending on certain features of the compression scheme and the resources available to the explicit construction algorithm, the specifics of the tradeoff assumption will vary, as we shall explain in Figure 3.1.

Initially, we consider the case of “poly-time compression schemes”, where both C and $\mathcal{D}$ are computable in polynomial time. We show that for such schemes, incompressible strings can be constructed deterministically in polynomial time assuming time-space tradeoffs for deterministic exponential time:

Theorem (Theorem 3.39). *Suppose there is some exponential time bound $T(n) \geq 2^{\Omega(n)}$ and some $\epsilon > 0$ such that it is impossible to simulate $T(n)$ -time RAM computations on 1-tape Turing machines that use $T(n)^{1+\epsilon}$ time and $T(n)^\epsilon$ space. Then for any poly-time compression scheme, there is a polynomial time algorithm that produces incompressible n -bit strings for this scheme on input 1^n (for infinitely many n).*

Constructing incompressible strings for poly-time compression schemes is a natural and quite broad derandomization problem. Theorem 3.39 gives a novel connection between derandomizing this task and proving *uniform* time-space tradeoffs— in contrast, the assumptions previously required to derandomize this sort of problem require a lower bound against *non-uniform* algorithms, such as circuits. To put this derandomization task in a more familiar context, in Section 3.3.2 we show that several well-studied explicit construction problems, such as the construction of large prime numbers or the construction of truth tables of high circuit complexity/formula size, can be reduced to the problem of finding incompressible strings for some uniform compression scheme. In each case, the compression scheme will either be computable efficiently, or efficiently with access to an oracle for some search problem in FNP which is not known to be NP-complete.

To state our other main results in their most interesting form, we focus our attention now on the construction of strings/truth tables of high complexity with respect to some non-uniform complexity measure, such as formula size, circuit size, or time-bounded Kolmogorov complexity. In each case there is an associated NP search problem, where we are given a string/truth table and

must find a small formula/circuit/program computing it (if one exists); we call this the “compression problem.” The following table shows how our three main theorems relate various time-space tradeoff hypotheses to such problems:

Hypothesis: For some exponential time bound T and some $\epsilon > 0...$	Implication :
$\mathbf{RAM-TIME}[T] \not\subseteq \mathbf{1-TISP}[T^{1+\epsilon}, T^\epsilon]$	If the compression problem has a polynomial time algorithm, then incompressible strings can be constructed in polynomial time.
$\mathbf{NTIME}[T] \not\subseteq \mathbf{1-NTISPG}[T^{1+\epsilon}, T^\epsilon, T^\epsilon]$	Incompressible strings can be constructed in polynomial time with an $\mathbf{NP}$ -oracle. In particular, $\mathbf{E}^{\mathbf{NP}} \not\subseteq \mathbf{size}[2^n/2n]$.
$\mathbf{RAM-TIME}[T] \not\subseteq \mathbf{1-NTISPG}[T^{1+\epsilon}, T^\epsilon, T^\epsilon]$	Incompressible strings can be constructed in polynomial time with an oracle for the compression problem.

Figure 3.1: Main Results. We use the shorthand $T = T(n)$.

The classes seen on the left hand side will be defined formally in Section 3.2.2, but we give a brief explanation here so that the table can be interpreted appropriately. $\mathbf{TIME}[T(n)]$ and $\mathbf{NTIME}[T(n)]$ are defined in the standard way using multitape machines; the prefixes 1 and RAM indicate, respectively, either a restriction of the model to 1-tape machines or a strengthening to random access machines. The class $\mathbf{TISP}[T(n), S(n)]$ consists of problems decidable simultaneously in time $T(n)$ and space $S(n)$. Finally, $\mathbf{NTISPG}[T(n), S(n), G(n)]$ consists of problems decidable by a nondeterministic machine running in time $T(n)$ which, on every computation path, uses at most $S(n)$ space and $G(n)$ nondeterministic guesses.

For a concrete example of how to apply these results, let's focus on the compression problem **CIRCUIT SYNTHESIS** (given a truth table find a small circuit for it if one exists), and the explicit construction problem of producing truth tables of exponential circuit complexity. The first hypothesis in the above table implies that if **CIRCUIT SYNTHESIS** is in $\mathbf{P}$ then there is a polynomial time construction of hard truth tables (i.e. $\mathbf{E}$ has exponential circuit complexity). The second hypothesis

implies unconditionally that there is an efficient NP oracle construction of hard truth tables (i.e. E^{NP} has exponential circuit complexity). Finally, the third and strongest hypothesis tells us that there is a polynomial time construction of hard truth tables using an oracle for CIRCUIT SYNTHESIS.

The case of large prime construction does not fit in as neatly to the above picture, as the scheme we devise for this problem will require a factoring oracle for both the compressor and decompressor (while the above problems have a polynomial-time computable decompressor). In this case we get the following result:

Theorem (Corollary 3.43). *Under the hypothesis in the top row of the above table, a polynomial time algorithm for factoring implies a polynomial time algorithm to construct $32n$ -bit primes of magnitude $> 2^n$ for infinitely many n .*

If we forgo the questionable assumption that factoring lies in P , we get:

Theorem (Corollary 3.44). *One of the following is true:*

1. *For every exponential time bound T and every $\epsilon > 0$, every language decidable in time $T(n)$ on a RAM machine can be decided in time $T(n)^{1+\epsilon}$ and space $T(n)^\epsilon$ by a 1-tape machine with a factoring oracle, which makes oracle calls of length at most $T(n)^\epsilon$.*
2. *There is a polynomial time algorithm with a factoring oracle that generates $32n$ -bit primes of magnitude $> 2^n$ for infinitely many n .*

The problem of deterministically generating large primes has been investigated previously in several works, and was notably the subject of the Polymath 4 project [138]. In the public discussion forums for this project, it was explicitly asked whether a polynomial time algorithm for factoring, or more generally an oracle for factoring, would help. More recently, Oliveira and Santhanam gave a subexponential time “pseudodeterministic” construction of large primes [132], using only the fact that primality is testable in P [3] and that primes occur with non-negligible frequency.

3.1.1.2 BPP and the Weak Pigeonhole Principle

In Section 3.6 we briefly consider the relationship between various search problems associated with the weak pigeonhole principle, and the “full derandomization task” characterized by the class prBPP . Observe that the problem introduced above of finding incompressible strings for uniform compression schemes can be generalized to a TFNP search problem – instead of considering compression schemes generated by uniform Turing machines, we can consider the search problem where a compression scheme of some fixed length is given as input in the form of a pair of boolean circuits:

Definition 3.1. In LOSSY CODE , we are given as input a pair of circuits $C: \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$, $D: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, and must output some $x \in \{0, 1\}^n$ such that $D(C(x)) \neq x$.

While finding a deterministic algorithm for LOSSY CODE appears to be a quite generic derandomization problem, it would be a major breakthrough to show that LOSSY CODE captures the “full derandomization problem.” since LOSSY CODE lies in TFNP , if prBPP reduces to LOSSY CODE then $\text{BPP} \subseteq \text{NP}$, which is a notorious open problem. In Section 3.6, we show that a natural generalization of LOSSY CODE , where we allow the compressor C to be randomized, is indeed strong enough to characterize prBPP precisely. The formal problem is as follows:

Definition 3.2. In R-LOSSY CODE , we are given as input circuits $C: \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^{n-1}$, $D: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, and must find some x such that $\Pr_r[D(C(x, r)) = x] < \frac{1}{2}$.

In this work we demonstrate:

Theorem (Theorem 3.48). R-LOSSY CODE is complete for prBPP under deterministic Turing reductions.

The relation between pigeonhole principle search problems and more standard derandomization problems is summarized in Figure 3.2.

The proof of Theorem 3.48 is mostly standard, following quite directly from Yao’s next bit predictor lemma. From the proof of this theorem we are able to extract the following interesting corollary:

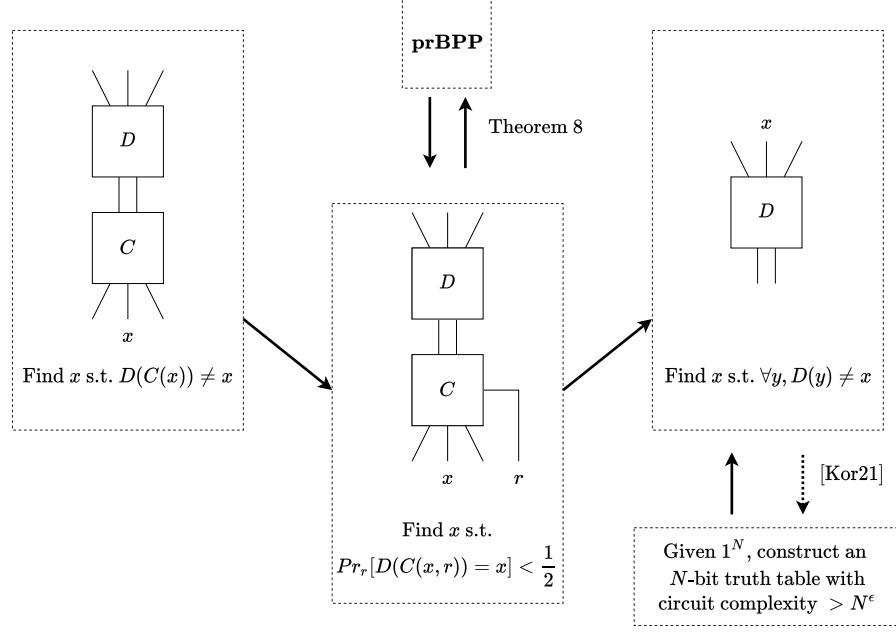


Figure 3.2: Relations between the total search problems associated with various weak pigeonhole principles, and standard derandomization problems. Solid arrows represent deterministic polynomial time reductions, while dotted arrows represent NP-oracle reductions.

Corollary (Corollary 3.49, Informal). *If the fixing of the leftover bits in Yao’s hybrid argument can be derandomized, then $\text{BPP} \subseteq \text{NP}$. Indeed, under this assumption, every problem in prBPP reduces to the search problem LOSSY CODE in $\text{PPP} \subseteq \text{TFNP}$.*

In other words, derandomizing a particular step in Yao’s classical argument implies a quite universal derandomization of prBPP .

3.1.2 Relation to Prior Work

Hardness vs. Randomness As mentioned above, the task of constructing incompressible strings can be naturally viewed as a derandomization problem, and is thus amenable to the standard hardness/randomness paradigm. In particular, when the compression scheme is polynomial time computable as in Theorem 3.39, the standard hardness assumptions used to derandomize BPP (e.g. [84]) would suffice to yield a polynomial time construction of incompressible strings. Such hardness assumptions require a lower bound for a language in E against some non-uniform model of computation, such as boolean circuits. In contrast, Theorem 3.39 gives a hardness/randomness

connection for this problem which only requires a *uniform lower bound* for a language in E , in particular a lower bound against low-space algorithms running in slightly more time using a weaker memory model.

The Easy Witness Lemma: Perhaps the closest prior work to our results is the “Easy Witness Method,” initiated by [92] and furthered in [82] and [169], which roughly says that assuming E^{NP} has small circuits, any nondeterministic exponential time computation must have witnesses of low circuit complexity. This immediately implies that unless E^{NP} requires large circuits, we can efficiently simulate $NTIME[2^n]$ using limited nondeterminism by “guessing a small circuit.” In Appendix 3.8, we show that this old method (along with one other well-known tool) is in fact enough to prove the second implication in Figure 3.1, although this connection to time-space tradeoffs does not seem to have been noted previously. However, this proof heavily utilizes the distinctive power of nondeterminism, whereby additional “guesses” allow us to vastly simplify the verification procedure, and does not seem to extend to our other two main results.

3.1.3 Known Time-Space tradeoffs:

Finally, we cover the known results on time-space tradeoffs and separations between the RAM and 1-tape models. We first emphasize the following: all three time-space tradeoff hypotheses stated on the left-hand side of Figure 3.1 are known to hold *unconditionally* when the time bound T is $O(n)$. In particular, an old result of Maass [117] shows that the set of palindromes is recognizable in quasilinear time on a deterministic RAM machine, but requires $\Omega(n^2)$ time on a nondeterministic 1-tape machine. However, Maass’s proof is essentially a counting argument which crucially relies on the entropy of the input being comparable in magnitude to the total computation time, which is no longer the case for exponential time computations. When it comes to separating the RAM and 1-tape models for generic time bounds, a result of [147] shows that there must be *some* slowdown when simulating a RAM on a 1-tape machine for *any time bound* greater than $n \log \log n$, but the slow-down is an astronomically small multiplicative factor, on the order of $\log \log T(n)$.

Another sequence of works ([50], [51], [167] among others) has shown unconditionally that nondeterministic linear time cannot be solved in $\text{TISP}[n^{1+\epsilon}, n^\epsilon]$ for certain fixed values of $\epsilon > 0$, even when the simulating machine is given access to RAM. These results can be scaled to larger time bounds as follows: for any time-constructible T , $\Sigma_2\text{TIME}[T(n)]$ is not contained in $\text{TISP}[T(n)^{1+\epsilon}, T(n)^\epsilon]$ for certain fixed values of $\epsilon > 0$. Such results are obtained by combining hierarchy theorems with a fast simulation result, whereby a Σ_2 machine can simulate a $\text{TISP}[T(n)^{1+\epsilon}, T(n)^\epsilon]$ computation in time $T(n)^{1-\delta}$ for some fixed $\epsilon, \delta > 0$. It is evident that such results will not be sufficient for our purposes, as they only rule out efficient low-space simulations of Σ_2 computations by deterministic machines.

3.1.4 Main Tool: the J-tree

TODO: update the terminology here to match other parts of paper? The basis of our main proofs is a construction which we call the “J-tree.” Roughly speaking, the J-tree is a data structure which allows us to store an array of T elements, subject to fast update/query operations, using significantly less than T bits. While this task is information-theoretically impossible (by the weak pigeonhole principle), the J-tree uses a compression scheme C, D to perform its operations, and whenever it fails in its role as a data structure, it is able to print out an incompressible string for this scheme. Thus, assuming no polynomial time algorithm can witness the weak pigeonhole principle for C, D , no polynomial time algorithm can witness the failure of this data structure. This will allow our low-space simulations to operate a RAM memory using low space on a 1-tape machine, in such a way that if the simulation fails, then a polynomial time algorithm can witness this failure, and thus witness the weak pigeonhole principle for C, D .

The core construction underlying the J-tree has an intriguing history dating back to the late 1970’s. In the course of a decade, this construction was used, seemingly independently, to prove three seminal results in cryptography and logic:

1. In 1979², Merkle showed how to iterate a cryptographically secure hash function $h: \{0, 1\}^{2^n} \rightarrow$

²Although the cited work is dated to 1988, Merkle filed a patent for the concept about a decade earlier.

$\{0, 1\}^n$ in a tree-like fashion to construct a secure digital signature scheme whose signature size increases logarithmically in the number of messages signed [121]³.

2. In 1986, Goldwasser, Goldreich and Micali showed how to iterate a pseudorandom generator $G: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ in a similar tree-like manner into a family of functions $\{G_x: \{0, 1\}^n \rightarrow \{0, 1\}^n\}_{x \in \{0, 1\}^n}$ whose input/output behaviours are indistinguishable from random, assuming G is a secure PRG [60].
3. In 1988, Paris, Wilkie, and Woods gave a novel proof of the weak pigeonhole principle in bounded arithmetic [135]. Again, the same process of iterating a function $f: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ is applied. Here, the function f is assumed to be a bijection between $\{0, 1\}^n$ and $\{0, 1\}^{2n}$, and the iteration procedure is the first step towards establishing a contradiction in the relevant proof system.

Following [135], Jeřábek [91] used this same construction within bounded arithmetic to establish a deep connection between the weak pigeonhole principle and circuit complexity. This work most closely informs our perspective, hence the name “J-tree.” In [101], the techniques of Jeřábek were translated to the language of traditional complexity theory, which yield the following interpretation of this tree-like iteration process: given a circuit $D: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, we can efficiently construct a new circuit $D^*: \{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$ satisfying the following: any string in the range of D^* is a truth table of low circuit complexity, and any string outside the range of D^* can be used to efficiently construct a string outside the range of D using an NP oracle. If we have a compression scheme $C: \{0, 1\}^{2n} \rightarrow \{0, 1\}^n$, $D: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, the same argument lets us iterate these to a scheme C^*, D^* of message length $2^k n$ and code length n , where again D^* always outputs low complexity truth tables, and now we have that any incompressible string for C^*, D^* can be transformed in polynomial time (without an oracle) into an incompressible string for C, D .

Thus, if it is hard to witness the weak pigeonhole principle for C, D , then we will be able to take any string (“message”) of length $2^k n$ and find a small circuit computing it (“codeword”) by

³The Merkle Tree is in some sense “dual” to the other two examples, as it iterates a function which *compresses* strings by a factor of two, while the others iterate a function that *extends* by a factor of two.

feeding it through C^* . A circuit can naturally be interpreted as a data structure storing an array subject to fast query operations: the truth table represents the contents of the array, and the circuit allows us to efficiently compute a particular bit in the truth table given its index. However, to use the J-tree to simulate RAM computations, we will also require fast update operations, which let us set a position in the array to a new value. Our main addition to this line of work is the “J-Tree Update Lemma” (Lemma 3.30), which gives a procedure that accomplishes this second task. The Update Lemma does not seem to have an analogue in any of the previously mentioned works.

3.2 Preliminaries

3.2.1 Basics

Throughout this chapter we will define the following precise notion of an “exponential time bound:”

Definition 3.3. *We say a function $T: \mathbb{N} \rightarrow \mathbb{N}$ is an “exponential time bound” if T is time constructible, and there exist constants $0 < \beta < B$ such that for sufficiently large n , $2^{\beta n} \leq T(n) \leq 2^{Bn}$.*

This is in contrast to the more general notion of exponential growth where we have $2^{n^\beta} \leq T(n) \leq 2^{n^B}$.

3.2.2 Machine Models

We start by precisely defining the various machine models and complexity classes that will be used. We begin with a definition of “random access memory” or “RAM” machines:

Definition 3.4 (RAM machines). *A RAM machine is a turing machine equipped with two binary tapes, one called the “auxilliary tape” and the other called the “addressing tape.” We collectively refer to these as the “linear tapes.” Both linear tapes admit the same operations as a standard Turing machine tape, where in one step we can move the head left/right and read or modify a cell of the tape. In addition, there is an associated “RAM memory” consisting of a sequence of binary variables $A_1, A_2, \dots$. The addressing tape has two additional operations that allow it to interact*

with the RAM memory. There is an Update operation, which in one step sets $A_i = b$, where i is the integer whose binary representation lies to the left of the addressing tapehead, and b is the bit currently read by the auxiliary tape. There is also a Load operation, which in one step sets the cell pointed to by the auxiliary tapehead to the value A_i , where again i is the integer whose binary representation lies to the left of the addressing tapehead. At the start of the computation on input x , the RAM cells $A_1, \dots, A_{|x|}$ are initialized to the bits of x in order, and $A_{|x|+1}, \dots$ are initialized to 0. The addressing tape is then initialized to $|x|$, so that the machine knows where the input ends.

We will use the following simplification lemma for RAM computations later, which follows easily from the use of balanced binary search trees:

Lemma 3.5. [63] *A RAM machine running in time $T(n)$ can be simulated in time $T'(n) = \tilde{O}(T(n))$ by a RAM machine that uses $O(\log T'(n))$ space on its addressing and auxiliary tapes, and uses only its first $T'(n)$ RAM cells.*

We will also make reference to k -tape Turing machines, for which we omit a formal definition as this model is standard. However, we will at some points concern ourselves with multi-tape machines equipped with oracles for languages/functions, and here we will need to be precise about the oracle access mechanism:

Definition 3.6 (Oracle Turing Machines). *For a function $F: \{0, 1\}^* \rightarrow \{0, 1\}^*$, an F -oracle k -tape Turing machine has k standard read/write “work tapes,” in addition to an oracle tape where in one step the machine can replace the leading cells of the oracle tape with $F(x)$, where x is the string lying to the left of the oracle tapehead prior to the oracle call; after this the oracle tapehead is moved to the first cell of the oracle tape. In some cases we will give a machine access to several oracles $F_1, F_2, \dots, F_k$ (for a fixed constant k), in which case each gets its own oracle tape. The oracle tape(s) and the first work tape share a single tapehead, and in one move this tapehead can move from the first cell of the work tape to the first cell of one of the oracle tapes, or vice-versa.*

The space usage of an oracle k -tape machine is defined to be the sum of the space used on all of its work tapes and oracle tapes.

The sharing of a single tapehead between the oracle tapes and the first work tape is only relevant for 1-tape machines, where we don't want the machine to "cheat" and use its oracle tapes as additional work tapes.

Definition 3.7 (Time Bounded Classes). *For a function $T: \mathbb{N} \rightarrow \mathbb{N}$, let $\text{TIME}[T(n)]$ denote the class of languages which are decidable by a deterministic multi-tape Turing machine running in time $O(T(n))$. Let $\text{RAM-TIME}[T(n)]$ be defined analogously for RAM machines.*

Definition 3.8 (Time-Space Classes). *For functions $T, S: \mathbb{N} \rightarrow \mathbb{N}$, let $\text{TISP}[T(n), S(n)]$ denote the class of languages which are decidable by a deterministic multi-tape Turing machine running simultaneously in time $O(T(n))$ and total space $O(S(n))$. Let $1\text{-TISP}[T(n), S(n)]$ be defined identically, but with the additional restriction that the machine uses only one tape.*

Finally, we define an analogous "time-space" class for nondeterministic time:

Definition 3.9 (Nondeterministic Time-Space Classes). *For functions $T, S, G: \mathbb{N} \rightarrow \mathbb{N}$, let $\text{NTISPG}[T(n), S(n), G(n)]$ ("nondeterministic time, space, guess") denote the class of languages decidable by a nondeterministic multi-tape Turing machine such that on any computation path on an input of length n , the machine spends time $O(T(n))$, uses space at most $O(S(n))$, and makes at most $O(G(n))$ non-deterministic guesses. Let $1\text{-NTISPG}[T(n), S(n), G(n)]$ be defined identically, but with the additional restriction that the machine uses only one tape. We define $\text{NTIME}[T(n)]$ in the standard way, which is analogous to the above but with no restriction on space usage or nondeterminism.*

We make note of the following result, which tells us that for nondeterministic computations the multi-tape and RAM models are roughly equivalent:

Lemma 3.10. [63] *Any time- $T(n)$ computation on a nondeterministic RAM machine can be simulated in $\tilde{O}(T(n))$ time on a nondeterministic multi-tape machine.*

We will thus not bother to explicitly define a nondeterministic RAM model, though the definition for the deterministic case naturally extends. Finally, we formally define our measures of circuit complexity and K^t complexity (time-bounded Kolmogorov complexity):

Definition 3.11. Let $x \in \{0, 1\}^N$. We say that x “has circuits of size s ” if there is a boolean circuit C of fan-in 2 over the basis $\wedge, \vee, \neg$ on $\lceil \log N \rceil$ variables such that C has at most s gates and for all $0 \leq i < N$, $C(i)$ equals the i^{th} bit of x (where we convert between integers and binary strings in the standard way). We say that “ x has circuit complexity s ” if x does not have circuits of size $s - 1$.

Definition 3.12. Fix some efficient universal Turing machine U . For $x \in \{0, 1\}^*$, we define $K^t(x)$ to be the length of the smallest string y such that U outputs x on input y in at most t steps.

3.2.3 Search Problems

At some points in this work we will be concerned with functions that are efficiently computable with access to an oracle for solving some search problem. A subtlety arises here, since an instance of a search problem may have many valid solutions, and this may cause our oracle machine to have a variety of possible outputs. To formalize the notion of search problem oracles correctly, we need to recall the definition of a function solving a search problem from the preliminaries:

Definition 3.13. Let R be a search problem. We say that $O: \{0, 1\}^* \rightarrow \{0, 1\}^* \sqcup \{\perp\}$ is a “functional oracle” for R if the following holds for all $x \in \{0, 1\}^*$:

1. If there exists a y such that $(x, y) \in R$ then $(x, O(x)) \in R$.
2. If there does not exist a y such that $(x, y) \in R$ then $O(x) = \perp$.

Functional oracles give us a natural way to complexity classes solved by R -oracle machines for some bounded relation R – an R -oracle machine computes a certain function, or solves some other search problem, if it has a valid output on every input when given access to an arbitrary functional oracle for R .

3.3 Pigeonhole Principles and Compression Schemes

TODO: reevaluate this section for redundancies during intro-unification stage We now define the basic formalization of the weak pigeonhole principle we will investigate in this work, and introduce the relevant terminology.

3.3.1 Compressors and Decompressors

Definition 3.14. Let $D: \{0, 1\}^m \rightarrow \{0, 1\}^n$, with $m < n$. We call such a map which extends its input length a “decompressor.” The “code length” of this decompressor is m , and its “message length” is n .

The terminology should be interpreted as follows: for a string x such that $D(y) = x$, y functions as an m -bit compressed representation (or “codeword”) for the n -bit “message” x , and D functions as an algorithm which lets us “decompress” this codeword into the message it represents.

By the *dual weak pigeonhole principle*, if $n > m$, any function mapping 2^m “pigeons” to 2^n “holes” must leave some hole empty. We thus know there must exist an $x \in \{0, 1\}^n$ such that $\forall y \in \{0, 1\}^m, D(y) \neq x$. We call such a string x an “empty pigeonhole” for the decompressor D . The primary subject of [101] was the problem EMPTY, originally introduced in [97], where we are given a decompressor specified by a boolean circuit and must output one of its empty pigeonholes. In this work, we will study a slight modification of this problem, where we are given both a compressor and a decompressor, and must find a witness to the fact that some message is not recoverable from its codeword. This was referred to by Jeřábek as the “retractive pigeonhole principle.”[90].

Definition 3.15. Let $C: \{0, 1\}^n \rightarrow \{0, 1\}^m, D: \{0, 1\}^m \rightarrow \{0, 1\}^n$, where $m < n$; we call such a C a “compressor”, and collectively we will refer to C, D as a “compression scheme.” Again, m and n are the “code length” and “message length” respectively. By the pigeonhole principle, we know that there must exist some $x \in \{0, 1\}^n$ such that $D(C(x)) \neq x$; we call such an x “incompressible” with respect to the scheme C, D .

Note that when $x \in \{0, 1\}^n$ is an empty pigeonhole for a decompressor D , it is necessarily incompressible for *all* schemes C, D which use D as the decompressor. The converse is not true in general, but when it is we use the following terminology:

Definition 3.16. We call C a “proper compressor” for D if C, D satisfy the following for all $x \in \{0, 1\}^n$: if there exists an $y \in \{0, 1\}^m$ such that $D(y) = x$, then $D(C(x)) = x$.

By definition, when C is a proper compressor for D , the incompressible strings for the scheme C, D correspond exactly to the empty pigeonholes for D . It should be noted that when D is polynomial time computable, or is specified by some boolean circuit, there always exists a proper compressor for D computable efficiently with an NP oracle, which simply finds the lexicographically first preimage of a string under D or else outputs something arbitrary when none exist. In this sense, the work of [101], which studies the complexity of EMPTY with respect to NP-oracle reductions, was essentially studying a special case of this problem, where the compressor is the “canonical proper compressor” which searches for the lexicographically first preimage.

For most of this work it will be convenient to focus on the special case of functions which exactly double their input length:

Definition 3.17. *When a compression scheme has code length n and message length $2n$, we will say that it has “stretch 2.”*

We will utilize the following lemma, which tells us that if our goal is to find an incompressible string with respect to some scheme, we can assume it has stretch 2 without loss of generality:

Lemma 3.18. *Let C, D be a compression scheme with code length n and message length m . Then there is another scheme C', D' of code length n and message length $2n$, such that C' is computable in $\text{poly}(m)$ time with an oracle for C , D' is computable in $\text{poly}(m)$ time with an oracle for D , and such that given an incompressible string for C', D' , we can construct an incompressible string for C, D in $\text{poly}(m)$ time with oracles for C, D .*

Proof. This is proven in [101] for the special case when C is a proper compressor for D , but the exact same proof extends to the more general case stated here. $\square$

In [101], the problem EMPTY was studied as a search problem, where the decompressor D is provided as input (in the form of a circuit), and the goal is to find one of its empty pigeonholes. We could equivalently define such a search problem for the retractive pigeonhole principle, where we are given circuits computing C, D as input, and must output an incompressible string for this scheme (this search problem is considered in Section 3.6). However, for the purposes of our

main results it will be more suitable to study a *uniform* version of this problem, where we have a sequence of schemes C_n, D_n of increasing size, and each are computable within some uniform complexity class.

Definition 3.19. *A uniform compression scheme is a pair $\mathcal{C} = \{C_n: \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}^{m(n)}\}_{n \in \mathbb{N}}$, $\mathcal{D} = \{D_n: \{0, 1\}^{m(n)} \rightarrow \{0, 1\}^{\ell(n)}\}_{n \in \mathbb{N}}$ for some pair of time constructible functions $m, l: \mathbb{N} \rightarrow \mathbb{N}$ such that $m(n) < \ell(n)$ for all n , and $\ell(n)$ is bounded above by a polynomial in n .*

In this work we will concern ourselves with uniform compression schemes where $\mathcal{C}, \mathcal{D}$ are computable in polynomial time, each with access to different oracles, hence the name “uniform.”

3.3.2 Particular Compression Schemes of Interest

We now introduce some uniform compression schemes whose incompressible strings have certain desirable properties for which no explicit constructions are currently known.

3.3.2.1 Compression Schemes for Non-Uniform Complexity Measures

We start with the case of hard truth tables. The following lemma is a well-known folklore result, for which a formal proof can be found in [101]:

Lemma 3.20. *For sufficiently large $N \in \mathbb{N}$, there is function $f_N: \{0, 1\}^{N-1} \rightarrow \{0, 1\}^N$ computable uniformly in $\tilde{O}(N^2)$ time such that if $x \in \{0, 1\}^N$ has circuits of size at most $\frac{N}{2 \log N}$, then x is in the range of f_N .*

We now define the search problem associated with finding preimages of strings under f_N , which has commonly been referred to as the “circuit synthesis problem:”

Definition 3.21 (CIRCUIT SYNTHESIS). *Given a string $x \in \{0, 1\}^N$, output an $N-1$ bit description of a circuit C of size at most $\frac{N}{2 \log N}$ on $\lceil \log N \rceil$ variables such that $C(i) = x_i$ for all $0 \leq i < N$ if such a circuit exists, or else determine that no such circuit exists.*

This problem is the search variant of the more well-studied “Minimum Circuit Size Problem” [93], which is not known to admit a search-to-decision reduction. By the same arguments underlying Lemma 3.20, given an instance $x \in \{0, 1\}^N$ of CIRCUIT SYNTHESIS, any circuit of the stated size can be represented using at most $N - 1$ bits via a standard encoding so this search problem is well defined. Further, requiring the output to be specified in such an encoding does not increase the complexity of the problem, since the standard encoding can be computed efficiently from any description of such a circuit.

By definition, we see that there is a proper compressor for f_N computable in polynomial time with a CIRCUIT SYNTHESIS oracle, and thus any incompressible string for this scheme is an N -bit string with high circuit complexity. As foreshadowed in Section 3.2.3, a subtlety arises here since there are many valid functional oracles O for the search problem CIRCUIT SYNTHESIS, and for any such oracle, $f_N, O \upharpoonright_{\{0,1\}^N}$ will be a uniform compression scheme satisfying the required properties. However, an important observation is that if CIRCUIT SYNTHESIS is in FP, then there is a particular functional oracle O for this problem which is computed by some polynomial time algorithm, and in this case we have a fixed compression scheme with the required properties where both the compressor and decompressor are computable in polynomial time.

We similarly have the following:

Lemma 3.22. *For any fixed polynomial p , there is a uniform compression scheme whose decompressor is computable in polynomial time, and whose compressor is computable with an oracle for $K^{p(n)}$ MINIMIZATION (given $x \in \{0, 1\}^n$ find a short program $y \in \{0, 1\}^{n-2}$ that prints x in $p(n)$ steps if one exists). The incompressible strings for this scheme have $K^{p(n)}$ complexity $\geq n - 1$.*

Lemma 3.23. *There is a uniform compression scheme whose decompressor is computable in polynomial time, and whose compressor is computable with an oracle for FORMULA SYNTHESIS (given a truth table find a short formula if one exists). The incompressible strings for this scheme are truth tables of exponential formula size.*

We provide these as basic examples without defining the problems too formally; more generally it can be verified that for most reasonable non-uniform measures of complexity (which are bounded

above by K^{poly}), such a uniform scheme exists whose decompressor is computable efficiently, whose compressor is computable with access to the relevant “compression problem,” and whose incompressible strings have high complexity with respect to this measure.

3.3.2.2 Large Primes

We next construct a compression scheme related to the large prime construction problem. This scheme is unlike the previous ones, in that both the compressor and decompressor have the same complexity: each will require an oracle for factoring.

Theorem 3.24. *There is a compression scheme R, P with code length $n + \lceil \log n \rceil + 3$ and message length $n + \lceil \log n \rceil + 4$, such that R, P are each computable uniformly in polynomial time with a factoring oracle, and given an incompressible string for this scheme, a $32n$ -bit prime of magnitude $> 2^n$ can be constructed in polynomial time with a factoring oracle.*

We can immediately note that we will not encounter the issue of dealing with different “functional oracles” here, since the factoring problem has a unique solution on all inputs. This theorem is an algorithmic analogue of a well known result of Paris, Wilkie, and Woods [135], and our proof will follow from analysing the computational resources needed to carry out their construction. We start with the following useful lemma:

Lemma 3.25. *For sufficiently large n there exists a pair of efficiently computable maps $g: \{0, 1\}^n \rightarrow \{0, 1\}^{n-2}$, $f: \{0, 1\}^{n-2} \rightarrow \{0, 1\}^n$ such that for all n -bit primes p we have $f(g(p)) = p$.*

The constant 2 is arbitrary and can be replaced with any positive integer; we give a proof of this more general case in Appendix 3.7. The encoding follows from the observation that the primes can be easily shown to have natural density $< \epsilon$ by considering their structure with respect to divisibility by the first $O(1)$ primes.

We will also use the following notation at various points in the proof:

Definition 3.26. *Let $x \in \mathbb{N}$, and let $\prod_{i \leq v} p_i^{e_i}$ be the prime factorization of x where the p_i are in increasing order of magnitude. We define I_x to be the interval $[\ell]$ where $\ell = \sum_{j \leq v} e_j \lceil \log p_j \rceil$. We*

will think of the interval I_x as being composed of e_1 intervals of length $\lceil \log p_1 \rceil$, followed by e_2 intervals of length $\lceil \log p_2 \rceil$, and so on. In this way, we see that every position in the interval I_x determines a prime power p^k dividing x and an index $i \in [\lceil \log p \rceil]$, and conversely every such p^k, i determines a position in I_x .

We are now ready to prove Theorem 3.24:

Proof of Theorem 3.24. We will use the shorthand $N = 2^n$ for the remainder of the proof. Consider an arbitrary string $xs \in \{0, 1\}^{n+\lceil \log n \rceil+4}$, where $|x| = n$ and $|s| = \lceil \log n \rceil + 4$. We will consider x as an integer in the range $[N]$, and s as representing a position in the interval $I_{N^{32}+x}$. Since $N^{32} + x > N^{32}$, we see that $I_{N^{32}+x}$ has length at least $32n$, and thus any $\lceil \log n \rceil + 4$ bit string can be identified uniquely with some position in $I_{N^{32}+x}$. As described above, this position is in turn fully defined by a prime power p^k dividing $N^{32} + x$, together with an index $i \in [\lceil \log p \rceil]$. Further, it is clear that from x and s we can determine p, k, i in $\text{poly}(n)$ time with a factoring oracle.

For some prime power p^m and $y \in [N]$, we will say that $p^m \sim y$ if the following holds: p^{m+1} has no multiple in the interval $[N^{32} + 1, N^{32} + N]$, and $N^{32} + y$ is the smallest integer in the interval $[N^{32} + 1, N^{32} + N]$ which is divisible by p^m . Clearly it can be tested in $\text{poly}(n)$ time whether $p^m \sim y$ using division. We can also see that for any prime $p \leq N$, there exists a unique $y \in [N]$ such that $p^m \sim y$ for some m , and such a y can be computed from p efficiently by testing increasing powers of p .

Now, our compressor R on input xs will start by computing the values p, k, i determined by s and $I_{N^{32}+x}$ as described above. During this computation, R will test if each prime in the decomposition of $N^{32} + x$ is of magnitude $\leq N$. If one of them is not, R will “fail” and simply output an arbitrary string (say $1^{n+\lceil \log n \rceil+3}$ for concreteness). Otherwise, R next tests if $p^k \sim x$. If this holds, we have that x can be recovered from p efficiently, and in this case R maps xs to $0ps$ where the prime p is encoded in $n - 2$ bits according to Lemma 3.25. Overall this can be done in the required $n + \lceil \log n \rceil + 3$ bits. Now say that $p^k \sim x$ does not hold. In this case, R computes the unique integer $y \in [N]$ such that $p^m \sim y$ holds for some m . We then set $z = |x - y|$, and $b = \text{sign}(x - y) \in \{0, 1\}$. By definition of m we see that $k \leq m$, since p^k does have a multiple in $[N^{32} + 1, N^{32} + N]$, namely

$N^{32} + x$. In addition we have that p^m divides $N^{32} + y$ and p^k divides $N^{32} + x$. So overall we conclude that p^k must divide $|(N^{32} + y) - (N^{32} + x)| = z$. Now, consider the interval I_z , which has length at most $2n$ since $z \leq N$. Since p^k divides z , there is some position in I_z which determines p, k, i , and this position can in turn be encoded in $\lceil \log n \rceil + 1$ bits, call this encoding s' . Now R outputs $1zbs'$. Again we see that this output has length $n + \lceil \log n \rceil + 3$ as required.

We now define the decompressor P, which simply unpacks the above encoding of an input xs whenever R does not fail. When P sees an input of the form $0ps$ (where p is encoded in $n - 2$ bits as per Lemma 3.25), it recovers the unique x such that $p^m \sim x$ for some m , and then outputs xs . Otherwise, if it sees an input of the form $1zbs'$, it computes the prime factorization z to determine the values p, k, i encoded by the position of s' in I_z . From p it then computes the unique $y \in [N]$ and m such that $p^m \sim y$, and sets $x = y + (-1)^b z$. Finally, it computes the factorization of $N^{32} + x$, and determines the position in $I_{N^{32}+x}$ representing p, k, i , call it j ; if $j \leq 2^{\lceil \log n \rceil + 4}$, it then encodes the position j as a $\lceil \log n \rceil + 4$ bit string and outputs xs ; otherwise it outputs something arbitrary ($1^{n+\lceil \log n \rceil + 4}$ for example).

By construction we see that whenever R does not “fail” on input xs , it outputs some value from which P then recovers xs . But the only way that R can fail on an input xs is when the prime factorization of $N^{32} + x$ contains a prime $> N$. Thus if xs is an incompressible string for P, R, $N^{32} + x$ must have prime factor $> N$, which we can compute given x using a factoring oracle. $\square$

3.4 J-Trees

In this section we develop the core tool used in our main result, namely the “J-tree.” The J-tree is, informally, a data structure “solving” the following information-theoretically impossible task: store an array of T elements, subject to efficient updates and queries, using significantly fewer than T bits. While such a data structure cannot exist unconditionally, the J-tree will take as input a compression scheme C, D , and will be set up so that when it fails in its capacity as a data structure, it prints out an incompressible string for C, D . In other words, if it is hard to witness the weak pigeonhole principle for the scheme C, D , then it is hard to find a sequence of updates/queries

which causes our data structure to fail.

Definition 3.27 (J-trees). *Let $D: \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ be a decompressor of code length n and stretch 2. We define $D_0, D_1: \{0, 1\}^n \rightarrow \{0, 1\}^n$ to be the maps obtained by computing D and taking the first n and last n bits of output respectively. Now, for any binary string $x \in \{0, 1\}^*$, we define $D_x: \{0, 1\}^n \rightarrow \{0, 1\}^n$ as follows. When $|x| = 0$, D_x is the identity, and when $|x| = 1$, D_0, D_1 are defined as above. In the general case, when $x = b_1 \cdots b_k$ for some $b_1, \dots, b_k \in \{0, 1\}$, D_x is defined as:*

$$D_x = D_{b_k} \circ \cdots \circ D_{b_1}$$

Now, for a particular input $y \in \{0, 1\}^n$, we can define the function $D[y]: \{0, 1\}^ \rightarrow \{0, 1\}^n$ as follows. For each $x \in \{0, 1\}^*$:*

$$D[y](x) = D_x(y)$$

For an integer k , we then define $D^k[y]: \{0, 1\}^k \rightarrow \{0, 1\}^n$, which is simply the restriction of $D[y]$ to the domain $\{0, 1\}^k$.

We will refer to $D^k[y]$ as a “J-tree,” D as its decompressor, y as its “seed,” and k as its “depth.”

It will be useful to visualize a J-tree as a binary tree (hence the name). Refer to Figure 3.3 which illustrates a J-tree $D^k[y]$ where $k = 3$. For a given D and depth k , we can construct a tree-like circuit which starts with one copy of D , then feeds each of its two n -bit output blocks into another copy of D , and so on for k iterations, ultimately yielding a circuit with n inputs and $2^k n$ outputs which has the structure of a perfect binary tree of depth k . If we now fix the inputs to some value y (the seed), by passing y through this tree-like circuit we obtain 2^k n -bit values at the output. As seen in the figure, each of the 2^k n -bit values is associated uniquely with a leaf in this binary tree of depth k , and thus can be specified by a k -bit index indicating whether to move left or right at each step along a root-to-leaf path. $D^k[y]$ is then the function which, given the description of such a path, returns the value at the corresponding leaf.

We will think of a J-tree $D^k[y]$ operationally as a data structure which stores an array of 2^k

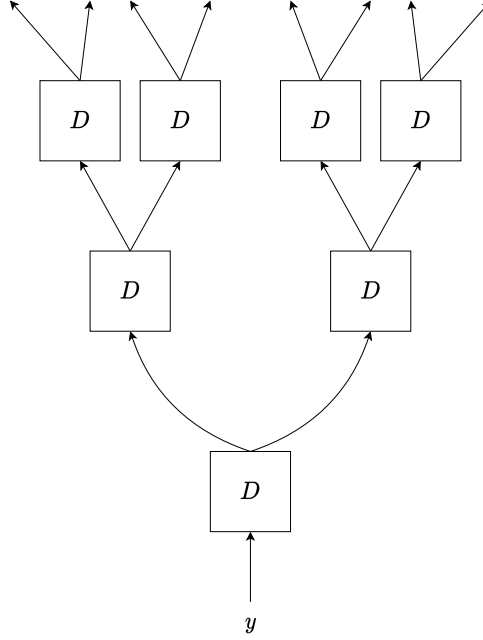


Figure 3.3: A J-tree of depth 3. Each arrow consists of n wires, where n is the code length of the decompressor D .

n -bit values, one for each of its 2^k “leaves”, where the i^{th} value is simply $D^k[y](i)$. The state of the data structure is described purely by y and D , which in our use cases will require far fewer bits than storing 2^k n -bit strings explicitly. In the following, we now prove that the J-tree data structure admits fast query and update operations, i.e. operations that allow us to read one entry of the data structure, or update the value of one entry. While the query operation will be computable efficiently by evaluating only the decompressor D $O(k)$ times, the update operation will require evaluating some compressor C , and might “fail” in a certain well-defined sense. However, when the update does not fail, there is a deterministic algorithm which can verify, using $O(k)$ evaluations of D , that the resulting J-tree is in fact the true updated version of the original. We begin with the query or “access” operation:

Lemma 3.28 (J-tree Access Lemma). *Let $D^k[y]$ be a J-tree. Then for any $i \in \{0, 1\}^k$ we can compute $D^k[y](i)$ in time $O(nk)$ given i, y and using $O(k)$ evaluations of D .*

Proof. This follows directly from the definition of $D^k[y]$. Letting $b_1, \dots, b_k$ be the individual bits

of i , we have that:

$$D^k[y](i) = D_{b_k} \circ D_{b_{k-1}} \circ \cdots \circ D_{b_1}(y)$$

So we can compute $D^k[y](i)$ by evaluating D k times successively starting with the input y . $\square$

Definition 3.29 (J-tree Modifications). *Let $D^k[y]$, $D^k[y']$ be J-trees of depth k and code length n . We say that the relation $\text{Modified}(y, y', i, s, D)$ holds if:*

1. $D^k[y'](i) = s$
2. For all $i' \in \{0, 1\}^k$, $i' \neq i$, $D^k[y'](i') = D^k[y](i')$

In other words, $\text{Modified}(y, y', i, s, D)$ asserts that the J-tree $D^k[y']$ represents a local modification of the J-tree $D^k[y]$ which changes its i^{th} value to s and leaves all other values the same.

Lemma 3.30 (J-tree Update Lemma). *There exist algorithms Find and Verify satisfying the following.*

Verify takes as input a decompressor D (specified as an oracle) of code length n and stretch 2, a pair of seeds $y, y' \in \{0, 1\}^n$, an index $i \in \{0, 1\}^k$, and a value $s \in \{0, 1\}^n$, and either accepts or rejects. Verify runs in deterministic time $O(nk)$ using $O(k)$ evaluations of D , and satisfies the property that if $\text{Verify}(y, y', i, s, D)$ accepts, then $\text{Modified}(y, y', i, s, D)$ must hold.

Find takes as input a compression scheme C, D of code length n and stretch 2 (again specified as oracles), a seed $y \in \{0, 1\}^n$, an index $i \in \{0, 1\}^k$, and a value $s \in \{0, 1\}^n$. It either “succeeds” and outputs a string $y' \in \{0, 1\}^n$, or else outputs $\text{FAIL}\langle e \rangle$, where $e \in \{0, 1\}^{2n}$. Find runs in $O(nk)$ time using $O(k)$ evaluations of C and D , and satisfies the property that for every input y, i, s, C, D , one of the following holds:

1. $\text{Find}(y, i, s, C, D)$ outputs a string y' such that $\text{Verify}(y, y', i, s, D)$ accepts.
2. $\text{Find}(y, i, s, C, D)$ outputs $\text{FAIL}\langle e \rangle$, where e is incompressible with respect to C, D .

Proof. We start by defining the procedure Find. Given inputs y, i, s, C, D , Find begins by computing a sequence of k values $z_1, \dots, z_k \in \{0, 1\}^n$ as follows. Let $b_1, \dots, b_k$ denote the bits of i in

order. For each $j \in [k]$, we set

$$z_j = D[y](b_1 b_2 \dots b_{j-1} \neg b_j)$$

It is clear that the list of z_j be computed in $O(nk)$ time using $O(k)$ evaluations of D , by storing the intermediate values of $D[y](b_1 \dots b_j)$ for each j and computing the z_j in increasing order of j .

Now, given this list of values, we compute a second sequence $v_1, \dots, v_k \in \{0, 1\}^n$. We compute the v_j for each $j \in [k]$ in decreasing order of k as follows. First we set:

$$v_k = \begin{cases} C(sz_k) & \text{if } b_k = 0 \\ C(z_k s) & \text{if } b_k = 1 \end{cases}$$

We then check that $D(C(sz_k)) = sz_k$ (resp. $D(C(z_k s)) = z_k s$). If this check fails we abort and return $\text{FAIL}\langle sz_k \rangle$ (resp. $\text{FAIL}\langle z_k s \rangle$). Now, for each $j \in \{k-1, k-2, \dots, 1\}$, we set:

$$v_j = \begin{cases} C(v_{j+1} z_j) & \text{if } b_j = 0 \\ C(z_j v_{j+1}) & \text{if } b_j = 1 \end{cases}$$

Again, each time we evaluate C on some input, we check that that input is indeed compressible with respect to C, D , and if not then we return $\text{FAIL}\langle e \rangle$ where e is the incompressible string we found. If we get to the end of this process without returning failure, we return the string v_1 . This completes the description of the Find procedure, which overall requires at most $O(nk)$ time to compute using at most $O(k)$ evaluations of C and D . Figures 3.4 and 3.5 illustrate this procedure for a J-tree of depth 3.

We now describe the Verify procedure on input y, y', i, s, D , which simply verifies that a given string y' is a possible successful output of $\text{Find}(y, i, s, C, D)$ for some C . Given y, y', i, s, D , again let $b_1, \dots, b_k$ denote the bits of i in order. First, Verify iterates over each value of $j \in [k]$,

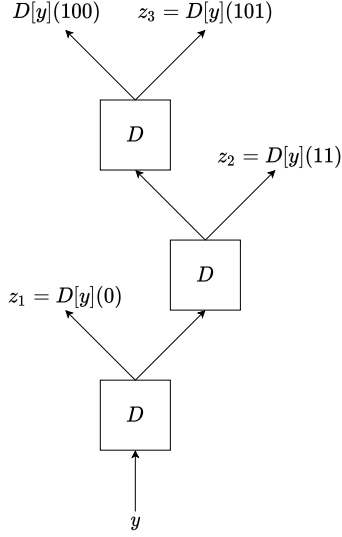


Figure 3.4: “Forward phase” of $\text{Find}(y, 100, s, C, D)$ (where $k = 3$), during which the $z_1, \dots, z_k$ are computed.

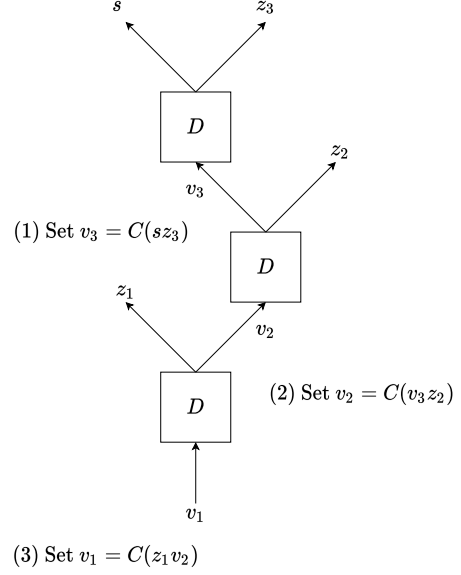


Figure 3.5: “Backward phase” of $\text{Find}(y, 100, s, C, D)$ (where $k = 3$), during which the $v_1, \dots, v_k$ are computed.

and checks that $D[y'](b_1, \dots, b_{j-1}, \neg b_j) = D[y](b_1, \dots, b_{j-1}, \neg b_j)$. Next, it checks that $D[y'](b_1, \dots, b_k) = D^k[y'](i) = s$. If all these conditions hold, the Verify procedure accepts, and otherwise it rejects. It is clear that these conditions can be verified in $O(nk)$ time using $O(k)$ evaluations of D , by storing the intermediate values $D[y](b_1, \dots, b_j), D[y'](b_1, \dots, b_j)$ at each step.

We now show that if $\text{Find}(y, i, s, C, D)$ succeeds and returns a string y' , then $\text{Verify}(y, y', i, s, D)$ accepts. By definition, if $\text{Find}(y, i, s, C, D)$ doesn't fail, then it is able to compute some list of values $v_1, \dots, v_k \in \{0, 1\}^n$ such that for all $j < k$, $D_{b_j}(v_j) = v_{j+1}$ and $D_{\neg b_j}(v_j) = z_j$, and it returns v_1 as its output. So then we have that $D[v_1](b_1, \dots, b_{j-1}, \neg b_j) = D[v_j](\neg b_j) = z_j$ for all $j \in [k]$. Further, we have that $D[v_1](b_1, \dots, b_{k-1}, b_k) = D[v_k](b_k) = s$. So overall $\text{Verify}(y, v_1, i, s, D)$ must accept if $\text{Find}(y, i, s, C, D)$ succeeds and returns v_1 .

It remains only to show that if $\text{Verify}(y, y', i, s, D)$ accepts, then $\text{Modified}(y, y', i, s, D)$ must hold. Recall that $\text{Modified}(y, y', i, s, D)$ asserts that the two J-trees $D^k[y]$, $D^k[y']$ agree on all indices $i' \neq i$, and that $D^k[y'](i) = s$. If Verify accepts then this second condition holds trivially, since Verify explicitly checks that $D^k[y'](i) = s$ and rejects if this does not hold. Now consider

the first condition of Modified. Let $i' \in \{0, 1\}^k$, $i' \neq i$. Let $\ell_1, \dots, \ell_k$ denote the bits of i' in order, and let $t \in [k]$ be the smallest index such that $\ell_t \neq b_t$. By our assumption that Verify accepted, we have that

$$D[y](\ell_1, \dots, \ell_t) = D[y](b_1, \dots, b_{t-1}, \neg b_t) = D[y'](b_1, \dots, b_{t-1}, \neg b_t) = D[y'](\ell_1, \dots, \ell_t)$$

But by definition we also know that

$$D[y](\ell_1, \dots, \ell_t, \ell_{t+1}, \dots, \ell_k) = D[D[y](\ell_1, \dots, \ell_t)](\ell_{t+1}, \dots, \ell_k)$$

and similarly

$$D[y'](\ell_1, \dots, \ell_t, \ell_{t+1}, \dots, \ell_k) = D[D[y'](\ell_1, \dots, \ell_t)](\ell_{t+1}, \dots, \ell_k)$$

so if $D[y](\ell_1, \dots, \ell_t) = D[y'](\ell_1, \dots, \ell_t)$ then it must be that $D[y](\ell_1, \dots, \ell_k) = D[y'](\ell_1, \dots, \ell_k)$. So we have established that $D^k[y](i') = D^k[y'](i')$, which completes the proof. $\square$

Next, we prove the following “initialization” lemma, which lets us efficiently set all leaves of a J-tree to a common value in time proportional to its depth:

Lemma 3.31. (*J-Tree Initialization Lemma*) *There is an algorithm Initialize which takes as input a compression scheme C, D of code length n and stretch 2 (specified as oracles), a value $s \in \{0, 1\}^n$, and a depth parameter k . It either succeeds and returns a seed $y \in \{0, 1\}^n$ such that for all $i \in \{0, 1\}^k$, $D^k[y](i) = s$, or else outputs $\text{Fail}\langle e \rangle$ where e is incompressible with respect to C, D . $\text{Initialize}(s, k, C, D)$ runs in time $O(nk)$ using $O(k)$ evaluations of C and D .*

Proof. We start by setting $v_1 = C(ss)$. We then check if $D(v_1) = ss$ and if not we return $\text{Fail}\langle ss \rangle$. Otherwise we continue for each value $j \in [k]$ in increasing order, setting $v_j = C(v_{j-1}v_{j-1})$, checking that C finds a valid compression at each step, and returning an incompressible string if it does not. We then return v_k if all checks pass.

Say $\text{Find}(s, k, C, D)$ succeeds and returns v_k . By induction we show that $D[v_j](i) = s$ for all $i \in \{0, 1\}^j$. For the base case, since $D(v_1) = ss$ have that $D[v_1](i) = s$ for all $i \in \{0, 1\}$. For the inductive case say $i = bi'$ for $b \in \{0, 1\}, i' \in \{0, 1\}^{j-1}$. So then we have that $D[v_j](i) = D[D_b(v_j)](i')$. By assumption that $\text{Find}(s, C, D)$ succeeds we have that $D_b(v_j) = v_{j-1}$, and by the inductive hypothesis we have that $D[v_{j-1}](i') = s$, completing the proof. $\square$

In the proofs to come it will also be useful to have the following “iterated compression” lemma. This Lemma is implicit in the proof of the main theorem in [101].

Lemma 3.32 (J-Tree Iterated Compression Lemma). *Let C, D be a compression scheme of code length n , and let $S = (s_1, s_2, \dots, s_\ell)$ be a sequence of strings, where $s_i \in \{0, 1\}^n$. There exists an algorithm Iter-Compress running in time $\text{poly}(n, \ell)$ and using $\text{poly}(\ell)$ evaluations of C and D which, given C, D and S , either “succeeds” and outputs a seed $y \in \{0, 1\}^n$ such that $D^{\lceil \log \ell \rceil}[y](i) = s_i$ for all $i \in \{0, 1\}^{\lceil \log \ell \rceil}, i \leq \ell$, or else “fails” and outputs a string e which is incompressible with respect to C, D .*

Proof. This follows from repeated application of the Find procedure in Lemma 3.30. We initialize $y = 0^n$. Now, for each $i \in [\ell]$, we perform $\text{Find}(y, i, s_i, C, D)$, and if Find fails and an incompressible string e , then Iter-Compress fails and outputs e . Otherwise we set $y = \text{Find}(y, i, s_i, C, D)$. We then repeat for the next value of i . If we succeed for all i , we output the final value of y . Correctness and runtime of this algorithm follow from Lemma 3.30. $\square$

3.5 Low-Space Simulations

In this section we prove our main set of theorems. In each case, we show how to simulate an exponential time computation with a drastic reduction in certain resources, using short oracle calls to some uniform compression scheme. We then show that either this simulation is successful, or there is an explicit construction algorithm that prints incompressible strings for this compression scheme.

3.5.1 Proofs of Main Theorems

Theorem 3.33. *Let $C, \mathcal{D} = \{C_n\}_{n \in \mathbb{N}}, \{D_n\}_{n \in \mathbb{N}}$ be a uniform compression scheme.*

Then one of the following must hold:

1. *There is polynomial time algorithm with oracle access to $C, \mathcal{D}$ which, for infinitely many n , outputs an incompressible string for C_n, D_n on input 1^n .*
2. *For every exponential time bound T , every language $L \in \text{RAM-TIME}[T(n)]$, and every $\epsilon > 0$, there is 1-tape Turing machine with oracle access to $C, \mathcal{D}$ which decides L in time $T(n)^{1+\epsilon}$, uses space at most $T(n)^\epsilon$, and makes oracle calls of length at most $T(n)^\epsilon$.*

Proof. Let $L \in \text{RAM-TIME}[T(n)]$, and let $\mathcal{M}$ be the deterministic random access oracle machine witnessing this inclusion. Let $C, \mathcal{D}$ be a uniform compression scheme. By Lemma 3.18, we can assume C_n, D_n have stretch 2 for all n . We will define a “simulator machine” which attempts to decide L using low space and small oracle calls to $C, \mathcal{D}$. We then define a second “checker machine” which checks the work of the simulator. We show that whenever the simulator fails to decide L , the checker will be able to witness this failure in the form of an incompressible string. We will thus conclude that if simulation of L fails for infinitely many inputs, the “checker” will constitute an explicit construction algorithm which prints incompressible strings for $C, \mathcal{D}$.

Step 1 - Defining the Simulator

Let $0 < \epsilon < 1$ be a fixed rational constant. We define a machine S_ϵ which will attempt to efficiently simulate $\mathcal{M}$ using low space, short $C, \mathcal{D}$ -oracle queries, and which operates on a 1-tape oracle Turing machine. Let $x \in \{0, 1\}^n$ be an input. For the remainder of this section we will keep a particular input length n fixed in our mind and thus drop the dependence of other terms on n in our notation; in particular we will use the abbreviation $T = T(n)$. Our machine will now fix a particular instance of $C, \mathcal{D}$, in particular $C_{\lceil 2^{\epsilon n} \rceil}, D_{\lceil 2^{\epsilon n} \rceil}$; again we simplify our notation from here on and simply write C, D for this scheme. By definition of a uniform compression scheme, we see

that the code length of C, D will be by $\text{poly}(2^{\epsilon n}) = T(n)^{O(\epsilon)}$, and by assumption its message length is twice its code length. We will use W to denote its code length for the remainder of the proof.

We start by invoking Lemma 3.5, which lets us assume without loss of generality that $\mathcal{M}$ uses its first T cells of RAM and uses at most $O(\log T)$ space on its linear tapes. This simplification will come at a $\log T$ multiplicative cost to run time, which is negligible here. Now, our simulating machine will initialize a variable $\text{Mem} \in \{0, 1\}^W$ which will be used as a seed in a J-tree with decompressor D and depth $\lceil \log T \rceil$. From now on we will fix $k = \lceil \log T \rceil$. The simulation $\mathcal{S}_\epsilon$ will run in T phases, and will maintain the invariant that if $\mathcal{M}$'s i^{th} memory cell has value $b \in \{0, 1\}$ at the start of $\mathcal{M}$'s t^{th} time step, then $D^k[\text{Mem}](i) = b^W$ at the start of $\mathcal{S}_\epsilon$'s t^{th} phase. Aside from Mem , $\mathcal{S}_\epsilon$ will also explicitly store a copy of $\mathcal{M}$'s linear tapes (which have total length $O(k)$), the input x , and descriptions of $\mathcal{M}$'s state and tape-head pointers, each requiring at most k bits. Thus the total space required to store all local variables between phases in the simulation is at most $O(W(n)^2 + n) = T(n)^{O(\epsilon)}$ (recall that T is an exponential time bound).

At the start of the first phase, $\mathcal{S}_\epsilon$ initializes Mem to a value such that $D^k[\text{Mem}](i) = 0^W$ for all $i \in \{0, 1\}^k$, which matches the initial state of $\mathcal{M}$'s RAM memory at the beginning of its computation. By Lemma 3.31, this can be accomplished in time $O(kW) = T^{O(\epsilon)}$ with oracle access to C, D ; if the initialization procedure fails and returns an incompressible string for C, D , the simulation halts and rejects its input x . Now, in phase $t \in [T]$, $\mathcal{S}_\epsilon$ will simulate the t^{th} step of $\mathcal{M}$ as follows:

1. Read the values at the current tapehead positions on all linear tapes, and the current state of $\mathcal{M}$.
2. Based on these values, determine which of $\mathcal{M}$'s transition rules to apply. Every rule involves a state update, which we can perform manually as we explicitly store $\mathcal{M}$'s state. If the new state is an accept/reject state for $\mathcal{M}$, then $\mathcal{S}_\epsilon$ halts and accepts/rejects accordingly. Otherwise:
 - (a) First, say the rule only involves updates to the linear tapes. In this case we just carry out the rule explicitly, which requires at most $\text{poly}(k) = T^{o(\epsilon)}$ operations since the

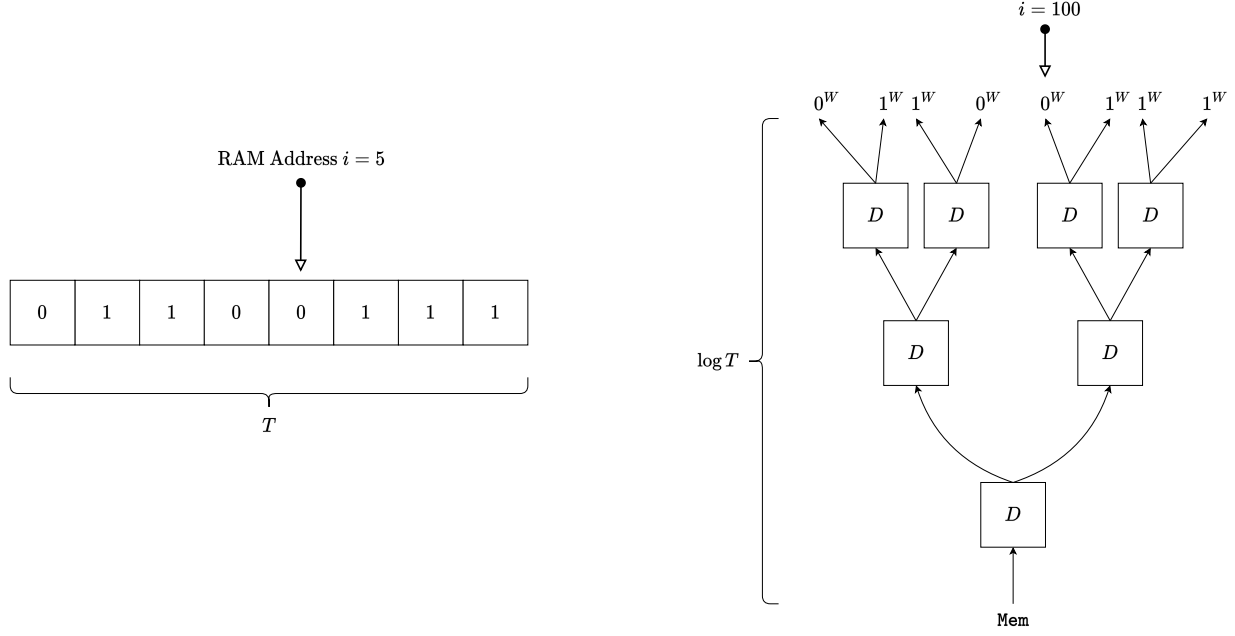


Figure 3.6: Simulating a RAM memory with a J-tree. While the RAM memory on the left requires T bits to store explicitly, the virtual RAM on the right can be stored with $|\text{Mem}| = W = T^{O(\epsilon)}$ bits.

linear tapes have length $O(k)$.

- (b) Next, say the rule involves a RAM operation. In this case we start by reading the entire contents of the addressing tape, which we denote $i \in \{0, 1\}^k$. Now, if the operation is a Load, we compute the first bit of $D^k[\text{Mem}](i)$ and update the auxiliary tape at its current tapehead position to this value. If the operation is an Update, we read the value at the current position of the auxiliary tape, call it $s \in \{0, 1\}$. We then compute $\text{Find}(\text{Mem}, i, s^W, C, D)$. If this procedure fails and returns an incompressible string, S_ϵ aborts its entire simulation and rejects its input. Otherwise, we update Mem to the value returned by this Find call.

If we get through all T phases without halting, we reject the input.

We now show that S_ϵ operates within the required resource bounds. First, we note that all oracle calls are of length $W = T^{O(\epsilon)}$. Second, it is clear that the Find/Initialize operations and the evaluations of $D^k[\text{Mem}]$ can be carried out in $\text{poly}(W)$ space, since in particular they require at most $\text{poly}(W)$ time by Lemmas 3.28, 3.30, and 3.31. So the space used within a phase is at

most $\text{poly}(W) = T^{O(\epsilon)}$ and the size of oracle calls are bounded identically. To bound the time complexity, we note that by the same arguments each phase can be completed in time $T^{O(\epsilon)}$, and overall there are T phases, so the total time complexity is $T^{1+O(\epsilon)}$. In the above description of each phase, we were informal about the number of work tapes required to carry out these computations. However, since any multi-tape machine running in space S and time T can be simulated on a one-tape machine in time $\text{poly}(S, T)$ and space $\text{poly}(S)$, we see that we can modify the algorithm within each phase to operate on a 1-tape machine with at most a polynomial blowup in space and time. So the above bounds still hold on a 1-tape machine, where the space is bounded by $\text{poly}(T^\epsilon) = T^{O(\epsilon)}$, and the time within each phase is bounded identically.

Step 2 - If Find Never Fails Then the Simulator Works

We now show that if $\mathcal{S}_\epsilon$ completes its computation on an input x without any Initialize or Find operation failing, then $\mathcal{S}_\epsilon$ accepts x if and only if $x \in L$. To prove this, we show by induction on $t \in [T]$ that at the beginning of the t^{th} phase of $\mathcal{S}_\epsilon$'s simulation on x , $\mathcal{S}_\epsilon$'s simulated configuration matches the configuration of $\mathcal{M}$ on input x at the beginning of time step t . This is clearly true for $t = 1$, since at the start of the simulation we initialize explicit representations of $\mathcal{M}$'s linear tapes to all zeroes, and do the same for the RAM memory using the Initialize operation, which succeeds by assumption.

Now assume the inductive hypothesis up to step t . So at the beginning of this time step on input x , $\mathcal{M}$'s linear tapes, tapehead pointers, machine state and RAM memory at the start of time step t are faithfully represented by $\mathcal{S}_\epsilon$ at the start of phase t . Thus, $\mathcal{S}_\epsilon$ is able to choose the correct transition rule to apply during this phase. It then updates the linear tapes, tapehead pointers, and machine state accordingly; all of these are stored explicitly so $\mathcal{S}_\epsilon$ has no potential for failure here. Next, if $\mathcal{M}$ uses a Load operation at this step, since we assumed the previous state of the virtual RAM is accurate, the correct value will be found by $\mathcal{S}_\epsilon$. Finally, if $\mathcal{M}$ uses a Update operation at this step, $\mathcal{S}_\epsilon$ makes the associated call to the Find procedure. By Lemma 3.30, if this does not fail, then the state of the virtual RAM after this step will accurately represent $\mathcal{M}$'s ram at the end of

time step t . This completes the inductive case.

Step 3 - Failure of the Simulator Witnesses the Pigeonhole Principle for $C, \mathcal{D}$

We now show that if the above simulation S_ϵ fails to decide L for infinitely many inputs, then there is an algorithm which, given 1^n , outputs an incompressible string for C_n, D_n in polynomial time using oracles for $C, \mathcal{D}$, for infinitely many n .

In particular, assume that there is an infinite set $R \subseteq \mathbb{N}$ such that for all $n \in R$, S makes a mistake on some length n input in its attempt to decide L . Recall that S_ϵ uses the compression scheme $C_{\lceil 2^{\epsilon n} \rceil}, D_{\lceil 2^{\epsilon n} \rceil}$ in its simulation on inputs of length n . Now, let $\mathbb{I} = \{\lceil 2^{\epsilon n} \rceil \mid n \in R\}$. We will give a construction algorithm that succeeds for all input lengths in $\mathbb{I}$.

Given an input 1^m , our construction algorithm $\mathcal{H}$ operates as follows. It starts by computing an n such that $\lceil 2^{\epsilon n} \rceil = m$; by construction we see $n = O(\log m)$. Next, $\mathcal{H}$ runs the simulation S_ϵ on all inputs of length n one after the other; by construction we see that S_ϵ uses the same compression scheme C_m, D_m on all such inputs. During each simulation, if S_ϵ fails on some Find or Initialize operation and returns an incompressible string for C_m, D_m , $\mathcal{H}$ halts and outputs that string. If $\mathcal{H}$ gets through all inputs of length n without any operation failing, it outputs something arbitrary, say 1^m . By definition we see that when $m \in \mathbb{I}$, $\mathcal{H}$ cannot get through all simulations without S_ϵ failing, since by definition of $\mathbb{I}$ we know that S_ϵ fails to compute L on some input of length n , and by the previous section we know this can only happen when S_ϵ fails on some Find/Initialize operation. So overall we have that $\mathcal{H}$ runs in $\text{poly}(2^n) = 2^{O(n)} = 2^{O(\log m)} = \text{poly}(m)$ time, and successfully finds an incompressible string for C_m, D_m on input 1^m for infinitely many m .

Step 4 - Wrapping Things Up

We now have that for any language $L \in \text{RAM-TIME}[T(n)]$ and every $\epsilon > 0$, there is a machine S_ϵ running in time $T(n)^{1+O(\epsilon)}$, space $T^{O(\epsilon)}$, and making $C, \mathcal{D}$ oracle calls of length $T^{O(\epsilon)}$, such that one of the following holds:

1. S_ϵ correctly decides L on all but finitely many inputs.

2. There is a polynomial time construction algorithm which finds incompressible strings for C_n, D_n given oracles for $C, \mathcal{D}$, which works for infinitely many inputs.

Clearly if the first possibility holds then we can get a simulation of the same complexity which decides L exactly. In addition, if the first possibility holds for all $\epsilon > 0$, then we can replace the $O(\epsilon)$ terms with ϵ as we take ϵ to zero. This yields the stated theorem. $\square$

We now prove a variant of the above theorem, which allows us to replace the oracle for the compressor C in our low-space simulation with nondeterminism. This will be relevant for schemes $C, \mathcal{D}$ where C is significantly harder to compute than $\mathcal{D}$.

Theorem 3.34. *Let T be an exponential time bound, and let $C, \mathcal{D}$ be a uniform compression scheme.*

Then one of the following must hold:

1. *There is polynomial time algorithm with oracle access to $C, \mathcal{D}$ which, for infinitely many n , outputs an incompressible string for C_n, D_n on input 1^n .*
2. *For every language $L \in \text{RAM-TIME}[T(n)]$ and every $\epsilon > 0$, there is nondeterministic 1-tape Turing machine with oracle access to $\mathcal{D}$ which decides L in time $T(n)^{1+\epsilon}$, uses space at most $T(n)^\epsilon$, and makes at most $T(n)^\epsilon$ nondeterministic guesses on all computation paths.*

Proof. We follow the same simulation S_ϵ of some language L in $\text{RAM-TIME}[T(n)]$, with a slight modification. The key observation is the following: the procedure Find is proven to work with access to both C and $\mathcal{D}$ oracles by Lemma 3.30. However, recall that Lemma 3.30 also defines separate procedure Verify, which is able to verify that a certain J-tree seed is the updated form of another, and this verification only needs the $\mathcal{D}$ oracle. Recall also the key property relating Find and Verify, which says that Verify will accept any successful output of a Find operation.

Thus, if we make our simulator S_ϵ nondeterministic, it can replace the Find operation by a non-deterministic guess as to the new value of the seed Mem during each phase, and then use Verify to check that this guess is correct, which requires only the $\mathcal{D}$ oracle. This immediately gives us a low

space nondeterministic simulator $\mathcal{S}_\epsilon$ with similar properties as that in the proof of Theorem 3.33. However, the total nondeterminism used by this simulation is $T(n)^{1+O(\epsilon)}$, since it has to guess a seed of length $T^{O(\epsilon)}$ during each of the T phases.

To get away with $T^{O(\epsilon)}$ bits of nondeterminism, we use the following trick⁴. At the beginning of $\mathcal{S}_\epsilon$'s simulation, it guesses a single seed for a separate J-tree of the same code length, call this seed Up . Now, during phase t , to simulate an Update operation which sets RAM cell $i \in \{0, 1\}^k$ to value $s \in \{0, 1\}$, we first set $\text{Mem}' = D^k[\text{Up}](t)$, and then check if $\text{Verify}(\text{Mem}, \text{Mem}', i, s^W, D)$ holds. If so we then set $\text{Mem} = \text{Mem}'$ and continue to the next phase, and if not then we halt the simulation and reject outright. It is straightforward to see that this simulation runs in time $T(n)^{1+O(\epsilon)}$, uses space $T(n)^{O(\epsilon)}$, and guesses at most $T(n)^{O(\epsilon)}$ nondeterministic bits.

By the same arguments as in Theorem 3.33, we have that if every Verify call accepts during $\mathcal{S}_\epsilon$'s computation on input x , then $\mathcal{S}_\epsilon$ correctly decides if $x \in L$. To finish the proof, it suffices to show that given some x such that $\mathcal{S}_\epsilon$ fails to decide if $x \in L$, we can construct an incompressible string for $C_{2^{\epsilon n}}, D_{2^{\epsilon n}}$ in $\text{poly}(T(n))$ time with C, D oracles; from here the theorem follows directly using the arguments at the end of the proof of Theorem 3.33.

So, say we have such an x . Since our simulation errs on the side of rejecting, we know that $x \in L$ but $\mathcal{S}_\epsilon$ rejects x . We start by running the *deterministic* low space simulation from Theorem 3.33 (corresponding to L, ϵ) on x , which can be done in $\text{poly}(T(n))$ time with C, D oracles. During this simulation, we keep track of all updated memory seeds constructed using Initialize and Find . If at any step a Find or Initialize procedure fails, we find an incompressible string and output it. Otherwise, we have found a sequence of seeds $\text{Mem}_1, \dots, \text{Mem}_T \in \{0, 1\}^W$ (where W is the code length used by the simulation), such that Mem_{t+1} represents a correct update to the RAM memory previously represented by Mem_t after time step t . Now, we use the Iter-Compress procedure to construct a seed $\text{Up} \in \{0, 1\}^W$ such that for all $t \in \{0, 1\}^k$, $D^k[\text{Up}](t) = \text{Mem}_t$. By Lemma 3.32, in $\text{poly}(T(n))$ time we can either find such a seed Up , or else find an incompressible string for our scheme. But if such a string Up exists, this is precisely the nondeterministic guess which would

⁴This trick is essentially the “easy witness method” of [92].

cause our nondeterministic simulation $\mathcal{S}_\epsilon$ to accept x . So by assumption that it rejects x , if we get to this point then Iter-Compress must find an incompressible string. $\square$

Finally, we show that if the explicit construction algorithm in the above theorem is also granted access to an **NP** oracle, we can get the same result, but where we simulate languages in the larger class $\text{NTIME}[T(n)]$.

Theorem 3.35. *Let T be an exponential time bound, and let $\mathcal{C}, \mathcal{D}$ be a uniform compression scheme.*

Then one of the following must hold:

1. *There is polynomial time algorithm with oracle access to $\mathcal{C}, \mathcal{D}$, and SAT which, for infinitely many n , outputs an incompressible string for $\mathcal{C}_n, \mathcal{D}_n$ on input 1^n .*
2. *For every language $L \in \text{NTIME}[T(n)]$ and every $\epsilon > 0$, there is nondeterministic 1-tape Turing machine with oracle access to $\mathcal{D}$ which decides L in time $T(n)^{1+\epsilon}$, uses space at most $T(n)^\epsilon$, and makes at most $T(n)^\epsilon$ nondeterministic guesses on all computation paths.*

Proof. This will follow quite directly from the proof of the previous theorem. Let $\mathcal{M}$ be some $\text{NTIME}[T(n)]$ machine which we are attempting to simulate. At the outset of its computation, in addition to guessing one seed Up which encodes a sequence of RAM-seed updates, our new simulator $\mathcal{S}_\epsilon$ also guesses a second seed Wit which will encode the nondeterministic choices made by $\mathcal{M}$ during its computation. In particular, after guessing this seed, at each phase t of our simulation the simulator reads the first $O(1)$ bits of $D^k[\text{Wit}](t)$, and uses this to determine which transition rule to apply at that step (in a nondeterministic machine there can be $O(1)$ such rules, which will be smaller than the code length of the decompressor for sufficiently large input lengths). Otherwise we run the simulation exactly as in the above proof.

By the same reasoning as in the previous proof, we see that if this machine accepts an input x then $x \in L$ (since the simulation errs on the side of rejection), but it is possible that $x \in L$ and the simulation fails to determine this. In such a case, the only possibility is that for all sequences $z \in$

$\{0, 1\}^{O(T)}$ of nondeterministic guesses causing $\mathcal{M}$ to accept x , there is no pair $\text{Up}, \text{Wit} \in \{0, 1\}^W$ such that Wit encodes z (as described above), and Up encodes a sequence of valid RAM seeds representing the deterministic computation of $\mathcal{M}$ on x with witness z . Thus, if we have some input x on which our simulation fails, we can use an NP oracle to compute a witness z causing $\mathcal{M}$ to accept x . We then run the deterministic low space simulation $\mathcal{M}$ on x with witness z , and either find an incompressible string for $C, \mathcal{D}$, or else find a sequence of valid RAM seeds $\text{Mem}_1, \dots, \text{Mem}_T$ for this computation. We then proceed as in the previous Theorem, attempting to compress the Mem_t to a single seed Up and the bits of z to another seed Wit , both using the Iter-Compress procedure. By assumption that our main nondeterministic simulation rejected x , if we get to this point we know that one of these Iter-Compress procedures must return an incompressible string. $\square$

3.5.2 Implications

Consider the following three hypotheses:

Hypothesis 3.36. *There exists some exponential time bound T and some $\epsilon > 0$ such that:*

$$\text{RAM-TIME}[T(n)] \not\subseteq 1\text{-TISP}[T(n)^{1+\epsilon}, T(n)^\epsilon]$$

Hypothesis 3.37. *There exists some exponential time bound T and some $\epsilon > 0$ such that:*

$$\text{NTIME}[T(n)] \not\subseteq 1\text{-NTISPG}[T(n)^{1+\epsilon}, T(n)^\epsilon, T(n)^\epsilon]$$

Hypothesis 3.38. *There exists some exponential time bound T and some $\epsilon > 0$ such that:*

$$\text{RAM-TIME}[T(n)] \not\subseteq 1\text{-NTISPG}[T(n)^{1+\epsilon}, T(n)^\epsilon, T(n)^\epsilon]$$

The first Hypothesis asserts that deterministic exponential time RAM computations cannot be simulated on 1-tape machines using low space and near-linear blowup in time. The second Hypothesis asserts roughly the same for nondeterministic computations, claiming that such a simulation

cannot occur which simultaneously uses a small amount of nondeterminism. Recall that for non-deterministic time, the multi-tape and RAM models are roughly equivalent, which is why we don't make a distinction here on the left-hand side. Finally, the third hypothesis says that deterministic RAM computations cannot be recognized by machines with short “proofs” verifiable in low space and near-linear time on a 1-tape machine.

While 3.36 and 3.37 seem incomparable and both quite reasonable, we see that 3.38 is formally stronger than the previous two, and we have significantly less intuition as to whether or not it should be true. However, recall from Section 3.1.3 that when the time bound T is linear, all three of these hypotheses are known to hold unconditionally (indeed they hold for *all* $\epsilon < 1$). In any case, the results of the previous section give us the following:

Theorem 3.39. *If Hypothesis 3.36 holds, then for any uniform compression scheme $C, \mathcal{D}$ where both C and $\mathcal{D}$ are computable in polynomial time, there is a polynomial time algorithm which, given 1^n , prints an incompressible string for C_n, D_n for infinitely many n .*

Proof. This follows from Theorem 3.33; since the C and $\mathcal{D}$ oracle calls are of length $T(n)^{O(\epsilon)}$, if both C and $\mathcal{D}$ are computable in polynomial time then the oracles can be abandoned without effecting the resource bounds of the simulation. $\square$

For Hypothesis 2, we have:

Theorem 3.40. *If Hypothesis 3.37 holds, then for any uniform decompressor $\mathcal{D}$ computable in polynomial time, there exists there is a polynomial time **NP**-oracle algorithm which, given 1^n , prints an empty pigeonhole for D_n for infinitely many n . In particular, $E^{\text{NP}} \not\subseteq \text{size}[2^n/2n]$.*

Proof. This follows from Theorem 3.35. As noted in Section 3.3, any polynomial time decompressor $\mathcal{D}$ has a canonical “proper compressor” C computable with an **NP** oracle, such that the incompressible strings for this scheme are empty pigeonholes for $\mathcal{D}$. The theorem then follows from the above proof, where we can abandon the $\mathcal{D}$ oracle in our simulation if $\mathcal{D}$ is computable in polynomial time since the oracle calls we make are small. $\square$

Finally, the strongest hypothesis gives us the following:

Theorem 3.41. *If Hypothesis 3.38 holds, then for any uniform compression scheme $C, \mathcal{D}$ where $\mathcal{D}$ is computable in polynomial time, there is a polynomial time algorithm using a C oracle which, given 1^n , prints an incompressible string for C_n, D_n for infinitely many n .*

Proof. This follows from Theorem 3.34, where again we use the fact that $\mathcal{D}$ is computable in polynomial time to eliminate the $\mathcal{D}$ oracle. $\square$

To illustrate the use of these theorems, we first consider their implications for the compression schemes related to non-uniform complexity measures described in Section 3.3.2.1. In these cases, we have some complexity measure on strings/truth tables, such as circuit complexity, formula size, or K^{poly} complexity, and wish to construct strings/truth tables of high complexity (for infinitely many input lengths), which we call the “construction problem.” In each case there is an associated “compression problem” in FNP, where we are given a string/truth table and wish to find a small circuit/formula/program computing it if one exists. For these sorts of problems, we now have the following:

Corollary 3.42. *For each of the above mentioned uniform compression schemes related to non-uniform complexity measures, we have:*

1. *If Hypothesis 3.36 holds, then an efficient algorithm for the compression problem implies an efficient algorithm for the construction problem.*
2. *If Hypothesis 3.37 holds, then there is an efficient NP-oracle algorithm for the construction problem.*
3. *If Hypothesis 3.38 holds, then there is an efficient algorithm for the construction problem using an arbitrary oracle for the compression problem. In other words, the construction problem reduces to the compression problem.*

Proof. The first and second implications follow directly from the above theorems. For the last implication, there is a slight subtlety related to the fact that the compression problem can generally

have many valid solutions per instance, and so an oracle for this problem must be defined according to our notion of “functional oracles” (defined in Section 3.2.3). Thus, for a particular compression problem we have the following dichotomy for every one of its function oracles O : either the compression scheme defined by O returns solutions that can be used as witnesses in the low space simulation for all but finitely many inputs, or else there is an explicit construction algorithm with access to O which prints strings of high complexity. So assuming Hypothesis 3.38, every O must fail to help the simulation for infinitely many inputs. Thus, given access to an arbitrary functional oracle for the compression problem, the explicit construction algorithm will work for infinitely many input lengths, although the choice of functional oracle might affect which infinite set of input lengths the algorithm works on. $\square$

We see here that the conclusion of the third implication implies the conclusion of the previous two. As noted in the introduction, the second implication can be proved by simpler techniques, utilizing the “Easy Witness Lemma.” We give an alternate proof using this method in Appendix 3.8 for the interested reader. However, for the first and third implication, the J-tree update lemma seems necessary.

The case of large prime construction does not quite fit in with the others, as both the compressor and decompressor given in Theorem 3.24 require a factoring oracle. In addition, the second implication of Corollary 3.42 above is trivial when applied to the construction of large primes, since primality testing is in P [3]. However we can still derive the following from Theorem 3.39:

Corollary 3.43. *If Hypothesis 3.36 holds, then a polynomial time algorithm for factoring implies a polynomial time algorithm to construct $16n$ -bit primes of magnitude $> 2^n$ (for infinitely many n).*

More generally we can conclude the following directly from Theorem 3.33:

Corollary 3.44. *One of the following is true:*

1. *For every exponential time bound T and every $\epsilon > 0$, every language decidable in time $T(n)$ on a RAM machine can be decided in time $T(n)^{1+\epsilon}$ and space $T(n)^\epsilon$ by a 1-tape machine with a factoring oracle, which makes oracle calls of length at most $T(n)^\epsilon$.*

2. *There is a polynomial time algorithm with a factoring oracle that generates $32n$ -bit primes of magnitude $> 2^n$ for infinitely many n .*

3.6 BPP, TFNP, and the Weak Pigeonhole Principle

Throughout this paper we have studied the problem of finding incompressible strings for *uniform* compression schemes. As mentioned in Section 3.3, it would also be natural to study the more general search problem, where a compression scheme of a fixed message/code length is given as input in the form of a boolean circuit:

Definition 3.45. *LOSSY CODE is the following problem: given circuits $C: \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$ and $D: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, find some x such that $D(C(x)) \neq x$.*

It follows from the definition that LOSSY CODE lies in TFNP (more specifically the class PPP defined in [134]), and reduces to the problem EMPTY studied in [97] and [101]. Further, we have seen that LOSSY CODE admits a randomized algorithm that outputs a solution with high probability. Since solutions can be verified efficiently, this fits it into a family of search problems studied by Goldreich [59], whose results imply the following:

Lemma 3.46. *LOSSY CODE is polynomial time Turing reducible to CAPP.*

Here, CAPP denotes the canonical complete problem for prBPP, where we are given as input a circuit $E: \{0, 1\}^n \rightarrow \{0, 1\}$, and must output an approximation of $Pr_x[E(x) = 1]$ that is accurate to within $\pm \frac{1}{6}$. A natural question is whether the converse holds, i.e. whether CAPP reduces deterministically to LOSSY CODE. As hinted towards in Section 3.3.2, a polynomial time algorithm for CIRCUIT SYNTHESIS would imply such a reduction, but an unconditional reduction would be a major breakthrough as it would place $BPP \subseteq NP$. We show here that if the compressor C in LOSSY CODE is allowed to be randomized, the associated total search problem is in fact Turing-equivalent to CAPP, and thus complete for prBPP. The problem is defined formally as follows:

Definition 3.47. *R-LOSSY CODE is the following problem: given $C: \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^{n-1}$, $D: \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$, find some $x \in \{0, 1\}^n$ such that $Pr_r[D(C(x, r)) = x] < \frac{1}{2}$.*

To clarify, this is precisely the same problem as LOSSY CODE, except that the “compressor circuit” C now uses some random coins. In this case, we seek a string which has a $< \frac{1}{2}$ probability of being recoverable from its description (where the probability is taken over the random coins of the compressor C). Just like LOSSY CODE this search problem is total, but verifying a solution is no longer in P . Indeed, it may be PP -hard to determine if any particular string is a solution, but there is an efficient randomized verification procedure that accepts only valid solutions, and accepts a random solution with high probability.

We now show the following:

Theorem 3.48. *R-LOSSY CODE and CAPP are Turing reducible to one another in deterministic polynomial time.*

Proof. A reduction from R-LOSSY CODE to CAPP follows from [59]. To see this, note that a random $x \in \{0, 1\}^n$ is an empty pigeonhole for D with probability $\geq \frac{1}{2}$, and whenever x is an empty pigeonhole we have $Pr_r[D(C(x, r)) = x] = 0$. In contrast, when x is a non-solution we have $Pr_r[D(C(x, r)) = x] \geq \frac{1}{2}$. So distinguishing empty pigeonholes from non-solutions is a problem in $prBPP$ (indeed it lies in $prRP$), there is an efficient randomized algorithm which generates empty pigeonholes with high probability (namely the trivial algorithm that outputs a random x), and any empty pigeonhole is a solution. This problem therefore falls into the category of search problems which were proven reducible to CAPP in [59].

To get a reduction in the other direction, we rely on Yao’s next-bit predictor lemma [173]. In particular, say we are given a circuit $E: \{0, 1\}^n \rightarrow \{0, 1\}$ whose acceptance probability we wish to estimate to within an additive error of $\frac{1}{6}$. By Yao’s lemma, for any sufficiently large table of n -bit strings $A \in \{0, 1\}^{m \times n}$ we have that if E ’s acceptance probability over the rows of A differs from its true acceptance probability over $\{0, 1\}^n$ by more than $\frac{1}{6}$, then there is a simple “predictor circuit” which uses E to derive one of the columns of A from its remaining columns. In particular, this argument shows that there is some small advice of length $< mn$ from which we can reconstruct A (whose original bit-length is mn) using E . This immediately implies a reduction to EMPTY as described in [101]. The key observation we make here (which is also noted in several other works

on the subject, notably [25] and [72]) is that the “compressed representation” of A described in this argument can indeed be *found efficiently given E* using randomness. This will imply a reduction of CAPP to R-LOSSY CODE, since any table A which cannot be recovered from this randomized compression procedure will then give a good approximation to E ’s acceptance probability.

More formally, given E as above we will construct an instance of R-LOSSY CODE with message length n^5 and code length $n^5 - 1$. The compressor circuit $C: \{0, 1\}^{n^5} \times \{0, 1\}^{\text{poly}(n)} \rightarrow \{0, 1\}^{n^5-1}$ interprets its n^5 -bit input as an $n^4 \times n$ matrix. C then iterates over all choices $i \in [n], b \in \{0, 1\}$ of a “hybrid index” and “bias.” For each such choice, let $x_1^{<i}, \dots, x_m^{<i}$ denote the rows of A truncated to their first $i-1$ bits. Our compressor now selects a random $z \in \{0, 1\}^{n-i+1}$, and attempts to predict the i^{th} column of A from the preceding columns as follows: it predicts the j^{th} value of the i^{th} column of A to be the first bit of z if $E(x_j^{<i}z) = b$, or else the negation of this first bit if $E(x_j^{<i}z) \neq b$. Given a particular selection of i, b and z , the compressor then evaluates the accuracy of this prediction, by counting the number of correctly guessed entries of the j^{th} column of A . If it guesses fewer than $\frac{n^4}{2} + \frac{n^3}{12}$ correctly, C aborts and outputs something arbitrary. If C guesses at least $\frac{n^4}{2} + \frac{n^3}{12}$ correctly, it then sets $q \in \{0, 1\}^{n^4}$ equal to the bit-wise XOR of the correct column and the predicted column. Since in this case q will be an n^4 -bit string with weight at most $\frac{n^4}{2} - \frac{n^3}{12}$, we can then encode q in $n^4 - \Omega(n^2)$ bits using a standard sparse-string encoding scheme [101]. Finally, the decompressor outputs an $n^5 - 1$ -bit string describing A with its i^{th} column removed, b, i, z , and the encoding of q ; it can be readily verified that $n^5 - 1$ (indeed $n^5 - \Omega(n^2)$) bits suffice to specify all of these.

It is straightforward to show that if the compressor successfully finds such an encoding, A can be recovered efficiently (and deterministically) from this encoding. In particular we can construct a decompressor $D: \{0, 1\}^{n^5-1} \rightarrow \{0, 1\}^{n^5}$ which takes an encoding of i, b, z, q , and A with a missing column, uses E, i, b, z to predict the missing column and insert it into the correct position, and then uses q to correct the errors in this prediction. On the other hand, by Yao’s lemma, whenever the acceptance probability of E over the rows of A differs from its acceptance probability over $\{0, 1\}^n$ by $> \frac{1}{6}$, there exists some choice of i, b such that a random z will give a successful encoding with

nonnegligible ($\geq \frac{1}{\text{poly}(n)}$) probability⁵. By repeatedly trying different random values z for each i, b until a successful encoding is found we can amplify this probability to $\geq \frac{1}{2}$. Thus we have that any solution to this instance of R-LOSSY CODE is a table of n -bit strings which can be used to approximate the acceptance probability of E to within $\frac{1}{6}$, completing the proof. $\square$

An interesting takeaway from the above proof is the following: choosing a good fixing of the “leftover bits” in Yao’s lemma is in some sense a *universal* probabilistic search problem, since derandomizing this step would give a reduction from prBPP to LOSSY CODE $\in$ TFNP (and in particular would imply $\text{BPP} \subseteq \text{NP}$). More formally:

Corollary 3.49. *Consider the following promise problem: we are given a circuit $E: \{0, 1\}^n \rightarrow \{0, 1\}$, a matrix $A \in \{0, 1\}^{m \times n}$, and an index $i \in [n]$, such that A fails Yao’s next-bit test with respect to E at index i with bias $\epsilon = \text{poly}(\frac{1}{m})$, and we must produce some $z \in \{0, 1\}^{n-i}$ such that fixing the last $n-i$ bits of E to z preserves a non-negligible bias for this next-bit test. If this problem can be solved in deterministic polynomial time, then prBPP reduces to LOSSY CODE.*

This follows directly from the above proof, since the only randomness the compressor uses is its random choice of these leftover bits.

3.7 Succinct Representation of Primes

We give here a proof of a generalization of Lemma 3.25.

Lemma. *Let $c \in \mathbb{N}$. For sufficiently large n there exists a pair of polynomial time computable maps $g: \{0, 1\}^n \rightarrow \{0, 1\}^{n-c}$, $f: \{0, 1\}^{n-c} \rightarrow \{0, 1\}^n$, such that for all n -bit primes p we have $f(g(p)) = p$.*

Proof. Let $p_1, \dots, p_k$ be the first k primes, and consider the probability that a uniformly random

⁵Most proofs of Yao’s Lemma only show the *existence* of a such a z , but a standard Markov argument shows that a random choice will suffice, as demonstrated in [72].

tuple in $\mathbb{Z}/p_1\mathbb{Z} \times \cdots \times \mathbb{Z}/p_k\mathbb{Z}$ has no zero-entry. This probability is:

$$\prod_{j=1}^k \frac{p_j - 1}{p_j}$$

We see that this probability approaches zero for large k , so we now fix some k such that this probability is at most 2^{-2c} . Let $M = \prod_{j \leq k} p_j$. By the Chinese remainder theorem, this implies the existence of a set $S \subseteq [M]$ of size at most $2^{-2c}M$, such that any integer x which is not divisible by any of $p_1, \dots, p_k$ can only be congruent to a value in S modulo $\mathbb{Z}/M\mathbb{Z}$. So in particular, any prime $p > p_k$ can only take on this small set of values modulo M . This immediately gives the required encoding of n bit primes larger than p_k for sufficiently large n : we simply divide by M which saves $\lfloor \log M \rfloor$ bits, and encode the remainder using $\lceil \log(2^{-2c}M) \rceil \leq \lfloor \log M \rfloor - 2c + 1$ bits, so overall we can use at most $n - c - 1$ bits to encode an n bit prime greater than p_k , provided $n \gg M$ (recall that M is a fixed constant). This encoding is not quite good enough, as it does not work for the primes $p_1, \dots, p_k$. However by adding a single bit to the encoding scheme we can trivially extend to it encode these $k = O(1)$ values, completing the proof. $\square$

3.8 Alternate Proof of Nondeterministic Low-Space Simulation

We give here an alternative proof of the following theorem using the well-known “Easy Witness Lemma:”

Theorem. *If, for some exponential time bound T and $\epsilon > 0$ we have:*

$$\text{NTIME}[T(n)] \not\subseteq \text{NTISP}[T(n)^{1+\epsilon}, T(n)^\epsilon, T(n)^\epsilon]$$

then $\text{E}^{\text{NP}} \not\subseteq \text{size}[2^n/2n]$.

We start by stating the “Easy Witness Lemma” for E^{NP} :

Lemma 3.50. [82] *If $\text{E}^{\text{NP}} \subseteq \text{size}[f(n)]$, then for any exponential time nondeterministic turing machine $\mathcal{M}$ and every input x that $\mathcal{M}$ accepts, there is a circuit of size $O(f(|x|)^k)$ (for some fixed*

universal constant k) whose truth table is a sequence of nondeterministic bits causing $\mathcal{M}$ to accept x .

The only other tool we need is the NP-completeness of 3-SAT under hyper-efficient reductions due to [51], which can be readily used to show the following:

Theorem 3.51. *Let $L \in \text{NTIME}[T(n)]$. There is a nondeterministic machine $\mathcal{M}$ deciding L in time $T'(n) = \tilde{O}(T(n))$ of the following special form. $\mathcal{M}$ guesses $T'(n)$ nondeterministic bits on a separate read-only random-access “guess tape” at the beginning of its computation in one step. $\mathcal{M}$ then runs in time $T'(n)$ and uses space at most $\tilde{O}(\log T(n) + n)$ on its work tapes and accepts or rejects. As usual, we say $\mathcal{M}$ accepts an input x if there is some initial guess tape configuration causing it to accept, and otherwise we say that it rejects x . We do not count the guess tape towards the space usage of $\mathcal{M}$.*

Proof. The stated simulator simply iterates over each clause of a 3-SAT instance produced by applying the efficient 3-SAT reduction from L . For each clause, the verifier confirms that the clause is satisfied by checking the variable assignments on the random access guess tape. Due to the efficiency/high degree of uniformity in the 3-SAT reduction of [51], it is straightforward to confirm that this verifier runs in the stated time and space bounds. $\square$

Combining this with the easy witness lemmas we get:

Theorem 3.52. *There is a universal constant k such that if $\text{E}^{\text{NP}} \subseteq \text{size}[f(n)]$, then for all exponential time bounds $T(n)$ we have:*

$$\text{NTIME}[T(n)] \subseteq \text{NTISPG}[T(n)f(n)^k, f(n)^k, f(n)^k]$$

Proof. Let L be in $\text{NTIME}[T(n)]$, and let $\mathcal{M}$ be the special machine deciding L which is given by Theorem 3.51. Note that although this machine is defined in a nonstandard way (guessing $\tilde{O}(T(n))$ bits in one step on a random access tape), it can be simulated by a standard multi-tape nondeterministic machine $\mathcal{M}'$ in $\text{poly}(T(n)) = 2^{O(n)}$ time in such a way that there is a one-to-one

correspondence between the nondeterministic guesses causing $\mathcal{M}$ and $\mathcal{M}'$ to accept on any given input $x \in L$. In particular, the machine $\mathcal{M}'$ simply guesses $\tilde{O}(T(n))$ bits one at a time and stores them on one of its tapes, and then simulates random access to these bits during the verification by walking along the tape, incurring a time overhead of $\tilde{O}(T(n))$. Now by the Δ_2 Easy Witness Lemma, assuming $E^{\text{NP}} \subseteq \text{size}[f(n)]$ we have that for any $x \in L$, there is a circuit of size $f(n)^k$ (for a fixed universal constant k) whose truth table is a sequence of guesses causing $\mathcal{M}'$ to accept x , and by the above relation between $\mathcal{M}$ and $\mathcal{M}'$ this truth table must also cause $\mathcal{M}$ to accept x .

We can then simulate $\mathcal{M}$ in low space and low nondeterminism as follows: at the start, instead of guessing a full random access tape of nondeterministic bits, we guess a circuit C of size $f(n)^k$ encoding a potential witness, which requires $\tilde{O}(f(n))$ nondeterministic bits. We then run $\mathcal{M}$'s random access verification procedure, replacing unit cost accesses to its random access guess tape with evaluations of the circuit C . The time overhead to simulate each step of $\mathcal{M}$'s verification is at most $\text{poly}(f(n)^k)$ (to evaluate C on a requested index), and the additional space usage is $\tilde{O}(f(n))$ to store C (on top of the original space usage of the verifier which is $\tilde{O}(\log T(n) + n)$). Since T is an exponential time bound and $f(n) \geq n$, overall this simulation runs in time $T(n)\text{poly}(f(n))$ and uses space and nondeterminism at most $\text{poly}(f(n))$, and by the easy witness lemma this simulation must correctly decide the language L . $\square$

The main theorem is then obtained by combining the above with the following amplification lemma for E^{NP} :

Lemma 3.53. *If $E^{\text{NP}} \subseteq \text{size}[2^n/2n]$, then $E^{\text{NP}} \subseteq \bigcap_{\epsilon > 0} \text{size}[2^{\epsilon n}]$.*

Recall that an almost identical theorem is proven in Chapter 1 for the case where both inclusions are of the “infinitely often” type. We give a proof of this version below for completeness.

Proof. We prove this statement by contrapositive. Say there exists a language $L \in E^{\text{NP}}$ and a constant $\epsilon > 0$ such that for infinitely many $n \in \mathbb{N}$, L^n has circuit complexity $2^{\epsilon n}$. By assumption L can be computed in time 2^{cn} with an NP oracle for some fixed $c \in \mathbb{N}$. Thus, given 1^n , we can also compute the truth table of L^n , which has length 2^n , in time $2^n 2^{cn} = 2^{(c+1)n}$ with an NP oracle.

We now define a language $H \in E^{NP}$ such that $H \notin \text{size}[2^n/2n]$. To compute H on inputs of length n , we start by constructing a circuit D with the following properties:

1. D has $W = 2^n - 1$ inputs and $2W$ outputs, and has size $O(W^3) = O(2^{3n})$.
2. Given a string outside the range of D , we can find an 2^n -bit truth table which cannot be computed by a circuit of size $2^n/2n$ in $\text{poly}(2^n)$ time with an NP oracle.

The existence of a such a D can follow directly from combining Lemmas 3.18 and 3.20. Now, for each $m \in \{\lceil \frac{6}{\epsilon} \rceil n, \dots, \lceil \frac{6}{\epsilon} \rceil (n+1)\}$, we do the following:

1. Compute the truth table of L^m , which takes time $2^{(c+1)m}$ with an NP oracle.
2. Attempt to find a seed $y \in \{0, 1\}^W$ such that for all $i \in \{0, 1\}^m$, $D^m[y](i) = b_i^W$ where b_i denotes the i^{th} bit of L^m . This can be accomplished using the Iter-Compress algorithm. By Lemma 3.32, in $\text{poly}(|D|m) = O(2^{3n}n)$ time this procedure will either find such a y , or else find an empty pigeonhole e for D . By construction of D , if we can find one of its empty pigeonholes this in turn allows us to construct a truth table of length 2^n and circuit complexity exceeding $2^n/2n$, also in $\text{poly}(2^n)$ time. If this happens, we halt the entire procedure and use this truth table to decide whether to accept or reject the input.

If we get through all values of m without finding an empty pigeonhole, we reject the input. This concludes the definition of the machine deciding H . By construction we have $H \in E^{NP}$.

By definition, for a particular input length n , if there is some $m \in \{\lceil \frac{6}{\epsilon} \rceil n, \dots, \lceil \frac{6}{\epsilon} \rceil (n+1)\}$ for which the above procedure finds an empty pigeonhole for D , then H^n will have circuit complexity at least $2^n/2n$. Now, say this does not hold for a particular n . Then we claim that for all input lengths $m \in \{\lceil \frac{6}{\epsilon} \rceil n, \dots, \lceil \frac{6}{\epsilon} \rceil (n+1)\}$, L^m has circuit complexity less than $2^{\epsilon m}$, provided n is sufficiently large. This follows directly from Lemma 3.28: if there is some $y \in \{0, 1\}^W$ such that the first bit of $D^m[y](i)$ equals the i^{th} bit of L^m for all $i \in \{0, 1\}^m$, then this directly gives a circuit of size $O(|D|m)$ allowing us to compute L^m using y as advice (namely the algorithm described in Lemma 3.28, which can be made non-uniform at no additional cost by [101]). For n sufficiently

large, this $O(|D|m)$ term will be strictly less than $2^{\epsilon m}$. Thus, if $H \in \text{size}[2^n/2n]$, then for all sufficiently large n , L^m has circuit complexity less than $2^{\epsilon m}$ for all $m \in \{\lceil \frac{6}{\epsilon} \rceil n, \dots, \lceil \frac{6}{\epsilon} \rceil (n+1)\}$. But since the intervals $\{\lceil \frac{6}{\epsilon} \rceil n, \lceil \frac{6}{\epsilon} \rceil (n+1)\}_{n \in \mathbb{N}}$ cover all but finitely many natural numbers, this implies L^m has circuit complexity less than $2^{\epsilon m}$ for all sufficiently large m , contradicting our hardness assumption for L . So $H \notin \text{size}[2^n/2n]$, completing the proof. $\square$

Chapter 4: How to Construct Random Strings

4.1 Introduction

As in Chapter 3, our main result in this chapter is a deterministic algorithm for solving a wide class of explicit construction problems, whose correctness hinges on a certain computational hardness assumption. In contrast to Chapter 3, which gives algorithms for explicit construction problems reducible to LOSSY CODE, in this chapter our algorithms apply more generally to explicit construction problems that reduce to AVOID. This increased generality will come at a cost: (1) we will only be able to handle reductions that produce sufficiently *stretching* instances of AVOID, and (2) the hardness assumptions we require here are *significantly* stronger than those required in Chapter 3. The results in this chapter are based on joint work with Rahul Santhanam originally appearing in [107].

Recall that in Chapter 1, we identified a wide range of important unsolved explicit construction problems which all have a common complexity upper bound (they are reducible to AVOID). The primary motivating question behind the work in this chapter is the following:

Question 1: Is there compelling complexity-theoretic evidence that the explicit construction problems considered in Chapter 1, such as the construction of Ramsey graphs or rigid matrices, should be solvable in deterministic polynomial time?

Now, as per the reductions in Chapter 1, one way to obtain a positive answer to Question 1 would be to find a plausible complexity-theoretic assumption implying that AVOID is in FP. Unfortunately, as discussed in the concluded section of Chapter 1, it was shown in [80] that under plausible cryptographic and complexity-theoretic assumptions, Range Avoidance is *intractable* in

the worst case, so this general approach to Question 1 seems ill-founded. However the intractability of Range Avoidance in the worst case doesn't give evidence for the intractability of the explicit construction problems mentioned above - it could be that a *weaker* assumption than solving Range Avoidance is sufficient for efficient explicit constructions. Indeed such an assumption was identified in [80] - solving Range Avoidance for *uniformly generated* circuits. In particular, recall that an explicit construction is defined by some property $\Pi \subseteq \{0, 1\}^*$, and the goal is to produce a string in $\Pi \cap \{0, 1\}^n$ given 1^n as input. If the explicit construction problem defined by Π reduces in polynomial time to AVOID, then there is some *uniform sequence* of AVOID instances $C_n : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}^{m(n)}$, such that C_n can be produced in $\text{poly}(n)$ time given 1^n ; any algorithm which can solve AVOID *only on this uniformly generated sequence of instances* would then suffice to produce explicit strings in Π . The tractability of Range Avoidance on uniform circuits is raised explicitly in [80]. It is also stated there that “some of the authors are skeptical that this special case of AVOID is easy.”

Aside from the application to explicit constructions and Range Avoidance, there is a second self-contained motivation for the results of this chapter in terms of algorithmic information theory. Time-bounded Kolmogorov complexity K^T [112] is a fundamental measure of complexity of a Boolean string. Given a string x and a time bound T , $K^T(x)$ is the size of the smallest program p such that $\mathcal{U}(p)$ outputs x within T steps, where $\mathcal{U}$ is a universal Turing machine fixed in advance. Intuitively, for feasible time bounds $T(n)$, $K^{T(n)}(x)$ measures the inherent compressibility or “structure” in x from the point of view of efficient algorithms, in the sense that any x with low $K^{T(n)}$ complexity has a short description from which it can be recovered efficiently. Thus a string with high K^T complexity can be considered unstructured or random-like.

Given functions $T, \ell : \mathbb{N} \rightarrow \mathbb{N}$ and a sequence $(x_n)_{n \in \mathbb{N}}$ of strings, where each x_n is of length n , let us call the sequence (K^T, ℓ) -hard if $K^{T(n)}(x_n) \geq \ell(n)$ for all n . We are interested in the complexity of producing a (K^T, ℓ) -hard sequence of strings, for polynomial time bounds T and for ℓ as large as possible. Three particularly important settings we focus on here will be $\ell(n) = n^{\Omega(1)}$, $\ell(n) = \tilde{\Omega}(n)$, and $\ell(n) = n - o(n)$. A simple argument shows that K^T -hard strings cannot be

produced in time $T' = o(T/\log(T))$ by an algorithm A which outputs x_n when given n in unary. Indeed, the existence of such an algorithm would imply that $K^T(x_n) = O(\log(n))$, as the universal machine $\mathcal{U}$ can output x_n in less than time T when given n in binary and a constant-size program for $\mathcal{A}$ ¹. But is it possible for such a sequence to be produced by an algorithm running in time $\text{poly}(T)$?

Question 2: Let $T : \mathbb{N} \rightarrow \mathbb{N}$ be any time-constructible function and let $\ell(n) \in \{n^{\Omega(1)}, \tilde{\Omega}(n), n - o(n)\}$. Is there a deterministic algorithm $\mathcal{A}$, which given input 1^n , runs in time $\text{poly}(T)$ and outputs a (K^T, ℓ) -hard string?

Question 2 is a fundamental question about Kolmogorov complexity, for which we currently do not have clear positive or negative complexity-theoretic evidence. A positive answer to Question 2 would be very interesting, as it would demonstrate the power of having polynomially more time: a $\text{poly}(T)$ time algorithm would be capable of producing highly T -incompressible strings, while a $o(T/\log(T))$ time algorithm can only produce strings that are highly T -compressible. Indeed Hypothesis 1.19 in a recent work of [145] asks a very similar question² to Question 2, and states that “The plausibility of Hypothesis 1.19 remains to be investigated”. Question 2 also has implications for the theory of meta-complexity, which has seen much recent work [6, 73, 116].

As the reader may have guessed by now, Questions 1 and 2 are in fact closely related; in particular, it was shown in [144] that a positive answer to Question 2 for polynomial time bounds T would imply a positive answer to Question 1 (this connection explained in detail in **TODO: later part**). We prefer to work with the Question 2 formulation for most of this chapter. A natural approach to Question 2 is via pseudo-randomness. Under the assumption that E requires non-deterministic circuits of size $2^{\epsilon n}$ for some constant $\epsilon > 0$, it is known [83, 99] that for any constant $c > 0$ there is a pseudo-random generator G_n with seed length $O(\log(n))$ and error $o(1)$ against

¹Here we use the standard fact that any algorithm $\mathcal{A}$ running in time T' can be simulated by $\mathcal{U}$ in time $O(T' \log(T'))$.

²The difference is that instead of only asking about producing K^T -hard strings for time-constructible functions T , [145] asks for an algorithm that gets T and n as input in unary, and outputs a K^T -hard string of length n .

co-non-deterministic circuits of size n^c , computable in time $\text{poly}(n)$. Consider $T(n) = n^d$, where $d < c$ is any constant. The property of being a K^T -incompressible string, i.e., a length- n string of K^T -complexity $\geq n - 1$, can be checked by a co-non-deterministic circuit of size at most n^c , and moreover at least half of all strings of length n satisfy the property. Hence close to half of the outputs of G_n satisfy the property of K^T -incompressibility, which means that the range of G_n is a $\text{poly}(n)$ size set S_n computable in time $\text{poly}(n)$ and containing at least one K^T -incompressible string for large enough n . However, it is unclear how to identify efficiently *which* of the strings in S_n is incompressible - this seems to require calls to an NP oracle. We could try and *combine* the strings in S_n together into a string that is K^t -hard, for example by concatenating them. This approach has two drawbacks:

1. Concatenating the strings yields a new string of length $\gg T(n)$ rather than length n , since the set S_n has size $\gg n^c$. Hence for any reasonable setting of $\ell(n)$, even $\ell(n) = n^{\Omega(1)}$, we do not get any interesting consequences for K^T -hardness when we are interested in time bounds as a *function* of input length.
2. Even if we don't mind destroying the relation between the time bound and input length, concatenating $\text{poly}(n)$ strings which are maximally incompressible can, at best, achieve incompressibility $\ell(n) = n^{\Omega(1)}$. If we are in the more stringent compressibility settings $\ell(n) = \tilde{\Omega}(n)$ or $\ell(n) = n - o(n)$ which occur more frequently in explicit construction applications, concatenating $\text{poly}(n)$ many strings appears to be useless.

Like most of the interesting problems in derandomization, answering Question 2 *unconditionally* would be difficult, as any $(K^T, n^{\Omega(1)})$ -hard string for $T > n^2$ can be transformed easily into the truth table of a hard Boolean function in E, i.e., a function with circuit complexity at least $2^{\epsilon m}$ for some ϵ , and hence would imply $E \not\subseteq P/\text{poly}$. Instead, we would simply like complexity-theoretic evidence in favor of or against Question 2. Preferably, this evidence should avoid exotic assumptions, and involve standard beliefs about the difficulty of simulating one resource efficiently by another, e.g., the belief that time cannot be simulated by much smaller space or by much smaller

non-uniformity.

4.1.1 Our Results

We give positive answers to Questions 1 and 2, under believable complexity assumptions. Our hardness assumptions are of the following two forms, each involving a parameter $d \in \mathbb{N}$:

Hypothesis (Hardness Assumptions for d^{th} -Exponential Time Bounds).

1. *(Strong Form) There is an absolute constant $\epsilon > 0$ so that the following holds. For any time constructible $T(n), m(n)$, with $T(n) \geq n$ having at most d^{th} -order exponential growth rate and $n \leq m(n) \leq \text{poly}(n)$, there is a function $f : \mathbb{N} \mapsto \{0, 1\}^*$, $|f(n)| = m(n)$ computable in $\text{poly}(T(n))$ time, which cannot be computed in time $T(n)^\epsilon$ with $m(n)^\epsilon$ bits of advice by any NP-oracle machine for more than finitely many n .*
2. *(Weak Form) Let $T(n) = \exp^{[d+1]}(n)$. There is some $\epsilon > 0$ and a language computable in time $T(n)$ which is not computable in time $T(n)^\epsilon$ with an NP oracle and $2^{\epsilon n}$ bits of advice even infinitely often.*

We are using $\exp^{[d]}(n)$ for the d -fold iteration of the exponential function $\exp(n) = 2^n$. Such assumptions, particularly of the second kind, are fairly standard in complexity theory. Indeed, when $d = 0$, the second assumption is a standard derandomization assumption, asserting that there is a language in $\text{TIME}[2^n]$ which does not have NP oracle circuits of size $2^{o(n)}$. The main novelty in our case is that we use these assumptions for larger values T , in particular for time bounds T which have iterated-exponential growth rate. While we state the strong form assumption by quantifying over all time constructible $T(n), m(n)$, in fact we only require the assumption to hold for quite a limited class of bounds: the bounds $T(n)$ needed are representable by constant length arithmetic expressions with $\exp$, $\log$, and ceiling/floor functions, while the bounds $m(n)$ needed are simply of the form $m(n) = \exp(c \lceil \log n \rceil)$ for constants c .

The first assumption is analogous to assumptions made in the recent literature on hardness vs randomness [32], and generalizes the second assumption in a natural way. Both the first and second

assumptions involve natural beliefs about the relationships between the fundamental resources of time, non-determinism and advice. Both assumptions formalize the intuition that time cannot be sped up by an arbitrary polynomial amount using non-determinism and advice. Indeed, note that there is no known way to speed up time even by a *super-constant* amount by using non-determinism and advice.

We believe that our assumptions hold relative to a *random oracle*, and here is an informal argument for the case of the second assumption. Given oracle A , define the time T bounded Turing machine M^A to accept x of length n if the x 'th string of length $T(n)$ is in A (where we consider the lexicographic order on strings). For a random oracle A , the truth table of A at length $T(n)$ is Kolmogorov-random even conditioned on truth tables below length $T(n)$. Since $T(n)^\epsilon$ time non-deterministic oracle machines can only access strings of A at length $T(n)^\epsilon$ or below, it should not be possible to compute the first 2^n bits of A at length $T(n)$ from this information together with arbitrary $T(n)^\epsilon$ bits of advice.

Our approach to using these assumptions in service of constructing random strings is via pseudo-randomness, and is a variation on the strategy we critiqued a few paragraphs ago: construct a pseudorandom generator, enumerate its range, and concatenate the constituent strings. The problems with this approach discussed in the last section boiled down to one issue: for a standard complexity-theoretic PRG fooling polynomial size nondeterministic circuits, we can at best achieve a seed length of $O(\log n)$, and hence will end up needing to concatenate a very long list of $\text{poly}(n)$ strings. To remedy the situation, it would suffice to use pseudorandom generators with much smaller seed length of $o(\log n)$ or even $\log \log n$. While it is known that $O(\log n)$ seed length is information-theoretically optimal for fooling *nonuniform* circuits of size $\text{poly}(n)$, we aim to exploit the uniformity of the adversary we are fooling to get seed length *as small as possible*; for uniform adversaries the lower bound of $\log n$ no longer applies. This yields a result which is interesting in its own right: efficient constructions of small pseudo-random sets for uniform (or even slightly non-uniform) algorithms, under assumptions of the form discussed in the previous paragraph.

Theorem 4.1 (Informal, see Theorems 4.18 and 4.20). *Under the Hypotheses above with $d = 1$, there is a pseudorandom generator with seed length $O(\log \log n)$ which fools $\text{TIME}^{\text{NP}}[\text{poly}(n)]$; if we assume the strong form the generator runs in polynomial time, and if we assume the weak form it runs in quasipolynomial time.*

More generally, under the above hardness assumption with large parameters d (i.e. for higher-exponential time bounds), we obtain pseudorandom generators fooling $\text{TIME}^{\text{NP}}[\text{poly}(n)]$ with seed length $O(\log^{[d+1]}(n))$ (for infinitely many n). If we assume the strong form hypothesis our generators will run in polynomial time, and if we assume the weak form they will run in iterated quasipolynomial time. When $d > 1$, our construction will require in addition that the input length n is of the form $n = \exp^{[d-1]}(n')$ for some n' , or more generally that $K^{\text{poly}(n)}(n) \leq \log^{[d]}(n)$.

Some terms in the above require clarification. We are using $\log^{[d]}(\cdot)$ to denote the d -fold iteration of the logarithmic function. Our notion of “iterated quasipolynomial time” will be defined in the preliminaries - it describes a family of runtime bounds which grows faster than polynomial and quasipolynomial, but is much closer to polynomial than to exponential. Finally, the notation $K^{\text{poly}(n)}(n)$ denotes the time bounded Kolmogorov complexity of the *number* n , when written as a string in standard binary notation.

Our PRG construction can actually handle advice (nonuniformity) up to $\log^{[d]}(n)$ when the seed length is $\log^{[d+1]}(n)$, which we observe is optimal via a standard argument (Lemma 4.24). It also holds for fooling larger time bounds $\text{TIME}^{\text{NP}}[T(n)]$ for $T(n) \gg \text{poly}(n)$ (at the cost of the time complexity of the generator being low as a function of $T(n)$ rather than of n), and for other oracles $\mathcal{O}$ in place of the NP oracle (provided we use $\mathcal{O}$ in place of NP in the hardness assumption).

We then turn our attention briefly to PRGs fooling uniform algorithms *without* NP oracles: what is the minimal seed length such that we can fool $\text{TIME}[T(n)]$ by a PRG with running time $T(n)^{O(1)}$? For this problem we show in Theorem 4.22 that the machinery developed above is unnecessary: if the PRG is capable of simulating the distinguishers it is trying to fool, there is a straightforward method which can reduce seed length $O(\log n)$ to arbitrarily small seed length. Hence under the standard assumptions that achieve logarithmic seed length for $\text{TIME}[T(n)]/n$

(namely that E requires exponential circuit size) we can obtain an arbitrarily strong reduction in seed length for $\text{TIME}[T(n)]$. Like our NP-oracle result, this result applies to distinguishers with mild nonuniformity, and the achievable seed length scales with the uniformity in a way that is provably optimal according to Lemma 4.24.

Random Strings and Explicit Constructions: We next use our short-seed pseudorandom generators for P^{NP} (Theorem 4.1) to give (conditional) constructions of K^{poly} -random strings. Our first construction of random strings is the following, which achieves incompressibility $\tilde{\Omega}(n)$:

Theorem 4.2 (Informal, see Theorems 4.25 and 4.26). *Under our main hardness assumption with $d = 1$, for any $k \in \mathbb{N}$ there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(1^n)$ outputs an n -bit string x satisfying $K^{n^k}(x) \geq \frac{n}{\text{polylog}(n)}$ for all n ; the algorithm will run in polynomial time if we use the strong form of the assumption, and quasipolynomial time if we use the weak form.*

More generally under our main hardness assumption for larger values of d , we obtain constructions of strings with $K^{n^k}(x) \geq \frac{n}{\text{poly}(\log^{[d]}(n))}$. Using the strong form of the assumption the construction runs in polynomial time, and under the weak form it runs in iterated quasipolynomial time. When $d > 1$ we require the integer n to be of the form $n = \exp^{[d-1]}(n')$ for some integer n' , or more generally to satisfy $K^{\text{poly}(n)}(n) \leq \log^{[d]} n$.

If we are satisfied with our construction algorithms succeeding on some unknown infinite set of input lengths, we are able to bootstrap the above constructions so that the $\log^{[d]}(n)$ loss in K -complexity becomes additive rather than multiplicative:

Theorem 4.3 (Informal, see Theorem 4.27). *Assume the first version of our main hardness assumption for all d . Then for any $k, d \in \mathbb{N}$ there is a polynomial time algorithm $\mathcal{A}$ such that $\mathcal{A}(1^n)$ outputs an n -bit string x satisfying $K^{n^k}(x) \geq n - \log^{[d]} n$ for infinitely many n .*

In Section 4.4.2 we then use the previous results to give conditional polynomial time explicit constructions of various important combinatorial objects shown reducible to Range Avoidance in [102]. We will defer most of the specifics to Section 4.4.2, but highlight the following two results:

Theorem (See Theorems 4.30 and 4.31).

1. *Under the strong form of our main hardness assumption with $d = 2$, there is a $\text{poly}(n)$ time algorithm which produces a matrix $M \in \mathbb{F}_2^{n \times n}$ which is Valiant-Rigid whenever n is a power of 2.*
2. *Assuming that there is a language in $\text{TIME}[2^{2^n}]$ that is not computable in $\text{TIME}^{\text{NP}}[2^{\epsilon 2^n}]/2^{\epsilon n}$ (even infinitely often)³, there is a language in EXP that requires Boolean circuits of size $\frac{2^n}{\text{poly}(n)}$ for all n .*

Crucially, our results give the first standard-form hardness assumptions under which the above explicit construction problems (and others in Section 4.4.2) have an efficient algorithm, and they do so via a universal approach: we have a single object, K^{poly} -random strings, whose efficient explicit construction follows from plausible hardness assumptions, and which automatically satisfies various other pseudorandom properties, e.g. rigidity as a matrix or maximal hardness as a Boolean function.

Further Applications: In Sections 4.4.3 and 4.4.4 we discuss two further applications of our random string constructions. The first shows a “hardness condensation” phenomenon for the main hardness assumptions we use in this paper (with the lower bound relativized to a PSPACE oracle), whereby we may amplify the degree of nonuniformity in the lower bounds from 2^{n^ϵ} to $2^{n-o(n)}$. The second addresses the question of the relative difficulty of *uniform* Range Avoidance and general (nonuniform) Range Avoidance posed in [145, 80], where we observe using our main results and some previous hardness results ([80, 1]) that under plausible hardness assumptions, nonuniform Range Avoidance is significantly harder than the uniform variant.

Barriers to Improving Our Main Results: In the final section we give two sets of results indicating barriers to improving our main PRGs and random string constructions. Our first result casts our

³This is a particular case of the weak form hardness assumption.

iterated-logarithmic seedlength PRG fooling P^{NP} (Theorem 4.1) as a reconstructive *multi-source* extractor, similar to Trevisan’s analysis [158] of the classical hardness randomness connection in [130, 83] in terms of reconstructive single-source extractors. We show that any such extractor must have iterated-logarithmic seed length, and thus that any improvement to the seed length of our PRG in Theorem 4.1 must use substantially different techniques.

Second, we give an oracle separation showing that it is not possible via black-box arguments to obtain our main results from more standard assumptions like $E \not\subseteq \text{SIZE}[2^{\epsilon n}]$. In [102] it is shown that explicit construction of $(K^{\text{poly}}, n - 1)$ random strings and $2^{\epsilon n}$ hard truth tables are equivalent with respect to NP oracle reductions. If such an equivalence held without NP oracles it would supersede our main results here: we could start with an assumption that E requires $2^{\epsilon n}$ -size circuits, and then use the reduction to produce from the hard truth table a string with high time bounded Kolmogorov complexity. We show that no such reduction exists which is *relativizing*. This justifies in some sense the need for more high-end hardness assumptions to achieve our main results.

4.1.2 Overview of Main Construction

We describe at a high level our construction of pseudorandom generator with small seed length secure against any $L \subseteq \{0, 1\}^n$ computable in $\text{TIME}^{NP}[\text{poly}(n)]$. Using the classical hardness-randomness connection, under the assumption that $\text{TIME}[2^{O(n)}]$ requires exponential size NP-oracle circuits we can obtain a generator $G^0 : \{0, 1\}^{s_0(n)} \rightarrow \{0, 1\}^n$ with seed length $s_0(n) \leq O(\log n)$. The key observation is that this first attempt at a generator significantly overshoots our primary goal in one respect: the generator G^0 obtained from the generic hardness-randomness connection would in fact be secure against $\text{TIME}^{NP}[\text{poly}(n)]/n$ machines which have access to n bits of advice. On the other hand we only want a generator which fools *uniform* algorithms.

To use this to our advantage, we consider a recursive argument. Let $n_1 = s_0(n)$, and consider n_1 as our *new input length*. Define the language $L_1 \subseteq \{0, 1\}^{n_1}$ consisting of the seeds z of G^0 such that $G^0(z) \in L$. We know that $\Pr_{z \sim \{0, 1\}^{n_1}}[z \in L_1] \approx \Pr_{x \sim \{0, 1\}^n}[x \in L]$ by the security of our first

generator G_0 , and therefore if we could find a second generator $G_1 : \{0, 1\}^{s_1(n_1)} \rightarrow \{0, 1\}^n$ which fools the language L_1 , we would find that the composition $G_0 \circ G_1$ fools the original language L . If $s_1(n_1) = O(\log n_1) = O(\log \log n)$ we would have made significant progress. For such a generator G^1 to fool L_1 , we would need it to fool NP-oracle algorithms with running time $\text{poly}(n)$; however the input length of L_1 is $n_1 = O(\log n)$, so as a function of the new input length we need G^1 to fool algorithms running in *exponential time* with an NP oracle. We may proportionally increase the allowable runtime of G^1 to exponential as well; so overall we have scaled the time complexities of our generator/distinguisher by an exponential. However, crucially, the *advice complexity* of deciding L_1 remains $O(1)$: this is where we use crucially the uniformity of our distinguisher. If L required n bits of advice compute, then the best advice upper bound we could place on L_1 is $n \approx 2^{n_1}$ which is trivial.

Continuing in this way, we are able to iteratively reduce the seed length of our original generator by a logarithmic composition, for an arbitrary constant number of phases. The cost is that we need to obtain, in each step, pseudorandom generators with logarithmic seed length that fool algorithms running in $\exp(\exp(\dots \exp(n) \dots))$ time with an NP oracle, and which are computable in a comparable amount of time (without an NP oracle). To achieve such generators we rely on the standard toolkit of hardness-randomness transformations [130, 83, 160]. In this way, we can argue the security of our construction based on the kind of hardness assumptions described above.

There are a few additional intricacies that we are skimming over here, which contribute to the two caveats in our main theorem statements: the weaker runtime bounds when using the second version of our hardness assumptions, and the requirement that the number n is of the form $n = \exp^{[d-1]}(n')$ for some n' when using the first version of our hardness assumptions. The first of these issues roughly stems from the fact that, when trying to apply our hardness assumption to get an $O(\log n_j)$ seed length generator on input length $n_j \approx \log^{[j]}(n)$ (which we will then compose with the outer generator to reduce the total seed length by another logarithm), we will actually need to apply our hardness assumption on some other input length $m_j = O(n_j)$; this will mean that we can at best hope for our generator to run in time $\text{poly}(\exp^{[j]}(O(n_j))) \approx \exp^{[j]}(O(\log^{[j]} n))$,

which will be superpolynomial when $j > 1$. This issue can be avoided by considering the more refined first version of our hardness hypothesis.

For the second issue, it turns out that we can only guarantee the security of our generator on inputs lengths n such that the *number* n itself has time-bounded Kolmogorov complexity $\log^{[d]}(n)$. For $d = 1$ this changes nothing, since every number n has $K^{O(n)}(n) \leq \log n$ via its binary representation, so we achieve a generator with seed length $O(\log \log n)$ which is valid on all input lengths, but for large values of d our construction is only valid for infinitely many n (in particular it will work for all n of the form $\exp^{[d-1]}(n')$ for some n'). The reason for this issue is as follows: in the above exposition, we considered the language $L_1 \subseteq \{0, 1\}^{n_1}$ of seeds mapping to strings in the base language L , and argued it is computable in time exponential in its input length. When we continue this argument for more steps and obtain languages $L_2, L_3, \dots$, we would like to argue at each step that they are uniformly computable with a small amount of advice (comparable to the input length they are defined on). However, the definition of these languages depends on the original input length n that we started on, and since $\log^{[d]}(\cdot)$ is not injective for any $d > 0$, we cannot determine n solely from the smaller input lengths, but will need to somehow supply the number n as advice. If we assume the number n is sufficiently compressible, we are able to skirt this issue and complete the argument.

4.1.3 Related Work

We have already mentioned connections of our work to meta-complexity and Range Avoidance. In terms of derandomizing uniform algorithms, there has been a lot of work on uniform hardness-randomness tradeoffs, beginning with [85], but much of that work is not relevant to our setting as the emphasis is on uniform assumptions for derandomization, while we are interested instead in decreasing the seed length, and are not concerned with uniformity of the assumption.

However the recent work of [32] on super-fast derandomization is indeed very relevant from a technical point of view. They give a (conditional) derandomization of $\text{BPTIME}[T(n)]$ into $\text{TIME}[T(n)^{1+\epsilon}]$. The classical approach to derandomizing $\text{BPTIME}[T(n)]$ would be to model

it as a circuit of size $T(n)$ with input length $T(n)$ (the input represents the randomness) and use a PRG fooling this class. To fool this class of circuits requires a PRG with seed length $\log T(n)$ and running time $T(n)$, and if $T(n) = n^3$ we could therefore not hope to achieve derandomization in time $T(n)^{1+\epsilon}n$ no matter how fast the PRGs runtime is. A key observation in [32] is that that this approach to derandomizing $\text{BPTIME}[T(n)]$ actually *overshoots* what's required: if we model the $\text{BPTIME}[T(n)]$ machine more precisely as a $\text{TIME}[T(n)]$ machine on input length $T(n)$ with nonuniformity $n \ll T(n)$, then we only need a PRG which fools $\text{TIME}[T(n)]/n$; for this task, it is possible (at least information-theoretically) to achieve a seed length of $\log n$ or more generally $(1 + \epsilon) \log n$ rather than $\log T(n)$, which means that the enumeration of seeds will only cost us $n^{1+\epsilon}$ time.

They then proceed to construct a PRG for $\text{TIME}[T(n)]/n$ with seed length $(1 + \epsilon) \log n$ and computable in time $T(n)^{1+\epsilon}$. In their case a great deal of work must be dedicated to optimizing the runtime of the PRG which has no relevant parallel in our work; however the question of reducing the seed length down to the true level of uniformity, and the assumptions under which they are able to achieve it, have similarities to ours. In particular, they require assumptions of the form $\text{TIME}[T(n)] \not\subseteq \text{TIME}[T(n)^{1-\delta}]/2^{(1-\delta)n}$ where $T(n) = 2^{kn}$ for a large constant k .

The innovation in our work is the use of recursion to achieve dramatically smaller seed length against inefficient uniform adversaries using a small amount of non-uniformity, which enables us to address Questions 1 and 2.

4.2 Preliminaries

We start with some notation for the classes of growth rates appearing in this chapter:

Definition 4.4 (Growth Rates). *Both $\log(\cdot)$, $\exp(\cdot)$ are base 2. For any function $f : \mathbb{N} \rightarrow \mathbb{N}$, $d \in \mathbb{N}$ we use $f^{[d]}$ to denote the d -ary composition of f with itself, with $f^{[0]}$ being the identity $n \mapsto n$. We say that a function f has elementary growth rate, or that it is “elementary,” if $f(n) \leq \exp^{[d]}(n)$ for some absolute constant d . We say that a function $f : \mathbb{N} \rightarrow \mathbb{N}$ is time-constructible if there is a Turing machine M such that $M(1^n)$ prints $f(n)$ in binary, and M has running time $O(f(n))$ on*

all inputs.

For each $d \in \mathbb{N}$ and $\alpha \in \mathbb{R}_+$ we define the function $\Phi_{d,\alpha}(n) = \exp^{[d]}(\log(\alpha \log^{[d-1]}(n)))$. Using this function we define a hierarchy of $O(\cdot)$ notations more relaxed than the standard as follows; for $f, g : \mathbb{N} \rightarrow \mathbb{N}$, we say $f(n) = O^{(d)}(g(n))$ if $f(n) \leq \Phi_{d,C}(g(n))$ for some $C \in \mathbb{N}$ and sufficiently large n , and $f(n) = \Omega^{(d)}(g(n))$ if $f(n) \geq \Phi_{d,\epsilon}(g(n))$ for some $\epsilon > 0$ and sufficiently large n . $o^{(d)}(g(n))$, $\omega^{(d)}(g(n))$ are defined analogously.

In this notation we have $O^{(1)}(n) = O(n)$, $O^{(2)}(n) = \text{poly}(n)$, $O^{(3)}(n) = \text{quasipoly}(n)$. This hierarchy of growth rates enumerates a class of strongly subexponential growth rates having magnitude much closer to n than to 2^n : indeed for each fixed d , $O^{(d)}(n)$ is closed under composition and grows slower than $\Omega^{(d')}(2^n)$ for any fixed d' . Similarly, $\Omega^{(1)}(n) = \Omega(n)$, $\Omega^{(2)}(n) = n^{\Omega(1)}$, $\Omega^{(3)}(n) = 2^{\log^{\Omega(1)} n}$, and for each fixed d the class $\Omega^{(d)}(n)$ has a growth rate much closer to n than to $\log n$, and in particular far exceeds the growth rate of $O^{(d')}(\log n)$ for any fixed d' .

Definition 4.5 (Languages and Complexity Classes). For functions $f, g : \mathbb{N} \rightarrow \mathbb{N}$ and language $A \subseteq \{0, 1\}^*$ we define the complexity class $\text{TIME}^A[f(n)]/g(n)$ consisting of those languages decidable in time $O(f(n))$ with an oracle for the language A and using $O(g(n))$ bits of advice on inputs of length n . When $g(n) = 0$ we omit this argument. For a language $L \subseteq \{0, 1\}^*$ we use $L_n : \{0, 1\}^n \rightarrow \{0, 1\}$ to denote the restriction of L to $\{0, 1\}^n$, and $\text{SIZE}[f(n)]$ to denote the set of languages L so that L_n is computed by Boolean circuits of size $O(f(n))$ for all n .

We next define our notation for time-bounded Kolmogorov complexity:

Definition 4.6 (Kolmogorov Complexity). We fix an efficient universal oracle turing machine $\mathcal{U}$ once and for all. For an oracle language $O \subseteq \{0, 1\}^*$, time bound $T \in \mathbb{N}$, $x, y \in \{0, 1\}^*$ we use $K^{T,O}(x \mid y)$ to denote the length of the shortest program $\pi \in \{0, 1\}^*$ so that $\mathcal{U}^O(\pi, y)$ halts with output x in at most T time steps. If $O = \{\}$ or y is the empty string we omit them from the notation as in $K^T(x \mid y)$, $K^{T,O}(x)$ respectively.

For a number $n \in \mathbb{N}$, we use $K^{T,O}(n)$ to denote $K^{T,O}(\text{bin}(n))$ where $\text{bin}(n) \in \{0, 1\}^{\lceil \log n \rceil}$ is the canonical binary representation of n .

Finally we introduce relevant notation for pseudorandom generators:

Definition 4.7 (Pseudorandom Generators). *For a family of “distinguishers” $\mathcal{D} \subseteq \{0, 1\}^{\{0,1\}^n}$ and $\epsilon \in [0, 1]$, we say that $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ is a pseudorandom generator (PRG) secure against $\mathcal{D}$ with error ϵ if, for all $D \in \mathcal{D}$, we have*

$$\left| \Pr_{x \sim \{0,1\}^n} [D(x) = 1] - \Pr_{z \sim \{0,1\}^s} [D(G(z)) = 1] \right| \leq \epsilon$$

We say that G is a hitting set generator (HSG) if it satisfies the weaker condition

$$\left(\Pr_{x \sim \{0,1\}^n} [D(x) = 1] > \epsilon \right) \Rightarrow \left(\Pr_{z \sim \{0,1\}^s} [D(G(z)) = 1] > 0 \right)$$

Let $s : \mathbb{N} \rightarrow \mathbb{N}$, $\epsilon : \mathbb{N} \rightarrow [0, 1]$. If $C \subseteq \{0, 1\}^{\{0,1\}^}$ is a complexity class (set of languages) and $G = (G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ is an ensemble of generators, we say that G is a PRG (resp. HSG) secure against C if, for all $L \in C$, there is $n_0 \in \mathbb{N}$ so that for all $n > n_0$, G_n is a PRG (resp. HSG) secure against $\{L_n\}$ with error $\epsilon(n)$.*

4.2.1 Range Avoidance and Construction of Random Strings

Recall the general definition of an explicit construction problem, defined by a language $\Pi \subseteq \{0, 1\}^*$, where given 1^n we are tasked with finding $x \in \{0, 1\}^n \cap \Pi$. We define here a certain class of algorithm for solving explicit construction problems:

Definition 4.8 (List Solution). *Let Π be an explicit construction problem. A function S is a “list solution” to Π if $S(1^n)$ outputs a list of strings in $\{0, 1\}^n$, such that $S(1^n) \cap \Pi_n \neq \emptyset$ for all n . The “list size” $\ell(\cdot)$ is defined as $\ell(n) = |S(1^n)|$.*

Next, we give some terminology for describing reducibility of an explicit construction problem to AVOID:

Definition 4.9. *We say that an explicit construction problem Π reduces to AVOID in polynomial time with stretch function $\ell(n)$, if there is a polynomial time algorithm which, given 1^n , outputs an*

AVOID instance $C_n : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}^n$ so that whenever x is a solution for C_n , we have $x \in \Pi$.

The key connection between K^{poly} random strings and explicit construction problems reducible to Range Avoidance is the following:

Observation 4.10 ([145]). *Say that for every $k \in \mathbb{N}$, there is a polynomial time algorithm which, for every n (resp. infinitely many n) outputs a string $x \in \{0, 1\}^n$ with $K^{n^k}(x) \geq \ell(n) + \omega(1)$. Then every explicit construction problem reducible to AVOID with stretch function $\ell(n)$ is solvable in polynomial time .*

Proof. The definition of the reduction implies that there is a polynomial time algorithm $C_n : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}^n$, so that whenever $x \notin \text{range}(C_n)$ we have $x \in \Pi$. If k is such that the algorithm constructing C_n runs in time n^k , then we observe that every string x in the range of C_n satisfies $K^{n^k}(x) \leq \ell(n) + O(1)$. $\square$

Recall that our approach to constructing K^{poly} random strings will consist of two steps. First, we will try only to construct a short list of strings, one of which is guaranteed to have K^{poly} complexity $\geq n - 1$: we then concatenate the list to obtain a single string whose complexity degrades by a factor proportional to the length of the list. The second step (concatenation) is justified by the following standard claim:

Observation 4.11. *Let $x^1, \dots, x^m \in \{0, 1\}^n$ and let $\hat{x} = (x^1, \dots, x^m)$ be their concatenation. Then for any time bound T and $i \leq m$ we have $K^{T-O(mn)}(\hat{x}) \geq K^T(x^i) - O(\log m)$.*

For the first step (obtaining a short list of candidate solutions), we rely on the following observation:

Observation 4.12. *Let $(G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ be a hitting set generator secure against $\text{TIME}^{\text{NP}}[O(T(n))]$ with error $\frac{1}{2}$. Then there exists a seed $z \in \{0, 1\}^{s(n)}$ so that $K^{T(n)}(G_n(z)) \geq n - 1$; in other words the range of G_n is a list solution for the set of strings with $K^{T(n)}(\cdot) \geq n - 1$.*

Proof. At least half of n bit strings have $K^{T(n)}(\cdot) \geq n - 1$. On the other hand $K^{T(n)}(\cdot)$ is computable in time $O(T(n))$ with an NP oracle. $\square$

Hence, we have reduced the problem of constructing random strings to constructing hitting generators against P^{NP} with short seed length. The next section is dedicated to the construction of such a generator under plausible hardness assumptions. We will in fact produce a pseudorandom generator (despite only needing a hitting set generator).

4.3 PRGs for Uniform Classes with Near-Optimal Seed Length

The main goal in this section is to produce a pseudorandom generator computable in a $\text{poly}(n)$ time, or more generally $O^{\langle d \rangle}(n)$ time for some fixed constant d , which is secure against uniform P^{NP} algorithms and has seed length significantly smaller than $\log n$. We will phrase all of our assumptions/generator construction with respect to an arbitrary oracle O in place of NP for the sake of generality.

We will consider here two qualitatively distinct classes of hardness assumption used to instantiate our generators, each parameterized by an oracle O (typically we set $O = NP$) and a constant $d \in \mathbb{N}$:

Hypothesis 4.13 (Strong Assumption for O, d , abbreviated $SH(O, d)$). *There is $\epsilon > 0$ so that for all time constructible $T(n) \leq (\exp^{[d]}(n))^{O(1)}$ and all time constructible $m : \mathbb{N} \rightarrow \mathbb{N}$, $n \leq m(n) \leq \text{poly}(n)$ the following holds: there is a sequence of strings $(f_n \in \{0, 1\}^{m(n)})_{n \in \mathbb{N}}$ so that the map $n \mapsto f_n$ is computable uniformly in $T(n)$ time, but no machine running in time $T(n)^\epsilon$ with an O oracle and $m(n)^\epsilon$ bits of advice can compute f_n for more than finitely many n . In the case $T(n) \leq \text{poly}(n)$, we only require the assumption to hold in the case $m(n) = n$.*

We refer to $m(n)$ as the “length bound” in the above.

Hypothesis 4.14 (Weak Assumption for parameters O, d, v , $d \geq 1$, abbreviated $WH(O, d, v)$). *Let $1 \leq d' \leq d$ and $T(n) = \exp^{[d']}(n)$. There is some constant $\epsilon > 0$ and a language L computable in $\text{TIME}[T(n)]$ which is not computable in $\text{TIME}^{NP}[\Phi_{v,\epsilon}(T(n))]/\Phi_{v,\epsilon}(2^n)$ on more than finitely many input lengths.*

Note that $\text{WH}(O, 1, 2)$ translates to the assumption that E requires $2^{\Omega(n)}$ circuit complexity with O oracles, which is the standard regime in which polynomial time generators fooling P^O with logarithmic seedlength can be constructed by known methods.

4.3.1 Fooling $\text{TIME}^{\text{NP}}[T(n)]$ with $O(\log n)$ Seed Length

The first step in our construction is to use classical hardness-randomness constructions to give pseudorandom generators which fool $\text{TIME}^{\text{NP}}[T(n)]$ with $O(\log n)$ seed length and runtime $\approx T(n)$ in the case $T(n)$ is very large, under the appropriate hardness assumptions. Depending on which assumption we use we get a generator with different parameters:

Lemma 4.15. *Assume $\text{SH}(O, d)$. Then for every time constructible $T(n) \leq (\exp^{[d]}(n))^{O(1)}$ there is a PRG $(G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ computable uniformly in time $\text{poly}(T(n))$ which fools $\text{TIME}^O[T(n)]/3n$.*

Lemma 4.16. *Assume $\text{WH}(O, d + 1, v)$, $d \geq 0$, $v \geq 2$. Then for any $k \in \mathbb{N}$ there is a PRG $(G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ computable uniformly in time $O^{\langle d+v \rangle}(n)$ which fools $\text{TIME}^O[n^k]/3n$ and has seed length $s(n) \leq O^{\langle v-1 \rangle}(\log n)$.*

We will prove the first case (Lemma 4.15) here, and reserve Lemma 4.16 for the Appendix as its proof is similar. We require the following standard tool from complexity-theoretic pseudorandomness:

Theorem 4.17 (Black-Box Hardness-Randomness Connection [83]). *For each $\epsilon > 0$ exists a constant $c \in \mathbb{N}, c \geq \frac{3}{\epsilon}$ and a uniform algorithm IW with the following behavior. On input n and given oracle access to a function $f : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}$ with $\ell(n) = c \cdot \lceil \log n \rceil$, IW^f computes a function $\text{IW}^f : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n$ in $\text{poly}(n)$ time for $s(n) \leq O(\ell(n))$ such that for any $D : \{0, 1\}^n \rightarrow \{0, 1\}$ with*

$$\left| \Pr_{x \sim \{0, 1\}^n} [D(x) = 1] - \Pr_{z \sim \{0, 1\}^{s(n)}} [D(\text{IW}^f(z)) = 1] \right| \geq \frac{1}{n}$$

there exists a circuit C with D oracle gates computing f whose total size is at most $2^{\epsilon \ell(n)}$.

Proof of Lemma 4.15. We will invoke $\text{SH}(O, d)$ on some yet to be determined time bound $T'(n)$ and length bound $m(n)$, with respect to oracle O and constant d ; let $\epsilon > 0$ be the implied constant guaranteed by the hypothesis (which does not depend on the choice of T', m).

Next, invoke Theorem 4.17 with parameter $\delta := \epsilon/2$, and let $c \in \mathbb{N}$ be the guaranteed constant from this Theorem. So there is a function $\text{IW}^f : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n$ which, given oracle access to a function $f : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}$ with $\ell(n) = c \cdot \lceil \log n \rceil$, runs in $\text{poly}(n)$ time with $\text{poly}(n)$ oracle calls to f , and such that whenever $D : \{0, 1\}^n \rightarrow \{0, 1\}$ distinguishes IW^f from uniform, there is a D -oracle circuit C of size $2^{\delta \ell(n)}$ computing f .

Now we set $m(n) = 2^{\ell(n)} = 2^{c \cdot \lceil \log n \rceil}$ and let $T'(n) = T(n)^{3k}$, which are both time constructible, and use these in our specific invocation of $\text{SH}(O, d)$ as hinted previously. We then obtain an algorithm which, in time $T(n)^{3k}$, computes a string $f_n \in \{0, 1\}^{m(n)}$, which cannot be computed by any machine running in time $T'(n)^\epsilon = T(n)^3$ with $m(n)^\epsilon = 2^{\epsilon \ell(n)}$ bits of advice. We then claim that, setting $G_n = \text{IW}^{f_n} : \{0, 1\}^{\ell(n)} \rightarrow \{0, 1\}^n$ gives the required generator. By assumption it is computable in time $T(n) \cdot \text{poly}(n) \leq \text{poly}(T(n))$. On the other hand if it were distinguished in time $T(n)$ with $3n$ bits of advice, we'd obtain a circuit computing every bit of f_n of size $3n + 2^{\frac{\epsilon}{2} \ell(n)}$ with oracle gates that can be evaluated in time $T(n)$, hence overall we would be able to compute f_n in time $T(n) \cdot m(n)^{\frac{\epsilon}{2}} + O(n \cdot T(n)) = T(n) \cdot n^{\frac{1}{2}} + O(T(n) \cdot n) < n^3$ with $3n + m(n)^{\frac{\epsilon}{2}} < m(n)^\epsilon$ bits of advice, provided $n = o(m(n)^\epsilon)$. Recalling from our application Theorem 4.17 that $c\delta \geq 3$, we have that $m(n)^{\frac{\epsilon}{2}} \geq \Omega(n^{\frac{c\epsilon}{2}}) = \Omega(n^{\frac{3}{2}})$ and we are done. $\square$

4.3.2 Recursive Generator Construction

We give here our main construction of a generator fooling languages decidable in polynomial time with an NP oracle under a natural computational hardness assumption, specifically the assumptions $\text{SH}(\text{NP}, d)$, $d \in \mathbb{N}$. Our construction will have seed iterated-logarithm type seed length $\log^{[O(1)]} n$ for infinitely many n ; in particular it will work for all n such that the number n has time-bounded Kolmogorov complexity comparable to the seed length. Afterwards we show that a similar construction gives a generator fooling the same class with a slower runtime under the

weak form hardness assumptions; the proof of this construction will be relegated to the appendix.

Theorem 4.18. *Let $O \subseteq \{0, 1\}^*$ be any oracle, $d, v \in \mathbb{N}$, and assume Hypothesis $\text{SH}(O, d + v)$. Let $T(n) \leq (\exp^{[v]}(n))^{O(1)}$ be time constructible. There exists a generator $(\mathcal{G}_n^d : \{0, 1\}^{s_d(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ computable uniformly in $\text{poly}(T(O(n)))$ time, so that the following hold:*

1. *For all n , $s_d(n) \leq O(\log^{[d+1]}(n))$*
2. *For all $L \in \text{TIME}^O[T(n)]/\log^{[d]}(n)$, we have*

$$| \Pr_{z \sim \{0,1\}^{s_d(n)}} [\mathcal{G}_n^d(z) \in L] - \Pr_{x \sim \{0,1\}^n} [x \in L] | \leq O\left(\frac{1}{\log^{[d]}(n)}\right)$$

for all but finitely many $n \in \{n \mid K^{T(n)}(n) \leq \log^{[d]}(n)\}$.

Proof. Using our hardness assumption $\text{SH}(O, d + v)$ in combination with Lemma 4.15, for each time constructible bound $R : \mathbb{N} \rightarrow \mathbb{N}$ bounded above by $(\exp^{[d+v]}(n))^{O(1)}$ there is a constant $c(R) \in \mathbb{N}$ and a pseudorandom generator $(G_n^{(R)} : \{0, 1\}^{c(R)\lceil \log n \rceil} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ computable in $\text{TIME}(R(n)^{c(R)})$ which fools $\text{TIME}^B[R(n)]/3n$ with error $\frac{1}{n}$. Using $c(\cdot)$ we may define a sequence of constants $(c_i)_{i \in \mathbb{N}}$ and functions $(T_d, f_d : \mathbb{N} \rightarrow \mathbb{N})_{d \in \mathbb{N} \cup \{0\}}$ as follows:

1. $T_0 = T, f_0 = (x \mapsto x)$.
2. Set $c_d = c(T_{d-1}), f_d = c_d \cdot \lceil \log f_{d-1}(\cdot) \rceil, T_d = 4 \cdot T_{d-1}(\exp(\lceil \frac{\cdot}{c_d} \rceil))^{c_d}$

Observe that $3T_{d-1}(f_{d-1}(n)) + T_{d-1}(f_{d-1}(n))^{c_d} \leq T_d(f_d(n))$.

Then for each d , we have the generator $(G_n^{(T_d)})_{n \in \mathbb{N}}$ defined for every input, which we abbreviate as G^d . We use this family of generators to construct the generators $(\mathcal{G}_n^d)_{n \in \mathbb{N}}$ promised in the theorem statement. In particular, we define for every $d \geq 0$ the generator

$$(\mathcal{G}_n^d : \{0, 1\}^{f_{d+1}(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}, \quad \mathcal{G}_n^d = G_n^0 \circ G_{f_1(n)}^1 \circ \dots \circ G_{f_d(n)}^d$$

We prove its security by induction, using the following stronger induction hypothesis for each $d \geq 0$:

1. $\mathcal{G}_n^d$ is computable in time $\leq T_{d+1}(f_{d+1}(n)) - 3T(n)$ with the number n and $O(1)$ additional bits as advice
2. $\mathcal{G}_n^d$ fools any $L \subseteq \{0, 1\}^n$, $L \in \text{TIME}^B[T_d(f_d(n))]/f_d(n)$ with error $\leq \sum_{j \leq d} f_j(n)^{-1}$ for all sufficiently large n satisfying $K^{T(n)}(n) \leq f_d(n)$

For the base case, we know that $\mathcal{G}_n^0 = G_n^0$ which, by assumption, fools $\text{TIME}^B[T(n)]/n$ with error $\frac{1}{n}$. Recall that $T_1(f_1(n)) \geq 3T_0(f_0(n)) + T_0(f_0(n))^{c_1} = 3T(n) + T(n)^{c_1}$, hence $\mathcal{G}_n^0 = G_n^0$ is computable in time $T(n)^{c_1} \leq T_1(f_1(n)) - 3T(n)$ (with no advice). Now assume the induction hypothesis up to $d - 1$. Let $L \subseteq \{0, 1\}^n$ be given, n sufficiently large, so that $L \in \text{TIME}^B[T_d(f_d(n))]/f_d(n)$ and $K^{T(n)}(n) \leq f_d(n)$. Define $L_d \subseteq \{0, 1\}^{f_d(n)}$, $L_d = \{x \mid \mathcal{G}^{d-1}(x) \in L\}$. By induction, $\mathcal{G}^{d-1}$ is computable in time $T_d(f_d(n)) - 3T(n)$ given the number n as advice, and $O(1)$ additional advice bits; hence L_d is computable in $\text{TIME}^B[T_d(f_d(n))]/3f_d(n)$: if we are given the number n as advice, then we may determine $f_0(n), \dots, f_d(n)$ using the number d as advice which costs $O(1)$, after which we can compute the generator $\mathcal{G}_n^{d-1}$, and the language L using $\leq f_d(n)$ additional bits of advice (together with some $O(1)$ bits to specify the algorithm described in the current sentence). Under the assumption $K^{T(n)}(n) \leq \log^{[i]}(n) \leq f_i(n)$ we may produce the number n from an additional $f_i(n)$ bits of advice, for a total advice cost $\leq 3f_i(n)$, and the time of the additional operations (beyond the original computation of $\mathcal{G}^{d-1}$) is bounded by $3T(n)$.

Now, using the second part of our induction hypothesis, we determine that $\mathcal{G}_n^{i-1}$ fools L , in particular we have:

$$\left| \Pr_{z \in \{0,1\}^{f_i(n)}} [\mathcal{G}_n^{i-1}(z) \in L] - \Pr_{x \in \{0,1\}^n} [x \in L] \right| \leq \sum_{j < i} f_j(n)^{-1}$$

On the other hand, using the security of the generator G_n^d on input length $f_{d+1}(n)$ (which is sufficiently large), we have that G_n^d fools L_d , in particular:

$$\left| \Pr_{w \in \{0,1\}^{f_{d+1}(n)}} [G_n^d(w) \in L_i] - \Pr_{z \in \{0,1\}^{f_d(n)}} [z \in L_d] \right| \leq f_d(n)^{-1}$$

Recalling that $\mathcal{G}_n^d = \mathcal{G}_n^{d-1} \circ G_n^d$ and combining the previous two inequalities using the triangle inequality we have:

$$\left| \Pr_{z \in \{0,1\}^{f_{d+1}(n)}} [\mathcal{G}_n^d(w) \in L] - \Pr_{x \in \{0,1\}^n} [x \in L] \right| \leq \sum_{j \leq d} f_j(n)^{-1}$$

which establishes the second condition in our inductive hypothesis. For the first, note that to compute $\mathcal{G}_n^d$, we need only recover the number n from its shortest $K^T(\cdot)$ description in time $T(n)$, compute $\mathcal{G}_n^{i-1}$ which takes time $\leq T_i(f_i(n)) - 3T(n)$ by induction, and finally compute G_n^i which takes time $T_i(f_i(n))^{c_{i+1}}$; overall this is bounded by $T_{i+1}(f_{i+1}(n)) - 3T(n)$.

At this point the theorem is proven; it remains to verify a few bounds:

1. $\sum_{j \leq d} f_j(n)^{-1} \leq O(f_d(n)^{-1})$ so that the error bound is as stated in the theorem.
2. $f_d(n) \leq O(\log^{[d]}(n))$
3. $T_d(f_d(n)) \leq \text{poly}(T(O(n)))$

The first and second hold trivially. For the last, we prove it by induction; more specifically we will show by induction that $T_d(O(f_d(n))) \leq T(O(n))^{O(1)}$. In the base case, $T_0(O(f_0(n))) = T(O(n))$.

We then have that

$$T_d(O(f_d(n))) = 4 \cdot T_{d-1} \left(O \left(\exp \left(\left\lceil \frac{c_d \lceil \log f_{d-1} \rceil}{c_d} \right\rceil \right) \right) \right)^{c_d} \quad (4.1)$$

$$\leq \left(T_{d-1} (O(\exp(\log f_{d-1}(n) + O(1)))) \right)^{O(1)} \quad (4.2)$$

$$= T_{d-1}(O(f_{d-1}(n)))^{O(1)} \leq T(O(n))^{O(1)} \quad (4.3)$$

where the last step uses the induction hypothesis. □

We highlight an important special case of the above:

Theorem 4.19. *Assume that for some $\epsilon > 0$ and every time constructible $T(n) \leq 2^{O(n)}$, $m(n) \leq \text{poly}(n)$, there is a function $n \mapsto \{0, 1\}^{m(n)}$ computable uniformly in time $T(n)$ which is not com-*

putable in $\text{TIME}^{\text{NP}}[T(n)^\epsilon]/m(n)^\epsilon$ even infinitely often. Then for every $k \in \mathbb{N}$, there is a pseudorandom generator family $(\mathcal{G}_n)_{n \in \mathbb{N}}$ with seed length $O(\log \log n)$ which fools $\text{TIME}^{\text{NP}}[n^k]/\log n$ for all sufficiently large n .

Proof. We will set $v = 0$, $d = 1$, $T(n) = n^k$ in Theorem 4.18. Observe that for all n , we have $K^{O(n)}(n) \leq \log n$ since we may encode the number n in binary. $\square$

If we rely instead on the weak form hypotheses $\text{WH}(\cdot)$ we get:

Theorem 4.20. *Assume Hypothesis $\text{WH}(O, d+1, v)$ with $d \geq 0$, $v \geq 2$ fixed constants, and $k \in \mathbb{N}$ a fixed constant. Then there is a pseudorandom generator with seed length $O^{\langle v-1 \rangle}(\log^{[d+1]}(n))$ and runtime $O^{\langle d+v \rangle}(n)$ that fools $\text{TIME}^O[n^k]/\log^{[d]}(n)$ with error $(2 \log^{[d]}(n))^{-1}$ whenever $K^{n^k}(n) \leq \log^{[d]}(n)$.*

As in the case of Lemma 4.16, we relegate the proof to the appendix since it uses essentially the same ideas as that of Theorem 4.18 and is in fact a bit simpler. As before we highlight an important special case, obtained immediately from Theorem 4.20 by setting $O = \text{NP}$, $d = 1$, $v = 2$:

Theorem 4.21. *Assume that there is some $\epsilon > 0$ and a language $L \in \text{TIME}[2^{2^n}]$ which is not computable in $\text{TIME}^{\text{NP}}[2^{\epsilon 2^n}]/2^{\epsilon n}$ on more than finitely many input lengths. Then for any $k \in \mathbb{N}$ there is a pseudorandom generator fooling $\text{TIME}^{\text{NP}}[n^k]$ with seedlength $O(\log \log n)$ and quasipolynomial runtime.*

4.3.3 Fooling Uniform Deterministic Time

For our application to construction of random strings, we will use the generator in the previous section with oracle setting $O = \text{NP}$. It is nonetheless natural to consider the implications of our generator in the case $O = \emptyset$; in this case we obtain a generator with $o(\log n)$ seed length fooling $\text{TIME}[T(n)]$ under plausible hardness assumptions. However, we demonstrate here that in the regime of a deterministic distinguisher, once $O(\log n)$ seed length is achieved for $\text{TIME}[\text{poly}(n)]$, we can reduce the seed length all the way down to an arbitrarily small super-constant value; more

generally, we can fool $\text{TIME}[\text{poly}(n)]/a(n)$ with essentially optimal seed length $O(\log a(n))$ for any time constructible $a(n) \leq \log n$. The discrepancy between the simplicity of the result here and the work required in the previous section is due to the key distinction that in this regime, the pseudorandom generator has enough resources to *simulate* the distinguishers it is trying to fool.

Theorem 4.22. *Let $a(n) \leq \log n$ be time constructible. Assuming E requires $2^{\Omega(n)}$ -size Boolean circuits for sufficiently large n , there is a polynomial time computable generator $\mathcal{G}^a : \{0, 1\}^{O(\log a(n))} \rightarrow \{0, 1\}^n$ which fools $\text{TIME}[n]/a(n)$ with error $\leq \frac{1}{3}$.*

Proof. Using the hardness assumption we obtain (uniformly in n) a pseudorandom generator with seed length $c \log n$ for some constant c . Setting $m = n^c$ and enumerating the outputs of our PRG, we obtain a list of n bit strings $x^1, \dots, x^m \in \{0, 1\}^n$ which is a pseudorandom generator for $\text{TIME}[n]/n$. Let $r = 2^{a(n)}$, let $L_1, \dots, L_r$ enumerate $\text{TIME}[n]/a(n)$ machines. Define the matrix $M \in \{0, 1\}^{m \times r}$ with $M(i, j) = L_j(x^i)$; we may compute M in $\text{poly}(n)$ time.

We now deterministically construct a set $I \subseteq [m]$ of size $a(n)^{O(1)}$ so that for every j we have

$$|\Pr_{i \sim I}[M(i, j) = 1] - \Pr_{i \sim [m]}[M(i, j) = 1]| \leq \frac{1}{3}$$

If we can accomplish this, we output the condensed PRG whose range consists of the strings $\{x^i \mid i \in I\}$ and has seed length $\lceil \log |I| \rceil = O(\log a(n))$ and are finished. The algorithm to find the set I is given in the next lemma. $\square$

Lemma 4.23. *There is a polynomial time algorithm which, given $M \in \{0, 1\}^{m \times r}$ with $r \leq m^{O(1)}$, outputs a set $I \subseteq [m]$ of rows so that $|I| \leq O(\log r)$, and for every column $j \in [r]$, we have*

$$|\Pr_{i \sim I}[M(i, j) = 1] - \Pr_{i \sim [m]}[M(i, j) = 1]| \leq \frac{1}{3}$$

Proof. Using known results on the so-called “set balancing problem” [127], for any $I \subseteq [m]$ we

may efficiently compute $I' \subseteq I$, $|I'| \leq \frac{|I|}{2} + \sqrt{|I| \log r}$ so that

$$| \Pr_{i \sim I} [M(i, j) = 1] - \Pr_{i \sim I'} [M(i, j) = 1] | \leq \sqrt{\frac{\log r}{|I|}}$$

for every j . In particular, provided $\sqrt{|I| \log r} \leq \frac{1}{4}|I|$, i.e. $|I| \geq 16 \log r$, we have that $|I'| \leq \frac{3}{4}|I|$.

Initializing $I_0 = [m]$, we may apply the above to obtain $I_0 \supseteq I_1 \supseteq \dots \supseteq I_q$, $|I_q| = \Theta(\log r)$, and for every j

$$| \Pr_{i \sim I_\ell} [M(i, j) = 1] - \Pr_{i \sim I_{\ell+1}} [M(i, j) = 1] | \leq \sqrt{\frac{\log r}{|I_\ell|}}$$

Hence

$$| \Pr_{i \sim [m]} [M(i, j) = 1] - \Pr_{i \sim I_q} [M(i, j) = 1] | \leq \sum_{\ell \leq q} \sqrt{\frac{\log r}{|I_\ell|}}$$

For a suitable choice of q we will have $\sqrt{\frac{\log r}{|I_q|}} = \epsilon$ for an arbitrarily small constant ϵ , and $\sqrt{\frac{\log r}{|I_\ell|}} \leq \sqrt{\frac{3}{4}} \cdot \sqrt{\frac{\log r}{|I_{\ell+1}|}}$, hence

$$| \Pr_{i \sim [m]} [M(i, j) = 1] - \Pr_{i \sim I_q} [M(i, j) = 1] | \leq \sum_{\ell \leq q} \sqrt{\frac{\log r}{|I_\ell|}} \leq \epsilon \sum_{\ell=0}^{\infty} \left(\sqrt{\frac{3}{4}}\right)^\ell < \frac{1}{3}$$

□

4.3.4 Optimality of the Seed Length

We include a simple (and basically standard) argument which indicates that the seed lengths obtained in the previous are essentially optimal with respect to the amount of advice they can handle:

Lemma 4.24. *Let $s(n) \leq \log n$ be time-constructible, and $G = (G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ an arbitrary family of generators which fools $\text{TIME}[n]/a(n)$ almost everywhere with error $\frac{1}{2}$. Then $s(n) \geq \log a(n)$.*

In particular, if G fools $\text{TIME}[n]$ then $s(n) \geq \lambda(n)$ for some inverse-total computable $\lambda(n) = \omega(1)$.

Proof. Let $\ell(n) = s(n) + 1$, and consider the family $\mathcal{L}$ of languages L with $L_n(x)$ depending only on the first $\ell(n)$ bits of x , and $|L_n| = 2^{n-1}$. For each $n \in \mathbb{N}$, we have that $|\text{range}(G_n)| \leq 2^{\ell(n)-1}$, hence there exists a language $L \in \mathcal{L}$ so that for all n sufficiently large, we have $\text{range}(G_n) \cap L_n = \emptyset$. On the other hand for every n we have $\Pr_{x \sim \{0,1\}^n}[x \in L] = \frac{1}{2}$, hence G_n fails to fool the language L_n for all n sufficiently large. Every language $L \in \mathcal{L}$ is decidable in deterministic time $n + 2^{\ell(n)}$ with $2^{\ell(n)}$ bits of advice (we only need that $s(n)$, and hence $\ell(n)$ are time constructible) so we get a contradiction if $2^{\ell(n)} \leq O(n)$ or $2^{\ell(n)} \leq a(n)$. $\square$

In the case of deterministic time (Section 4.3.3) this implies that the stated construction is optimal for advice complexity $\leq \log n$. For our main construction fooling TIME^{NP} , we have no indication that to fool uniform languages (no advice) requires iterated-logarithmic seed length. However Theorem 4.18 is able to fool languages with advice complexity $\log^{[d]}(n)$ with seed length $O(\log^{[d+1]}(n))$, so in this sense we are able to achieve the maximum advice complexity for the given seed length $O(\log^{[d+1]}(n))$. It is consistent with our current knowledge that fooling completely uniform algorithms, even with an NP oracle, is achievable with arbitrarily slow-growing time constructible seed lengths, as we can achieve in the case of deterministic adversaries (under standard hardness assumptions).

4.4 Construction of Random Strings and Applications

In this section we discuss the applications of our main result to explicit constructions of random strings and circuit lower bounds.

4.4.1 Random String Construction Via PRG Concatenation

We start by applying our PRG constructions to give polynomial time explicit constructions of highly random strings. The first is a direct combination of our main PRG constructions with Observations 4.11 and 4.12 from the preliminaries. We start with the situation in which we use the strong form of our hardness assumptions, and hence Theorem 4.18 as our PRG:

Theorem 4.25. *Let $d \in \mathbb{N}$, $T(n) \leq (\exp^{[c]}(n))^{O(1)}$, and assume Hypothesis SH(NP, $d + c$). Then for any constant $k \in \mathbb{N}$, there is a polynomial time algorithm $\mathcal{A}$ so that, for all n with $K^{T(n)}(n) \leq \log^{[d]}(n)$, $\mathcal{A}(1^n) \in \{0, 1\}^n$ is an n -bit string x with $K^{T(n)}(x) \geq \Omega(\frac{n}{(\log^{[d]}(n))^{O(1)}})$.*

In the special case $d = 1$, $c = 0$, under a hardness assumption for singly exponential time bounds and NP oracles we obtain an algorithm $\mathcal{A}$ which produces, for every $n \in \mathbb{N}$, a string x of length n with $K^{T(n)}(x) \geq \frac{n}{\text{poly}(\log n)}$ for all n .

More generally, a hardness assumption for exponential time bounds and $\mathcal{O}'$ oracles yields an efficient algorithm for computing strings with $\tilde{\Omega}(n)$ $\mathcal{O}$ -oracle time-bounded Kolmogorov complexity, provided there is a $\text{poly}(T(n))$ time $\mathcal{O}'$ oracle algorithm which can test the $K^{T(n), \mathcal{O}}$ complexity of an n -bit string.

Proof. This is a rather direct corollary of Theorem 4.18. Let n be given with $K^{T(n)}(n) \leq \log^{[d]}(n)$, and let $s_d(\cdot) \leq O(\log^{[d+1]}(\cdot))$ be the seed length bound of the generator $\mathcal{G}^d$ from Theorem 4.18 using the base time bound $T(n)$. Let n' be the largest integer such that $n'2^{s_d(n')} \leq n$; clearly $K^{T(n)}(n') \leq K^{T(n)}(n) + O(1) \leq \log^{[d]}(n)$, hence we may apply Theorem 4.18 to compute (uniformly) a pseudorandom generator $\mathcal{G}_{n'}^d : \{0, 1\}^{s_d(n')} \rightarrow \{0, 1\}^{n'}$ which fools $\text{TIME}^{\text{NP}}[T(n)]/O(1)$ on input length n' . By Observation 4.12 we conclude that some string z in the range of this generator has $K^{T(n)}(z) \geq n' - 1$, hence the concatenation of all strings in the range of $\mathcal{G}_{n'}^d$ (padded at the end with 0s to bring the length up to n) will have $K^{T(n)}(\cdot)$ complexity at least $n' - s_d(n') \geq \frac{n}{(\log^{[d]}(n))^{O(1)}}$ (recall Observation 4.11).

The second part of the theorem is the special parameter setting given by Theorem 4.19, and for the last sentence in the statement, we use the fact that Theorem 4.19 works for any oracle $\mathcal{O}'$. $\square$

If we instead rely on the weaker form hardness assumptions and the associated PRG from Theorem 4.20 we obtain via the exact same argument:

Theorem 4.26. *Assume Hypothesis WH(NP, $d + 1, v$) with $d \geq 0$, $v \geq 2$. Then for every $k \in \mathbb{N}$, there is a $O^{(d+v)}(n)$ time algorithm $\mathcal{A}$ such that for all sufficiently large n , $\mathcal{A}(1^n) \in \{0, 1\}^n$ is a string x satisfying $K^{n^k}(x) \geq \frac{n}{O^{(v)}(\log^{[d]}(n))}$.*

In particular, assuming that there is a language in $\text{TIME}[2^{2^n}]$ which cannot be decided in $\text{TIME}^{\text{NP}}[2^{\epsilon 2^n}]/2^{\epsilon n}$ infinitely often, there is a quasipolynomial time algorithm which outputs strings $x \in \{0, 1\}^n$ with $K^{n^k}(x) \geq \frac{n}{\text{poly}(\log n)}$.

More generally, a hardness assumption for exponential time bounds and $\mathcal{O}'$ oracles yields an efficient algorithm for computing strings with $\tilde{\Omega}(n)$ $\mathcal{O}$ -oracle time-bounded Kolmogorov complexity, provided there is a $\text{poly}(T(n))$ time $\mathcal{O}'$ oracle algorithm which can test the $K^{T(n), \mathcal{O}}$ complexity of an n -bit string.

We now move on to a second class of constructions which achieves a far superior degree of randomness, but the cost is that our construction only works for infinitely many n , and unlike the previous we have no control over which specific values of n our construction succeeds on.

Theorem 4.27. *Assume Hypothesis $\text{SH}(\text{NP}, d)$. Then for any constant $k \in \mathbb{N}$, there is a polynomial time algorithm $\mathcal{A}$ so that, for infinitely many n , $\mathcal{A}(1^n) \in \{0, 1\}^n$ is an n -bit string x with $K^{n^k}(x) \geq n - \text{poly}(\log^{[d]} n)$.*

Proof. Applying the assumption with Theorem 4.18 and the argument in the previous proof, we have an algorithm $\mathcal{A}$ which prints, for all sufficiently large n in $N_d := \{n \mid n = \exp^{[d+1]}(n') \text{ for some } n'\}$, a list of $\text{poly}(\log^{[d]}(n))$ strings, one of which must have $K^{n^k}(n) \geq n - 1$. We now define a second algorithm $\mathcal{A}'$, which, given n , computes the largest value $m \leq n$ with $m \in N$, and prints the $(n-m)^{\text{th}}$ element of the list $\mathcal{A}(1^m)$ (if $(n-m)$ is larger than the length of the list, it does something arbitrary). For every $m \in N$, there is some $n \leq m + \text{poly}(\log^{[d]}(n))$ which prints a string x of length m with $K^{n^k}(x) \geq m - 1 \geq n - \text{poly}(\log^{[d]}(n))$. $\square$

As before, a modified variant of this construction can be done in the case of the alternative weak form hardness assumptions, where the cost of using the weaker kind of assumption is that the run time of the construction will degrade correspondingly; we leave the details to the interested reader as we will not use this variant in what follows.

4.4.2 Explicit Constructions Under Plausible Hardness Assumptions

By our main result, under reasonable hardness assumptions we are able to obtain explicit constructions of strings with time-bounded Kolmogorov complexity $\frac{n}{\beta(n)}$, where $\beta(n)$ grows like an arbitrarily slow iterated logarithm function. We show here how to use this construction to build various important pseudorandom objects under our main hardness assumptions. We start with a list of fundamental explicit construction problems, focusing on those studied originally in [102], in addition to so-called “incompressible functions” considered in [9]. For background on substantial literature dedicated to these problems see [102, 9],

Definition 4.28 (List of Explicit Construction Problems).

- *(s, r)-Rigidity: A matrix $M \in \{0, 1\}^{n \times n}$ such that $M \neq S + R$ whenever $S, R \in \mathbb{F}_2^{n \times n}$, S has at most s nonzero entries, R has rank at most r*
- *s-Hard Boolean Functions: A Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ not computed by any Boolean circuit of size $\leq s$*
- *(s, k)-Incompressible Boolean Functions: A Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ which cannot be expressed in the form $f(x) = g(C(x))$ where $C : \{0, 1\}^n \rightarrow \{0, 1\}^k$ is a circuit of size s and $g : \{0, 1\}^k \rightarrow \{0, 1\}$ is an arbitrary Boolean function*
- *s-PSPACE^{cc}-Complexity: A communication matrix $M \in \{0, 1\}^{2^n \times 2^n}$ such that M cannot be solved by space- s communication protocols*
- *(s, t)-Bit-Probe Complexity: A data structure problem $D \in \{0, 1\}^{2^n \times 2^n}$ which requires space $\geq s$ or time $\geq t$ in the bit-probe model.*
- *Ramsey: A graph $G \in \binom{[n]}{2}$ with no clique or independent set of size $\geq 2.1 \cdot \log n$*

We then have:

Lemma 4.29. *Say that an AVOID instance has “stretch $n \mapsto m$ ” if it is of the form $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$. The following uniform reductions exist from the problems in Definition 4.28 to Range Avoidance:*

1. *(s, r) -Rigidity reduces uniformly to AVOID with stretch $(2s \log n + 2nr) \mapsto n^2$*
2. *s -Hard Boolean Function reduces uniformly to AVOID with stretch $(1 + o(1))s \log s \mapsto 2^n$*
3. *(s, k) -Incompressible Boolean Functions reduces uniformly to AVOID with stretch $2^k + (1 + o(1))s \log s \mapsto 2^n$*
4. *s -PSPACE^{cc}-Complexity reduces uniformly to AVOID with stretch $2^{O(s)+n} \mapsto 2^{2n}$*
5. *(s, t) -Bit-Probe Complexity reduces uniformly to AVOID with stretch $s2^n + 2^{t+n} \mapsto 2^{2n}$*
6. *Ramsey reduces uniformly to AVOID with stretch $n - \Omega(\log n) \mapsto n$*

Proof. All of these proofs occur in [102], with the exception of Incompressible Functions. For this, if $f(x) = g(C(x))$ for a circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^k$ of size s and $g : \{0, 1\}^k \rightarrow \{0, 1\}$, we may represent the circuit C (gate by gate) using $(1 + o(1))s \log s$ bits via a standard encoding, and the function g via its 2^k -bit truth table. From these it is clear how we may reproduce the function f efficiently. □

Combining this with our first generator construction from the previous section and Observation 4.10, we have:

Theorem 4.30. *Assume Hypothesis SH(NP, d). Then for all n with $K^{\text{poly}(n)}(n) \leq \log^{[d]}(n)$, we have the following:*

1. *Polynomial time construction of matrices in $\mathbb{F}_2^{n \times n}$ which are $(\frac{n^2}{\log n \cdot \text{poly}(\log^{[d]}(n))}, \frac{n}{\text{poly}(\log^{[d]}(n))})$ -rigid. In particular, for any $d > 2$ such matrices achieve Valiant rigidity.*
2. *Boolean functions in $E = \text{TIME}[2^{O(n)}]$ with circuit complexity at least $\frac{2^n}{n \cdot \text{poly}(\log^{[d-1]}(n))}$*

3. *Boolean functions in $E = \text{TIME}[2^{O(n)}]$ which are $(\Omega(\frac{2^n}{n \cdot \text{poly}(\log^{[d-1]}(n))}), n - O(\log^{[d]} n))$ incompressible*
4. *Polynomial time construction of communication matrices $M \in \{0, 1\}^{2^n \times 2^n}$ which require space $\Omega(n)$ (for any setting of d)*
5. *Polynomial time construction of data structure problems $D \in \{0, 1\}^{2^n \times 2^n}$ which require space $\Omega(2^n)$ or time $\Omega(n)$ in the bit probe model (for any setting of d)*

In the important case $d = 1$, each of the constructions works for all n and requires the hardness assumption only for singly-exponential time bounds.

We arrive at similar conclusions if we use the weak form hypothesis in combination with Theorem 4.20 instead, at the cost of a slow-down in our construction algorithms. We highlight below the results obtained by using the special case of Theorem 4.20 given in Theorem 4.21:

Theorem 4.31. *Assume that there is some $\epsilon > 0$ and a language $L \in \text{TIME}[2^{2^n}]$ which is not computable in $\text{TIME}^{\text{NP}}[2^{\epsilon 2^n}]/2^{\epsilon n}$ on more than finitely many input lengths (this is Hypothesis $\text{WH}(\text{NP}, 1, 2)$). Then we have the following for all n :*

1. *Quasipolynomial time construction of matrices in $\mathbb{F}_2^{n \times n}$ which are $(\frac{n^2}{(\log n)^{O(1)}}, \frac{n}{(\log n)^{O(1)}})$ -rigid.*
2. *Boolean functions in $\text{EXP} = \text{TIME}[2^{n^{O(1)}}]$ with circuit complexity at least $\frac{2^n}{\text{poly}(n)}$*
3. *Boolean functions in EXP which are $(\Omega(\frac{2^n}{\text{poly}(n)}), n - O(\log n))$ incompressible*
4. *Quasipolynomial time construction of communication matrices $M \in \{0, 1\}^{2^n \times 2^n}$ which require space $\Omega(n)$ (for any setting of d)*
5. *Quasipolynomial time construction of data structure problems $D \in \{0, 1\}^{2^n \times 2^n}$ which require space $\Omega(2^n)$ or time $\Omega(n)$ in the bit probe model (for any setting of d)*

The above results cover the vast majority of explicit construction problems considered originally in [102]. A notable exception is the construction of near-optimal Ramsey graphs; here

the stretch $n - \Omega(\log n) \mapsto n$ given by Lemma 4.29 is too small to apply Theorem 4.25, and we must appeal instead to Theorem 4.27. Using this approach, we will be able to construct near-optimal Ramsey graphs infinitely often, provided we modify appropriately our encoding of graphs by strings to be well-defined on every input length:

Definition 4.32 (Ramsey Graphs of Every Length). *We associate every string $x \in \{0, 1\}^*$ to a graph G_x as follows. Let $n = |x|$ and set n' to be the greatest integer so that $\binom{n'}{2} \leq n$. Set $m = \binom{n'}{2}$, truncate x to the first m bits and interpret it as a graph on $\binom{n'}{2}$ vertices canonically. Under this encoding, we say that x encodes a Ramsey graph if G_x contains no clique or independent set of size $\geq 2.1 \cdot \log n'$.*

We then have:

Lemma 4.33. *Assume Hypothesis SH(NP, 2). Then there is a polynomial time algorithm which, for infinitely many n , outputs a string $x \in \{0, 1\}^n$ so that G_x is Ramsey.*

Proof. We claim that we may uniformly construct a Range Avoidance instance $C : \{0, 1\}^{n-\Omega(n)} \rightarrow \{0, 1\}^n$ so that for all n , $\text{range}(C_x)$ contains every x such that G_x fails to be Ramsey. In particular, given n we may compute efficiently the parameters n' , m being the number of vertices and edges for the graphs $\{G_x \mid x \in \{0, 1\}^n\}$; we then apply Lemma 4.29 to obtain (uniformly) a Range Avoidance instance with stretch $m - \Omega(\log m) \mapsto m$ covering all strings $z \in \{0, 1\}^m$ which fail to encode Ramsey graphs. If $x \in \{0, 1\}^n$ fails to be Ramsey, then $x = zy$ for some $z \in \{0, 1\}^m$ which fails to be Ramsey, hence we may uniformly construct an AVOID instance covering every non-Ramsey string $x \in \{0, 1\}^n$ with stretch $n - \Omega(\log n') \mapsto n$, which is $n - \Omega(\log n) \mapsto n$. We thus conclude that $K^{\text{poly}(n)}(x) \leq n - \Omega(\log n)$ whenever G_x fails to be Ramsey. Applying Theorem 4.27 and our hardness assumption yields the lemma. $\square$

4.4.3 Hardness Condensation

We also apply Theorem 4.25 to derive a new *hardness condensation* result. Hardness condensation is a phenomenon where a mild hardness assumption can be transformed into a much

stronger hardness assumption of the same type. It was first studied in [21], who showed hardness condensation results for complexity classes with advice. It is more interesting to get hardness condensation for uniform classes, and indeed [102] achieves this for E^{NP} - he shows that E^{NP} requires exponential-size circuits iff it requires almost maximum-size circuits.

We obtain a new hardness condensation result for deterministic time *without* an NP-oracle. Here the condensation is with respect to non-uniform complexity - the stronger hardness assumption can handle almost a maximum non-trivial amount of non-uniformity, while the weaker hardness assumption only involves an exponential amount of non-uniformity.

Theorem 4.34. *The following are equivalent:*

1. *For every d there exists v so that, for all time constructible $T(n) \leq \exp^{[d]}(n)$, we have $\text{TIME}[T(n)]$ is not contained even infinitely often in $\text{SPACE}[o^{(v)}(T(n))]/o^{(v)}(2^n)$*
2. *For every d there exists v so that, for all time constructible $T(n) = \exp^{[d]}(n)$, we have $\text{TIME}[T(n)]$ is not contained even infinitely often in $\text{SPACE}[o^{(v)}(T(n))]/2^{n-\omega(\log n)}$*

Proof. Clearly the second item implies the first. We show that the first item implies the second. Assuming (1), we have that every d there exists v so that $\text{WH}(\text{QBF}, d, v)$ holds. Applying Theorem 4.26, for every d there is v so that for any time constructible $T \leq \exp^{[d]}(n)$, there is an algorithm A running in time $O^{(v)}(T(2^n))$ which, given 1^{2^n} , outputs a string of length 2^n which cannot be produced by any algorithm running in space $T(2^n)$ with $2^{n-\omega(\log n)}$ bits of advice⁴. Interpreting this string as a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, we obtain a language in $\text{TIME}[O^{(v)}(T(2^n))]$ which is not contained even infinitely often in $\text{SPACE}[T(2^n)]/2^{n-\omega(\log n)}$. Reparameterizing the time bounds yields (2). $\square$

4.4.4 Range Avoidance vs Uniform Range Avoidance

We use our results together with previous work to argue that the Range Avoidance problem becomes much more tractable for uniformly computable maps. While there is compelling

⁴We are using here the fact that $\text{SPACE}[T(n)]$ can be simulated in $\text{TIME}^{\text{QBF}}[\text{poly}(T(n))]$

complexity-theoretic evidence in various settings that Range Avoidance is intractable in general, our results show that Uniform Range Avoidance is tractable under plausible complexity-theoretic assumptions.

The first setting we consider is where the Range Avoidance instance is an arbitrary polynomial-size circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, and we wish to solve Range Avoidance in polynomial time.

Ilango, Li and Williams [80] showed that in this setting, Range Avoidance is infeasible, under the standard assumption that $\text{NP} \neq \text{coNP}$ and the plausible assumption that subexponentially-secure indistinguishability obfuscation exists [87].

Theorem 4.35. *[80] If $\text{NP} \neq \text{coNP}$ and subexponentially-secure indistinguishability obfuscation exists, then for any $c \geq 0$, there is no polynomial-time algorithm which solves Range Avoidance for polynomial-size circuits mapping n to n^c bits.*

Corollary 4.36. *Assume that $\text{NP} \neq \text{coNP}$, subexponentially-secure indistinguishability obfuscation, and Hypothesis $\text{SH}(\text{NP}, 1)$. There is a constant c such that for all $d \geq c$, there is a polynomial-time algorithm which solves Range Avoidance on uniform circuits of size n^d mapping n bits to n^d bits but no polynomial-time algorithm which solves Range Avoidance on all circuits of size n^d mapping n bits to n^d bits.*

Proof. We show that we can take c to be $1 + \delta$, for arbitrarily small $\delta > 0$. Indeed, for such c and $d \geq c$, the intractability of general Range Avoidance follows from Theorem 4.35, under the given assumptions. We show that uniform Range Avoidance can be solved by applying Theorem 4.25 and using the third assumption. Indeed, by this assumption, for every constant k , there exists a polynomial-time algorithm A_k which for all n , on input 1^n outputs a length- n string of $\mathcal{K}^{n^k}$ complexity at least $n/\text{polylog}(n)$.

Let $\{C_n\}$ be a sequence of uniform circuits mapping n bits to n^d , where d is any constant greater than 1. Here our notion of uniformity is standard DLOGTIME-uniformity, but the argument can be adapted to work for more relaxed notions of uniformity. Note that any output $y = C_n(x)$ has $\mathcal{K}^{n^k}$ complexity at most $n + O(\log(n))$ for any $k > d$, as we can first generate C_n given n using the

uniformity condition and then generate y from C_n and x by simulating C_n . By running A_k on input 1^{n^d} , we obtain a string of length n^d and $K^{n^{kd}}$ complexity at least $n^d/\text{polylog}(n)$, which is therefore a non-output of C_n when n is large enough. $\square$

We also consider the setting where the Range Avoidance instance is an arbitrary polynomial-size TQBF-oracle circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, and we wish to solve Range Avoidance in polynomial time. The negative evidence for this is even stronger - the task is intractable under the standard assumption $\text{PSPACE} \neq \text{NP}$. Somewhat surprisingly, we show that uniform Range Avoidance is tractable under plausible assumptions even though the uniform Range Avoidance instance C is allowed to use a TQBF-oracle.

Theorem 4.37. *[1] Suppose $\text{PSPACE} \neq \text{NP}$ (resp. $\text{PSPACE} \not\subseteq \text{NTIME}[2^{\log^{O(1)} n}]$). There is a constant c such that for all $d \geq c$, there is no polynomial-time (resp. quasipolynomial time) algorithm which solves Range Avoidance on all TQBF-oracle circuits of size n^d .*

In fact, [1] showed that computing an KS-incompressible string x of length n conditional on a given string y is hard for polynomial time if $\text{PSPACE} \neq \text{NP}$, where KS is Kolmogorov space-bounded complexity. This is easily seen to imply the result above by considering the Range Avoidance instance which takes a TQBF-oracle program as input and evaluates it. The extension to quasipolynomial time bounds is not stated in [1] but follows immediately from the proof.

Corollary 4.38. *Assume $\text{SH}(\text{PSPACE}, 1)$, and that $\text{PSPACE} \not\subseteq \text{NTIME}[2^{\log^{O(1)} n}]$. Then there is a constant c such that for all $d \geq c$, there is a polynomial-time algorithm which solves Range Avoidance on uniform TQBF-oracle circuits of size n^d but no polynomial-time algorithm which solves Range Avoidance on all TQBF-oracle circuits of size n^d .*

Proof. For the negative result on Range Avoidance, we simply apply Theorem 4.37. For the positive result, we apply Theorem 4.25 (relativized to a TQBF oracle) and then use the same argument as in the proof of the previous corollary. $\square$

4.5 Barriers to Improvements

In this section we present two barriers to improving our main results. First we show that the seed length $\log^{[O(1)]} n$ achieved in our PRG from Section 4.3 is in some sense optimal for hardness/randomness approaches which use the same overall structure as our argument: specifically, we show that our PRG construction corresponds to a construction of a special kind of seeded extractor with seed length $\log^{[O(1)]} n$, and prove unconditionally that this is the minimal achievable seed length in any such construction. Second we show that there is no relativizing argument that directly reduces the construction of $K^{\text{poly}(n)}$ -random strings to the construction of hard truth tables.

4.5.1 Multi-Reconstructive Extractors for Block Sources

The classical approach to turning hardness into randomness [130, 83] was famously shown by Trevisan [158] to be essentially equivalent to the task of constructing an explicit seeded extractor. Roughly speaking, Trevisan showed that any method for producing a pseudorandom generator using a hard boolean function whose correctness is proven in a sufficiently *black box* fashion must actually produce a seeded extractor, which treats the hard function as a high min-entropy source and the seed of the PRG as the seed in the seeded extractor.

In Section 4.3 a PRG was constructed against *uniform* algorithms (or more generally, algorithms with extremely small advice), whose seed length was much smaller than $\log n$, using a different kind of hardness assumption applied across many different input lengths. In the section we cast such a hardness/randomness construction in terms of certain seeded *multi-source* extractors, where each source corresponds to a hardness assumption used at a different input length. Through this connection we are able to show that the iterated logarithmic ($\log^{[d]}(n)$, $d = O(1)$) seed length in our PRGs from Section 4.3 is necessary for any hardness/randomness tradeoff that uses the same general framework as ours, and in particular which uses “only $O(1)$ many” hardness assumptions.

We start by recalling the proof of correctness of the PRG from Section 4.3, first in the case

where $O(\log \log n)$ seed length is achieved using a hardness assumption at two different input lengths. We first used a hard function whose truth table had size $\text{poly}(n)$ in order to get a PRG with seed length $O(\log n)$; call this f_1 . We then bounded the run time of this PRG by some $T(n) = \text{poly}(n)$, and in the next step required a hard function f_2 of truth table size $\approx \log n$, which was hard for algorithms with running time $T(n)$; in particular f_2 needed to be hard even in the regime where the computation of f_1 is easy. For this reason, we may roughly interpret this as saying, f_2 is a function (of much smaller input length) which is hard *conditioned on* f_1 . When we iterate the argument to reduce the seed length to $\log^{[d]}(n)$, we require a sequence of functions $f_1, \dots, f_d$, of smaller and smaller input lengths, with f_j being hard for algorithms with enough running time to compute all of $f_1, \dots, f_{j-1}$, i.e. which is hard conditioned on $f_1, \dots, f_{j-1}$. This is naturally analogous to the following well-studied information-theoretic notion of a sequence of random variables, each with high min-entropy conditioned on all of the previous:

Definition 4.39 (Block Sources [36]). *A random variable $\bar{X} = (X_1, \dots, X_d)$ is said to be a block source with d blocks and entropy sequence $(k_1, \dots, k_d)$, if for every $(x_1, \dots, x_d) \in \text{supp}(\bar{X})$ and every $j \leq d$,*

$$H_\infty(X_j \mid X_1 = x_1, \dots, X_{j-1} = x_{j-1}) \geq k_j$$

In the case $j = 1$, this means $H_\infty(X_1) \geq k_1$.

A (seeded) block source extractor is a function which can produce nearly uniform randomness from a block source and an independent uniform seed:

Definition 4.40 (Seeded Block Source Extractors). *We say that a function $E : \prod_{j \leq d} \{0, 1\}^{m_j} \times \{0, 1\}^s \rightarrow \{0, 1\}^n$ is a block source extractor for entropy sequence $(k_1, \dots, k_d)$ and error ϵ if, for every block source $\bar{X} = (X_1, \dots, X_d)$ supported on $\prod_{j \leq d} \{0, 1\}^{m_j}$ with entropy sequence $(k_1, \dots, k_d)$, we have that $E(\bar{X}, \mathcal{U}_s)$ is ϵ -close to $\mathcal{U}_n$ in TV distance, where $\mathcal{U}_s$ is the uniform distribution on $\{0, 1\}^s$ (independent of the block source) and $\mathcal{U}_n$ is uniform on $\{0, 1\}^n$. The parameter s is called the seed length of the extractor.*

We will now show that our PRG construction can be viewed as giving a specific kind of explicit *reconstructive* extractor for block sources (with a seed), in the same sense that the standard hardness randomness constructions can be viewed as reconstructive single-source extractors via Trevisan's connection. We define such reconstructive seeded block source extractors as follows:

Definition 4.41 (Multi-Reconstructive Extractors). *Let $E : \prod_{j \leq d} \{0, 1\}^{m_j} \times \{0, 1\}^s \rightarrow \{0, 1\}^n$. We say that E is a multi-reconstructive extractor for entropy sequence $(k_1, \dots, k_d)$ and error ϵ if there are $\text{poly}(m_1, \dots, m_d)$ -time algorithms $R_1, \dots, R_d$, each taking oracle access to some $D : \{0, 1\}^n \rightarrow \{0, 1\}$, with the following behavior*

1. *Each R_j takes strings $x_1, \dots, x_{j-1} \in \{0, 1\}^{m_1}, \dots, \{0, 1\}^{m_{j-1}}$ and an advice string $a_j \in \{0, 1\}^{k_j}$ and outputs a string in $\{0, 1\}^{m_j}$ (in the case $j = 1$ the only input to R_1 is $a_1 \in \{0, 1\}^{k_1}$).*
2. *For every $\bar{x} \in \prod_{j \leq d} \{0, 1\}^{m_j}$, if*

$$\left| \Pr_{z \sim \{0, 1\}^s} [D(E(\bar{x}, z)) = 1] - \Pr_{y \sim \{0, 1\}^n} [D(y) = 1] \right| \geq \epsilon$$

then there exists $j \leq d$ and some $a_j \in \{0, 1\}^{k_j}$ such that $R_j^D(x_1, \dots, x_{j-1}, a_j) = x_j$

Note that in the case $d = 1$, we obtain the standard definition of reconstructive single source extractors. As shown in the single-source case by Trevisan [158], we may observe that any multi-reconstructive extractor is automatically a block source extractor with similar parameters:

Lemma 4.42. *If $E : \prod_{j \leq d} \{0, 1\}^{m_j} \times \{0, 1\}^s \rightarrow \{0, 1\}^n$ is a multi-reconstructive extractor for entropy sequence $(k_1, \dots, k_d)$ and error ϵ , then it is a block source extractor for entropy sequence $(2k_1, \dots, 2k_d)$ and error $\epsilon' = \epsilon + \sum_{j \leq d} 2^{-k_j}$.*

Proof. Let $\bar{X}$ be a block source with entropy sequence $(2k_1, \dots, 2k_d)$. The deviation of $E(\bar{X}, \mathcal{U}_s)$ from uniform is bounded by $\epsilon + \delta$, where δ is the maximum over all distinguishers D of the probability that any of the d reconstruction procedures associated with E succeed on a random

sample $(x_1, \dots, x_d) \sim \bar{\mathcal{X}}$ from the source. We bound the probability for each reconstruction procedure separately and take a union bound over $j \leq d$. For a fixed D and any fixing of $\mathcal{X}_1 = x_1, \dots, \mathcal{X}_{j-1} = x_{j-1}$, there are at most 2^{k_j} values in $\{0, 1\}^{m_j}$ that $R_j^D(x_1, \dots, x_{j-1}, a_j)$ can output as we range over a_j ; if $H_\infty(\mathcal{X}_j \mid \mathcal{X}_1 = x_1, \dots, \mathcal{X}_{j-1} = x_{j-1}) \geq 2k_j$ then the probability that $\mathcal{X}_j$ lies in this set is at most 2^{-k_j} . $\square$

We can now recast the central step in our main construction in Theorem 4.18 in the language of reconstructive extractors:

Theorem 4.43 (Essentially in [131]). *Let $E^1, \dots, E^d$ be single source extractors $E^j : \{0, 1\}^{m_j} \times \{0, 1\}^{s_j} \rightarrow \{0, 1\}^{n_j}$, so that E^j is a reconstructive single-source extractor for min entropy k_j and error ϵ_j and is computable in polynomial time. Say that for each $j < d$, we have $s_j = n_{j+1}$. Define $\tilde{E}^j : \prod_{j' \leq j} \{0, 1\}^{m_{j'}} \times \{0, 1\}^{s_j} \rightarrow \{0, 1\}^{n_1}$ by induction on j , with $\tilde{E}^1 = E^1$, and $\tilde{E}^{j+1}(x_1, \dots, x_{j+1}, z) = \tilde{E}^j(x_1, \dots, x_j, E^{j+1}(x_{j+1}, z))$. Then $\tilde{E}^d$ is a multi-reconstructive extractor for entropy sequence $(k_1, \dots, k_d)$ and error $\sum_{j \leq d} \epsilon_j$.*

In [131], the same method of composing single source extractors to form a seeded block source extractor is analyzed in the information theoretic setting (rather than the computational/reconstructive setting). The proof of this lemma in the reconstructive framework is essentially identical to the core argument justifying the correctness of our pseudorandom generator construction in Theorem 4.18:

Proof. We prove by induction on j that $\tilde{E}^j$ is a multi-reconstructive extractor with error $\delta_j := \sum_{j' \leq j} \epsilon_{j'}$. By definition it is true for $\tilde{E}^1 = E^1$. Now, say that $\tilde{E}^j$ is a reconstructive extractor for entropy sequence $(k_1, \dots, k_j)$ and error δ_j . So there are reconstruction procedures $R_1, \dots, R_j$ taking oracle access to a function $D : \{0, 1\}^{n_1} \rightarrow \{0, 1\}$ so that, given any D distinguishing $\tilde{E}^j(\bar{x}, z)$ with error δ_j , there exists $j' \leq j$ so that $R_{j'}^D(x_1, \dots, x_{j'-1}, a)$ produces $x_{j'}$ for some $a \in \{0, 1\}^{k_{j'}}$. We then consider $\tilde{E}^{j+1}(x_1, \dots, x_{j+1}, z')$; our reconstruction procedures for $R_1, \dots, R_j$ will be as they were for $\tilde{E}^j$, and for R_{j+1} we use the construction procedure for the single source extractor E^{j+1} . Let D be given. For any $x_1, \dots, x_{j+1}$ such that the first $R_1, \dots, R_j$ reconstruction procedures all fail to reconstruct a symbol of $x_1, \dots, x_j$ from its prefix using D , we know that

$\tilde{E}^j(x_1, \dots, x_j, \mathcal{U}_{s_j})$ fools D with error δ_j . Now define the test $D' : \{0, 1\}^{n_{j+1}} \rightarrow \{0, 1\}$ (recall $n_{j+1} = s_j$), with $D'(z) = D(\tilde{E}^j(x_1, \dots, x_j, z))$. Then, for a uniformly random $z \in \{0, 1\}^{s_j}$ with have that $D'(z) = 1$ with probability in the range $\Pr_{y \sim \{0, 1\}^{n_1}} [D(y) = 1] \pm \delta_j$. Observe that D' is efficiently computable with oracle access to D and the values $x_1, \dots, x_j$, since each of the $E^{j'}$ extractors are efficiently computable. Thus, using the reconstruction procedure for E^{j+1} , either R_{j+1} succeeds in reconstructing x_{j+1} using oracle access to D , the previous inputs $x_1, \dots, x_j$, and k_{j+1} bits of advice, or else we must have that $E^{j+1}(x_{j+1}, \mathcal{U}_{s_{j+1}})$ fools D' with error ϵ_{j+1} , and hence the overall construction fools D with error $\delta_j + \epsilon_{j+1}$, completing the proof. $\square$

Using the above in combination with explicit families of single-source reconstructive extractors (e.g. Trevisan's extractor [158]) we observe:

Corollary 4.44. *For any n and fixed constants $d \in \mathbb{N}$, $\gamma > 0$, there is an explicit multi-reconstructive extractor $E : \prod_{j \leq d} \{0, 1\}^{m_j} \times \{0, 1\}^s \rightarrow \{0, 1\}^n$ for entropy sequence $(k_1, \dots, k_d)$ and error ϵ , where:*

$$\sum_{j \leq d} m_j \leq \text{poly}(n), \quad k_j = m_j^\gamma, \quad s \leq O(\log^{[d]} n), \quad \epsilon \leq (\log^{[d-1]} n)^{-1}$$

Moreover, $k_j \geq \log^{[j]} n$ for each j , and hence by Lemma 4.42 this construction is also a block source extractor with the same parameters stated above.

We now prove that the seed length $\log^{[d]} n$ achieved above is optimal as a function of the output length n , regardless of how we choose the source lengths $m_1, \dots, m_d$:

Theorem 4.45. *Say $E : \prod_{j \leq d} \{0, 1\}^{m_j} \times \{0, 1\}^s \rightarrow \{0, 1\}^n$ is a block source extractor for min entropy sequence $(k_1, \dots, k_d)$, $k_j \leq \frac{1}{2} m_j$, and error $\frac{1}{2}$. Then $s \geq \Omega(\log^{[d]} n)$.*

To prove this we rely on the following lemma:

Lemma 4.46. *Let $A \subseteq \prod_{j \leq d} \{0, 1\}^{m_j}$ with $\log |A| \geq (\sum_{j \leq d} m_j) - q$. Then there is a block source with entropy sequence $(k_1, \dots, k_d)$ supported on A , with $k_j \geq m_j - q - d$.*

Proof of Lemma 4.46. We prove by induction on d ; the case $d = 1$ is trivial. Let A be given; for each $x_1 \in \{0, 1\}^{m_1}$ define $A_{x_1} = \{(x_2, \dots, x_d) \mid (x_1, x_2, \dots, x_d) \in A\}$, $\deg(x_1) = |A_{x_1}|$. So $\sum_{x_1 \in \{0, 1\}^{m_1}} \deg(x_1) = |A|$. Define the set $V \subseteq \{0, 1\}^{m_1}$ by

$$V = \{x_1 \mid \deg(x_1) \leq \exp((\sum_{j>1} m_j) - q - 1)\} \subseteq \{0, 1\}^{m_1}$$

Let $U = \{0, 1\}^{m_1} \setminus V$; if $|U| < \exp(m_1 - q - 1)$ then we must have

$$|A| \leq |V| \cdot \max_{x_1 \in V} \deg(x_1) + |U| \cdot \max_{x_1 \in U} \deg(x_1) \quad (4.4)$$

$$< 2^{m_1} \cdot \exp((\sum_{j>1} m_j) - q - 1) + \exp(m_1 - q - 1) \cdot \exp(\sum_{j>1} m_j) \quad (4.5)$$

$$= \exp((\sum_{j \leq d} m_j) - q - 1) + \exp((\sum_{j \leq d} m_j) - q - 1) \leq \exp((\sum_{j \leq d} m_j) - q) \quad (4.6)$$

and we get a contradiction. Hence, we know that $|U| \geq \exp(m_1 - q - 1)$. For every $x_1 \in U$, we also know that $A_{x_1} \subseteq \prod_{j>1} \{0, 1\}^{m_j}$ has size at least $\exp(\sum_{j>1} m_j - q - 1)$, so by induction there is a block source $\mathcal{X}_{x_1}$ with min entropies $(k_2, \dots, k_d)$, $k_j \geq m_j - (q + 1) - (d - 1) \geq m_j - q - d$ supported on A_{x_1} . We then construct our overall distribution $\mathcal{X}$ as follows: sample $x_1 \sim U$ uniformly, then sample $(x_2, \dots, x_d) \sim \mathcal{X}_{x_1}$ and output $(x_1, \dots, x_d)$. We know that the first coordinate of this distribution has min entropy $k_1 := \log |U| \geq m_1 - q - 1$, and that the remaining entries are a block source with min entropy sequence $(k_2, \dots, k_d)$ conditioned on any fixing of the first coordinate; so overall $\mathcal{X}$ is a block source with entropy sequence $(k_1, \dots, k_d)$. $\square$

The proof of Theorem 4.45 combines the above lemma with the argument of [131] used to show an $\Omega(\log n)$ lower bound on the seed length of single source extractors:

Proof of Theorem 4.45. When $d = 0$, E is of the form $E : \{0, 1\}^s \rightarrow \{0, 1\}^n$, so that $E(\mathcal{U}_s)$ is $\frac{1}{2}$ close to uniform on $\{0, 1\}^n$; in this case we clearly require $s \geq n - 1$. For $d > 0$, we will show how to choose some $j \leq d$ and construct from E a second extractor $E' : \prod_{j \neq j'} \{0, 1\}^{m_j} \times$

$\{0, 1\}^{s'} \rightarrow \{0, 1\}^n$ using only $d - 1$ sources, so that E' is a block source extractor for entropy sequence $(k_1, \dots, k_{j'-1}, k_{j'+1}, \dots, k_d)$, the same error ($\frac{1}{2}$), and at most exponentially larger seed length $s' \leq s + 2^{s+2} + 2d$. This will yield the theorem by induction.

We will set j' to be any index such that $m_{j'} \leq 2^{s+2} + 2d$; if we can find such an index, we construct E' by simply moving input j' of E into the seed, i.e.

$$E'(x_1, \dots, x_{j'-1}, x_{j'+1}, x_d, (x_{j'}, z)) = E(x_1, \dots, x_{j'-1}, x_{j'}, x_{j'+1}, \dots, x_d, z)$$

Clearly for any choice of j' , E' remains a block source extractor for the contracted entropy sequence $(k_1, \dots, k_{j'-1}, k_{j'+1}, \dots, k_d)$ and the same error as E . To argue that $m_j \leq 2^{s+2} + 2d$ for some j we rely on Lemma 4.46 above. Let $\mathcal{D} \subseteq \{0, 1\}^{\{0,1\}^n}$ consist of all functions $D : \{0, 1\}^n \rightarrow \{0, 1\}$ which depend only on the first $s + 1$ bits. So $|\mathcal{D}| \leq 2^{2^{s+1}}$. For every $\bar{x} \in \prod_{j \leq d} \{0, 1\}^{m_j}$, there is some $D_{\bar{x}} \in \mathcal{D}$ such that

$$| \Pr_{z \sim \mathcal{U}_s} [D(E(\bar{x}, z)) = 1] - \Pr_{y \sim \mathcal{U}_n} [D(y) = 1] | \geq \frac{1}{2}$$

hence there exists a fixed D , so that for the set $\bar{X}_D = \{\bar{x} \mid D_{\bar{x}} = D\}$, we have $\log |\bar{X}_D| \geq (\sum_{j \leq d} m_j) - 2^{s+1}$. We then apply Lemma 4.46 with $q = 2^{s+1}$ to conclude that there is a block source $(X_1, \dots, X_d)$ supported on $\bar{X}_D$ with min entropy sequence $(k'_1, \dots, k'_d)$, $k'_j := m_j - 2^{s+1} - d$. If $k'_j \geq k_j$ for all j this will contradict the correctness of the extractor E , since $E(\bar{X}, \mathcal{U}_s)$ will be distinguished with error $\frac{1}{2}$ by the test D . Hence there exists j such that $k'_j < k_j$, i.e. $m_j - k_j < 2^{s+1} + d$. Recalling that $k_j \leq \frac{1}{2}m_j$, we get $m_j < 2^{s+2} + 2d$ which concludes the proof. $\square$

Note that the proof yields more than just a lower bound on s :

Observation 4.47 (Follows from Proof of Theorem 4.45). *In any construction achieving the optimal seed length $s = O(\log^{[d]} n)$, there must be some smallest source length m_{j_1} with $m_{j_1} = \Omega(\log^{[d-1]} n)$, then a second smallest source length of $\Omega(\log^{[d-2]} n)$, and so on with the longest source having length on the order $\Omega(n)$*

Hence our construction uses essentially the only possible sequence of source lengths $(m_1, \dots, m_d)$ which can be made to achieve a an optimal seed length of $\log^{[d]} n$.

4.5.2 No Relativizing Reduction from K^{poly} Randomness to Hard Truth Tables

In [102] it is shown that if we are permitted the use of an NP oracle in our explicit construction algorithms we have the following appealing equivalence:

Theorem 4.48. *The following are equivalent:*

1. *There is a polynomial time NP-oracle algorithm constructing strings $x \in \{0, 1\}^n$ with $K^{\text{poly}(n)}(x) \geq n - 1$*
2. *There is a polynomial time NP-oracle algorithm constructing truth tables $f : \{0, 1\}^n \rightarrow \{0, 1\}$ with hardness $2^{\Omega(n)}$ (i.e. $\text{poly}(2^n) = 2^{O(n)}$).*

If this kind of equivalence could be shown without the aid of an NP oracle, it would supersede all of the main results in this paper. The proof of Theorem 4.48 is relativizing, and more specifically it gives a black-box reduction from the problem of constructing high Kolmogorov-complexity strings to constructing hard truth tables (the nontrivial direction). We give some indication here that an NP oracle is necessary for such an argument to work.

Observe that if $f : \{0, 1\}^n \rightarrow \{0, 1\}$ has circuit complexity $o(2^n/n)$, then for $N = 2^n$ and interpreting f as an N -bit string, we have $K^{N^2}(f) \leq N/2$; this is because evaluating a circuit on an input takes at most $2^n = N$ time, and hence reconstructing f from a circuit computing it takes at most quadratic time. Hence if there were an analogue to Theorem 4.48 in the polynomial time regime, it would give, in particular, a reduction from constructing strings of large $K^{n^c}(\cdot)$ to strings of large $K^{n^2}(\cdot)$ complexity.

We show here that there is no such reduction which is “black box” in the same sense as Theorem 4.48. We first need to define the suitable notion of a black box reduction:

Definition 4.49. *Let c, d be fixed constants and n large. A black box reduction from $K^{n^c}(\cdot)$ -construction to $K^{n^2}(\cdot)$ -construction is an algorithm, given access to some oracle $O : \{0, 1\}^* \rightarrow$*

$\{0, 1\}$, which has the following behavior:

1. There is a procedure A_n^O which, given strings $y_1, \dots, y_{\text{poly}(n)}$ with $y_m \in \{0, 1\}^m$, makes $\text{poly}(m)$ additional queries to O and outputs a string $x \in \{0, 1\}^n$
2. For any oracle O , if $y_1, \dots, y_{\text{poly}(n)}$ are strings such that $K^{m^d, O}(y_m) \geq \frac{m}{2}$ for each m , $A^O(y_1, \dots, y_{\text{poly}(n)})$ outputs a string $x \in \{0, 1\}^n$ with $K^{n^c, O}(x) \geq n^{\Omega(1)}$.

Theorem 4.50. For $c > d$ fixed constants, $c - d > 1$, there is no black box reduction from d -AVOID to c -AVOID.

Proof. Consider the Range Avoidance instance $C^O : \{0, 1\}^{n^\epsilon} \rightarrow \{0, 1\}^n$, defined by $C^O(z) = (O((z, 1^{n^{c-1}}, 1)), \dots, O((z, 1^{n^{c-1}}, n)))$ where $(\cdot, \cdot, \cdot)$ is some standard pairing function. Clearly for any oracle O and any x such that $x \in \text{range}(C^O)$ we have $K^{n^c, O}(x) \leq n^\epsilon$. Hence if we can find an oracle O and supply strings $y_1, \dots, y_{\text{poly}(n)}$ so that $K^{m^d, O}(y_m) \geq \frac{m}{2}$ for each m , but $A^O(y_1, \dots, y_{\text{poly}(n)})$ outputs a string in $\text{range}(C^O)$ then we are done. Initially we fix the value of the oracle O to be zero on all strings of length at most n^{c-1} . We then conclude that for any $m \leq 4n$, we may determine a string y_m so that $K^{m^d, O}(y_m) \geq \frac{m}{2}$ is already forced by the current information about the oracle; this holds since a machine running in time $m^d \leq O(n^d) < n^{c-1}$ cannot access the any unfixed part of the oracle. We will fix these strings $y_1, \dots, y_{4n}$ for the remainder of the proof.

Now, consider the algorithm $\mathcal{A}$ with the first $4n$ arguments fixed, as a function of the remaining arguments $y_{4n+1}, \dots, y_{\text{poly}(n)}$. By the correctness of $\mathcal{A}$, for any extension of the partially defined oracle O and any valid solutions for these remaining arguments to $\mathcal{A}$, $\mathcal{A}$ will find a solution to the Range Avoidance instance C^O . On the other hand, observe that if $y_{4n+1}, \dots, y_{\text{poly}(n)}$ are chosen uniformly at random, then for any oracle O the probability that any y_m fails to have $K^{m^d, O}(y_m) \geq \frac{m}{2}$ is bounded by $\text{poly}(n)2^{-\frac{m}{2}} \leq 2^{-2n+o(n)}$ since for each such m we have $m \geq 4n$. Hence we obtain a randomized query procedure, making $\text{poly}(n)$ queries to the oracle O , which outputs a solution to AVOID on C^O with failure probability $\leq 2^{-2n+o(n)}$. Since we have not fixed the behavior of O above input length n^{c-1} , C^O can take on any value $\{0, 1\}^{n^\epsilon} \rightarrow \{0, 1\}^n$, and is thus an arbitrary oracle-presented Range Avoidance instance. It then remains only to show that a randomized query

algorithm making $\text{poly}(n)$ queries to an AVOID instance with stretch $n^\epsilon \mapsto n$ cannot succeed with probability as high as $1 - 2^{-2n+o(n)}$.

To obtain this randomized query lower bound, we appeal to Yao's lemma, and consider the best success probability of a deterministic $\text{poly}(n)$ -query algorithm Q on a uniformly random instance $C : \{0, 1\}^{n^\epsilon} \rightarrow \{0, 1\}^n$. For each possible sequence of $\text{poly}(n)$ queries we argue the probability the supplied answer is incorrect conditioned on this query sequence occurring is at least 2^{-n} . This holds trivially, since $\text{poly}(n) < 2^{n^\epsilon}$, and hence a random C extending a fixed sequence of $\text{poly}(n)$ mappings $C(x_1) = y_1, \dots$, has probability at least 2^{-n} of hitting any fixed string in $\{0, 1\}^n$. $\square$

4.6 Proof of Lemma 4.16

We prove here Lemma 4.16, which we restate below for convenience:

Lemma (Lemma 4.16, Restated). *Assume Hypothesis $\text{WH}(O, d + 1, v)$, $d \geq 0$, $v \geq 2$. Then for every time constructible $\exp^{[d]}(n) \leq T(n) \leq \text{poly}((\exp^{[d]}(n)))$ there is a PRG $(G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ computable uniformly in time $O^{\langle d+v \rangle}(T(n))$ which fools $\text{TIME}^O[T(n)]/3n$ and has seed length $s(n) \leq O^{\langle v-1 \rangle}(\log n)$.*

For this we require a strengthening of Theorem 4.17 due to Umans [160] (following a similar result of Shaltiel-Umans [152]) given as follows:

Theorem 4.51 ([160]). *There is a fixed universal constant $\gamma > 0$ so that the following holds. Let $h : \mathbb{N} \rightarrow \mathbb{N}$ be any time-constructible "hardness" parameter, $h(n) \leq 2^n$. On input n and given oracle access to a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, UM^f computes a function $\text{UM}^f : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^{m(n)}$ in $2^{O(n)}$ time for some time constructible $m(n) = \Theta(h(n)^\gamma)$, $s(n) \leq O(n)$, such that for any $D : \{0, 1\}^{m(n)} \rightarrow \{0, 1\}$ with*

$$\left| \Pr_{x \sim \{0,1\}^{m(n)}} [D(x) = 1] - \Pr_{z \sim \{0,1\}^{s(n)}} [D(\text{UM}^f(z)) = 1] \right| \geq \frac{1}{m}$$

there exists a circuit C with D oracle gates computing f whose total size is at most $h(n)$.

Proof of Lemma 4.16. Assuming Hypothesis $\text{WH}(O, d + 1, v)$, we conclude that for some $\epsilon > 0$, there is a language in $\text{TIME}[\exp^{[d+1]}(n)]$ which is not in $\text{TIME}^O[\Phi_{v,\epsilon}(\exp^{[d+1]}(n))]/\Phi_{v,\epsilon}(2^n)$ even infinitely often. Let $h(n) = \Phi_{v,\epsilon^2}(2^n)$, and define $\tilde{G}_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^{m(n)}$ by $G_n = \text{UM}^{L_n}$ where $s(n) \leq O(n)$, $m(n) = \Theta(h(n)^\gamma)$ are the time constructible bounds guaranteed in Theorem 4.51. By Theorem 4.51, if $\tilde{G}_n$ is distinguished by some $D : \{0, 1\}^{m(n)} \rightarrow \{0, 1\}$ then there is an oracle circuit of total size $\leq h(n)$ computing L_n ; hence if a distinguisher for $\tilde{G}_n$ exists which is computable in time $t(n)$ with an O oracle and $O(m(n))$ bits of advice for infinitely many n , then L is decidable with an O oracle in time $t(n) \cdot h(n)$ with $O(m(n)) + h(n) = O(h(n))$ bits of advice for infinitely many n .

Let $T(n) = (\exp^{[d]}(n))^k$ for some constant k . We then define our final generator G in terms of $\tilde{G}$ by a simple reparameterization of input lengths; for $n \in \mathbb{N}$, we determine the least $\tilde{n}$ such that $m(\tilde{n}) \geq n$ and $T(n) \cdot h(\tilde{n}) < \Phi_{v,\epsilon}(\exp^{[d+1]}(\tilde{n}))$ and set $G_n = \tilde{G}_{\tilde{n}}$ (truncating the output length of $\tilde{G}_{\tilde{n}}$ as necessary). We want to show that G_n fools time $T(n)$ algorithms using an O oracle and $O(n)$ bits of advice, and runs in time $O^{\langle d+v \rangle}(T(n))$ with seed length $O^{\langle v-1 \rangle}(\log n)$. For the first point, observe that for any $D : \{0, 1\}^n \rightarrow \{0, 1\}$ running in time $T(n)$ with $O(n)$ bits of advice, we obtain a computation of $L_{\tilde{n}}$ running in time $T(n) \cdot h(\tilde{n})$ time with $O(h(\tilde{n}))$ bits of advice, which contradicts our hardness assumption provided $T(n) \cdot h(\tilde{n}) < \Phi_{v,\epsilon}(\exp^{[d+1]}(\tilde{n}))$ and $O(h(\tilde{n})) < \Phi_{v,\epsilon}(2^{\tilde{n}})$ which both hold by construction.

On the other hand the runtime and seedlength of G_n are bounded by $2^{O(\tilde{n})} \cdot \exp^{[d+1]}(\tilde{n})$ and $O(\tilde{n})$ respectively, so it remains to bound the growth rate of $\tilde{n}$. Recall that we chose $\tilde{n}$ to be the least integer such that $m(\tilde{n}) \geq n$ and $T(n) \cdot h(\tilde{n}) < \Phi_{v,\epsilon}(\exp^{[d+1]}(\tilde{n}))$, where $m(\tilde{n}) = \Theta(h(\tilde{n}))^\gamma$. For the first point, we have $h(x) \leq 2^x \leq T(x)$ hence we can bound $\tilde{n}$ by the least integer satisfying $T(n)^2 < \Phi_{v,\epsilon}(\exp^{[d+1]}(\tilde{n}))$, i.e. $(\exp^{[d]}(n))^{2k} < \Phi_{v,\epsilon}(\exp^{[d+1]}(\tilde{n}))$, so it suffices here to take $\tilde{n} = O^{\langle v-1 \rangle}(\log n)$. On the other hand, to have $m(\tilde{n}) \geq n$ it suffices to have $h(\tilde{n}) \geq n$, i.e. $\Phi_{v,\epsilon^2}(2^{\tilde{n}}) \geq n$, so setting $\tilde{n} \leq O^{\langle v-1 \rangle}(\log n)$ suffices. Hence in the end we obtain a runtime of $\exp^{[d+1]}(O^{\langle v-1 \rangle}(\log n)) \leq O^{\langle d+v \rangle}(T(n))$ and seed length $O^{\langle v-1 \rangle}(\log n)$.

□

We will use a slight generalization of Lemma 4.16 that follows directly by padding (the difference compared to the previous lemma is merely that we allow a slightly more general upper bound on $T(n)$):

Corollary 4.52. *Assume Hypothesis $\text{WH}(O, d + 1, v)$, $d > 0$, $v \geq 2$ and let $\ell \geq d$ be a fixed constant. Then for every time constructible $\exp^{[d]}(n) \leq T(n) \leq O^{\langle d+v \rangle}(\exp^{[d]}(n))$ there is a PRG $(G_n : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ computable uniformly in time $O^{\langle d+v \rangle}(T(n))$ which fools $\text{TIME}^O[T(n)]/3n$ and has seed length $s(n) \leq O^{\langle v-1 \rangle}(\log n)$.*

Proof. Using Lemma 4.16, from our hardness assumption we get a generator fooling $\text{TIME}^O[\exp^{[d]}(n)]/3n$ with runtime $O^{\langle d+v \rangle}(T(n))$ and seed length $s(n) \leq O^{\langle v-1 \rangle}(\log n)$. On input length n , we choose some $n' = n'(n)$ such that $\exp^{[d]}(n') \geq T(n)$ and apply our generator on input length n' ; clearly we may use this generator for input length n as well, by considering a language $L \subseteq \{0, 1\}^n$ as $L' \subseteq \{0, 1\}^{n'}$ depending on only the first $n \leq n'$ bits. It suffices to take $n' = O^{\langle v \rangle}(n)$, in which case the seed length as a function of n is $O^{\langle v-1 \rangle}(\log O^{\langle v \rangle}(n)) \leq O^{\langle v-1 \rangle}(\log n)$ and the runtime is $O^{\langle d+v \rangle}(\exp^{[d]}(O^{\langle v \rangle}(n))) \leq O^{\langle d+v \rangle}(\exp^{[d]}(n)) = O^{\langle d+v \rangle}(T(n))$. $\square$

We are now ready to prove Theorem 4.20:

Theorem (Theorem 4.20, Restated). *Assume Hypothesis $\text{WH}(O, d + 1, v)$ with $d \geq 0$, $v \geq 2$, fixed constants, and $k \in \mathbb{N}$ a fixed constant. Then there is a pseudorandom generator with seed length $O^{\langle v-1 \rangle}(\log^{[d+1]}(n))$ and runtime $O^{\langle d+v \rangle}(n)$ that fools $\text{TIME}^O[n^k]/\log^{[d]}(n)$ with error $(2 \log^{[d]}(n))^{-1}$ whenever $K^{n^k}(n) \leq \log^{[d]}(n)$.*

Proof. Let $k \geq 1$, $d \geq 0$, $v \geq 2$ be fixed constants and let $T(n) = n^k$. We prove by induction on d that there is a pseudorandom generator $(\mathcal{G}_n^d : \{0, 1\}^{s_d(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ which runs in time $O^{\langle d+v \rangle}(n)$ and fools $\text{TIME}^O[n^k]/\log^{[d]}(n)$ with error $\leq \sum_{j \leq d} (\log^{[j]}(n))^{-1}$ and has seed length $s_d(n) \leq O^{\langle v-1 \rangle}(\log^{[d+1]}(n))$, provided the input length n satisfies $K^{n^k}(n) \leq \log^{[d]}(n)$.

In the case $d = 0$ we may apply Lemma 4.16 directly. Now, say that the generator $(\mathcal{G}_n^{d-1} : \{0, 1\}^{s_{d-1}(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ is given, computable in time $T_{d-1}(n) := O^{\langle d-1+v \rangle}(n)$ with $s_{d-1}(n) \leq O^{\langle v-1 \rangle}(\log^{[d]}(n))$ and fooling $\text{TIME}^O[n^k]/\log^{[d-1]}(n)$ with error $\sum_{j \leq d-1} (\log^{[j]}(n))^{-1}$. Let t_d be a

time constructible function such that $T_{d-1}(n) \leq 3t_d(s_{d-1}(n))$; we may set $t_d = O^{\langle d-1+v \rangle}(\exp^{[d]}(n))$. Now, let $(G_n : \{0, 1\}^{s'(n)} \rightarrow \{0, 1\}^n)_{n \in \mathbb{N}}$ be the generator guaranteed by Corollary 4.52 for time bound $t_d(\cdot)$; so G_n fools $\text{TIME}^O[t_d(n)]/3n$, runs in time $t'_d(n) = O^{\langle d+v \rangle}(t_d(n))$, and has error n^{-1} and seed length $s'(n) = O^{\langle v-1 \rangle}(\log n)$. We then set $\mathcal{G}_n^d$ to be the generator with seed length $s_d(n) = s'(s_{d-1}(n))$ given by $\mathcal{G}_n^d(z') = \mathcal{G}_n^{d-1}(G_{s_{d-1}(n)}(z'))$. The run time of $\mathcal{G}_n^d$ is bounded by some constant times the sum of the run times of the two constituent generators, which overall is bounded by $O^{\langle d-1+v \rangle}(n) + O(t'_d(s_{d-1}(n)))$. Let $L \subseteq \{0, 1\}^n$ be decided by a $\text{TIME}^O[n^k]/\log^{[d]}(n)$ machine; recall that $n = \exp^{[d-1]}(\ell)$ for some ℓ . Define $L' \subseteq \{0, 1\}^{s_{d-1}(n)}$ given by $L' = \{z \mid \mathcal{G}_n^{d-1}(z) \in L\}$; since $\mathcal{G}_n^{d-1}$ fools L with error $\epsilon := \sum_{j \leq d-1} \log^{[j]}(n)$, we have that $\Pr[y \in L] \in \Pr[z \in L'] \pm \epsilon$. On the other hand L' is decidable in time $\leq 3t_d(s_d(n))$ with $2\log^{[d]}(n) + O(1)$ bits of advice provided n is of the form $K^{n^k}(n) \leq \log^{[d]}(n)$: we use the advice for deciding L , together with $\log^{[d]}(n)$ bits of advice describing the number n , and $O(1)$ advice to specify the code for $\mathcal{G}^{d-1}$ and the procedure explained herein. Hence $G_{s_{d-1}(n)}$ fools L' to within error $\delta := (s_{d-1}(n))^{-1} \leq (\log^{[d]}(n))^{-1}$, so overall we must have that $\mathcal{G}_n^d = \mathcal{G}_n^{d-1} \circ G_{s_{d-1}}$ fools L to within error $\epsilon + \delta \leq \sum_{j \leq d} (\log^{[j]}(n))^{-1}$. It remains to bound the seed length and runtime of $\mathcal{G}_n^d$. The seed length is $O^{\langle v-1 \rangle}(\log s_{d-1}(n))$, and $s_{d-1} = O^{\langle v-1 \rangle}(\log^{[d]}(n))$, so overall the seed length is bounded by $O^{\langle v-1 \rangle}(\log^{[d+1]}(n))$. On the other hand the runtime is dominated by $t'_d(s_{d-1}(n)) = O^{\langle d+v \rangle}(\exp^{[d]}(s_{d-1}(n))) = O^{\langle d+v \rangle}(\exp^{[d]}(O^{\langle v-1 \rangle}(\log^{[d]}(n)))) \leq O^{\langle d+v \rangle}(n)$.

□

Chapter 5: Stronger Cell Probe Lower Bounds via Local PRGs

5.1 Introduction

In the final chapter of this thesis, based on joint work with Toniann Pitassi and Russell Impagliazzo originally appearing in [106], we explore connections between Range Avoidance for NC^0 circuits and several other well studied topics in theoretical computer science. Utilizing this connection we are able to make progress on a longstanding problem: improving state-of-the-art lower bounds in the cell probe model of static data structures. We also obtain the best known algorithms for NC^0 -AVOID, and establish a *natural proofs barrier* for data structure lower bounds. We start by briefly introducing the three main subjects of this chapter: static data structure lower bounds, NC^0 pseudorandom generators, and Range Avoidance for NC^0 circuits.

Static Data Structure Lower Bounds: The cell probe model of data structures introduced in [45, 173] is a fundamental computational model in which to study tradeoffs between storage space and access time for information retrieval problems. In the *static* cell probe model, a data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ is fixed, where $[M]$ is an abstract set of “datapoints,” $[N]$ is an abstract set of “queries” one might wish to make about a data point, and $F(x, y)$ is the correct answer to the x^{th} query about the datapoint y . The goal is to preprocess any given datapoint $y \in [M]$ into a data structure $\mathcal{E}(y) \in \Sigma^S$ consisting of S “cells,” each cell holding a character from some finite alphabet Σ , so that later we may answer different queries $x \in [N]$ about y by reading at most t cells from the data structure $\mathcal{E}(y)$. S is called the “space complexity,” t the “time complexity,” and $w = \log |\Sigma|$ the “word-length” (number of bits to store the contents of a single cell). As a prototypical example (which our main lower bounds will apply to), let $d \leq n$ be given, $N = 2^n$, $M = 2^{\binom{n}{\leq d}}$. Identifying $[N]$ with $\mathbb{F}_2^n$, and $[M]$ with the set of degree $\leq d$ multilinear polynomials

over $\mathbb{F}_2^n$, we can define the *polynomial evaluation problem* $\mathbb{F}_2\text{-Eval}_n^d : [N] \times [M] \rightarrow \{0, 1\}$ with $\mathbb{F}_2\text{-Eval}_n^d(x, p) = p(x)$; in this case to solve the data structure problem, we want a method for preprocessing polynomials p into small space data structures $\mathcal{E}(p)$, so that upon any query $x \in \mathbb{F}_2^n$ we may evaluate the polynomial p at the point x by querying only a few cells of the data structure.

The basic problem in static data structure lower bounds is to prove, for a particular problem F , that any data structure solution for F must use a large amount of either space, time, or word length (word length is generally of secondary importance). For *any explicit problem* $F : [N] \times [M] \rightarrow \{0, 1\}$, the best lower bound provable by techniques known prior to this work, for any $t > 1$ and any setting of the word length w , is at best:

$$S \geq N^{\frac{1}{t}} \cdot (\log M)^{1-\frac{1}{t}} \quad (5.1)$$

Lower bounds of this kind were first achieved by [154] using a method that was rediscovered in later work and which is now known as *cell sampling*. In the special case of *nonadaptive bit probe data structures* (queries chosen nonadaptively, word length $w = 1$), a lower bound exceeding (5.1) has been shown in two recent works. First, [164] achieved a near-maximal space lower bound $S \geq \frac{N}{2}$ in the case $t = 2$ for a problem with $M \leq \text{poly}(N)$ datapoints. Subsequent work of [61] improved this to $S \geq (1 - \epsilon)N$ for arbitrarily small $\epsilon > 0$ (the lower bound is for a different explicit problem), and for all larger $t > 2$ proved a bound like 5.1 with the exponent $\frac{1}{t}$ replaced with $\frac{1}{(t-1)}$. In Section 5.1.5 we will discuss these works, as well as the broader history of static data structure lower bounds, in some more detail.

NC⁰ Pseudorandom Generators: A prominent line of work initiated in [58, 40] has studied the existence of cryptographic PRGs in NC⁰. An NC_t⁰-PRG is function $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$, $m > n$, so that: (1) each output depends on only t inputs, and (2) G is a cryptographic pseudorandom generator: no polynomial time algorithm can distinguish a random output of G from a truly random m bit string. We say that G is an NC⁰ generator if it is NC_t⁰ for some $t = O(1)$. The existence of such

PRGs has been the subject of a great deal of work over the past two decades; we refer the reader to a comprehensive survey [8] and the references therein.

An important parameter is the *stretch* of the generator, which denotes the relation between n, m . We say that G has nontrivial stretch if $m > n$, and polynomial stretch if $m \geq n^{1+\Omega(1)}$. NC^0 generators with nontrivial stretch can be shown to exist under standard number theoretic cryptographic assumptions [10], however their utility has so far been limited. On the other hand, NC^0 generators with polynomial stretch are not known to exist under any more standard hardness assumptions, but have been found to have a wide range of advanced cryptographic applications [86], most notably in the construction of Indistinguishability Obfuscation [88, 89]. For this reason, a great deal of work has gone into studying the existence of polynomial stretch NC^0 PRGs as a foundational assumption in its own right, and various algorithmic methods for distinguishing such PRGs have been studied. The current state of the art shows that an NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ can be broken in polynomial time once $m \gg n^{\frac{t}{2}}$ ([126, 5, 2]).

NC^0 Range Avoidance: We finally turn to our third main subject of study, NC^0 Range Avoidance. The object in question here is the same as above: an NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with $m > n$ and $t = O(1)$. However, instead of trying to distinguish the output of G from random, our goal now is to solve the Range Avoidance problem for G . [144] was the first to investigate the Range Avoidance problem for NC^0 circuits, and showed that a FP^{NP} algorithm for this problem would imply the breakthrough circuit lower bound $\text{EXP}^{\text{NP}} \not\subseteq \text{NC}^1$. The same question of *stretch* arises here: the aforementioned result of [144] requires us to solve instances with barely nontrivial stretch. On the other hand unconditional algorithms have been given once the stretch is sufficiently large, in particular once $m \gg \frac{n^{t-1}}{\log n}$ ([65, 55]). An important question posed in [144] is whether unconditional FP^{NP} algorithms for NC^0 Range Avoidance can be found in the general polynomial stretch regime.

The Main Correspondence: The starting point of our work is the observation that the 3 prob-

lems discussed above are tightly related to one another. The relation between NC^0 PRGs and NC^0 Range Avoidance is rather obvious (although still insufficiently explored prior to this work), and so the key point is the relation we observe between these two topics pertaining to NC^0 circuits on the one hand, and the project of proving cell probe lower bounds on the other. This relationship follows directly from the definitions of the objects in question, and is more appropriately understood as a *perspective shift* rather than a new result.

To state the connection in its simplest form, we focus our attention for now on a special setting of the cell probe model: the word length is 1 (i.e. $\Sigma = \{0, 1\}$, often called the *bit probe* model), and the data structures in question are *nonadaptive*. Here, nonadaptive means that the data structure will decide which cells to probe as a function only of the query, in contrast to a general *adaptive* data structure which may decide on the next cell to probe based on the outcomes of previous probes. In this setting we have:

Observation. *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem, which we interpret as a matrix $F \in \{0, 1\}^{N \times M}$ with N rows and M columns below. The following are equivalent:*

1. *F does not have nonadaptive bit probe data structures with space complexity S and time complexity t .*
2. *The columns of F are a set of M strings $\mathcal{F} \subseteq \{0, 1\}^N$, $|\mathcal{F}| = M$, such that for any NC_t^0 generator $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$, there exists a string $f \in \mathcal{F}$ with $f \notin \text{range}(G)$.*

Proof. Say that F has nonadaptive bit probe data structures of space S and time t . Then, to every $y \in [M]$ we may associate an encoding $E_y \in \{0, 1\}^S$, so that for every $x \in [N]$ we may determine $F(x, y)$ by probing (nonadaptively) t bits of E_y . Considering the y^{th} column of F as a string $f_y \in \{0, 1\}^N$, we see that there is a NC_t^0 function $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ so that $G(E_y) = f_y$ for every $y \in [M]$; in particular, the x^{th} output of G runs the data structure's nonadaptive query procedure corresponding to the query x . The same reasoning applies in the reverse direction. $\square$

In other words, finding an explicit data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ which requires large space for time t in the nonadaptive bit probe model is *exactly equivalent* to con-

structuring explicit *list-solutions* for NC_t^0 Range Avoidance. By a list-solution, we mean a list of candidate strings, so that for *any instance* of NC_t^0 Range Avoidance G (with appropriate parameters), one of the strings in this list must be a Range Avoidance solution for G . We emphasize that an analogous connection exists for the general cell probe model (adaptive and with arbitrary word length), which is explained formally in Section 5.2.2. The problem of finding list-solutions for NC^0 Range Avoidance was considered already in [65]. Indeed, via the above correspondence the results of [65] recover a nonadaptive bit-probe lower bound of [61] in a certain range of parameters.

The connection between NC^0 -PRGs and cell probe lower bounds is more difficult to state formally, but even more central to our main results. In one direction, we will show how to use the best known methods for distinguishing NC^0 PRGs to achieve state of the art lower bounds in the cell probe model; this is the most significant contribution in this work, and after stating our main results below we will describe this methodology in more detail in Section 5.1.3 (“Our Techniques”). In the other direction we will use the conjectured security of various NC^0 PRGs to exhibit a *natural properties barrier* for data structure lower bounds, in the spirit of Razborov-Rudich [143].

We note that connections of a similar flavor have been utilized in the literature on sampling lower bounds initiated by Viola [163]. Connections between restricted variants of Range Avoidance and data structures were first observed in [65], who used a cell probe data structure from [136] to develop more efficient reductions to Range Avoidance; this proof was our original inspiration to explore more deeply the connections between NC^0 Range Avoidance and the cell probe model.

5.1.1 Our Results

State of the Art Cell Probe Lower Bounds: Using the perspective shift introduced above, we are able to break a decades-old barrier in static data structure complexity, and prove cell probe lower bounds for a explicit problems which are quadratically better for space as a function of time than all previous methods. Our first lower bound holds any data structure problem supporting a k -wise independent distribution; this is the class same problems originally considered by [154] and in various follow up works (including [133, 109]), and we are able to obtain tight lower bounds

across essentially the entire range of values $k \leq N$, provided the time complexity t is an even number:

Theorem 5.1. *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem such that there exists a k -wise independent distribution supported on its columns. Let $S, t, w \in \mathbb{N}$ be given with t an even number and assume $k > tw + 1$. Then, for any space S , time t , word length w cell probe data structure solving F we must have:*

1. $S \geq \left(\frac{N}{\log N}\right)^{\frac{2}{t}} \cdot \left(\frac{k}{\log N}\right)^{1-\frac{2}{t}} \cdot t^{-1} 2^{-O(w)}$ for all $t \log N \leq k \leq N 2^{-O(tw)} t^{-\frac{t}{2}}$
2. $S \geq \left(\frac{N}{\log N}\right)^{\frac{1}{t(\frac{1}{2}+\frac{2}{k})}} \cdot t^{-1} 2^{-O(w)}$ for all $tw + 1 \leq k \leq t \log N$

Remember that we often interpret $F : [N] \times [M] \rightarrow \{0, 1\}$ as a boolean matrix with N rows and M columns, which explains our use of the phrase “the columns of F .” If t is an odd number, the above lower bound holds with $t + 1$ in place of t ; hence this lower bound gives a polynomial improvement (for space as a function of time) over previous methods for all $t > 3$, gives a near-quadratic improvement for all even t , and approaches a quadratic improvement for large odd values of t . Using any of the various explicit examples of k -wise independent distributions with support size $N^{O(k)}$ (e.g. univariate polynomials over $\mathbb{F}_{2^n}$), we obtain explicit problems $F : [N] \times [M] \rightarrow \{0, 1\}$ satisfying the lower bound

$$S \geq \tilde{\Omega}\left(N^{\frac{2}{t}} \cdot (\log M)^{1-\frac{2}{t}} \cdot 2^{-O(w)}\right)$$

across essentially the entire spectrum of nontrivial values $M \in \{1, \dots, 2^N\}$. We thus achieve a near-quadratic improvement in the state of the art explicit cell probe lower bounds across this entire spectrum of relations between M, N . We emphasize that, in addition to giving a quadratic space improvement over the *cell sampling* bound (the first improvement of any kind), our lower bound is also the first to achieve any polynomial improvement over the much simpler *communication bound* $S \gtrsim N^{\frac{1}{t}}$ for a problem with N queries and $M \leq \exp(N^{o(1)})$ datapoints (by setting $k = \log N$ in the above); this is discussed further in the Section 5.1.5.

By known constructions [154, 155] (see also [111]), the lower bound in Theorem 5.1 is tight up to $\log N$ factors and the factors depending only on w provided t is not too large. Hence, for all small enough even values of t and small enough w , we essentially completely resolve the cell probe complexity of representing k -wise independent distributions. The almost-matching upper bounds are nonconstructive, relying on unbalanced bipartite graphs with very strong unique-expansion properties; they yield data structures which are nonadaptive and have word length $w = 1$, so the tightness of Theorem 5.1 holds even in this restricted setting of nonadaptive bit probe data structures.

Our lower bound for k -wise independent problems is actually a bit stronger than stated above, as we are able to establish the same bound for problems which only exhibit γ -almost k -wise independence, for a nontrivially large value of γ . The stronger form (which is the form which will appear as Theorem 5.1 in Section 5.3.2) is as follows:

Theorem (Theorem 5.1, Strengthened To Almost Independence). *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem such that there exists a γ -almost k -wise independent distribution supported on its columns. Then the same lower bound as in the first presentation of Theorem 5.1 (above) holds for F provided $\gamma \leq N^t 2^{-O(ktw)}$.*

An important consequence of this improvement is that, using known constructions of almost independent distributions [128], in the case $t, w = O(1)$ we can set $k = O(\log N)$, $\gamma = N^{-O(1)}$ and obtain an explicit problem $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M \leq \text{poly}(N)$ and satisfying the lower bound $S \geq \tilde{\Omega}(N^{\frac{2}{t}})$. This improvement will be necessary later on for obtaining polynomial time Range Avoidance algorithms with minimum possible stretch.

We next establish a lower bound that improves on the previous theorems in the special setting of *nonadaptive bit probe* data structures; remember that “bit probe” denotes the word length setting $w = 1$. For this lower bound, the hardness property of our data structure problems is *small bias* [128]: a distribution over $\{0, 1\}^N$ is γ -biased if it fools all parity functions with error $\leq \frac{\gamma}{2}$.

Theorem 5.2. *Let $t > 1$, $t = O(1)$ be a fixed odd number. Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem such that there exists a N^{-c} -biased distribution supported on its columns,*

for $c = O(1)$ a sufficiently large constant (depending on t). Then any nonadaptive bit probe data structure for F with space S and time t must satisfy:

$$S \geq \tilde{\Omega}(N^{\frac{1}{\frac{t}{2} - \frac{t-2}{2(t+2)}}}) \geq N^{\frac{2}{t} + \epsilon_t}$$

where $\epsilon_t > 0$ is a positive constant depending only on t .

Using explicit constructions of small bias probability spaces (again [128]), we obtain explicit problems $F : [N] \times [\text{poly}(N)] \rightarrow \{0, 1\}$ which satisfy the lower bound $S \geq N^{\frac{2}{t} + \epsilon_t}$ for every odd constant t , where $\epsilon_t > 0$ depends only on t . Small bias probability spaces were first used as a hard data structure problem in [164] to obtain near-maximal space lower bounds for nonadaptive bit probe data structures making $t = 2$ queries. We emphasize that, because of the optimality of the bound in Theorem 5.1 (there is an upper bound almost matching the bound in Theorem 5.1 even in the nonadaptive bit probe setting and even for odd values of t), the above Theorem 5.2 is provably unattainable using *only limited independence*. More formally, to construct any data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ satisfying a lower bound $S \geq N^{\frac{2}{t} + \epsilon}$ for any constants $t > 1$, $\epsilon > 0$, and $M \leq \exp(N^{o(1)})$, we provably cannot use k -wise independence as our only hardness property for the problem F . This result is therefore of special interest, as it the first example of a cell probe lower bound that is provably stronger than the best bound provable using limited independence alone: all of the previous best lower bound methods [154, 133, 109, 164, 61]¹ can be carried out using limited independence as the sole hardness condition.

NC⁰ Range Avoidance Algorithms: In Section 5.5 we give the best known unconditional algorithms for NC⁰ Range Avoidance. Using our primary observation connecting Range Avoidance to data structure lower bounds together with known constructions of limited independence and small bias spaces [128], we can immediately obtain state of the art P^{NP} algorithms for NC⁰ Range Avoidance using our new cell probe lower bounds as a black box. In the case of Theorem 5.2 we

¹Viola's lower bound for $t = 2$ nonadaptive bit probe data structures [164] does not use k -wise independence, but k -wise independence was shown to achieve the same quantitative lower bound in the follow up work of [61].

get:

Theorem 5.3. *Let $t = O(1)$ be a fixed odd number. There is a polynomial time algorithm which outputs a list of strings in $\{0, 1\}^m$, such that for any NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with*

$$m \geq \tilde{O}(n^{\frac{t}{2} - \frac{t-2}{2(t+2)}})$$

one of the strings in the list must be a range-avoidance solution for G . In particular, there is a polynomial time NP-oracle algorithm for Range Avoidance on instances of the above form.

For our lower bound in Theorem 5.1, we are able to analyse it more effectively (rather than applying it as a black box) and remove the use of an NP oracle in the associated Range Avoidance algorithm:

Theorem 5.4. *Let t, n be given with t even. There is a $m^{O(t^2)}$ -time algorithm which solves Range Avoidance given any NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with*

$$m \geq n^{\frac{t}{2}} \log n \cdot t^{\frac{t}{2} + o(t)}$$

More generally, the algorithm works if each output of G is computed by a depth t decision tree.

We emphasize the last point; not only do we decrease the required stretch quadratically for NC_t^0 functions compared to previous results [65, 55], but our algorithm also works in the more general setting that every output of G is a depth t decision tree over the inputs. The full statement of Theorem 5.4 is more general in two other ways which we didn't highlight above. First, it solves the harder *remote point* problem for G : for any constant $\epsilon > 0$, our algorithm will output a string which is $(\frac{1}{2} - \epsilon)$ -far in relative hamming distance from every string in the range of G . Second, it works if G is of the form $G : \Sigma^n \rightarrow \{0, 1\}^m$ for some nonboolean alphabet $|\Sigma| > 2$, and each output is computable by an adaptive decision tree querying at most t input symbols.

Small Bias Generators in NC^0 : In [126] it was shown that, if $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is an

NC_t^0 generator such that $G(\mathbf{x})$ is $\text{poly}(n)^{-1}$ biased when $\mathbf{x}$ is uniformly distributed on $\{0, 1\}^n$, then we must have $m \leq O(2^t n^{\lceil \frac{t}{2} \rceil})$; in other words, once $m \gg n^{\lceil \frac{t}{2} \rceil}$ there is an $\mathbb{F}_2$ -linear test that can distinguish $G(\mathbf{x})$ from random. Our Theorem 5.2 improves this result for all odd values of t ; in particular, as an immediate corollary we obtain:

Corollary 5.5. *For every odd constant t there is $\epsilon_t > 0$, so that every NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is distinguished with $\text{poly}(n)^{-1}$ advantage by an $\mathbb{F}_2$ -linear test once $m \geq n^{\frac{t}{2} - \epsilon_t}$.*

We note that Theorem 5.2 in fact implies something much stronger, that $G(\mathbf{x})$ can be distinguished from uniform by linear tests for *any* distribution of $\mathbf{x} \in \{0, 1\}^n$. The original result in [126] is specially tailored to the case where the random “seed” $\mathbf{x}$ is uniformly distributed on $\{0, 1\}^n$, and hence cannot be used to give a lower bound in the bit probe model. An interesting feature of this is the following: because $\mathbb{F}_2$ -linear tests can be performed by polynomial-size circuits, our result implies that every NC_t^0 PRG can be distinguished efficiently by *nonuniform* algorithms in the stretch regime $n \mapsto n^{\frac{t}{2} - \epsilon_t}$. However, in the typical approach to NC^0 PRG constructions pioneered by Goldreich [58], the generator G is sampled at random from a distribution of large ($\approx m$) entropy; in particular, such a cryptographic primitive does not consist of a single NC^0 PRG, but rather a *sampleable ensemble* of NC^0 PRGs. Since our Corollary 5.5 gives no indication as to how to *find* the linear distinguisher for G given the description of G , we seemingly cannot use it to attack Goldreich’s generator in the stretch regime $n \mapsto n^{\frac{t}{2} - \epsilon_t}$, even if we allow nonuniformity in our attacker.

Complexity-Theoretic Barriers to Higher Cell Probe Lower Bounds: Next, in Section 5.6 we use our main correspondence between cell probe lower bounds and local generators to establish *barrier results* for improving state of the art cell probe lower bounds. Our focus here is on the bit probe model (word length $w = 1$), and on nonadaptive data structures with time complexity $t = O(1)$. We show the following two barrier-type results:

Theorem 5.6 (Consequences of Explicit Data Structure Lower Bounds).

1. *If there is an explicit data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M \leq \exp(N^{o(1)})$*

which requires space $S \geq N - N^{o(1)}$ for data structures making 4 nonadaptive bit probes, then $\text{EXP}^{\text{NP}} \not\subseteq \text{NC}^1$.

2. If there is a universal constant $\epsilon > 0$ and an explicit data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M = \text{poly}(N)$ requiring space $S \geq N^\epsilon$ for nonadaptive data structures making $O(1)$ bit probes, then there is a polynomial time NP oracle algorithm for NC^0 Range Avoidance with polynomial stretch.

The conclusion in (1) above would be a monumental breakthrough in complexity theory and is considered extremely hard. The conclusion in (2) above was directly posed in [144] as an important direction in unconditional Range Avoidance algorithms. We note that (1) of Theorem 5.6 has a strong similarity to results in [164] and [43]. [164] shows that improved data static data structure lower bounds imply breakthrough circuit lower bounds for multi-output functions. The implied circuit lower bounds are of two kinds: lower bounds against low-depth circuits over a complete basis of large fan-in gates, and lower bounds against linear size log-depth circuits with bounded fan-in gates. On the other hand, [43] shows that improved data structure lower bounds (of a different kind) imply breakthrough constructions of rigid matrices; these results are specific to *linear data structures* rather than general data structures as considered in this work and in [164].

Next, we turn toward a different and more novel kind of barrier in the realm of data structure lower bounds, based on the *natural proofs* paradigm of Razborov and Rudich [143]. Following Razborov-Rudich, we define a *natural data structure lower bound* to be some efficiently testable property of functions $F : [N] \times [M] \rightarrow \{0, 1\}$, so that random functions have the property, and any function with the property satisfies a corresponding cell probe lower bound (see Section 5.6.2 for a formal definition). We argue in Section 5.6.2 that the strongest lower bound methods in the cell probe model known prior to this work can be made *natural* in this sense. This continues to hold for our new lower bound in Theorem 5.1. On the other hand, we show that under two widely believed cryptographic assumptions, certain lower bounds in the bit probe model are unachievable using proofs of this nature:

Theorem 5.7 (Natural Proofs Barriers for Data Structure Lower Bounds).

1. *Assume the existence of NC^1 PRGs. Then there is no natural proof of a space lower bound $S \geq N - N^{0.99}$ for data structures making 4 nonadaptive bit probes solving a problem $F : [N] \times [\text{poly}(N)] \rightarrow \{0, 1\}$.*
2. *Assume existence of polynomial stretch NC^0 PRGs. Then for every $\epsilon > 0$ there is $t \in \mathbb{N}$ so that no natural proof can establish a lower bound $S \geq N^\epsilon$ for data structures making t nonadaptive bit probes solving a problem $F : [N] \times [\text{poly}(N)] \rightarrow \{0, 1\}$.*

As mentioned above our lower bound in Theorem 5.1 can be made natural, as well as all lower bounds proven prior using the cell sampling method or communication arguments. However, interestingly, we *do not* know if the lower bound for small bias distributions in Theorem 5.2 can. It is therefore unclear at the moment whether or not the “natural properties barrier” for data structures, according to the definitions used above, has already been broken by the new results in this paper. Regardless, we argue in Section 5.6.2 that the lower bound in Theorem 5.2 is still “natural” in a weaker and less formal sense, and that significantly stronger lower bounds will probably have to differ from it in this regard assuming the nonuniform security of NC^0 PRGs. We discuss further the question of the “naturalness” of Theorem 5.2 in Section 5.5.

Approaches to Stronger Lower Bounds via Communication Complexity: Finally, in Section 5.6.3 we discuss possible approaches to proving lower bounds in the nonadaptive bit probe model of the form $S \geq N^\epsilon$ for time $t = O(1)$, where $\epsilon > 0$ is a fixed universal constant not depending on t ; this corresponds precisely to the regime in which the existence of cryptographically secure polynomial-stretch NC^0 PRGs yields a natural proofs barrier. We formulate cell probe lower bounds in this regime in terms of an equivalent communication model, reminiscent of PIR schemes, in which a single party Alice holding an input $x \in X$ communicates with a council of t Bobs holding a shared input $y \in Y$. We discuss an approach to analyzing this model based on “lifting” a query complexity measure investigated by [76, 110], and relate this model to other *high*

end communication complexity classes such as PH^{cc} .

5.1.2 Subsequent Developments:

After the posting of an initial preprint of this work, two improvements to Theorem 5.2 were communicated to us by Weiqiang Yuan [174]:

1. Our lower bound $S \geq \tilde{\Omega}(N^{\frac{1}{\frac{t}{2} - \frac{t-2}{2(t+2)}}})$ for small bias problems can be extended to hold for *adaptive* bit probe data structures.
2. In the case of *nonadaptive* bit probe data structures, the lower bound for small bias problems can be strengthened quantitatively to $S \geq \tilde{\Omega}(N^{\frac{2}{t-1}})$.

In both cases, it is required as in Theorem 5.2 that $t = O(1)$ is an odd constant.

5.1.3 Our Techniques: Derandomizing Semirandom CSP Refutations

We now discuss at a high level our approach to proving our main cell probe lower bound for k -wise independent problems (Theorem 5.1). We will narrow down our discussion in three respects: (1) we focus on the case $t = O(1)$ and ignore multiplicative factors depending on t in our space lower bound, (2) we restrict attention to *nonadaptive bit probe* data structures, and (3) we focus on achieving a lower bound of the form $S \geq \tilde{\Omega}(N^{\frac{2}{t}})$ for k -wise independent problems in the special case $k = O(\log N)$.

In this setting, using our main observation connecting NC^0 generators and data structures, we need to show that any $O(\log N)$ -wise independent random variable $f^{\text{pseud}} \in \{0, 1\}^N$ has the following property: for any NC_t^0 generator $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ with $S = \tilde{o}(N^{\frac{2}{t}})$, with nonzero probability $f^{\text{pseud}} \notin \text{range}(G)$. Say for a moment that we view a candidate generator G with the above parameters as a *cryptographic PRG*. In this case, a body of work dedicated to the cryptanalysis of NC^0 PRGs has shown that any such G can be distinguished by polynomial time algorithms; more strongly, there is a polynomial time algorithm which can generate certificates of the fact

$f^{\text{unif}} \notin \text{range}(G)$ with very high probability when $f^{\text{unif}} \in \{0, 1\}^N$ is a uniformly random vector². Our goal is to look at the certificates generated by these algorithms, and argue that they will still be produced with high probability when we replace the truly random vector f^{unif} with the *pseudorandom* vector f^{pseud} .

The task of certifying $f^{\text{unif}} \notin \text{range}(G)$ for a worst case NC_t^0 generator G is a special case of *semirandom CSP refutation*. In particular, given an NC_t^0 generator $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ and any $f \in \{0, 1\}^N$, we may define a t -CSP instance with S variables and N constraints, which is satisfiable if and only if $f \in \text{range}(G)$: for each $x \in [N]$ with G_x depending on input cells $i_x^1, \dots, i_x^t \in [S]$, we add the constraint

$$G_x(E(i_x^1), \dots, E(i_x^t)) = f_x$$

where $E(1), \dots, E(S)$ are boolean-valued variables. If G is fixed and f^{unif} is uniformly random, we end up with a random distribution of t -CSP instances, where the *left hand side* of each constraint is fixed (worst-case), while the *right hand side* is sampled uniformly at random (average case). If we can give an efficient *refutation algorithm* which, with high probability over this distribution, outputs a certificate that the given CSP is *unsatisfiable*, then we can use it to distinguish the generator G : for a uniformly random right hand side the refutation algorithm will typically output “UNSAT,” while for $f \in \text{range}(G)$ the algorithm can never output “UNSAT.”

This hybrid worst-case/average-case model is known as the *semirandom CSP* model³. This model was first introduced in [47] and has been the subject of many followups including [47, 5, 2, 64] (see the theses of Witmer and Manohar [172, 120] for a more in depth literature review). State of the art semirandom refutation algorithms ([2, 64, 77]) can certify $f^{\text{unif}} \notin \text{range}(G)$ for an NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ whenever $m \geq \tilde{\omega}(n^{t/2})$. Taking $m = N$, $n = S$ this is precisely the parameter regime $S \leq \tilde{o}(N^{\frac{2}{t}})$ we aim to investigate in our cell probe lower bound for

²Some NC^0 cryptanalysis techniques, e.g. Theorem 6 in [126], do not yield certificates $f \notin \text{range}(G)$, and only give statistical distinguishers which rely crucially on the fact that the *seed* of G is generated uniformly; such techniques seem to have little relevance to cell probe lower bounds.

³The semirandom CSP model is more general than the “random right hand side” variant introduced here, see e.g. [2] for a general definition and comparison to the special case presented here.

$O(\log N)$ -wise independent problems. It is considerably easier to carry out the arguments in [2, 64, 77] in the case t is even; we will only prove lower bounds for even values of t in this work and hence avoid the technical difficulties that arise for odd t .

In the case t is even, the refutation procedures in [2, 64, 77] can be divided into roughly three steps:

1. Generate from G a second NC_t^0 generator $G' : \{0, 1\}^{O(S)} \rightarrow \{0, 1\}^N$ which is $\mathbb{F}_2$ -linear (each output of G' XORs together t inputs); reduce the task of certifying $f \notin \text{range}(G)$ for an arbitrary NC_t^0 PRG G , to certifying that f is $(\frac{1}{2} - \epsilon)$ far in Hamming distance from $\text{range}(G')^4$ for some small constant $\epsilon > 0$. This is referred to as *strong XOR refutation*.
2. Generate from G' a sequence of real matrices $A_1, \dots, A_N \in \mathbb{R}^{m \times m}$ for some $m = \text{poly}(N)$; reduce the the task of certifying that f is far from $\text{range}(G')$ (strong XOR refutation) to certifying that the $\infty \rightarrow 1$ norm of the matrix

$$A[f] = \sum_{x \in [N]} (2f_x - 1)A_x$$

is small. The reader unfamiliar with the $\infty \rightarrow 1$ norm should think of it for now as close relative of the spectral norm which can be approximated in polynomial time (a formal definition is in Section 5.2.1).

3. Show that the $\infty \rightarrow 1$ norm of $A[f^{\text{unif}}]$ is small with high probability where f^{unif} is uniformly distributed on $\{0, 1\}^N$.

To carry out our lower bound, we need to modify the final point, and prove that it continues to hold with high probability when we replace the truly random f^{unif} with our pseudorandom f^{pseud} . This is argued in Section 5.3.3 and consists of two steps. The first is a reweighting trick from [64], which replaces the $A_1, \dots, A_N$ with a second sequence $B_1, \dots, B_N$ so that for any $f \in \{0, 1\}^N$, whenever $A[f]$ has a large $\infty \rightarrow 1$ norm, $B[f]$ has a large spectral norm ($B[f]$ is

⁴We actually need both f and $\neg f$ to be far in Hamming distance from the range.

defined analogously to $A[f]$). We then need to prove that $B[f^{\text{pseud}}]$ has small spectral norm with high probability. For this we use the fact that, for any matrix D in m dimensions and any even number k , $\|D\|$ is approximated by the trace-moment polynomial $\text{tr}((D^\top D)^{k/2})^{1/k}$ within a factor $m^{1/k}$. Since in our case $B[f]$ lies in $\mathbb{R}^{m \times m}$ with $m \leq \text{poly}(N)$ and $k = O(\log N)$, we have that $\text{tr}((B[f]^\top B[f])^{k/2})^{1/k}$ approximates the spectral norm of $B[f]$ to within a constant factor. Since $\text{tr}((B[\cdot]^\top B[\cdot])^{k/2})^{1/k}$ is (the k^{th} root of) a degree k polynomial, we can argue that its distribution with respect to f^{pseud} and f^{unif} will be similar which will complete the proof.

There are several technical difficulties that we are leaving out of our discussion. First, to make our lower bound work in the setting of a general locally computable generator G (with larger word length and adaptive decision trees at its outputs) we need a more involved argument in place of (1) above. Second, we need to exercise a bit more care in our analysis of the value of the trace moment polynomial $\text{tr}((B[f]^\top B[f])^{k/2})^{1/k}$ to achieve the same bounds in the case that our distribution on f is only $N^{-O(1)}$ -wise $O(\log N)$ -wise independent, rather than perfectly $O(\log N)$ -wise independent. Finally, to deal with the case $k \gg \log N$ we need to generalize (2) above, using another construction from semirandom CSP refutation called the “Kikuchi Matrix” [166, 64].

We will not sketch in any detail our proof of Theorem 5.2 which gives a lower bound $N^{\frac{t}{2} + \Omega(1)}$ for odd constants t in the nonadaptive bit probe model, however we note that it follows the same overall form: we take a semirandom refutation procedure (in this case, the refutations in Section 9 of [64] based on earlier work of [48]), and show that we can still find the associated certificates in the pseudorandom case, for some appropriate notion of pseudorandomness. In this case we rely instead on small bias (fooling parity functions) as our pseudorandom property in place of k -wise independence. As mentioned earlier in the introduction, it is provably necessary to move beyond k -wise independence to achieve a lower bound of this form.

5.1.4 Open Problems

TODO: update w venkat paper We present here three potentially tractable open problems we believe are of particular interest. We start with the (seemingly) most accessible problem:

Problem 5.8. *Extend the bound in Theorem 5.1 to hold for odd values of t , and the bound in Theorem 5.2 to hold for even values of t .*

The natural approach here is to utilize the CSP refutation procedures for odd order CSPs developed in [2, 64] and analyze them in the “pseudorandom case” as we have done for even order CSPs. It seems likely that a solution to the first part of Problem 5.8 would automatically solve the second part (see the discussion at the end of Section 5.4).

Our second problem relates to Theorem 5.2. While our lower bounds for k -wise independent data structure problems are known to be optimal up to lower order terms (for even t), our nonadaptive bit probe lower bounds for ϵ -biased distributions have no matching upper bound. We highlight closing this gap as an important open problem:

Problem 5.9. *For $t \in \mathbb{N}$, determine the largest constant $\delta_t > 0$ so that, in the nonadaptive bit probe model, we have the lower bound $S \geq N^{\delta_t - o(1)}$ for any data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ which supports an $\exp(-N^{o(1)})$ -biased distribution on its columns.*

Our Theorem 5.2 yields the lower bound $\delta_t \geq \left(\frac{t}{2} - \frac{t-2}{2(t+2)}\right)^{-1}$, in particular $\delta_t > \frac{2}{t}$, for every odd value $t \geq 3$. Theorem 5.1 (the second form described above) also implies $\delta_t \geq \frac{2}{t}$ for even values of t since ϵ -almost k -wise independence is a special case of ϵ -biasedness; this latter lower bound holds in the more general adaptive setting and with larger word lengths. Both of these lower bounds hold even if we only aim to achieve bias $\text{poly}(N)^{-1}$ (rather than $\exp(-N^{o(1)})$ as stated above). On the other hand a construction of [126] implies that $\lim_{t \rightarrow \infty} \delta_t = 0$, in particular $\delta_t \leq O(\sqrt{\frac{1}{t}})$. Closing this gap in either direction remains an intriguing open problem.

Finally, we highlight two closely related problems: removing the NP oracle in Theorem 5.3 and removing the nonuniformity in Corollary 5.5:

Problem 5.10. *For NC_t^0 generators $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$:*

- (a) *for any $t \geq 3, \epsilon > 0$, give a subexponential time deterministic algorithm that solves Range Avoidance for G whenever $m \geq n^{\frac{1}{2} - \epsilon}$*

(b) for any $t \geq 5, \epsilon > 0$, give a subexponential time uniform algorithm that distinguishes G with subexponential advantage whenever $m \geq n^{\frac{t}{2}-\epsilon}$

One approach to this problem is to prove an *algorithmically constructive* variant of the hypergraph Moore bound of [64, 77]. Such an algorithm would also improve the best known polynomial time refutation algorithm for random t -CSPs, and has been an open problem (in the case $t = 3$) since the original work of [48]; there appears to be some significant possibility that no such algorithm exists. A different approach, tailored specifically to (b), is to show that the lower bound in Theorem 5.2 can be made *natural* in the sense of Razborov-Rudich. More specifically, it would actually be sufficient to develop an efficient algorithm with the following behavior: for some absolute constant $c \in \mathbb{N}$, the algorithm takes as input a uniformly random set $A \subseteq \mathbb{F}_2^n$, $|A| = n^c$, and with nonnegligible probability must output a certificate that A is a $\frac{1}{4}$ -biased probability space. See Section 5.5 for more details.

5.1.5 Prior Work on Data Structure Lower Bounds

We give here a slightly more involved historical account of lower bound techniques that can prove the strongest lower bounds in the static cell probe model for some explicit problem⁵. As mentioned earlier, the best known lower bound for general static data structures (for any $t > 1$ and any w) prior to our work was $S \geq N^{\frac{1}{t}}(\log M)^{1-\frac{1}{t}}$, first achieved by [154] for k -wise independent problems ($M \approx N^k$). Lower bounds of the form $S \gtrsim N^{\frac{1}{t}}$ can be established by a direct reduction to deterministic two party communication complexity. A more precise communication complexity analysis (keeping track of the number of rounds and the number of bits sent by each party in each round) developed in [124, 125] gives a similar but more precise lower bound in various settings. To achieve a lower bound which includes the extra factor $\approx \log M$ requires techniques more specially adapted to the cell probe model. This can be achieved either by expansion-based arguments known as *cell sampling* ([154, 155, 133, 109]), or in some cases by a more involved communication complexity argument using direct sum theorems [137].

⁵A great deal of work in the field is dedicated to adapting these techniques to prove the best possible lower bounds for particular *natural* problems of interest, which we will not cover here.

In the special setting of *nonadaptive bit probe* data structures (the setting in which our Theorem 5.2 applies), two works have achieved lower bounds improving on the cell sampling bound. In [164], when the time is $t = 2$ a near-maximal space lower bound $S \geq \frac{N}{2}$ is shown for a problem with $M \leq N^{O(1)}$ datapoints. The lower bound applies to any $F : [N] \times [M] \rightarrow \{0, 1\}$ whose columns support a γ -biased distribution for γ a sufficiently small constant. In subsequent work of [61], a lower bound of the form $S \gtrsim N^{\frac{1}{t-1}} (\log M)^{1-\frac{1}{t-1}}$ is obtained for k -wise independent problems ($M \approx N^k$). Their method is a deterministic variant of cell sampling, and they prove that the given lower bound is essentially the best that cell-sampling type methods can achieve. [61] also improves quantitatively Viola's bound for $t = 2$, showing a lower bound $S \geq (1 - \epsilon)N$ for any $N^{-O(1)}$ -almost $O(\log N)$ -wise independent problem where $\epsilon > 0$ is any constant (and the $O(\cdot)$ terms hide linear dependencies on ϵ). The results in [61] can be applied to larger word lengths $w > 1$ provided we transition to data structure problems with nonboolean output, of the form $F : [N] \times [M] \rightarrow \{0, 1\}^w$.

Observe that, in the setting where M is not much larger than N , e.g. $M \leq \exp(\text{poly log } N)$, the factor $(\log M)^{1-\frac{1}{t}}$ has negligible effect on the cell sampling bound $S \geq N^{\frac{1}{t}} (\log M)^{1-\frac{1}{t}}$. In this setting, no previously known method (even cell sampling) gives a lower bound significantly stronger than the direct communication reduction, which yields $S \gtrsim N^{\frac{1}{t}}$. We explain this reduction here since it is so simple. Say F has a time t space S data structure with word length w . Consider a protocol in which Alice holds x , Bob holds y ; Bob constructs the data structure encoding $\mathcal{E}(y)$, Alice simulates the query procedure on x , requesting cells of $\mathcal{E}(y)$ from Bob who responds with their contents. Alice sends $\log S$ bits in each round to request a cell, Bob sends w bits to reveal the cell's contents, and communication lasts t rounds, so the overall communication is $t \log S + tw$. For any problem F with maximal two party communication complexity, e.g. $F : [N] \times [N] \rightarrow \{0, 1\}$ given by $F(x, y) = \mathbb{1}\{x = y\}$, we then have $t \log S + tw \geq \log N$, i.e. $S \geq N^{\frac{1}{t}} \cdot 2^{-w}$.

Finally it is important to keep in mind that for *non-explicit* problems, counting arguments show that most functions $F : [N] \times [N] \rightarrow \{0, 1\}$ require space very close to N or time very close to $\log N$ [123]. The situation here is similar to many other areas in complexity; a nonconstructive

counting argument implies that random functions exhibit an extremely high lower bound, and the heart of the subject is to prove similar bounds for explicit (and ideally natural) problems.

5.2 Preliminaries

5.2.1 Basics

In this chapter we will use boldface variables e.g. $\mathbf{v}$ exclusively for random variables. When $\mathbf{v}$ is a random variable and A is a set, we write $\mathbf{v} \in A$ to indicate that the support of $\mathbf{v}$ is contained in A . When an expression references only a single random variable $\mathbf{v}$ we will use $\mathbb{E} \phi(\mathbf{v})$ for some $\phi : \text{supp}(\mathbf{v}) \rightarrow \mathbb{R}$ to mean that the expectation is taken with respect to $\mathbf{v}$; this is in contrast to the notation $\mathbb{E}_{x \sim X} \phi(x)$, where we always include the subscript. At one point we will use the set theoretic operator Δ denoting symmetric difference $a \Delta b = (a \setminus b) \cup (b \setminus a)$. For a collection of sets $(a_j)_{j \in J}$ we define $\bigtriangleup_{j \in J} a_j$ in the natural way using the commutativity/associativity of Δ as a binary operation.

In Section 5.3.2 we will deal with various vector and matrix norms, whose (standard) definitions we review here:

Definition 5.11 (Vector and Matrix Norms). *For a vector $u \in \mathbb{R}^m$ we use $\|u\|$ to denote the euclidean (L_2) norm of u , and $\|u\|_\infty$ to denote its L_∞ norm. For a matrix $A \in \mathbb{R}^{n \times n}$, we will use $\|A\|$ to denote the spectral norm given by*

$$\|A\| = \max_{\|u\|, \|v\| \leq 1} |u^\top A v|$$

and use $\|A\|_{\infty \rightarrow 1}$ to denote the $\infty \rightarrow 1$ operator norm given by

$$\|A\|_{\infty \rightarrow 1} = \max_{\|u\|_\infty, \|v\|_\infty \leq 1} |u^\top A v| = \max_{u, v \in \{\pm 1\}^m} |u^\top A v|$$

We now define the notions of limited independence and small-bias probability spaces which we will use extensively. In our main lower bounds we will work over the signed hypercube $\{\pm 1\}^N$,

and we present the following definitions in this setting; each definition has an equivalent form over $\{0, 1\}^N$ via the natural correspondence $\{0, 1\} \leftrightarrow \{\pm 1\}$.

Definition 5.12. *Let $f \in \{\pm 1\}^N$ be a random variable, $\gamma \in (0, 1)$, $k \leq N$. We say that f is γ -almost k -wise independent if, for all $X \subseteq [N]$, $|X| = k$ and all $g \in \{\pm 1\}^X$ we have*

$$\Pr[f|_X = g] \in [2^{-k} - \gamma, 2^{-k} + \gamma]$$

We say that f is k -wise independent if it is 0-almost k -wise independent.

In other words, a distribution over $\{\pm 1\}^N$ is γ -almost k -wise independent if its projection to any k coordinates is γ -close to the uniform distribution on $\{\pm 1\}^k$ in L_∞ distance; the statement applies without change in the $\{0, 1\}^N$ basis. Turning our attention to small bias distributions:

Definition 5.13 ([128]). *Let $f \in \{\pm 1\}^N$ be a random variable, $\gamma \in (0, 1)$, $k \leq N$. We say that f is (γ, k) -biased if, for all $X \subseteq [N]$, $|X| \leq k$ we have*

$$\mathbb{E} \prod_{x \in X} f_x \in [-\gamma, \gamma]$$

We say that f is γ -biased if it is (γ, N) -biased.

In the $\{0, 1\}$ basis, the property of being γ -biased has the following interpretation: for any parity function $\bigoplus_X : \{0, 1\}^N \rightarrow \{0, 1\}$ given by $\bigoplus_X(f) = \sum_{x \in X} f_x \pmod 2$ for some $\emptyset \neq X \subseteq [N]$, we have $|\Pr[\bigoplus_X(f) = b] - \frac{1}{2}| \leq \frac{\gamma}{2}$ for each $b \in \{0, 1\}$. In other words a γ -biased distribution over $\{0, 1\}^N$ fools all parity functions to within error $\frac{\gamma}{2}$. We will use the following well-known relationship:

Lemma 5.14 ([128]). *If f is (γ, k) -biased then it is γ -almost k -wise independent.*

Finally, we rely on the following construction of explicit small-bias probability spaces:

Theorem ([128]). *For every γ, k, N there exists an explicit distribution on $\{\pm 1\}^N$ with support size $(\frac{k \log N}{\gamma})^{O(1)}$.*

We elaborate on our notion of explicit: for the purposes of our Range Avoidance algorithms in Section 5.5, we can take explicit to mean, there is a uniform algorithm which, given k, N, ϵ will run in time $\text{poly}(N, k, \frac{1}{\epsilon})$ and output a list of strings in $\{\pm 1\}^N$, such that the uniform distribution on these strings is (γ, k) -biased. The distributions in [128] are also explicit in the more standard/informal sense, being supplied by a direct construction.

5.2.2 Generators and Data Structures

We now define the two main objects of study in our work: the cell probe model of static data structures on the one hand, and locally-computable generators on the other. We then state the tight connection between them, giving some more detail than was supplied in the introduction. Broader implications of this connection will be explored in more depth in Section 5.6.

Definition 5.15 (Cell Probe Model). *Let $F : [N] \times [M] \rightarrow \{0, 1\}$, which we refer to as a “data structure problem.” Elements $x \in [N]$ are referred to as “queries,” $y \in [M]$ as “datapoints.” We say that F has cell probe data structures with space complexity S , time complexity t , and word length w if there exist procedures of the following form:*

1. (Encoding:) *A computationally-unbounded encoding procedure which, given $y \in [M]$, preprocesses it into a data structure $\mathcal{E}(y) \in \Sigma^S$, $|\Sigma| \leq 2^w$*
2. (Query Procedure:) *A computationally-unbounded t -query algorithm Q which, given $x \in [N]$ and some encoding $E \in \Sigma^S$, queries t indices (“cells”) of E and outputs a bit $Q(x, E) \in \{0, 1\}$*

The data structure correctly solves the problem F if, for all $(x, y) \in [N] \times [M]$, $Q(x, \mathcal{E}(y)) = F(x, y)$. We say the data structure is nonadaptive if the query procedure decides ahead of time, as a function only of x , on the set of t cells which it will query.

As mentioned in the introduction, for a problem $F : [N] \times [M] \rightarrow \{0, 1\}$, we will interchangeably view it as a matrix $F \in \{0, 1\}^{N \times M}$ and refer to its set of *columns*; the columns of F consist of the vectors of the form $F_y = (F(x, y))_{x \in [N]} \in \{0, 1\}^N$ for each $y \in [M]$.

Next we turn to the formal definition of locally-computable generators:

Definition 5.16 (Locally-Computable Generators). *We say that $G : \Sigma^S \rightarrow \{0, 1\}^N$ is a locally-computable generator with time complexity t and space complexity S , provided that each output $(G_x)_{x \in [N]}$ is computed by a depth t decision tree T_x over the alphabet Σ . The word length of the generator is $\lceil \log |\Sigma| \rceil$. We will refer to strings $E \in \Sigma^S$ as “encodings” which are decoded to $G(E)$ by the generator G . Each index $i \in [S]$ is referred to as a “cell,” and for an encoding E the contents of the i^{th} cell is $E(i)$. If the decision trees $(T_x)_{x \in X}$ are nonadaptive, we refer to G as a non-adaptive generator. In the case $\Sigma = \{0, 1\}$ and G is nonadaptive, we also refer to G as an NC_t^0 generator.*

We emphasize that in the literature on cryptographic NC^0 generators, the word “local generator” typically refers to the special case of nonadaptive generators. We use the term “locally-computable” to refer to the more general notion based on decision tree depth, and always use the qualifier “nonadaptive” to refer to the special case of nonadaptive decision trees, or else NC_t^0 in the nonadaptive case when $\Sigma = \{0, 1\}$.

We now arrive at the observation at the heart of our investigations, whose proof is a matter of definitional manipulation:

Observation 5.17. *Let $F : [N] \times [M] \rightarrow \{0, 1\}$ be a data structure problem, interpreted also as a matrix $F \in \{0, 1\}^{N \times M}$ below. The following are equivalent:*

1. *F has time t , space S , word length w cell probe data structures (resp. nonadaptive)*
2. *There is a (resp. nonadaptive) locally-computable generator $G : \Sigma^S \rightarrow \{0, 1\}^N$ with $|\Sigma| \leq 2^w$ and time complexity t , such that every column of F lies in $\text{range}(G)$*

The proof is essentially identical to that of the special case discussed in the introduction.

Proof. Say that F has nonadaptive bit probe data structures of space S and time t . Then, to every $y \in [M]$ we may associate an encoding $E_y \in \{0, 1\}^S$, so that for every $x \in [N]$ we may determine $F(x, y)$ by probing (resp. nonadaptively probing) t bits of E_y . Considering the y^{th} column of

F as a string $f_y \in \{0, 1\}^N$, we see that there is a (resp. nonadaptive) locally-computable function $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ with time complexity t , so that $G(E_y) = f_y$ for every $y \in [M]$. In particular, the x^{th} output of G runs the data structure's query procedure corresponding to the query x . Every step in this argument in fact utilized a direct equivalence between premise and conclusion; reading it in reverse yields a proof for the other direction. $\square$

5.3 Cell Probe Lower Bounds via CSP-Pseudorandomness

Using our main observation from the previous section (Observation 5.17), proving cell probe lower bounds for a data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$ is equivalent to showing that, for the set $\text{cols}(F) = \{(F(x, y))_{x \in [N]} \mid y \in [M]\} \subseteq \{0, 1\}^N$ consisting of the columns of F , there does not exist a locally-computable generator $G : \Sigma^S \rightarrow \{0, 1\}^N$ whose range contains every vector $f \in \text{cols}(F)$. If we instead interpret a given locally-computable generator $G : \Sigma^S \rightarrow \{0, 1\}^N$ as a candidate *pseudorandom generator*, a series of works on semirandom CSP refutation [47, 5, 2, 64, 77] have developed techniques which can efficiently generate *certificates* $f \notin \text{range}(G)$ for randomly chosen $f \sim \{0, 1\}^N$. In this section, we show how to apply these techniques to *pseudorandom* vectors $f \sim \mathcal{F} \subseteq \{0, 1\}^N$ with $\mathcal{F}$ being a distribution with suitable *limited independence* properties. This will imply that if $\text{cols}(F)$ supports such a pseudorandom distribution, then we can argue $\text{cols}(F) \not\subseteq \text{range}(G)$ for any generator G with appropriate parameters, which will yield a cell probe lower bound.

For the remainder of the section it will be more convenient to work over $\{\pm 1\}$ rather than $\{0, 1\}$; hence our data structure problems will be of the form $F : [N] \times [M] \rightarrow \{\pm 1\}$ and our generators of the form $G : \Sigma^S \rightarrow \{\pm 1\}^N$.

5.3.1 From Locally-Computable Generators to XOR Schemes

The first step in the semirandom refutation procedures of [47, 2, 64] is a reduction from refuting arbitrary t -CSPs to refuting max- t -XOR CSPs. We prove a kind of analogue of this argument for locally-computable generators. For any time t , space S locally-computable generator G and any

$f \in \{\pm 1\}^N$, we will associate a system of N different t -XOR equations over $\approx S$ variables. This system of equations will have one part which depends only on G , and another which depends only on f . We will argue that, in the case $f \in \text{range}(G)$, the system of equations will be “moderately satisfiable,” in the sense that there is some assignment which satisfies significantly more than half of the equations, or else falsifies significantly more than half. In subsequent sections, we will show that if f is sampled from a distribution with suitable limited independence properties, any such CSP arising from G, f will not be moderately satisfiable in this sense, which will give us our lower bound.

The above discussion leaves out some complications of the argument; for a given G, f we will actually construct a small collection of t -XOR CSPs, and can only guarantee that *one of them* has a large value in the case $f \in \text{range}(G)$. To obtain good parameters in our main reduction (Theorem 5.20) we also need some additional arguments more specially tailored to the cell probe model which do not seem to have a direct analogue in the CSP-refutation literature.

Definition 5.18. *We define a “ t -XOR scheme” $C = (X, V, c)$ to consist of the following components:*

1. *A finite “index set” X and a finite “variable set” V*
2. *$c : X \rightarrow \binom{V}{t}$ identifies each $x \in X$ with a set of variables (“constraint”) $c_x \subseteq V$, $|c_x| = t$.*

We say that $C = (X, V, c)$ is “indexed by X .” For a sign-pattern $f : X \rightarrow \{\pm 1\}$ and assignment $R \in \{\pm 1\}^V$ to the underlying variables, we define

$$\text{Val}(C, f, R) := \left| \mathbb{E}_{x \in X} f_x \prod_{i \in c_x} R(i) \right|$$

In some cases, we will have a ground set $[N]$, and a subset $X \subseteq [N]$ with an XOR scheme C indexed by X ; in this case, for $f \in \{\pm 1\}^N$ and an assignment R , we define $\text{Val}(C, f, R)$ to equal $\text{Val}(C, f', R)$, where f' is the restriction of f to the coordinates in X .

Finally, for an XOR scheme C and sign pattern f , we define

$$\text{OPT}(C, f) := \max_{R \in \{\pm 1\}^V} \text{Val}(C, f, R)$$

As foreshadowed in the discussion preceding this definition, the name “XOR scheme” stems from an interpretation in terms MAX- t -XOR CSPs. For a scheme $C = ([N], V, c)$ and some $f \in \{\pm 1\}^N$, we may think of the *pair* (C, f) as representing a t -XOR CSP in the following way. Under the natural identification $(\{\pm 1\}, \times) \leftrightarrow (\mathbb{F}_2, +)$, consider the system of N linear equations over $\mathbb{F}_2$ -valued variables $\{v_i \mid i \in V\}$, given by

$$\left(\sum_{i \in c_x} v_i = f_x \pmod{2} \right) \quad \text{for each } x \in [N]$$

We then see that $\text{OPT}(C, f)$ is the maximum, over all assignments of the variables to values in $\mathbb{F}_2$, of the difference between the fraction of satisfied and unsatisfied equations. In particular, if some assignment can satisfy (resp. falsify) every equation the value is 1; if the value is small, then every assignment will have close to the same number of equations being satisfied/unsatisfied. If we fix C , then we are fixing the left-hand sides of the above system of equations, and as we range over f , we range over different possible right hand sides which may cause the system to be more or less satisfiable. This is the source of the word *scheme*: a single scheme C defines a family of 2^N different t -XOR CSP instances as we range over the 2^N possible values of the “right hand side” $f \in \{\pm 1\}^N$.

Returning to the $\{\pm 1\}$ viewpoint, observe that we may extend the definition of $\text{Val}(C, f, R)$ to any $R \in \mathbb{R}^V$. In this view, $\text{Val}(C, f)$ defines a multilinear polynomial in $|V|$ many real variables, such that the absolute value of the polynomial on any assignment $R \in \mathbb{R}^V$ is equal to $\text{Val}(C, f, R)$. This leads to the following:

Observation 5.19. For any C, f ,

$$\max_{R \in \{-1,1\}^V} \text{Val}(C, f, R) = \max_{R \in [-1,1]^V} \text{Val}(C, f, R) \geq \max_{R \in \{0,1\}^V} \text{Val}(C, f, R)$$

Proof. The second inequality is trivial ($\{0,1\} \subseteq [-1,1]$). The first is a standard consequence of multilinearity. To confirm it, we may start with any assignment $R \in [-1,1]^V$ achieving a value $\delta \in [0,1]$, and replace it by an assignment in $\{\pm 1\}^V$ achieving value $\geq \delta$ by iterating over $i \in V$ and greedily replacing each $R(i)$ by an element in $\{\pm 1\}$ which does not decrease the value. At a given step, we have

$$\text{Val}(C, f, R) = \left| R(i) \cdot P((R(j))_{j \neq i}) + Q((R(j))_{j \neq i}) \right|$$

for some polynomials P, Q depending only on $(R(j))_{j \neq i}$. This is the absolute value of a linear function in $R(i)$ whose maximum is achieved at the boundary of $[-1,1]$. $\square$

The right-hand side in Observation 5.19 may be seen as a kind of hypergraph discrepancy parameter. Viewing $(c_x)_{x \in X}$ as a t -uniform (multi-)hypergraph over vertex set V with hyperedges indexed by X , we may think of $f \in \{\pm 1\}^X$ as assigning a sign to each hyperedge. Then $\max_{R \in \{0,1\}^V} \text{Val}(C, f, R)$ is the maximum, over all subsets of vertices $U \subseteq V$, of the sum of the values f_x with x ranging over hyperedges in the sub-hypergraph induced by U . The smallness of $\max_{R \in \{0,1\}^V} \text{Val}(C, f, R)$ indicates that every large induced subhypergraph has its edges colored by f in an approximately balanced way.

We now show that, for any locally-computable generator over output space $\{\pm 1\}^N$, we can associate a small collection of XOR schemes indexed by $[N]$ (or by large subsets $X \subseteq [N]$), so that for any $f \in \text{range}(G)$, we have that $\text{OPT}(C, f)$ is large for one of the schemes C in our collection.

Theorem 5.20. Let $G : \Sigma^S \rightarrow \{\pm 1\}^N$ be a locally-computable generator with time complexity t , word length $w = \log |\Sigma|$. There exists a collection of sets $(X_j \subseteq [N])_{j \leq 2^{tw}+1}$, $|X_j| \geq 2^{-tw-1}N$,

and a collection of t -XOR schemes $(C_j)_{j \leq 2^{tw+1}}$, with C_j a t -XOR scheme indexed by X_j and having $\leq tS$ variables, so that for every $f \in \text{range}(G)$ there exists $j \leq 2^{tw+1}$ with $\text{Val}(C_j, f) \geq 2^{-tw-1}$. In particular, if $f \in \{\pm 1\}^N$ is a random variable supported on $\text{range}(G)$, then there exists an XOR scheme C over some $X \subseteq [N]$, $|X| \geq 2^{-tw-1}N$, so that $\Pr[\text{OPT}(C, f) \geq 2^{-tw-1}] \geq 2^{-tw-1}$.

Proof. By definition, for every $x \in [N]$, $G_x : \Sigma^S \rightarrow \{\pm 1\}$ is computed by a decision tree T_x of depth t over the alphabet Σ . We develop here some more detailed notation to describe the trees $(T_x)_{x \in [N]}$. Let $\mathcal{L}(T_x)$ denote the leaves of T_x and $\mathcal{I}(T_x)$ denote the internal nodes. Each internal node $v \in \mathcal{I}(T_x)$ is associated with a cell $p_x^v \in [S]$ which it probes, i.e. if T_x reaches the internal node v at some point in its computation on the input $E \in \Sigma^S$, then it will query $E(p_x^v)$ next and then take a corresponding step down the tree T_x . Each leaf $\ell \in \mathcal{L}(T_x)$ corresponds uniquely to an element $\pi \in \Sigma^t$, where π is the label of the unique path from the root to ℓ : the first cell queried by T_x on input $E \in \Sigma^S$ has value π_1 , the second has value π_2 , and so on. Thus overall, $T_x(E)$ will follow the unique path π that is consistent with E to the leaf ℓ . Finally, each $\pi \in \Sigma^t$ is labeled by a value $\phi_x(\pi) \in \{\pm 1\}$, so that if T_x reaches the leaf $\ell \in \mathcal{L}(T_x)$ associated with π on input $E \in \Sigma^S$, then $G_x(E) = \phi_x(\pi)$. For a given value $\pi \in \Sigma^t$, define the sequence $q_x^{(\pi,1)}, \dots, q_x^{(\pi,t)} \in [S]$ as follows: let $v_1, \dots, v_t \in \mathcal{I}(T_x)$ be the internal nodes found on the root to leaf path in T_x with leaf corresponding to π , and set $q_x^{(\pi,j)} = p_x^{v_j}$.

Using the structure of the generator G we now define the collection of XOR schemes guaranteed in the Theorem statement. Let $V^1, \dots, V^t$ be sets of size S , each associated canonically with $[S]$, and $V = V^1 \cup \dots \cup V^t$. For each $x \in [N]$, $\pi \in \Sigma^t$, let $c_x^\pi = \{q_x^{(\pi,1)}, \dots, q_x^{(\pi,t)}\}$, where $q_x^{(\pi,j)}$ is considered to live inside V^j . For each $\pi \in \Sigma^t$ and each $b \in \{\pm 1\}$ let $X^{(\pi,b)} = \{x \mid \phi_x(\pi) = b\}$. For each π, b , we define the XOR scheme $C^{(\pi,b)} = (X^{(\pi,b)}, V, c^\pi)$; we will ignore any $C^{(\pi,b)}$ with $|X^{(\pi,b)}| < 2^{-tw-1}|X|$. We emphasize that the variables V in our XOR schemes are $\{\pm 1\}$ -valued (as per the definition of XOR schemes) rather than Σ -valued, despite being derived from some correspondence to the domain of G , whose cells are Σ -valued.

Now, let $f \in \text{range}(G)$, and choose some canonical preimage E_f with $G(E_f) = f$. Then there exists $\pi^f \in \Sigma^t$ so that, for the set $\tilde{X}^{(\pi^f, f)}$ consisting of all $x \in [N]$ such that $T_x(E_f)$ reaches the

leaf corresponding to π^f , we have $|\tilde{X}^{(\pi^f, f)}| \geq 2^{-tw}N$. There must then exist a value $b^f \in \{\pm 1\}$ such that $|\tilde{X}^{(\pi^f, f)} \cap X^{(\pi^f, b^f)}| \geq 2^{-tw-1}N$; clearly $|X^{(\pi^f, b^f)}| \geq 2^{-tw-1}N$ as well.

At this point, we have

$$\sum_{x \in X^{(\pi^f, b^f)}} \prod_{j \leq t} \mathbb{1}\{E_f(q_x^{(\pi, j)}) = \pi_j^f\} = |\tilde{X}^{(\pi^f, f)} \cap X^{(\pi^f, b^f)}| \geq 2^{-tw-1}N \geq 2^{-tw-1}|X^{(\pi^f, b^f)}|$$

Now define the boolean assignment $R_f \in \{0, 1\}^V$, where for $i \in V^j \subseteq V$, we set $R_f(i) = \mathbb{1}\{E_f(i) = \pi_j\}$. In this new notation we have $\prod_{i \in c_x^{(\pi^f)}} R_f(i) = \prod_{j \leq t} \mathbb{1}\{E_f(q_x^{(\pi, j)}) = \pi_j\}$ for every x . Recall also that $G_x(E_f) = b^f$, and hence $f_x = b^f$, whenever $\prod_{j \leq t} \mathbb{1}\{E_f(q_x^{(\pi, j)}) = \pi_j\} = 1$, $x \in X^{(\pi^f, b^f)}$.

So overall we conclude

$$\begin{aligned} \text{Val}(C^{(\pi^f, b^f)}, f, R_f) &= |X^{(\pi^f, b^f)}|^{-1} \cdot \left| \sum_{x \in X^{(\pi^f, b^f)}} f_x \prod_{i \in c_x^{(\pi^f)}} R_f(i) \right| \\ &= |X^{(\pi^f, b^f)}|^{-1} \cdot \left| b^f \sum_{x \in X^{(\pi^f, b^f)}} \prod_{j \leq t} \mathbb{1}\{E_f(q_x^{(\pi, j)}) = \pi_j^f\} \right| \geq 2^{-tw-1} \end{aligned}$$

Using Observation 5.19, we may replace the boolean assignment $R_f \in \{0, 1\}^V$ by a $\{\pm 1\}$ -assignment $\tilde{R}_f \in \{\pm 1\}^V$ achieving a value at least as large, so overall we conclude that $\text{OPT}(C^{(\pi^f, b^f)}, f) \geq 2^{-tw-1}$.

This concludes the proof of the initial claim in the theorem. The last sentence of the Theorem (“in particular...”) follows from the first part by taking a union bound over $j \leq 2^{tw+1}$. $\square$

We focus our attention on the “in particular” clause at the end of the statement of Theorem 5.20. For a data structure problem $F : [N] \times [M] \rightarrow \{\pm 1\}$, we have reduced the problem of proving cell probe lower bounds for F to the following: find a random vector $\mathbf{f} \in \{\pm 1\}^N$ supported on the columns of F , so that for *any* t -XOR scheme C defined over a large subset of $[N]$, we have that $\text{OPT}(C, \mathbf{f})$ is very small with very high probability. This may be seen as a specialized notion of pseudorandomness; in particular we may define:

Definition 5.21 (CSP-Pseudorandomness). *Let $\mathbf{f} \in \{\pm 1\}^N$ be a random variable. We say that $\mathbf{f}$*

is “ (N, K, δ, ϵ) -CSP-Pseudorandom” if, for any XOR scheme C indexed by $[N]$ with K variables, we have

$$\Pr[\text{OPT}(C, f) \geq \delta] < \epsilon$$

We will not use this extra notation in our formal arguments later on, and will instead refer directly to Theorem 5.20, but we isolate the definition here for the purpose of discussion. We may think of each C indexed by $[N]$ with K variables and each δ as defining a “statistical test” $\mathcal{T} : \{\pm 1\}^N \rightarrow \{0, 1\}$, with $\mathcal{T}(f) = \mathbb{1}\{\text{OPT}(C, f) < \delta\}$. Provided $K \ll N$ and δ is not too small, we can easily argue that a *truly random* f will pass the test with overwhelming probability (chernoff bound + union bound over $\{\pm 1\}^K$); our goal is then to define a pseudorandom distribution which is similar to the uniform distribution with respect to passing this class of tests. This is a standard way to define a notion of pseudorandomness in terms of a class of statistical tests which it “fools.” Many examples of such distributions have been constructed in the case that the family of tests in question are of very low computational complexity. However, in this case the statistical test defining $\mathcal{T}$ is NP hard to compute in general: it references the optimal value of a CSP defined by f . For this reason, analyzing this class of tests directly and constructing a suitable pseudorandom distribution appears extremely difficult. This is precisely where *efficient CSP refutation algorithms* come in to play: if we can come up with a more tractable test $\mathcal{T}'$, so that $\mathcal{T}'(f) \rightarrow \mathcal{T}(f)$ and still $\mathcal{T}'$ happens with high probability for a random f , we may reduce the task at hand to fooling the simpler test $\mathcal{T}'$. Technically speaking the notions of “tractability” needed here vs. in the setting CSP refutation are quite different. For us, it is neither necessary nor sufficient that $\mathcal{T}'$ be computable in polynomial time; rather we would like the proof of the statement “a random f satisfies $\mathcal{T}'(f) = 1$ ” to hold over some pseudorandom distribution with significantly smaller entropy.

We make a final note on the discrepancy between Definition 5.21 and Theorem 5.20: in Theorem 5.20 we may have to pass to some large subset $X \subseteq [N]$ and define an XOR scheme indexed only by X , while in Definition 5.21 we refer only to XOR schemes over the original set of indices $[N]$. This distinction will be essentially irrelevant to us, since we will deal with notions of

pseudorandomness for vectors $f \in \{\pm 1\}^N$ which are preserved under passing to subsets of indices $X \subseteq [N]$. Hence, for most of the following we will state results in terms of XOR schemes indexed by $[N]$ and random vectors in $\{\pm 1\}^N$, with the understanding that we may have started with some original $[N']$, $f' \in \{\pm 1\}^{N'}$, passed to $X \subseteq [N']$ of size N , and then identified X with $[N]$ in some canonical way. We will then remember that this “passing to a subset” operation has occurred only when we get to our ultimate proof of the lower bound in Theorem 5.1, where the density of the subset we passed to will have some mild effect on the parameters.

5.3.2 CSP-Pseudorandomness via Limited Independence

In this section we prove that if $f \in \{\pm 1\}^N$ satisfies suitable *limited independence* properties, then for any t -XOR scheme C with suitable parameters, $\text{OPT}(C, f)$ will be very small with very high probability. In the terminology of Definition 5.21, we aim to prove that limited independence implies CSP-pseudorandomness (within a certain regime of parameters).

We accomplish this in the following way: starting with any t -XOR scheme C indexed by $[N]$, we construct a certain ensemble of matrices indexed by $[N]$, which has the property that whenever we take a *signed sum* of these matrices using f as a signing, we have that the $\infty \rightarrow 1$ norm of the resulting matrix is lower bounded by $\text{OPT}(C, f)$. We then prove that, when f is chosen from a pseudorandom distribution with suitable limited-independence properties, the operator norm of a random matrix sum signed by f will be small with high probability. The proof of this latter claim is delayed to the next subsection (Section 5.3.3). We start with the definition of a matrix ensemble:

Definition 5.22. A matrix ensemble $\mathcal{A} = (A_x \in \mathbb{R}^{m \times m})_{x \in [N]}$ is a collection of matrices of the same dimension; we say that $\mathcal{A}$ lives in dimension m and is indexed by the set $[N]$. For $f \in \{\pm 1\}^N$, we define the matrix

$$\mathcal{A}[f] = \sum_{x \in [N]} f_x A_x$$

If each matrix A_x is a subpermutation matrix (boolean with at most one nonzero entry per row and column), we say that $\mathcal{A}$ is a permutation ensemble, and we define $d(\mathcal{A}) = m^{-1} \sum_{x,i,j} A_x(i, j)$ to

be the “average degree” of the ensemble.

We will construct matrix ensembles from XOR schemes via the “Kikuchi-Matrix Method” pioneered by [166] and developed further in [64]. We describe first a special case of the construction, which will suffice for our lower bound on k -wise independent problems in the important case $k \approx \log N$. The idea is simple: we create a new “meta variable” for each $t/2$ -tuple of original variables, and each original t -ary constraint $c_x = \{i_1, \dots, i_t\}$ becomes a 2-ary constraint $c'_x = \{(i_1, \dots, i_{t/2}), (i_{t/2+1}, \dots, i_t)\}$ over the new variables. This reduces things to the case of a 2-XOR scheme, while blowing up the number of variables by an exponent of $t/2$. For 2-ary XOR schemes, the optimization problem associated with $\text{OPT}(C, f)$ can be interpreted as maximizing the quadratic form of some matrix over the signed hypercube; this optimization problem is precisely captured by the $\infty \rightarrow 1$ norm. In the proof below we will cut out the intermediate step of defining a 2-CSP, and construct the relevant matrices directly from the original t -CSP.

Lemma 5.23. *Let $C = ([N], V, c)$ be a t -XOR scheme with $|V| = K$ and t even. There exists a matrix ensemble $\mathcal{A} = (A_x)_{x \in [N]}$ so that:*

1. *Each A_x lives in $\mathbb{R}^{m \times m}$ for $m = \binom{K}{t/2}$, and has exactly 1 nonzero entry with value 1*
2. *$d(\mathcal{A}) \geq K^{-t/2} \cdot N$*
3. *For any $f \in \{\pm 1\}^N$, we have $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq d(\mathcal{A})m \cdot \text{OPT}(C, f)$*

Proof. Associate $[m]$ with $\binom{V}{t/2}$, so $m \leq K^{t/2}$. For $x \in [N]$, write its associated constraint in C as $c_x = \{i_x^1, \dots, i_x^t\} \subseteq V$ with its variables ordered in some canonical way. We define our matrix ensemble by

$$A_x(a, b) = \mathbb{1}\{a = \{i_x^1, \dots, i_x^{t/2}\} \text{ and } b = \{i_x^{t/2+1}, \dots, i_x^t\}\}$$

Observe that each A_x has exactly one nonzero entry with value one, and hence $d(\mathcal{A}) = m^{-1}N \geq K^{-t/2} \cdot N$. Now, let $f \in \{\pm 1\}^N$ be given, and let $R \in \{\pm 1\}^V$ be such that $\text{OPT}(C, f) = \text{Val}(C, f, R)$.

Consider the vectors $\tilde{R} \in \{\pm 1\}^m$, given by $\tilde{R}(a) = \prod_{i \in a} R(i)$. Then

$$\begin{aligned} \|\mathcal{A}[f]\|_{\infty \rightarrow 1} &\geq \tilde{R}^\top \mathcal{A}[f] \tilde{R} = \sum_{a,b \in \binom{V}{t/2}} \tilde{R}(a) \tilde{R}(b) \mathcal{A}[f](a,b) = \sum_{x \in [N]} f_x \sum_{a,b} A_x(a,b) \prod_{i \in a} R(i) \prod_{j \in b} R(j) \\ &= \sum_{x \in [N]} f_x \prod_{i \in c_x} R(i) = N \cdot \text{Val}(C, f, R) = N \cdot \text{OPT}(C, f) \end{aligned}$$

In other words $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq d(\mathcal{A})m \cdot \text{OPT}(C, f)$. $\square$

Now, say that we started with a time- t generator $G : \Sigma^S \rightarrow \{0, 1\}^N$ with $t = O(1)$, $S = o(\frac{N}{\log N})^{\frac{2}{t}}$, and had some random vector $f \in \{\pm 1\}^N$ supported on the range of G which is $O(\log N)$ -wise independent. Applying Theorem 5.20 followed by Lemma 5.23 above, we would obtain a matrix ensemble $\mathcal{A}$ indexed by some large subset $X \subseteq [N]$, $|X| \geq \Omega(N)$ with the following properties: the ensemble lives in $\mathbb{R}^{m \times m}$ for $m = N^{O(1)}$, has average degree $\omega(\log N) = \omega(\log m)$, and with high probability $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq \Omega(N)$. In the following we will argue that for any f which is $O(\log N)$ -wise independent, we will have $\|\mathcal{A}[f]\| = o(N)$ for any $\mathcal{A}$ in dimension $m = N^{O(1)}$ and with $d(\mathcal{A}) = \omega(\log N)$; we would then reach a contradiction with our initial assumption $S \leq o(\frac{N}{\log N})^{\frac{2}{t}}$, which would give us the desired cell probe lower bound $S \geq \tilde{\Omega}(N^{\frac{2}{t}})$ for $O(\log N)$ -wise independent problems.

In the case $k \gg \log N$ we need a generalization of the construction in Lemma 5.23 from [166, 64]. We will reproduce the proof (in the appendix) since our notation is somewhat different. The parameter “ L ” occurring in this Theorem is called the “Kikuchi level” of the construction, and we will set it to be $\approx k$ in our subsequent arguments when proving lower bounds for k -wise independent problems for larger values of k :

Theorem 5.24 ([166, 64]). *Let $C = ([N], V, c)$ be a t -XOR scheme, $|V| = K$, and t even. For any $t \leq L \leq \frac{K}{8}$, we may associate with C a permutation ensemble $\mathcal{A}$ indexed by $[N]$ with the following properties:*

1. $\mathcal{A}$ lives in $\mathbb{R}^{m \times m}$ for $m = \binom{K}{L}$
2. $d(\mathcal{A}) \geq (\frac{L}{K})^{t/2} N$

3. For any $f \in \{\pm 1\}^N$, we have $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq d(\mathcal{A})m \cdot \text{OPT}(C, f)$

The construction is a straightforward generalization of that in Lemma 5.23: identifying $[m]$ with $\binom{V}{L}$, we set $A_x(a, b) = \mathbb{1}\{c_x = a\Delta b\}$ where Δ denotes symmetric difference. The proof is also essentially identical to that of the special case Lemma 5.23 and so we relegate it to the appendix (Section 5.7). From now on we will refer only to the more general Theorem 5.24 and leave behind the special case Lemma 5.23.

It remains to show that if $\mathcal{A}$ is a permutation ensemble with sufficiently small dimension and large average degree, then for any random vector f exhibiting suitable *limited independence* properties, we will have $\|\mathcal{A}[f]\|_{\infty \rightarrow 1}$ small with high probability. We defer the proof to the next subsection (Section 5.3.3), but state the bounds here:

Theorem 5.25. *Let $\mathcal{A}$ be a permutation ensemble in $\mathbb{R}^{m \times m}$ indexed by $[N]$, let $f \in \{\pm 1\}^N$ be a random vector which is γ -almost k -wise independent for k even, and let $\delta > 0$ be sufficiently small. Assume the following relations are satisfied:*

1. $d(\mathcal{A}) \geq C \frac{1}{\delta^2} \log(\frac{1}{\delta}) m^{2/k} (k + \log m)$ for a sufficiently large universal constant C
2. $\gamma \leq 2^{-2k-1} \delta^k m^{-1}$

Then we have the tail bound:

$$\Pr[\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq \delta m d] \leq 2^{-k}$$

Using Theorem 5.24 we may directly translate this to a statement about XOR schemes:

Theorem 5.26. *Let $C = ([N], V, c)$ be a t -XOR scheme, $|V| = K$, t even, $L \in \mathbb{N}$ with $t \leq L \leq \frac{K}{8}$, and say that $f \in \{\pm 1\}^N$ is γ -almost k -wise independent, $k \leq L \log N$. Then for any $\delta > 0$ sufficiently small we have*

$$\Pr[\text{OPT}(C, f) \geq \delta] \leq 2^{-k}$$

provided that $\frac{N}{\log N} \geq C \frac{1}{\delta^2} \log \frac{1}{\delta} K^{\frac{t}{2} + \frac{2L}{k}} \cdot L^{1-\frac{t}{2}}$ where C is a universal constant, and $\gamma \leq 2^{-2k-1} \delta^k m^{-1}$.

Finally applying the above Theorem together with Theorem 5.20, we obtain our main cell probe lower bound:

Theorem (Theorem 5.1 restated). *Let $F : [N] \times [M] \rightarrow \{\pm 1\}$ be a data structure problem such that there exists a γ -almost k -wise independent random vector $\mathbf{f} \in \{\pm 1\}^N$ supported on the columns of F . Let $S, t, w \in \mathbb{N}$ be given with t an even number and assume $k > tw + 1$. Then, for any space S , time t , word length w cell probe data structure solving F , we must have:*

1. $S \geq \left(\frac{N}{\log N} \right)^{\frac{2}{t}} \cdot \left(\frac{k}{\log N} \right)^{1-\frac{2}{t}} \cdot t^{-1} 2^{-(6-o(1))w}$ for all $t \log N \leq k \leq N 2^{-(3-o(1))tw} t^{-\frac{t}{2}}$
2. $S \geq \left(\frac{N}{\log N} \right)^{\frac{1}{t(\frac{1}{2}+\frac{2}{k})}} \cdot t^{-1} 2^{-(6-o(1))w}$ for all $t \leq k \leq t \log N$,

provided $\gamma \leq N^{-t} 2^{-O(ktw)}$ in each case.

Proof. Set $\delta = 2^{-tw-1}$. Say that F has a time t , space S data structure with t even. Fix some $L \in \mathbb{N}$ satisfying $t \leq L \leq \frac{tS}{8}$ and $k \leq L \log N$ to be specified later, and assume that $\gamma \leq (tS)^{-L} 2^{-2k-1} \delta^{-k}$. Applying Theorem 5.20 we may find $X \subseteq [N]$ of size $|X| \geq \delta N$ and an XOR scheme $C = (X, V, c)$, $|V| = K := tS$ indexed by X so that, with probability at least δ over $\mathbf{f}$, $\text{OPT}(C, \mathbf{f}) \geq \delta$. Applying Theorem 5.26 and using the fact that $2^{-k} < \delta$ (since $k > tw + 1$), this is only possible provided

$$\delta \frac{N}{\log N} \leq \frac{|X|}{\log |X|} < C \frac{1}{\delta^2} \log \frac{1}{\delta} K^{\frac{t}{2} + \frac{2L}{k}} \cdot L^{1-\frac{t}{2}}$$

We have $C \frac{1}{\delta^3} \log \frac{1}{\delta} \leq 2^{3tw+o(tw)}$. We also may safely assume $S \leq \frac{N}{t}$ and $\log K \leq \log N$. Plugging in these estimates and tS in place of K we obtain the bound:

$$S > \left(\frac{N}{\log N} \right)^{\frac{1}{\frac{t}{2} + \frac{2L}{k}}} \cdot L^{\frac{\frac{t}{2}-1}{\frac{t}{2} + \frac{2L}{k}}} \cdot t^{-1} 2^{-(6-o(1))w} \quad (5.2)$$

To finish the proof, we set $L = \frac{k}{\log N}$ in case (1) and $L = t$ in case (2); in both cases the condition $t \leq L \leq \frac{tS}{8}$ will be satisfied, and $\gamma \leq (tS)^{-L} 2^{-2k-1} \delta^{-k}$ will hold provided $\gamma \leq N^{-t} 2^{-Cktw}$ for a sufficiently large constant C .

□

5.3.3 Proof of Theorem 5.25

We now prove Theorem 5.25 which is reproduced below for convenience:

Theorem (Theorem 5.25, restated). *Let $\mathcal{A}$ be a permutation ensemble in $\mathbb{R}^{m \times m}$ indexed by $[N]$, let $\mathbf{f} \in \{\pm 1\}^N$ be a random vector which is γ -almost k -wise independent for k even, and let $\delta > 0$ be sufficiently small. Assume the following relations are satisfied:*

1. $d(\mathcal{A}) \geq C \frac{1}{\delta^2} \log(\frac{1}{\delta}) m^{2/k} (k + \log m)$ for a sufficiently large universal constant C
2. $\gamma \leq 2^{-2k-1} \delta^k m^{-1}$

Then we have the tail bound:

$$\Pr[\|\mathcal{A}[\mathbf{f}]\|_{\infty \rightarrow 1} \geq \delta m d] \leq 2^{-k}$$

The first half of our proof will follow a strategy of [77]: we will pass from $\mathcal{A}$ to a second matrix ensemble $\mathcal{B}$ over the same index set $[N]$ so that whenever $\|\mathcal{A}[\mathbf{f}]\|_{\infty \rightarrow 1}$ is large, $\|\mathcal{B}[\mathbf{f}]\|$ is large, where $\|\cdot\|$ denotes the spectral norm. We then prove that limited independence fools the spectral norm. For this we use a standard technique, which approximates the spectral norm of a matrix by the trace of a suitable matrix power; the trace of this power will be low degree polynomial in the underlying variables $\{f_x \mid x \in [N]\}$, and hence takes on a similar value when $\mathbf{f}$ is chosen truly uniformly, or chosen from distribution with limited independence. We remark that a similar argument is made in passing in the introduction of [41] in the context of refuting fully-random CSPs. Finally, it remains to prove that the spectral norm of $\mathcal{B}[\mathbf{f}]$ is low when $\mathbf{f}$ is truly uniform; for this we rely on matrix concentration inequalities as in various prior works on semirandom refutation [2, 64].

We start with some additional definitions and inequalities necessary for the proof:

Definition 5.27. *For a matrix A and even integer k , define $A^{\bowtie k} := (A^\top A)^{k/2}$. For a matrix*

ensemble $\mathcal{B} = (B_x)_{x \in [N]}$, let

$$\nu_{\text{row}}(\mathcal{B}) = \left\| \sum_x B_x^\top B_x \right\|, \quad \nu_{\text{col}}(\mathcal{B}) = \left\| \sum_x B_x B_x^\top \right\|, \quad \nu(\mathcal{B}) = \max\{\sigma_{\text{row}}(\mathcal{B}), \sigma_{\text{col}}(\mathcal{B})\}$$

and let $\chi(\mathcal{B}) = \max_{x \in [N]} \|B_x\|$.

We need the following well-known fact:

Lemma 5.28 (Trace/Norm Inequality). *For $A \in \mathbb{R}^{m \times m}$ and any even k ,*

$$m^{-1/k} \cdot \text{tr}(A^{\boxtimes k})^{1/k} \leq \|A\| \leq \text{tr}(A^{\boxtimes k})^{1/k}$$

Proof. For the PSD matrix $A^{\boxtimes 2}$, we have $\sqrt{\lambda_{\max}(A^{\boxtimes 2})} = \|A\|$, and $\lambda_{\max}((A^{\boxtimes 2})^{k/2}) = (\lambda_{\max}(A^{\boxtimes 2}))^{k/2}$, hence overall we have $\|A\| = \lambda_{\max}((A^{\boxtimes 2})^{k/2})^{1/k} = \lambda_{\max}(A^{\boxtimes k})^{1/k}$. On the other hand for any PSD matrix $D \in \mathbb{R}^{m \times m}$ we have $m^{-1} \text{tr}(D) \leq \lambda_{\max}(D) \leq \text{tr}(D)$; apply this to $D = A^{\boxtimes k}$ and take k^{th} roots. $\square$

As described above, the first step in our proof is to pass from a permutation ensemble $\mathcal{A}$ to another ensemble $\mathcal{B}$, so that the spectral norm of $\mathcal{B}[f]$ controls the $\infty \rightarrow 1$ norm of $\mathcal{A}[f]$. For this we use a strategy laid out in the introduction of [77], which multiplies each $A_x(i, j)$ by some global scaling factors $w_i \cdot w_j$ so as to normalize the total entry weight lying in each row and column.

Lemma 5.29. *Let $\mathcal{A} = (A_x)_{x \in [N]}$ be a permutation ensemble in $\mathbb{R}^{m \times m}$. Then there exists an ensemble $\mathcal{B} = (B_x)_{x \in [N]}$ of the same dimensions, so that*

1. *For every $f \in \{\pm 1\}^N$, we have $\frac{1}{2md(A)} \|\mathcal{A}[f]\|_{\infty \rightarrow 1} \leq \|\mathcal{B}[f]\|$*
2. *$\nu(\mathcal{B}), \chi(\mathcal{B}) \leq d(\mathcal{A})^{-1}$*
3. *$\text{tr}(B^{\boxtimes k}) \leq m$ for all even integers k , where $B = \sum_{x \in [N]} B_x$*
4. *$\|\mathcal{B}[f]\| \leq 1$ for all $f \in \{\pm 1\}^N$*

Proof. Let $d = d(\mathcal{A})$. For each $i \in [m]$ let $d_{\text{row}}(i) = \sum_{x,j} A_x(i, j)$ and let $\Gamma_i = d + d_{\text{row}}(i)$; we define $d_{\text{col}}(j), \Lambda_j$ symmetrically w.r.t. rows and columns. Define the ensemble $\mathcal{B} = (B_x)_{x \in [N]}$ by

$$B_x(i, j) = \Gamma_i^{-1/2} \Lambda_j^{-1/2} A_x(i, j)$$

Let $u, v \in [-1, 1]^m$ be such that $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} = |u^\top \mathcal{A}[f] v|$. Define

$$\hat{u} = (u(i) \cdot \Gamma_i^{1/2})_{i \in [m]}, \quad \hat{v} = (v(j) \cdot \Lambda_j^{1/2})_{j \in [m]}$$

Then we have $|\hat{u}^\top \mathcal{B}[f] \hat{v}| = |u^\top \mathcal{A}[f] v| = \|\mathcal{A}[f]\|_{\infty \rightarrow 1}$, and hence

$$\|\mathcal{B}[f]\| \geq \frac{|\hat{u}^\top \mathcal{B}[f] \hat{v}|}{\|\hat{u}\| \cdot \|\hat{v}\|} \geq \|\mathcal{A}[f]\|_{\infty \rightarrow 1} \cdot \left(\sum_{i \leq m} (d_{\text{row}}(i) + d) \right)^{-1/2} \left(\sum_{j \leq m} (d_{\text{col}}(j) + d) \right)^{-1/2} = \frac{\|\mathcal{A}[f]\|_{\infty \rightarrow 1}}{2md}$$

which gives (1). For (2), we have $\chi(\mathcal{B}) \leq \max_{i,j} |\Gamma_i^{-1/2} \Lambda_j^{-1/2}| \leq d^{-1}$ since the spectral norm of each B_x , being a reweighted subpermutation matrix, is bounded above by the magnitude of its largest entry. For $\nu(\mathcal{B})$, note that $B_x^\top B_x$ is a diagonal matrix with $(i, i)^{\text{th}}$ entry equal to $\sum_j B_x(i, j)^2$. Hence $\sum_x B_x^\top B_x$ is diagonal as well, with spectral norm equal to the magnitude of its largest entry, in particular:

$$\nu_{\text{row}}(\mathcal{B}) = \max_i \sum_{x,j} B_x(i, j)^2 = \max_i \sum_{x,j} \Gamma_i \Lambda_j A_x(i, j)^2 \quad (5.3)$$

$$\leq \max_i \Gamma_i^{-1} \sum_{x,j} \frac{A_x(i, j)^2}{d} = \frac{1}{d} \max_i \frac{d_{\text{row}}(i)}{\Gamma_i} \leq \frac{1}{d} \quad (5.4)$$

The same argument applies to columns, and we conclude $\nu(\mathcal{B}) \leq d^{-1}$. Next we verify (3). Let $2k$ be an even integer and let $B = \sum_x B_x, A = \sum_x A_x$. We argue here exactly as in the introduction of

[77]. Expanding the definition of $\text{tr}(B^{\bowtie 2k})$:

$$\begin{aligned}
\text{tr}(B^{\bowtie 2k}) &= \sum_{i_1} \sum_{j_1} B(i_1, j_1) \sum_{i_2} B(i_2, j_1) \cdots \sum_{j_k} B(i_k, j_k) \sum_{i_{k+1}} B(i_{k+1}, j_k) \mathbb{1}\{i_{k+1} = i_1\} \\
&= \sum_{i_1} \sum_{j_1} A(i_1, j_1) \Gamma_{i_1}^{-1/2} \Lambda_{j_1}^{-1/2} \sum_{i_2} A(i_2, j_1) \Gamma_{i_2}^{-1/2} \Lambda_{j_1}^{-1/2} \cdots \sum_{i_{k+1}} A(i_{k+1}, j_k) \Gamma_{i_1}^{-1/2} \Lambda_{j_k}^{-1/2} \mathbb{1}\{i_{k+1} = i_1\} \\
&= \sum_{i_1} \Gamma_{i_1}^{-1} \sum_{j_1} A(i_1, j_1) \Lambda_{j_1}^{-1} \sum_{i_2} A(i_2, j_1) \Gamma_{i_2}^{-1} \cdots \Lambda_{j_k}^{-1} \sum_{i_{k+1}} A(i_{k+1}, j_k) \mathbb{1}\{i_{k+1} = i_1\} \\
&\leq \sum_{i_1} 1 \leq m
\end{aligned}$$

where the domain of each summand i, j is $[m]$, and in the second to last inequality we are using the fact that for any i we have $\Gamma_i^{-1} \sum_j A(i, j) \leq 1$ (and symmetrically for columns, Λ_j). Finally for (4), observe that since $\text{tr}(\mathcal{B}[f]^{\bowtie 2k})$ is a polynomial in the variables $\{f_x \mid x \in [N]\}$ with nonnegative coefficients, (3) implies that $\text{tr}(\mathcal{B}[f]^{\bowtie 2k}) \leq m$ for all f and all k . Applying the Trace/Norm inequality (Lemma 5.28) we have that $\|\mathcal{B}[f]\|^{2k} \leq m$ for arbitrarily large k , hence $\|\mathcal{B}[f]\| \leq 1$. $\square$

We now wish to argue that, for the kind of matrix ensemble $\mathcal{B}$ obtained in the last lemma, the upper tail of $\|\mathcal{B}[\hat{f}]\|$ is controlled by that of $\|\mathcal{B}[f]\|$, where $\hat{f}$ and f are k -wise independent and truly uniform vectors respectively. This follows directly from the trace norm inequality and the fact that $\text{tr}(\mathcal{B}[\cdot]^{\bowtie k})$ is a degree k polynomial. If we assume only that $\hat{f}$ is γ -almost k -wise independent, for $\gamma > 0$, some error term involving γ and the total monomial weight of the associated polynomial will be introduced. In the end we get:

Lemma 5.30. *Let $\mathcal{B} = (B_x)_{x \in [N]}$ be a matrix ensemble in $\mathbb{R}^{m \times m}$ such that $\|\mathcal{B}[f]\| \leq 1$ for all $f \in \{\pm 1\}^N$ and let $k > 3$ be an even integer. Say that $f, \hat{f} \in \{\pm 1\}^N$ are random variables, with f uniform on $\{\pm 1\}^N$ and $\hat{f}$ sampled from a γ -almost k -wise independent distribution. Then for any $\delta > 0$ with $\gamma \cdot \text{tr}((\sum_x B_x)^{\bowtie k}) \leq 2^{-2k-1} \delta^k$ we have:*

$$\Pr[\|\mathcal{B}[\hat{f}]\| \geq \delta] \geq 2^{-k} \longrightarrow \Pr[\|\mathcal{B}[f]\| \geq \delta(8m)^{-1/k}] \geq 2^{-k-2} \delta^k m^{-1}$$

Proof. Set $\epsilon := 2^{-k}$, $r := \text{tr}((\sum_x B_x)^{\text{pa}k})$. Say that $\Pr[\|\mathcal{B}[\hat{f}]\| \geq \delta] \geq \epsilon$, hence $\Pr[\|\mathcal{B}[\hat{f}]\|^k \geq \delta^k] \geq \epsilon$ and therefore $\mathbb{E} \|\mathcal{B}[\hat{f}]\|^k \geq \epsilon \delta^k$. By the trace/norm inequality, we know that for all f , $\text{tr}(\mathcal{B}[f]^{\text{pa}k}) \geq \|\mathcal{B}[f]\|^k$, hence we have $\mathbb{E} \text{tr}(\mathcal{B}[\hat{f}]^{\text{pa}k}) \geq \epsilon \delta^k$. Since $\text{tr}(\mathcal{B}[\cdot]^{\text{pa}k})$ is a degree k polynomial with total monomial weight $\leq r$, we have that $|\mathbb{E} \text{tr}(\mathcal{B}[\hat{f}]^{\text{pa}k}) - \mathbb{E} \text{tr}(\mathcal{B}[f]^{\text{pa}k})| \leq \gamma r 2^k$, hence $\mathbb{E} \text{tr}(\mathcal{B}[f]^{\text{pa}k}) \geq \epsilon \delta^k - \gamma r 2^k \geq \frac{1}{2} \epsilon \delta^k$ and therefore, applying the other side of the trace/norm inequality, $\mathbb{E} \|\mathcal{B}[f]\|^k \geq \frac{\epsilon \delta^k}{2m}$. Thus $\Pr[\|\mathcal{B}[f]\|^k \geq \frac{\epsilon \delta^k}{4m}] \geq \frac{\epsilon \delta^k}{4m}$ (we use here that $\|\mathcal{B}[f]\| \leq 1$ probability 1), and so $\Pr[\|\mathcal{B}[f]\| > \delta(\frac{\epsilon}{4m})^{1/k}] \geq \frac{\epsilon \delta^k}{4m}$. Recall that $\epsilon = 2^{-k}$, so $\delta(\frac{\epsilon}{4m})^{1/k} = \delta(8m)^{-1/k}$. $\square$

It then remains only to argue that, for the ensemble $\mathcal{B}$ arising in our argument, $\|\mathcal{B}[f]\|$ has a suitably decaying upper tail when f is a uniformly random signing. For this we rely on the Matrix Bernstein inequality.

Theorem 5.31 (Matrix Bernstein [159]). *Let $\mathcal{B} = (B_x)_{x \in X}$ be an ensemble in $\mathbb{R}^{m \times m}$ and $f \in \{\pm 1\}^X$ a uniformly random sign vector. For any $\alpha > 0$ we have*

$$\Pr[\|\mathcal{B}[f]\| \geq \alpha] \leq 2m \exp\left(-\frac{1}{6} \cdot \frac{\alpha^2}{\nu(\mathcal{B}) + \alpha \chi(\mathcal{B})}\right)$$

We are now ready to prove Theorem 5.25:

Proof of Theorem 5.25. Let $\mathcal{A} = (A_x)_{x \in [N]}$ be a permutation ensemble in $\mathbb{R}^{m \times m}$ with average degree $d = d(\mathcal{A})$, let $\delta > 0$ be sufficiently small. Let $f, \hat{f} \in \{\pm 1\}^X$ be random vectors, with f uniform and $\hat{f}$ γ -almost k -wise independent for some even k , and assume $\gamma \leq 2^{-2k-1} \delta^k m^{-1}$. Assume that $\Pr[\|\mathcal{A}[\hat{f}]\|_{\infty \rightarrow 1} \geq \delta m d] > 2^{-k}$; we then wish to establish that $d \leq C \frac{1}{\delta^2} \log(\frac{1}{\delta}) m^{2/k} \log m$ for a suitably large constant C . Applying Lemma 5.29, we construct from $\mathcal{A}$ an ensemble $\mathcal{B}$, so that $\nu(\mathcal{B}), \chi(\mathcal{B}) \leq d^{-1}$, $\text{tr}((\sum_x B_x)^{\text{pa}k}) \leq m$, $\|\mathcal{B}[f]\| \leq 1$ for all f , and $\|\mathcal{B}[f]\| \geq (2md)^{-1} \|\mathcal{A}[f]\|_{\infty \rightarrow 1}$ for all f . In particular this implies

$$\Pr[\|\mathcal{B}[\hat{f}]\| \geq \frac{\delta}{2}] \geq \Pr[\|\mathcal{A}[\hat{f}]\|_{\infty \rightarrow 1} \geq \delta m d] > 2^{-k}$$

We then apply Lemma 5.30 and conclude that:

$$\Pr[\|\mathcal{B}[f]\| \geq \frac{\delta}{2}(8m)^{-1/k}] \geq 2^{-2k-2}\delta^k m^{-1}$$

Now setting $\alpha := \frac{\delta}{2}(8m)^{-1/k}$, from the Bernstein inequality we know that

$$\Pr[\|\mathcal{B}[f]\| \geq \alpha] \leq 2m \exp\left(-\frac{1}{6} \cdot \frac{\alpha^2}{\frac{1}{d} + \frac{\alpha}{d}}\right) \leq 2m \exp\left(-\frac{\alpha^2 d}{12}\right) \leq 2^{\log m - \frac{\alpha^2 d}{12}}$$

We then have $2^{\log m - \frac{\alpha^2 d}{12}} \geq 2^{-2k-1}\delta^k m^{-1}$, which implies $d \leq C \frac{1}{\delta^2} \log \frac{1}{\delta} m^{2/k} (k + \log m)$ for C a suitable absolute constant and δ sufficiently small. $\square$

5.3.4 Lower Bounds for Concrete Problems

We may apply the lower bound in Theorem 5.1 to any problem $F : [N] \times [M] \rightarrow \{0, 1\}$ exhibiting suitable limited independence properties. If our goal is simply to minimize M as a function of N , subject to proving the best quantitative bound and subject to F being *explicit*, our best course of action is to use any explicit construction of γ -almost k -wise independent distributions with minimum possible support size (i.e. minimum seed length), for example [128] or any of its subsequent improvements. However, it is also of interest to prove lower bounds for *natural* problems which have some more concrete interpretation as an information retrieval task. We highlight here two such natural problems to which we can apply the lower bound in Theorem 5.1. Both pertain to evaluating low degree polynomials of some kind over a finite field.

5.3.4.1 Evaluating Univariate Polynomials over a large field $\mathbb{F}$

The first problem we discuss is standard in data structure complexity (e.g. [109, 61]): the evaluation of low degree univariate polynomials over a large finite field. The k -wise independence properties of this problem are well known and this lower bound does not require much elaboration. We define the polynomial evaluation problem formally below.

Definition 5.32. For any finite field $\mathbb{F}$ and $d \in \mathbb{N}$ we define the data structure problem $\mathbb{F}\text{-Eval}^d$. Datapoints are univariate polynomials p of degree $\leq d$ over $\mathbb{F}$; queries are elements $x \in \mathbb{F}$; the correct answer is the evaluation of p on x , i.e. $\mathbb{F}\text{-Eval}^d(x, p) = p(x)$. For any function $\chi : \mathbb{F} \rightarrow \{0, 1\}$, define the restricted problem $\chi\text{-}\mathbb{F}\text{-Eval}^d$, which has the same datapoints and queries, but the required answer is only the bit $\chi(p(x))$.

Note that the problem $\mathbb{F}\text{-Eval}^d$ does not have a boolean output, however it is clearly at least as hard as the boolean-valued problem $\chi\text{-}\mathbb{F}\text{-Eval}^d$ for any χ . A typical choice of χ is as follows: say that $\mathbb{F} = \mathbb{F}_{2^n}$ is identified with $\{0, 1\}^n$ under some natural encoding (e.g. $x \in \mathbb{F}$ is interpreted as a degree $\leq n - 1$ univariate polynomial over $\mathbb{F}_2$ modulo some fixed irreducible and is coded by its vector of coefficients), and set $\chi(x)$ to be the first bit of x 's encoding. We then have, as an immediate consequence of Theorem 5.1 and the well known d -wise independent properties of degree d polynomials:

Theorem 5.33. Let $\mathbb{F}$ be a finite field of characteristic 2 and let $\chi : \mathbb{F} \rightarrow \{0, 1\}$ be any function which is balanced, i.e. $|\chi^{-1}(0)| = |\chi^{-1}(1)|$. Let $d, S, t, w \in \mathbb{N}$ be given with t an even number and assume $d > tw + 1$. Then, for any space S , time t , word length w cell probe data structure solving $\chi\text{-}\mathbb{F}\text{-Eval}^d$ we must have:

1. $S \geq \left(\frac{|\mathbb{F}|}{\log |\mathbb{F}|}\right)^{\frac{2}{t}} \cdot \left(\frac{d}{\log |\mathbb{F}|}\right)^{1-\frac{2}{t}} \cdot t^{-1} 2^{-(6-o(1))w}$ for all $t \log |\mathbb{F}| \leq d \leq |\mathbb{F}| 2^{-(3-o(1))tw} t^{-\frac{t}{2}}$
2. $S \geq \left(\frac{|\mathbb{F}|}{\log |\mathbb{F}|}\right)^{\frac{1}{t(\frac{1}{2}+\frac{1}{k})}} \cdot t^{-1} 2^{-(6-o(1))w}$ for all $t \leq d \leq t \log |\mathbb{F}|$,

5.3.4.2 Evaluating Multivariate Polynomials over $\mathbb{F}_2$

The second and perhaps less known problem we highlight pertains to evaluating low degree multivariate polynomials $p : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$, defined formally below:

Definition 5.34. For any $d \leq n$ we define the data structure problem $\mathbb{F}_2\text{-Eval}_n^d$. Datapoints are multilinear polynomials $p : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ of degree $\leq d$; queries are elements $x \in \mathbb{F}_2^n$; the correct answer is the evaluation of p on x , i.e. $\mathbb{F}_2\text{-Eval}_n^d(x, p) = p(x)$.

We use the following result of [141], improved quantitatively by [151]:

Lemma 5.35 ([141, 151]). *Let $k \leq n$. There exists a random function variable $f : \{0, 1\}^n \rightarrow \{0, 1\}$ which is k -wise independent, so that with probability 1 f is computed by a degree $\leq \log k + 1$ polynomial $p : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ (under the natural correspondence $\mathbb{F}_2 \leftrightarrow \{0, 1\}$).*

The above statement does not occur explicitly in [141, 151]; rather, [151] shows that for every $k, \gamma > 0$, there exists a random function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ which is γ -almost k -wise independent and computed (with probability 1) by a formula of the form:

$$\phi(x) = \bigoplus_{i \leq m} \bigwedge_{j \leq \log k + 1} \bigoplus_{u \leq 2n+2} \ell_{i,j,u} \quad (5.5)$$

for $m = 2 \log(\frac{1}{\gamma}) \log k$, where each $\ell_{i,j,u}$ is a literal in $\{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n, 0, 1\}$. Clearly such a formula is representable by a polynomial of degree at most $\log k + 1$ regardless of the value of γ ; taking γ suitably small (below the granularity of the implied distribution, which is bounded away from zero by a function of only n, k) we obtain the desired result. Combining Lemma 5.35 and Theorem 5.1 we arrive at the following cell probe lower bound for $\mathbb{F}_2\text{-Eval}_n^d$:

Theorem 5.36. *Let n, d, t, w be given, and assume that d lies in the range:*

$$\log n + \log t + \log w + O(1) \leq d \leq n - O(wt \log t)$$

Then the problem $\mathbb{F}_2\text{-Eval}_n^d$ satisfies the space/time/word length tradeoff

$$S \geq \left(2^{n-\log n}\right)^{\frac{2}{t}} \cdot \left(2^{d-\log n}\right)^{1-\frac{2}{t}} \cdot t^{-1} 2^{-(6-o(1))w}$$

We make some observations about this lower bound. Consider the setting $w, t = O(1)$ and $d = \log n + O(1)$ and let F be the data structure problem $F = \mathbb{F}_2\text{-Eval}_n^d$. Then F is of the form $F : [N] \times [M] \rightarrow \{0, 1\}$ for $N = 2^n$, $M = 2^{n \log n + O(1)}$, i.e. $M = 2^{2^{O(\log \log N)^2}}$ which is close to polynomial in N (we may say that it is quasi-quasi-polynomial). This problem F satisfies the (new

as of this paper) strongest known cell probe lower bound for any problem with this relation between M, N : we require space $S \geq \tilde{\Omega}(N^{\frac{2}{t}})$ for time t , word length w data structures. On the other hand, we can observe that F has rank at most $2^{O(\log \log N)^2}$ over $\mathbb{F}_2$, which is very close to *logarithmic* in the number of queries N ; this is because the columns of F are spanned by the $n^{\log n + O(1)}$ monomials of degree $\leq d$. In the language of communication complexity, this places F in the communication class $\oplus P^{\text{cc}}$. This is of some interest in relation to our later discussions in Section 5.6.3, where we relate cell probe complexity to other “high-end” communication complexity classes such as PH^{cc} and $(\text{PH}[\oplus_p])^{\text{cc}}$.

Second, we note that we could have defined directly an “evaluation problem” for depth 3 $\text{AC}^0[\oplus]$ formula of the form occurring in (5.5) above. For an appropriate setting of γ and using our bounds for almost k -wise independence we would obtain a problem $F : \{0, 1\}^n \times \{0, 1\}^{\text{poly}(n)} \rightarrow \{0, 1\}$ satisfying the new state of the art time space tradeoff, which in addition has the property that $F(x, y)$ is computable by a $\text{poly}(n)$ -size $\text{AC}^0[\oplus]$ formula.

5.4 Beating the Limited Independence Bound

As discussed in the introduction, the lower bounds in the previous section are essentially tight for k -wise independent data structure problems, even in the special case of nonadaptive bit probe data structures and even when t is odd⁶. In particular for any constant $t > 1$, if we wish to obtain a data structure problem $F : [N] \times [M] \rightarrow \{\pm 1\}$ with $M \leq \exp(N^{o(1)})$ and satisfying a space/time tradeoff $S > N^{\frac{2}{t} + \epsilon}$ for some absolute constant $\epsilon > 0$ (again, even for nonadaptive bit probe data structures), we *provable cannot* rely solely on k -wise independence as our hardness property for the problem F . In this section we achieve lower bounds of this kind for every $t \in \mathbb{N}$ which is an odd number using a method from t -CSP refutation developed by [48] in the case $t = 3$ and generalized to all t by [64]. To establish the lower bounds here, we will have to pass from k -wise independent distributions to small-bias probability spaces.

For starters, we will need a variant of Theorem 5.20 which is specialized to nonadaptive gener-

⁶We cannot really say “tight” in the case t is odd since we have not proven the lower bound in this case; what we mean is that there is an upper bound matching the form of Theorem 5.1 even for odd values of t .

ators with word length 1; remember from Section 5.2.2 that such generators, with time complexity t , are referred to as NC_t^0 generators. The proof of this theorem more or less follows arguments in Section 9 of [64], adapted to the setting of NC_t^0 generators (in fact the argument becomes simpler in this setting):

Theorem 5.37. *Let $G : \{\pm 1\}^S \rightarrow \{\pm 1\}^N$ be an NC_t^0 generator. Then there is $X \subseteq [N]$, $|X| \geq 2^{-2^t} N$, and a collection of 2^t different XOR schemes indexed by X , $(C_u)_{u \subseteq [t]}$ so that:*

1. *Each XOR scheme has $\leq tS$ variables, and C_u is a $|u|$ -XOR scheme.*

2. *For every $f \in \text{range}(G)$ we have $\max_{u \subseteq [t], |u| < t} \text{Val}(C_u, f) \geq (1 - \text{Val}(C_{[t]}, f))2^{-t}$*

Proof. For each $x \in [N]$, $G_x : \{\pm 1\}^S \rightarrow \{\pm 1\}$ depends on t input cells which we list in some fixed order $p_x^1, \dots, p_x^t$. For each x there is a predicate $h_x : \{\pm 1\}^t \rightarrow \{\pm 1\}$ so that if, on input $E \in \{\pm 1\}^S$, $G_x(E)$ sees the query outcomes $\xi_1 = E(p_x^1), \dots, \xi_t = E(p_x^t)$, then $G_x(E) = h_x(\xi)$. Hence there exists $h : \{\pm 1\}^t \rightarrow \{\pm 1\}$ so that, for the set $X = \{x \in [N] \mid h_x = h\}$ we have $|X| \geq 2^{-2^t} N$. Fix this choice X, h for the remainder of the proof.

Let $V^1, \dots, V^t$ be copies of $[S]$. For each $u \subseteq [t]$, we define the XOR scheme $C_u = (X, V^u, c^u)$ indexed by X with variable set $V^u = \cup_{j \in u} V^j$ and constraints $c_x^u = \{p_x^j \mid j \in u\}$ where p_x^j is considered to live inside V^j ; note that this is well-defined even for $u = \emptyset$, in which case we get a 0-XOR scheme $C_\emptyset$ over 0 variables with $\text{OPT}(C_\emptyset, f) = \mathbb{E}_{x \in X} f_x$. The function h may be expressed uniquely as $h(\xi) = \sum_{u \subseteq [t]} \hat{h}(u) \cdot \prod_{j \in u} \xi_j$, where $(\hat{h}(u))_{u \subseteq [t]}$ are its Fourier coefficients. Now let $f \in \text{range}(G)$ with encoding E_f , we have:

$$\begin{aligned} 1 &= \mathbb{E}_{x \in X} f_x \cdot h(E_f(p_1(x)), \dots, E_f(p_t(x))) = \mathbb{E}_{x \in X} f_x \sum_{u \subseteq [t]} \hat{h}(u) \prod_{j \in u} E_f(p_j(x)) \\ &\leq \sum_{u \subseteq [t]} |\hat{h}(u)| \cdot \left| \mathbb{E}_{x \in X} f_x \prod_{j \in u} E_f(p_j(x)) \right| = \sum_{u \subseteq [t]} |\hat{h}(u)| \cdot \left| \mathbb{E}_{x \in X} f_x \prod_{i \in c_x^u} E_f^u(i) \right| \\ &\leq \sum_{u \subseteq [t]} |\hat{h}(u)| \cdot \text{Val}(C_u, f) \leq \sum_{u \subseteq [t]} \text{Val}(C_u, f) \end{aligned}$$

where, in the second to last line, we use E_f^u to denote the vector in $\{0, 1\}^{V_u}$ obtained from E_f by setting $E_f^u(i) = E_f(i)$ for every $i \in V^j \subseteq V_u$ ($E_f^\emptyset$ is the empty vector $\emptyset \mapsto \{0, 1\}$).

□

Using Theorem 5.37, we obtain from any NC_t^0 generator a sequence of r -XOR schemes with various orders $r \leq t$. We will deal with the schemes $(C_u)_{|u| < t}$ of order $< t$ using limited independence as in Section 5.3.2; for these we can hope to obtain upper bounds on $\text{Val}(C_u, f)$ which approach 0 at a reasonably fast rate, provided $S \ll N^{\frac{2}{(t-1)}}$. For the highest order scheme $C_{[t]}$, we will only be able to obtain an upper bound on $\text{Val}(C_{[t]}, f)$ that is somewhat bounded away from one; however, if these two upper bounds match up correctly, we still have room to complete the lower bound according to Theorem 5.37.

The highest order term is dealt with using a technique developed in [48, 64]. The main technical ingredient is the following theorem:

Theorem 5.38 ([64]). *Let $t > 1$, $t = O(1)$ be fixed. Let $e : [N] \rightarrow \binom{[K]}{t}$ be a t -uniform multi-hypergraph with N edges and K vertices, and let $L \in \{t, \dots, \frac{K}{8}\}$, such that $N \geq C \cdot K \left(\frac{K}{L}\right)^{\frac{t}{2}-1} \log^{4t+1} K$ for some constant C (depending on t). Then there exist a collection of disjoint nonempty sets $E_1, \dots, E_r \subseteq [N]$, $r \geq \Omega(L^{-1} \left(\frac{K}{L}\right)^{\frac{t}{2}} \log^{4t} K)$, so that for every $j \leq r$ we have $\bigtriangleup_{x \in E_j} e_x = \emptyset$, where $\bigtriangleup$ denotes the symmetric difference operator.*

In [48] a somewhat similar bound is obtained in the special case $t = 3$. We now use Theorem 5.38 to give a nontrivial upper bound on $\text{OPT}(C, f)$ when f has small bias:

Theorem 5.39. *Fix $t > 1$, $t = O(1)$. Let $C = ([N], V, c)$ be a t -XOR scheme, $|V| = K$, and L be any integer with $t \leq L \leq \frac{K}{8}$. Assume that $N \geq C \cdot K \left(\frac{K}{L}\right)^{\frac{t}{2}-1} \log^{4t+1} K$ for a constant C and that K is sufficiently large. Let $f \in \{\pm 1\}^N$ be a random vector which is $\frac{1}{4}$ -biased. Then*

$$\Pr[\text{OPT}(C, f) \leq 1 - \Omega\left(\frac{K^{\frac{t}{2}} \log^{4t} K}{NL^{\frac{t}{2}+1}}\right)] \geq \frac{1}{5}$$

Proof. For $R \in \{\pm 1\}^V$, let $\text{Sat}(C, f, R) = \Pr_{x \sim [N]} [f_x = \prod_{i \in c(x)} R(i)]$, $\text{Sat}(C, f) = \max_R \text{Sat}(C, f, R)$. Then we have $\text{Val}(C, f, R) = |\text{Sat}(C, f, R) - \text{Sat}(C, -f, R)|$ for every R . We will show that

$\Pr[\text{Sat}(C, f) > 1 - \delta] \leq \frac{2}{5}$ for $\delta = \epsilon \cdot \frac{K^{\frac{t}{2}} \log^{4t} K}{10NL^{\frac{t}{2}+1}}$ for a sufficiently small constant $\epsilon > 0$ depending on t . Since the property of being γ -biased is invariant under negation, the same bound will hold for $-f$ and so the theorem will follow by a union bound. By Theorem 5.38, there exists a collection of disjoint nonempty sets $E_1, \dots, E_r \subseteq [N]$, $r \geq \epsilon \cdot L^{-1}(\frac{K}{L})^{\frac{t}{2}} \log^{4t} K$, such that for each $j \leq r$ we have $\Delta_{x \in E_j} c_x = \emptyset$ where Δ denotes the symmetric difference operator. We now observe, as in [48, 64], the following: for any f , if $\prod_{x \in E_j} f_x = -1$, then for every $R \in \{\pm 1\}^V$, there exists $x \in E_j$ so that $f_x \neq \prod_{i \in c_x} R(i)$. This follows since

$$\prod_{x \in E_j} \prod_{i \in c_x} R(i) = \prod_{i \in (\Delta_{x \in E_j} c_x)} R(i) = \prod_{i \in \emptyset} R(i) = 1$$

Hence, if we define $v(f) = \sum_{j \leq r} \mathbb{1}\{\prod_{x \in E_j} f_x = -1\}$, then for every f we have $\text{Sat}(C, f) \leq 1 - \frac{v(f)}{N}$. We will show that with probability at least $\frac{2}{5}$, we have $v(f) \geq \frac{r}{10}$, in which case we have $\text{Sat}(C, f) \leq 1 - \frac{r}{10N}$ and the theorem is proven. Now let $\gamma = \frac{1}{4}$ and define the random vector $\mathbf{g} \in \{\pm 1\}^r$, $\mathbf{g}_j = \prod_{x \in E_j} f_x$. Observe that $\mathbf{g}$ is γ -biased inside $\{\pm 1\}^r$: for any $\emptyset \neq u \subseteq [r]$ we have

$$\mathbb{E} \prod_{j \in u} \mathbf{g}_j = \mathbb{E} \prod_{j \in u} \prod_{x \in E_j} f_x = \mathbb{E} \prod_{x \in \cup_{j \in u} E_j} f_x \in [-\gamma, \gamma]$$

using in the last step that f is γ -biased. So it suffices to show that, for γ -biased $\mathbf{g} \in \{\pm 1\}^r$, we have $|\sum_{j \in r} \mathbf{g}_j| \leq \frac{8r}{10}$ with high probability. This is standard and can be verified by inspecting the second moment:

$$\Pr[|\sum_j \mathbf{g}_j| > \frac{8r}{10}] \leq \left(\frac{8r}{10}\right)^{-2} \mathbb{E}(\sum_j \mathbf{g}_j)^2 = \left(\frac{8r}{10}\right)^{-2} \sum_{j, j'} \mathbb{E} \mathbf{g}_j \mathbf{g}_{j'} \leq \frac{100}{64r^2} (\gamma r^2 + r) = \frac{100}{256} + 9r^{-1} \leq \frac{2}{5}$$

Where the final inequality follows provided r is sufficiently large, which in turn follows provided K is sufficiently large. $\square$

We also need the following easy fact:

Observation 5.40. *Let $C = ([N], V, c)$ be a t -XOR scheme. Then for any $t' \geq t$ there is a t' -XOR*

scheme $C' = ([N], V', c')$ with $|V'| \leq |V| + (t' - t)$ so that $\text{OPT}(C, f) \leq \text{OPT}(C', f)$ for every $f \in \{\pm 1\}^N$.

Proof. Let $r = t' - t$ and set $V' = V \cup \{1, \dots, r\}$, $c'_x = c_x \cup \{1, \dots, r\}$. $\square$

Observe that the above argument is valid even in the degenerate case $t = 0$. Finally we obtain:

Theorem (Theorem 5.2, restated). *Let $t > 2$, $t = O(1)$ be a fixed odd number. Let $F : [N] \times [M] \rightarrow \{\pm 1\}$, be a data structure problem whose columns support a distribution which is γ -almost $\log N$ -wise independent and $\frac{1}{4}$ -biased, for $\frac{1}{\gamma} \geq N^{O(1)}$. Then any nonadaptive bit probe data structure for F with space S and time t must satisfy:*

$$S \geq \tilde{\Omega}(N^{\frac{1}{\frac{t}{2} - \frac{t-2}{2(t+2)}}})$$

Observe that for every $t > 2$, $\frac{t-2}{2(t+2)} > 0$, hence we obtain lower bounds of the form $S \geq N^{\frac{t}{2} + \Omega(1)}$ for every odd t . Recall also Lemma 5.14 from Section 5.2.1 in the preliminaries, which says that if a distribution is γ -biased then it is also γ -almost k -wise independent for any k . So we may strengthen our assumption on F to the simpler statement that its columns support an $N^{-O(1)}$ -biased distribution. Our proof here will follow very closely the arguments in Section 9 of [64].

Proof. Assume F has a data structure of space S and time $t = O(1)$, and let f be the random vector supported on the columns of F which is $\frac{1}{4}$ -biased and N^{-c} -almost $\log N$ -wise independence for c a suitably large constant. By Theorem 5.37 we may find $X \subseteq [N]$, $|X| \geq 2^{-2^t} N$, and XOR schemes $(C_u)_{u \subseteq [t]}$ all indexed by X , each having $O(S)$ variables, so that C_u is a $|u|$ -XOR scheme, and $\max_{u \subseteq [t], |u| < t} \text{Val}(C_u, f) \geq (1 - \text{Val}(C_{[t]}, f))2^{-t}$ for every column f of F . Using Observation 5.40 we may assume, up to increasing the number of variables of each XOR scheme by at most t , that C_u is a $(t-1)$ -XOR scheme for every $|u| < t$. Set $L = S^{\frac{1}{t+2}}$, and assume that $N = \tilde{O}(1) \cdot \left(\frac{S}{L}\right)^{\frac{t}{2}} L$ where $\tilde{O}(1)$ is some sufficiently large poly $\log N$ term. Applying Theorem 5.39, with probability at least $\frac{1}{5}$, we have

$$1 - \text{OPT}(C_{[t]}, f) \geq \tilde{O}(1) \cdot \frac{S^{t/2}}{\left(\frac{S}{L}\right)^{\frac{t}{2}} L} = \tilde{O}(1) \cdot L^{-1}$$

On the other hand, using the fact that f is γ -almost $\log N$ -wise independent and supported on the columns of F , with $\gamma \leq 2^{-O(tk)} = N^{-O(1)}$, we conclude using Theorem 5.26 (and the fact that $t - 1$ is even) that for every $|u| < t$ we have:

$$\Pr[\text{OPT}(C_u, f) \geq \delta] \leq N^{-1}, \quad \text{provided } N \geq \omega\left(\frac{1}{\delta^2} \log \frac{1}{\delta} S^{\frac{t-1}{2}} \log N\right)$$

Plugging in $\delta = \tilde{O}(1) \cdot 2^{-t} \cdot L^{-1}$, and using the fact $N = \omega(L^2 S^{\frac{t-1}{2}} \log N)$ as per our setting of L , there is a positive probability that

$$1 - \text{OPT}(C, f) \geq \tilde{O}(1) \cdot L^{-1} > 2^{-t} \cdot \max_{0 < |u| < t} \text{Val}(C_u, f)$$

and we reach a contradiction. Hence we cannot have $N \geq \tilde{O}(1) \cdot \left(\frac{S}{L}\right)^{\frac{t}{2}} L$, which by our setting of L yields the lower bound

$$S \geq \tilde{\Omega}\left(N^{\frac{1}{\frac{t}{2} - \frac{t-2}{2(t+2)}}}\right)$$

□

The only reason we needed to assume t is odd is in our application of Theorem 5.26; if this theorem could be generalized to odd-arity XOR schemes (which seems attainable using methods for odd-arity CSPs, e.g. [64]), the results here would be valid for all $t > 2$. **TODO: mention subsequent work that solved this**

5.5 Algorithmic Implications of Our Lower Bounds

5.5.1 Range Avoidance

As referenced in the introduction, our main correspondence (Observation 5.17) immediately implies that explicit cell probe lower bounds yield NP oracle algorithms for Range Avoidance. We give a more formal version of this statement here:

Lemma 5.41. *Let $G : \Sigma^S \rightarrow \{0, 1\}^N$ be a locally computable generator (resp. nonadaptive) with*

word length $w = \lceil \log |\Sigma| \rceil$ and time complexity t . There is a polynomial time NP oracle algorithm which, given G and a data structure problem $F \in \{0, 1\}^{N \times M}$ which does not have space S , time t , word length w (resp. nonadaptive) data structures, solves the Range Avoidance problem for G .

Proof. By Observation 5.17, the data structure lower bound for F implies that one of its columns must lie outside the range of G . We may use a NP oracle to test each column in order and find the lexicographically first which lies outside the range of G . $\square$

Hence, our new explicit cell probe lower bounds immediately imply new polynomial time NP-oracle algorithms for Range Avoidance in a completely black box sense. For one of our lower bounds (Theorem 5.1), we will be able to analyze the argument in a non black box way and remove the NP oracle. For Theorem 5.2 we are unable to make such an improvement. We start with this latter result, where using known constructions of small bias probability spaces ([128]) we get:

Theorem (Theorem 5.3, restated). *Let $t = O(1)$ be a fixed odd number. There is a polynomial time algorithm which outputs a list of strings in $\{0, 1\}^m$, such that for any NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with*

$$m \geq \tilde{O}(n^{\frac{t}{2} - \frac{t-2}{2(t+2)}})$$

one of the strings in the list must be a range-avoidance solution for G . In particular, there is a polynomial time NP-oracle algorithm for Range Avoidance on instances of the above form.

We now move on to our first lower bound from Theorem 5.1, which applies to more general adaptive generators with larger word lengths. Compared to Theorem 5.3, we will be able to remove the use of an NP oracle, and we will also be able to solve even the *remote point* problem for G , finding a string which is $(\frac{1}{2} - \epsilon)$ -far in relative hamming distance from every string in the range of G for an arbitrarily small constant ϵ .

Theorem (Theorem 5.4, restated). *Let t, w, n be given with $\log n > tw + 1$ and t even, and let $\epsilon \in (0, \frac{1}{2}]$ be given. There is a $m^{O(t^2 w)}$ -time algorithm which, given any time- t locally-computable*

generator $G : [2^w]^n \rightarrow \{0, 1\}^m$ with

$$m \geq n^{\frac{t}{2}} \log n \cdot t^{\frac{t}{2}} 2^{(3+o(1))tw} \cdot \text{poly}\left(\frac{1}{\epsilon}\right)$$

outputs a string in $\{0, 1\}^m$ which is $\geq (\frac{1}{2} - \epsilon)$ -far in relative hamming distance from every string in the range of G .

We will set $N = m$, $S = n$ in the following, where m, n are as above. To obtain the hamming distance bound we require a strengthening of Theorem 5.20. Recall that Theorem 5.20 showed how to construct from a generator $G : \Sigma^S \rightarrow \{0, 1\}^N$ a small collection of XOR schemes indexed by large subsets of $[N]$, so that whenever $f \in \text{range}(G)$, f will induce a large value in one of the CSPs. The following strengthening will give us a similar conclusion under the weaker assumption that f is close in hamming distance to $\text{range}(G)$. In the following, for two strings $f, g \in \{\pm 1\}^N$ and $X \subseteq [N]$, we use $\text{Cor}_X(f, g) = |\mathbb{E}_{x \sim X} f_x g_x|$; by default $\text{Cor}(\cdot, \cdot) = \text{Cor}_{[N]}(\cdot, \cdot)$.

Theorem 5.42 (Correlation Variant of Theorem 5.20). *Let $G : \Sigma^S \rightarrow \{\pm 1\}^N$ be a locally-computable generator with time complexity t , word length $w = \log |\Sigma|$. There exists a collection of sets $(X_j \subseteq [N])_{j \leq 2^{tw+1}}$, $|X_j| \geq \epsilon 2^{-tw-1} N$, and a collection of t -XOR schemes $(C_j)_{j \leq 2^{tw+1}}$, with C_j a t -XOR scheme indexed by X_j and having $\leq tS$ variables, so that for every $f, g \in \{\pm 1\}^N$ with $g \in \text{range}(G)$, $\text{Cor}(f, g) \geq \epsilon$, there exists $j \leq 2^{tw+1}$ with $\text{Val}(C_j, f) \geq \epsilon 2^{-tw-1}$.*

Proof of Theorem 5.42. We follow the proof of Theorem 5.20 and indicate only those parts which need to change. The definitions of $C^{(\pi, b)}$, $X^{(\pi, b)}$ remain as before; in this case we discard any such pair with $|X^{(\pi, b)}| < \epsilon 2^{-tw-1} N$. Let f, g be as in the statement of the theorem, and let E_g be a canonical preimage of g under G . For each π let $\tilde{X}^{(\pi, g)} \subseteq [N]$ consist of those x such that $T_x(E_g)$ reaches the leaf corresponding to π . These sets (ranging over $\pi \in \Sigma^t$) partition $[N]$ and so we have

$$\epsilon \leq \text{Cor}(f, g) = \left| \sum_{\pi \in \Sigma^t} \frac{|\tilde{X}^{(\pi, g)}|}{N} \mathbb{E}_{x \sim \tilde{X}^{(\pi, g)}} f_x g_x \right| \leq \sum_{\pi \in \Sigma^t} \frac{|\tilde{X}^{(\pi, g)}|}{N} \text{Cor}_{\tilde{X}^{(\pi, g)}}(f, g)$$

There must then exist $\pi^{f, g}$ so that $\frac{|\tilde{X}^{(\pi^{f, g}, g)}|}{N} \text{Cor}_{\tilde{X}^{(\pi^{f, g}, g)}}(f, g) \geq \epsilon 2^{-tw}$, in particular $|\tilde{X}^{(\pi^{f, g}, g)}| \geq$

$\epsilon 2^{-tw} N$ and $\text{Cor}_{\tilde{X}(\pi^{f,g})}(f, g) \geq \epsilon 2^{-tw}$. We take $\pi^{f,g}$ in place of “ π^f ” from the previous proof and $\tilde{X}(\pi^{f,g})$ in place of “ $\tilde{X}(\pi^f, f)$,” and otherwise carry out essentially the same argument. $\square$

Using this in conjunction with the main ideas in Section 5.3.2 we can now prove Theorem 5.4.

Proof of Theorem 5.4. Given the generator G , we may apply Theorem 5.42 to compute a list of 2^{tw+1} XOR schemes, so that whenever a string in $\{0, 1\}^m \leftrightarrow \{\pm 1\}^m$ makes the value of all XOR schemes $< \epsilon 2^{-tw-1}$, it must be $(\frac{1}{2} - \epsilon)$ -far in relative hamming distance from $\text{range}(G)$. We will use [128] to construct a polynomial size list of candidate strings, such that one of them must make all such values small simultaneously which will guarantee that it is far from $\text{range}(G)$. To make our algorithm run in polynomial time, we need to certify that one of these strings from the list accomplishes this goal. To perform this certification, we transform the XOR schemes into matrix ensembles using Theorem 5.24 followed by Lemma 5.29, and then compute the associated spectral norms. We can observe easily that each of the transformations prescribed in Theorems 5.20 and 5.24 and in Lemma 5.29 can be performed efficiently. Using [128] as indicated above, we can construct in time $2^{O(ktw)}$ a list of $\leq m^{O(tw)}$ strings which support a $2^{-O(ktw)}$ -almost k -wise distribution for $k = t \log N$. By Theorem 5.26, one of the strings in this list will pass all of the 2^{tw+1} spectral tests defined above; our algorithm outputs the first such string. $\square$

5.5.2 Linear Attacks on Local PRGs

Here we state formally our improvement to an old result of [126] on ϵ -biased generators in NC^0 referenced in the introduction:

Corollary (Corollary 5.5, restated). *Let $t = O(1)$ be a fixed odd number. If $m \geq \tilde{O}(n^{\frac{t}{2} - \frac{t-2}{2(t+2)}})$, then for any NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ and any random variable $\mathbf{x} \in \{0, 1\}^n$, $G(\mathbf{x})$ fails to be γ -biased for some $\gamma \geq n^{-O(1)}$.*

This is simply a reparameterization of Theorem 5.2 and does not require any further argument. Corollary 5.5 implies that for any odd $t = O(1)$ and any candidate NC_t^0 PRG $G : \{0, 1\}^n \rightarrow$

$\{0, 1\}^{n^{\frac{1}{2}-\epsilon_t}}$, there exists a *nonuniform distinguisher* for G , computable by a single parity gate, where $\epsilon_t > 0$ is a positive constant depending only on t . However, as noted in the introduction, our proof does not indicate how to find such a linear distinguisher, and hence has no bearing on the security of Goldreich's PRG in the stretch regime $n \mapsto n^{\frac{1}{2}-\epsilon_t}$ (even against nonuniform adversaries). It is therefore of interest to determine whether we can use our methods to find a *uniform distinguisher* for NC_t^0 PRGs in this stretch regime.

One approach is to use the natural proofs paradigm suggested in Section 5.6.2. In particular, if we can define a uniformly computable natural property which proves nonadaptive bit probe lower bounds of the form $S \geq N^{\frac{1}{2}-\epsilon_t}$, we could use this in conjunction with a hybrid argument as in Lemma 5.48 to uniformly distinguish such generators⁷. In particular, we showed that for any odd $t = O(1)$, if F supports a distribution which is $N^{O(1)}$ -almost $O(\log N)$ -wise independent and $\frac{1}{4}$ biased, then it must satisfy a lower bound of the form $S \geq N^{\frac{1}{2}-\epsilon_t}$. For $M = \text{poly}(N)$ sufficiently large, we can easily certify the property of $N^{O(1)}$ -almost $O(\log N)$ -wise independence with all but negligible probability for a random $F : [N] \times [M] \rightarrow \{0, 1\}$ in quasipolynomial time (iterate over every subset of $O(\log N)$ variables). Hence a natural proof (computable uniformly in $\text{poly}(N)$ time) that a matrix $F : [N] \times [N^{O(1)}] \rightarrow \{0, 1\}$ is $\frac{1}{4}$ biased would yield a distinguisher. If we are a bit more careful in our argument, computing spectral norms of matrices depending on the given generator G as in the proof of Theorem 5.4 instead of naively certifying limited independence by brute force, we arrive at:

Lemma 5.43. *Say that there is a constant c and a polynomial time algorithm which can certify with nonnegligible probability that a random $F : [N] \times [N^c] \rightarrow \{0, 1\}$ supports a $\frac{1}{4}$ -biased distribution. Then there is a uniform polynomial time distinguisher for NC_t^0 generators $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ whenever t is odd and $m \geq n^{\frac{1}{2}-\epsilon_t}$ for some $\epsilon_t > 0$ depending only on t . In particular, Goldreich's generator is insecure in this parameter regime.*

⁷We proved Lemma 5.48 for nonuniform distinguishers, however the hybrid argument is also valid in the uniform setting (with randomized distinguishers).

5.6 Towards Stronger Lower Bounds

In this section we discuss some of the broader implications of Observation 5.17 connecting locally-computable generators to cell probe lower bounds. Our focus here will be on the setting of nonadaptive bit probe data structures with time complexity $t = O(1)$, which correspond via Observation 5.17 to the class of NC^0 generators. More specifically, we are interested in the problem of obtaining lower bounds of two kinds, which we refer to as *strong* and *super-strong* respectively. In each setting, our goal is to construct lower bounds for a problem F with N queries; N will be the main complexity parameter. We seek lower bounds on the space as a function of N and the time t ; subject to obtaining such a lower bound and the problem being explicit, we'd like to minimize the number of datapoints M of our problem as a function of N . We will parameterize each goal by a growth rate $M(N)$, where the goal gets more difficult the slower M grows:

Goal 5.44 (Strong Lower Bounds with $M(N)$ Datapoints). *Exhibit an explicit data structure problem $F : [N] \times [M(N)] \rightarrow \{0, 1\}$, and some universal constant $\epsilon > 0$, so that for any $t = O(1)$, F requires space $S \geq N^\epsilon$ for time- t nonadaptive bit probe data structures.*

Goal 5.45 (Super-Strong Bit Lower Bounds with $M(N)$ Datapoints). *Exhibit an explicit data structure problem $F : [N] \times [M(N)] \rightarrow \{0, 1\}$ so that F requires space $S \geq N - N^{o(1)}$ for data structures making 4 nonadaptive bit probes.*

In the most ambitious case we aim for $M(N) = \text{poly}(N)$, and in the most general case we aim for $M(N) = \exp(N^{o(1)})$. We observe that nonconstructive counting arguments imply that Goal 5.44 is achievable by a nonexplicit problem once $M \geq O(t \log N)$, while Goal 5.45 is achievable by a nonexplicit problem once $M \geq O(tN \log N)$, so even in the ambitious regime $M = \text{poly}(N)$ neither goal is vacuous in an information-theoretic sense. Note that our main lower bounds in Sections 5.3 and 5.4 are of the form $S \geq N^{\delta_t}$ where $\lim_{t \rightarrow \infty} \delta_t = 0$ and hence they do not achieve either of these goals. In this section we discuss some barriers and approaches to achieving lower bounds of these kinds, and connect these problems to other topics in complexity theory. More specifically we will discuss barriers for both problems, while our “approaches” will only

apply to the first (Goal 5.44); we will see shortly that super-strong lower bounds (Goal 5.45) are likely to be completely out of reach of current mathematical methods.

Before proceeding, we make a note of our use of the word *explicit*; for the rest of this section, we will use this word in the following formal sense:

Definition 5.46. *We say that a sequence of strings $(x_n)_{n \in \mathbb{N}}$ is explicit if there is a uniform algorithm which, given 1^n as input, will print x_n in time $\text{poly}(n, |x_n|)$. In particular, if $(F_N : [N] \times [M(N)] \rightarrow \{0, 1\})_{N \in \mathbb{N}}$ is a family of data structure problems, we say that F_N is explicit if there is a $\text{poly}(N, M(N))$ time algorithm which, given 1^N , produces the truth table of F_N , i.e. the $N \times M(N)$ matrix consisting of all values $F(x, y)$.*

In this more formal terminology, our statement of Goal 5.44 should be interpreted as follows: define an explicit *family* of problems $F_N : [N] \times [M(N)] \rightarrow \{0, 1\}$ for each N , so that $M(N) \leq \exp(N^{o(1)})$, and F_N requires space $\Omega(N^\epsilon)$ for time t data structures, where ϵ, t are as in the statement of Goal 5.44, and N goes to infinity after these are fixed. In all of our main results here, we may replace the assumption that F is defined for all N by the assumption it is defined only for N of some special forms, e.g. prime numbers, powers of 2, etc., provided this class of allowable values is efficiently recognizable, and not too sparse as a subset of $\mathbb{N}$. We stick to the above definition which requires the lower bound to hold for all N only for the purpose of simplicity.

As has been observed repeatedly in complexity theory, this formal notion of *explicitness* is perhaps overly broad compared to the typical *informal* use of the word; indeed, in many places it is preferred to use the more stringent notion of explicitness, requiring $F(x, y)$ to be computable uniformly in $\text{poly}(\log N, \log M)$ time, which still captures apparently every data structure problem in the literature for which an interesting cell probe lower bound is currently known.

5.6.1 Consequences of Explicit Bit Probe Lower Bounds

Recall Lemma 5.41 from the previous section, which was a direct corollary of our main correspondence (Observation 5.17): every explicit data structure lower bound yields a NP oracle Range

Avoidance algorithm for locally computable generators with corresponding parameters. We apply this as follows:

Theorem (Theorem 5.6, restated).

1. *If explicit super-strong bit probe lower bounds with subexponentially many datapoints (Goal 5.45, $M = \exp(N^{o(1)})$) are achievable, then $\text{EXP}^{\text{NP}} \not\subseteq \text{NC}^1$.*
2. *If explicit strong bit probe lower bounds with polynomially many datapoints (Goal 5.44, $M = \text{poly}(N)$) are achievable, then there is a polynomial time NP oracle algorithm for NC^0 Range Avoidance with polynomial stretch.*

Proof. (1) If Goal 5.45 with $M(N) = \exp(N^{o(1)})$ is achievable than we can solve NC_t^0 Range Avoidance in the stretch regime $n \mapsto n + n^{o(1)}$ in subexponential time with an NP oracle using Lemma 5.41. By [144], this implies the separation $\text{EXP}^{\text{NP}} \not\subseteq \text{NC}^1$.

(2) By well known composition reductions (see [101]), we may reduce any NC^0 Range Avoidance $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n^{1+\Omega(1)}}$ to a second NC^0 instance $G' : \{0, 1\}^n \rightarrow \{0, 1\}^{n^c}$ where $c = O(1)$ is an arbitrarily large constant in polynomial time with an NP oracle. If Goal 5.44 is achievable than by Lemma 5.41 we can solve Range Avoidance in this regime using an NP oracle in polynomial time. $\square$

In (2) above, if we instead achieved Goal 5.44 with $M = \exp(\text{poly log } N)$ we would obtain a quasipolynomial time algorithm; if we achieved it with $M = \exp(N^{o(1)})$ we would obtain a subexponential time algorithm. Obtaining P^{NP} algorithms for NC^0 Range Avoidance in the polynomial stretch regime was explicitly posed as an open problem in [144].

5.6.2 Natural Bit Probe Lower Bounds Break NC^0 PRGs

We present here a natural proofs barrier to both “strong” and “super-strong” bit probe lower bounds. The natural proofs paradigm, introduced in the seminal work of Razborov and Rudich [143], identified the following pattern in all preceeding known lower bounds in complexity theory.

For a lower bound problem, we have in mind in some resource-bounded computational model C for computing boolean valued functions $f : U \rightarrow \{0, 1\}$, where U is a finite set with some structure (e.g. $U = \{0, 1\}^n$ or $U = [N] \times [N]$). We would like to present an explicit function $f_{\text{explicit}} : U \rightarrow \{0, 1\}$ which we can unconditionally prove requires a large amount of resources for computations in the class C . Razborov and Rudich observed that, in those cases where we *can* prove such an explicit lower bound, the proof can be extended (with only minimal-to-moderate additional work) to have the following features: (1) the proof defines an efficiently decidable “property of functions $f : U \rightarrow \{0, 1\}$ ” so that f_{explicit} has the property, and whenever f has this property it is hard for the class C , and (2) not only does f_{explicit} have this property, but so does a *uniformly random* function $f : U \rightarrow \{0, 1\}$ (with high probability). The “property of functions” referenced above is called a *natural property* in [143], and by “efficiently” we mean time polynomial/subexponential in the description size of f , which in this case is $|U|$. Razborov and Rudich then went on to show:

1. All previously known lower bounds are natural.
2. Assuming widely believed cryptographic assumptions, no natural proof can give superpolynomial lower bounds on circuit size.

Together these results give a strong indication that, to prove $f \notin P/\text{poly}$ for some explicit function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, we must develop methods of a fundamentally different nature than all results known up to the point of [143]. The situation has not changed much since the publication of [143], and this barrier is still considered to be of great significance in complexity theory. We now define formally the corresponding notion of natural proofs for the bit probe complexity measure:

Definition 5.47 (Natural Bit Probe Lower Bounds). *A (N, M, S, t) natural property is an algorithm $\mathcal{A}$ which takes as input the description of a data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$, and outputs a value in $\{\text{Hard}, ?\}$, such that:*

1. *If $\mathcal{A}(F) = \text{Hard}$, then F requires space $\geq S$ or time $> t$ for nonadaptive bit probe data structures.*

2. If F is chosen uniformly at random amongst all functions $[N] \times [M] \rightarrow \{0, 1\}$, then with probability at least $(NM)^{-O(1)}$, $\mathcal{A}(F) = \text{Hard}$.

We are interested in the existence of natural properties $\mathcal{A}$ which are computable by circuits of size $\ll 2^N$, i.e. $\text{poly}(N)$ or $\exp(N^{o(1)})$. In our discussion we will have N as our main asymptotic complexity parameter, t fixed, and $S = S(N)$, $M = M(N)$ growing as a function of N . We will then refer to a $(N, M(N), S(N), t)$ -natural property in plain english as a “natural proof of a space lower bound $\geq S(N)$ against time t data structures for a problem with $M(N)$ datapoints.” If the natural property is computable by size k circuits, we say that it is a “ k size computable” natural proof.

In the following, we will say that a cryptographic PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ has “hardness λ ” if circuits of size $\leq \lambda$ fail to distinguish G with advantage better than λ^{-1} .

Lemma 5.48. *Say that $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ is a cryptographic NC_t^0 PRG with hardness λ . Then there does not exist an (N, M, S, t) natural property computable by circuits of size k unless $\lambda \leq k \cdot \text{poly}(NM)$.*

Proof. Let G be a cryptographic NC_t^0 PRG with hardness λ . Consider the generator $G^{\otimes M} : \{0, 1\}^{S \cdot M} \rightarrow \{0, 1\}^{N \cdot M}$ given by $G(E_1, \dots, E_M) = (G(E_1), \dots, G(E_M))$. A standard application of the hybrid argument implies that $G^{\otimes M}$ is also a cryptographic PRG with hardness $M^{-1}\lambda$. On the other hand, consider any string in the range of $G^{\otimes M}$. Interpreted as data structure problem $F : [N] \times [M] \rightarrow \{0, 1\}$, this problem has time t nonadaptive bit probe data structures of space $\leq S$: if $G^{\otimes M}(E_1, \dots, E_M) = F$, then $E_1, \dots, E_M$ exhibit preimages under G for each column of F .

Say there existed a (N, M, S, t) natural property $\mathcal{A}$ computed by circuits of size k . Then $\mathcal{A}$ outputs Hard on a random $N \cdot M$ bit string with probability $\geq (NM)^{-O(1)}$ over the uniform distribution, and with probability zero over the range of our generator $G^{\otimes M}$. Hence $k^{-1}(NM)^{-O(1)}\lambda \leq O(1)$, i.e. $\lambda \leq k(NM)^{O(1)}$. $\square$

To establish our natural properties barriers we need two results from NC^0 cryptography. The

first is folklore and follows from standard PRG composition:

Lemma 5.49. *If there exists an NC^0 PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with arbitrary polynomial stretch ($m \geq n^{1+\Omega(1)}$) and hardness λ , then for every $c \in \mathbb{N}$ there exists an NC^0 PRG $G' : \{0, 1\}^n \rightarrow \{0, 1\}^{n^c}$ with hardness $\geq n^{-O(1)}\lambda$.*

Hence, from the existence of polynomial stretch NC^0 PRGs with superpolynomial hardness we obtain NC^0 PRGs with superpolynomial hardness of the form $G : \{0, 1\}^{N^\epsilon} \rightarrow \{0, 1\}^N$ for arbitrarily small constants $\epsilon > 0$. We also need the following from [10]:

Lemma 5.50. *If there exists an NC^1 PRG with hardness λ then there exists an NC_4^0 PRG $G : \{0, 1\}^m \rightarrow \{0, 1\}^{m+m^{0.99}}$ for some $m \leq \text{poly}(n)$ with hardness $\geq n^{-O(1)}\lambda$.*

Combining these two lemmas with our Lemma 5.48 we arrive at:

Theorem (Theorem 5.7, restated: Natural Proofs Barriers for Bit Probe Lower Bounds).

1. *If NC^1 PRGs with superpolynomial hardness exist, then there is no polynomial-size computable natural proof of a lower bound $S \geq N - N^{0.99}$ against bit probe data structures making 4 nonadaptive bit probes for a problem with $M \leq \text{poly}(N)$ datapoints.*
2. *If NC^0 generators with superpolynomial hardness exist, then for every ϵ there exists t so that there is no polynomial-size computable natural proof of a lower bound $S \geq N^\epsilon$ for against time t bit probe data structures for a problem with $M \leq \text{poly}(N)$ datapoints.*

As in the last section, if we increase the hardness parameter in our cryptographic assumptions we obtain stronger conclusions in terms of the runtime of our natural proofs and the allowable number of datapoints M . In both cases it is plausible that the stated PRGs exists with hardness $\lambda = \exp(N^{\Omega(1)})$, in which case we conclude that natural proofs computable by circuits of size $\exp(N^{o(1)})$ cease to exist already in the regime $M = \exp(N^{o(1)})$.

As in [143], to give a convincing argument that this “barrier” ought to be taken seriously we need some indication that the known lower bound techniques have not already broken it. To this

end we consider known techniques that can prove the strongest lower bounds for explicit problems $F : [N] \times [M] \rightarrow \{0, 1\}$ with $M \leq \exp(N^{o(1)})$ in the regime $t = O(1)$.

First we consider the communication bound, which can prove a space lower bound $S \geq \Omega(N^{\frac{1}{t}})$ already in the regime $M \leq \text{poly}(N)$. This lower bound holds provided F has maximal $(\log N)$ deterministic communication complexity, so it suffices to give a natural proof of strong communication lower bounds. For this we can use as our natural property “ F has full rank” where the rank is measure either over $\mathbb{R}$ or $\mathbb{F}_2$. This property is computable in polynomial time, equals N for a random F with nonnegligible (actually very high) probability, and yields a maximal lower bound $\log N$ on the communication complexity of F . We make a note that lower bounds for stronger complexity measures for two party communication, including randomized and nondeterministic communication, can be made natural as well using the discrepancy bound and related techniques. The original work of [143] proves this for the standard discrepancy bound which lower bounds both randomized and nondeterministic complexity. In the case of randomized communication one can use the more precise γ_2^∞ -norm instead to obtain natural lower bounds with stronger parameters, see [115].

Next we turn to the cell sampling method used in [154, 155, 133, 109] and our new lower bounds of the form $S \geq N^{\frac{2}{t}}$ from Section 5.3. Note that in the regime $M \leq \exp(N^{o(1)})$ the cell sampling bound does not give a significant improvement over the communication bound so it does not really require a separate treatment. Nonetheless, for both the cell sampling bound and our Theorem 5.1, the property we may use for the problem F is the following: the uniform distribution on columns of F is N^{-c} -almost $c \log N$ -wise independent for some constant c depending on t . Setting $M = N^{O(c)}$, we can certify this property for a random F with high probability in time $N^{O(\log N)}$ by iterating over every subset of $c \log N$ rows of F . Note that this natural property runs in quasipolynomial time (rather than polynomial), however this is still interesting as the cryptographic assumptions in question are widely believed to have quasipolynomial (and much higher) hardness⁸.

⁸Actually, we only need to assume that our cryptographic PRGs require super-quasipolynomial size circuits to be distinguished with polynomial advantage for a natural proof with these parameters to contradict their security, since $M = \text{poly}(N)$.

Finally we get to the nonadaptive bit probe lower bounds in Section 5.4 of the form $S \geq N^{\frac{1}{2} + \epsilon_t}$; here we actually *do not* know whether the lower bound in question can be made natural in the strict sense of Definition 5.47; this question is discussed further in Section 5.5. Nonetheless the lower bound in Theorem 5.2 still has the following form: for any random f which is *fools* $\mathbb{F}_2$ linear tests, we have $f \notin \text{range}(G)$ with nonzero probability, for any NC_t^0 generator G with large enough stretch. Stated contrapositively, we conclude that for any candidate NC_t^0 generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n^{t/2 - \epsilon_t}}$, there exists a $\mathbb{F}_2$ -linear test distinguishing its output from random, so in particular G cannot be secure as a PRG (this was Observed in Corollary 5.5 of the previous section). In other words, our lower bound still yields as a byproduct a method for distinguishing the associated PRGs, although in this case the existence of a distinguisher is shown nonconstructively. For this reason it seems that the lower bounds in Section 5.4 still have a degree of algorithmic content which cannot be present in any lower bound achieving Goals 5.44 and 5.45 (assuming the corresponding cryptographic generators exist).

5.6.3 Approaches to Strong Lower Bounds via Communication Complexity

Here we discuss some possible approaches to Goal 5.44: proving space lower bounds for time $t = O(1)$ nonadaptive bit probe data structures of the form $S \geq N^\epsilon$, where ϵ is a universal constant not depending on t . The reader may observe that we would have arrived at an equivalent problem if we had used adaptive in place of nonadaptive bit probe data structures: any adaptive bit probe data structure with time complexity t can be converted into a nonadaptive data structure with time complexity 2^t by unrolling each query decision tree. We will make another simplifying assumption on our data structures, having no effect on Goal 5.44, which will streamline some of our arguments:

Definition 5.51. *We say that an NC_t^0 generator is homogeneous if there is a single predicate $h : \{0, 1\}^t \rightarrow \{0, 1\}$, so that each output $G_x : \{0, 1\}^S \rightarrow \{0, 1\}^N$ is of the form $G_x(E) = h(E(p_x^1), \dots, E(p_x^t))$ for some nonadaptive probe sequence $p_x^1, \dots, p_x^t \in [S]$. We say that F has homogeneous data structures with time t and space S if there exists a homogeneous NC_t^0 generator whose range contains every column of F .*

Just as the distinction between adaptive and nonadaptive data structures is irrelevant with respect to lower bounds achieving Goal 5.44, so too is the distinction between general and homogeneous data structures:

Lemma 5.52. *Let F be any data structure problem, $t \in \mathbb{N}$. There exist constants t', c depending only on t , so that if F has space S time t bit probe data structures, then F has space $c \cdot S$ and time t' homogeneous data structures.*

We will therefore focus specifically on homogeneous data structures with time $t = O(1)$. Homogeneous data structures have a rather appealing formulation in terms of communication complexity:

Definition 5.53 (Parallel Channel Model). *Let $F : X \times Y \rightarrow \{0, 1\}$ be a two-party communication problem. We say that F has a (t, k) -parallel protocol if it can be computed in the following way:*

1. *Alice receives $x \in X$; a council of t Bobs $B_1, \dots, B_t$ receive (the same input) $y \in Y$; a referee Charlie receives no input.*
2. *(Round 1:) Alice sends a different k -bit message privately to each Bob*
3. *(Round 2:) Each Bob sends 1 bit to Charlie*
4. *(Round 3:) Charlie outputs the answer $F(x, y)$.*

The equivalence of this model (up to minor quantitative losses) to the homogeneous data structure model is a matter of definitional translation; the relation closely mirrors that between locally decodable codes and private information retrieval schemes:

Observation 5.54. *If F has homogeneous bit probe data structures with space S and time t , then F has a $(t, \log S)$ -parallel protocol. Conversely, if F has a (t, k) -parallel protocol then it has a homogeneous data structure with time t and space $t \cdot 2^k$.*

We refer to this as equivalence, since w.r.t. Goal 5.44 the difference between a space lower bound S and tS is irrelevant.

Proof. We prove only the first direction. Let $G : \{0, 1\}^S \rightarrow \{0, 1\}^N$ be a homogeneous NC_t^0 generator whose range covers the columns of F . Prior to communication, all of the Bobs prepare the data structure encoding E_y for y . Alice sends to B_j the message $p_x^j \in [S]$ which costs $\log S$ bits. After receiving this value, B_j sends Charlie the bit $E_{p_x^j}(B_j)$. Charlie, upon receiving bits $b_1, \dots, b_t$ from the t Bobs, outputs $h(b_1, \dots, b_t)$. $\square$

We can then restate Goal 5.44 as follows:

Goal 5.55 (Strong Parallel Communication Lower Bounds for a problem with M Columns). *Exhibit an explicit $F : \{0, 1\}^n \times [M] \rightarrow \{0, 1\}$ and some universal constant $\epsilon > 0$, so that F does not have $(O(1), \epsilon n)$ -parallel protocols.*

In this setting we will set $M = 2^m$ for some m , and identify $[M]$ with $\{0, 1\}^m$. In this case, achieving $m \leq O(n)$ for Goal 5.55 is equivalent to our most ambitious goal of achieving $M \leq \text{poly}(N)$ for Goal 5.44; we would also be happy with $m \leq 2^{o(n)}$ in Goal 5.55, which corresponds to our most general acceptable form of Goal 5.44 with $M \leq \exp(N^{o(1)})$.

While our primary intention in investigating the parallel channel model is as an avenue through which to derive new cell probe lower bounds, using Observation 5.54 we can use our lower bounds from Sections 5.3 and 5.4 to derive new lower bounds in this communication model, for example:

Lemma 5.56. *Let $t = O(1)$ be an even number. Consider the following communication problem: Alice receives $x \in \mathbb{F}_2^n$, and t Bobs receive a polynomial $p : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ of degree $\log n + \log \log n$; their goal is to compute $p(x)$. In the parallel channel model, Alice must send some Bob at least $\frac{2n}{t} - O(\log n)$ bits to solve this problem.*

We now discuss two avenues for approaching strong lower bound in the parallel channel model with $O(1)$ channels.

5.6.3.1 A Lifting-Based Approach To the Parallel Channel Model

For starters, we consider a restricted form of (t, k) -parallel protocol for a problem $F : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, which we call a *projection protocol*. A (t, k) projection protocol is a special

kind of (t, k) -parallel protocol, in which Alice is restricted to chose in advance sets $I_1, \dots, I_t \subseteq [n]$ with $|I_j| \leq k$, and upon input x , she will always send the j^{th} Bob the values $x|_{I_j} = (x_i)_{i \in I_j}$. In other words, instead of sending the j^{th} Bob an arbitrary k -bit message depending on x , she is restricted to sending him some fixed set of k coordinates of x . For protocols of this form, we may observe the following:

Claim 5.57. *If F has (t, k) projection protocols then every column of $f : \{0, 1\}^n \rightarrow \{0, 1\}$ of F is expressible in the form $f(x) = h(g_1(x), \dots, g_t(x))$ where $h : \{0, 1\}^t \rightarrow \{0, 1\}$ and g_j depends on at most k variables amongst $\{x_1, \dots, x_n\}$.*

Boolean functions expressible in this way have been studied in [76, 110]. In particular, the minimum value t such that f can be expressed as $f = h(g_1, \dots, g_t)$ for g_j depending on $\leq k$ variables is referred to as $C_k^2(f)$ in [76]. The “2” in the superscript refers to the interpretation of $C_k^2(f)$ as the smallest top-fanin of a depth 2 circuit with arbitrary gates and bottom fanin k computing f . Intriguingly, strong lower bounds on $C_k^2(f)$ for explicit functions f were proved unconditionally in [76]. For example, as a direct corollary of a classical result in [26] on the number-on-forehead complexity of threshold functions, they observe:

Lemma ([26, 76]). *For any constant γ , $C_{(1-\gamma)n}^2(\text{MAJ}_n) = \omega(1)$.*

We are using MAJ_n to denote the n bit majority function. Quantitatively stronger lower bounds on $C_k^2(\cdot)$ in the same regime $k = (1 - \gamma)n$ are derived in [76] for other more complicated explicit functions. In addition, an optimal lower bound on $C_k^2(\text{MAJ}_n)$ is derived in [110] in a quite different parameter regime $k \leq n^{1-\epsilon}$. The above result implies that even the problem $F : \{0, 1\}^n \times \{0, 1\}^0 \rightarrow \{0, 1\}$, given by $F(x, \emptyset) = \text{Maj}_n(x)$ is hard for $(O(1), 0.99n)$ projection protocols. This lower bound quantitatively matches (and in fact exceeds) the kind sought in Goal 5.55. Obviously in the general parallel channel model we cannot hope to obtain a lower bound for a problem F with a single column (Alice could precompute $f(x)$ in this case); nonetheless we believe this toy model presents a promising avenue towards lower bounds for general protocols, via the machinery of *query-to-communication-lifting*.

Lifting is a powerful method used to obtain lower bounds in communication complexity (see [68] for a prototypical example). The central idea is to start with an arbitrary base function, and “lift it” via function composition to get a new composed function whose *communication complexity* is essentially the same as the *query complexity* of the original base function. Proofs of lifting theorems typically involve a simulation theorem, showing from *any* protocol for the composed function, how to extract a simple protocol for the base function. Thus lifting simulation theorems give a constructive proof that the most efficient communication for the composed problem is essentially the protocol that simply mimics the optimal simple protocol, implying that separating the simpler query classes is not only necessary but also sufficient for proving the stronger communication separation.

We define here a standard lifting setup where the inner “gadget” function is the index function.

Definition 5.58 (Lifting with Index). *Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and ℓ be given. Let $X = [\ell]^n$, $Y = (\{0, 1\}^\ell)^n$. We define the lifted function $f \circ \text{Ind}_n^\ell : X \times Y \rightarrow \{0, 1\}$ by $f \circ \text{Ind}_n^\ell(x, y) = f(y_1(x_1), \dots, y_n(x_n))$.*

With this definition, it is natural to view the parallel-channel model as the “communication analogue” of the complexity measure $C_k^2(\cdot)$. In particular, we can observe that F is computed by a (t, k) parallel protocol if and only if we can express it in the form:

$$F(x, y) = h(g_1(x, y), \dots, g_t(x, y))$$

where $h : \{0, 1\}^t \rightarrow \{0, 1\}$ and $g_1, \dots, g_t : X \times Y \rightarrow \{0, 1\}$ have one way communication complexity (rows speaking to columns) $\leq k$. Compare this to the definition of $C_k^2(f) \leq t$, which means f can be expressed as:

$$f(x) = h(g_1(x), \dots, g_t(x))$$

with $g_j : \{0, 1\}^n \rightarrow \{0, 1\}$ depending on at most k variables, i.e. g_j having *nonadaptive decision tree depth* at most k . Prior work has established that one-way communication complexity can be naturally viewed as a communication complexity analogue of “nonadaptive decision tree depth,”

and in particular very strong lifting theorems have been established (by quite simple arguments) which transfer nonadaptive decision tree lower bounds to one way communication lower bounds [119]. This intuition leads us to make the following conjecture:

Conjecture 5.59 (Lifting C_k^2 To Parallel Channel Complexity). *There is a fixed constant $\epsilon > 0$ and some $\ell = \text{poly}(n)$ so that the following holds. For any $f : \{0, 1\}^n \rightarrow \{0, 1\}$, if $C_{(1-o(1))n}^2(f) = \omega(1)$, then $f \circ \text{Ind}_m^n$ requires communication $\geq \epsilon n \log \ell$ in the parallel model with $O(1)$ Bobs.*

The above conjecture would imply a lower bound achieving Goals 5.44/5.55 for the specific explicit problem $\text{MAJ}^n \circ \text{Ind}_\ell^n$, $\ell \leq \text{poly}(n)$. In this parameter setting, we would have a problem with $N = \ell^n = 2^{O(n \log n)}$ rows and $M = 2^{\text{poly}(n)}$ columns, i.e. M is at most quasipolynomial in N . It would be reasonable to strengthen the conjecture, so that under the same assumption we conclude stronger lower bound $(1 - o(1))n \log m$ on communication; we state it in the weaker form above so as not to be too greedy.

5.6.3.2 Communication with Alternation and Modular Counting

Here we discuss two more standard complexity measures arising in two party communication complexity which provably capture the parallel-channel model; in principle, good enough lower bounds on either of these measures would be sufficient to achieve Goal 5.44. Via the natural properties connection in Section 5.6.2, this also implies that there is a natural proofs barrier to obtaining strong lower bounds for either of these complexity measures. The first measure we examine is a subclass of the communication polynomial hierarchy PH^{cc} defined in [14], which captures alternating protocols with two levels of alternation:

Definition 5.60. For $F : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, we define $\Sigma_2^{\text{cc}}(F)$ as the minimal value $k_1 + k_2$ such that

$$F(x, y) = \bigvee_{i \leq 2^{k_1}} \bigwedge_{j \leq 2^{k_2}} L_{i,j}(x) \vee R_{i,j}(y)$$

for some $L_{i,j} : \{0, 1\}^n \rightarrow \{0, 1\}$, $R_{i,j} : \{0, 1\}^m \rightarrow \{0, 1\}$

Lemma 5.61. If $F : \{0, 1\}^n \times \{0, 1\}^m$ has (t, k) -parallel protocols then $\Sigma_2^{\text{cc}}(F) \leq k + t + \log t$

Proof. Say that $F(x, y) = h(g_1(x, y), \dots, g_t(x, y))$ with g_j having one way communication complexity at most k . So $g_j(x, y) = R_j(\Pi_j(x), y)$ for some $\Pi_j : \{0, 1\}^n \rightarrow \{0, 1\}^k$, $R_j : \{0, 1\}^k \times \{0, 1\}^m \rightarrow \{0, 1\}$. Hence

$$F(x, y) = \bigvee_{\xi \in h^{-1}(1)} \bigwedge_{\substack{j \leq t, \\ z \in \{0, 1\}^k}} \mathbb{1}\{\Pi_j(x) \neq z\} \vee \mathbb{1}\{R_j(z, y) = \xi_j\}$$

□

This means that any explicit problem F satisfying $\Sigma_2^{\text{cc}}(F) \geq \Omega(n)$ would automatically achieve Goal 5.44. This also yields, via Lemma 5.48, a natural properties barrier for the measure $\Sigma_2^{\text{cc}}(\cdot)$:

Lemma 5.62. *Assuming the existence of polynomial stretch NC^0 generators, there is no natural proof of a lower bound $\Sigma_2^{\text{cc}}(F) \geq \Omega(n)$ for $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$.*

We note that it is a notorious open problem to prove even a lower bound $\Sigma_2^{\text{cc}}(F) \geq \omega(\log n)$ for an explicit problem $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$. It is also an open problem to prove lower bounds of the form $2^{\Omega(n)}$ for depth 3 AC^0 circuits computing n variable functions, which lower bounds of the above form would also imply. These facts gives some indication that proving $\Omega(n)$ lower bounds on Σ_2^{cc} complexity may far overshoot Goal 5.44 in terms of difficulty. We make an additional note that, using basically the same proof as above, it is also possible to strengthen Lemma 5.61 to general adaptive data structures with arbitrary word length:

Lemma 5.63 (Strengthening of Lemma 5.61). *If F has general data structures with time t , word-size w , and space S , then $\Sigma_2^{\text{cc}}(F) \leq \log S + tw + \log t$*

With this strengthening, it follows that any lower bound $\Sigma_2^{\text{cc}}(F) \geq (3 + \Omega(1))\sqrt{n}$ for a problem $F : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$ with $m \leq 2^{o(\sqrt{n})}$ would yield a new state of the art bit probe lower bound for general adaptive data structures in the setting $w = 1$, $t = \sqrt{n}$; this is a very different setting then Goal 5.44 and we do not know of a natural properties barrier which applies in such a regime. Nonetheless the cell probe lower bound would still be of interest in its own right.

Next we discuss a complexity measure associated to a subclass of $(\text{AC}^0[\oplus_p])^{\text{cc}}$:

Definition 5.64. Let p be a prime number and $F \in \{0, 1\}^{N \times M}$ a boolean matrix. Define $\text{r-rank}_p(F)$, the “replacement-rank of F over $\mathbb{F}_p$,” to be the least rank of a matrix $\tilde{F} \in \mathbb{F}_p^{N \times M}$ such that

$$F(x, y) = \phi(\tilde{F}(x, y))$$

for some $\phi : \mathbb{F}_p \rightarrow \{0, 1\}$.

In other words, $\text{r-rank}_p(F) \leq r$ if and only if there is a rank $\leq r$ matrix $\tilde{F} \in \mathbb{F}_p^{N \times M}$ such that the value of $F(x, y)$ is completely determined by the value $\tilde{F}(x, y)$.

Lemma 5.65. Say that $F \in \{0, 1\}^{N \times M}$ has $(O(1), \log S)$ parallel protocols. Then there exists a prime $p \leq 2^{t+2}$ so that $\text{r-rank}_p(F) \leq tS$.

Proof. Say that $F(x, y) = h(g_1(x, y), \dots, g_t(x, y))$ with $g_j \in \{0, 1\}^{N \times M}$ having one way communication complexity $\leq \log S$. Choose $p > 2^{t+1}$ prime and consider each g_j to live in $\mathbb{F}_p^{N \times M}$. Then g_j has at most S distinct rows, and hence its rank is at most S . Define $\tilde{F} \in \mathbb{F}_p^{N \times M}$ by $\tilde{F} = \sum_{j \leq t} 2^j \cdot g_j$. Then $\text{rank}_{\mathbb{F}_p}(\tilde{F}) \leq \sum_j \text{rank}_{\mathbb{F}_p}(g_j) \leq tS$. On the other hand for any $\xi \in \{0, 1\}^t$, $\sum_j 2^j \xi_j$ completely determines ξ (here we use that $p > 2^{t+1}$). $\square$

Hence, exhibiting an explicit matrix $F \in \{0, 1\}^{N \times M}$ satisfying $\text{r-rank}_p(F) \geq N^\epsilon$ for some fixed $\epsilon > 0$ and all primes $p = O(1)$ would solve Goal 5.44. Currently the best lower bound we know of for $\text{r-rank}_p(\cdot)$ is via the inequality $\text{r-rank}_p(A) \leq p \cdot \text{rank}(A)^{(p-1)}$ which follows by expressing the predicate $\phi : \mathbb{F}_p \rightarrow \{0, 1\}$ as a degree $p - 1$ polynomial over $\mathbb{F}_p$ using Fermat’s little theorem; this never gives a bound better than $N^{\frac{1}{p-1}}$ which is clearly insufficient for our purposes. As in the case of Σ_2^{cc} complexity, combining Lemma 5.65 with Lemma 5.48 yields a natural properties barrier for lower bounding $\text{r-rank}_p(F)$.

5.7 Kikuchi Method

We prove here Theorem 5.24 from Section 5.3. This argument occurs in [166, 64] and we only need to adapt things to our notation:

Theorem 5.66 (Theorem 5.24). *Let $C = ([N], V, c)$ be a t -XOR scheme, $|V| = K$, and t even. For any $t \leq L \leq \frac{K}{8}$, we may associate with C a permutation ensemble $\mathcal{A}$ indexed by $[N]$ with the following properties:*

1. $\mathcal{A}$ lives in $\mathbb{R}^{m \times m}$ for $m = \binom{K}{L}$
2. $d(\mathcal{A}) \geq \left(\frac{L}{K}\right)^{t/2} N$
3. For any $f \in \{\pm 1\}^N$, we have $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq d(\mathcal{A})m \cdot \text{OPT}(C, f)$

Proof. Let $C = ([N], V, c)$ be t -XOR scheme, $|V| = K$, t even. Let $m = \binom{K}{L}$, and identify $[m]$ with $\binom{V}{L}$ canonically. We define a permutation ensemble in $\mathbb{R}^{m \times m}$, given by $A_x(a, b) = \mathbb{1}\{a \Delta b = c_x\}$ where $a \Delta b$ denotes the symmetric difference of a, b as elements of $\binom{V}{L}$. Observe that by symmetry, $\sum_{a,b} A_x(a, b)$ is the same value for all x ; call this value r . Indeed we have $r = 2 \binom{t}{t/2} \binom{K-t}{L-t/2}$. To see this, note that if $|a| = |b| = L$, $a \Delta b = c$, $|c| = t$ then we must have $a = (w \cup u)$, $b = (w \cup v)$ for some $|w| = L - t$, $|u| = |v| = \frac{t}{2}$, $u, v \subseteq c$. There are $\binom{t}{t/2}$ ways to choose disjoint subsets $u, v \subseteq c_x$, $|u| = |v| = \frac{t}{2}$, and $\binom{K-t}{L-t/2}$ ways to choose $w \in \binom{V \setminus \{u \cup v\}}{L-t/2}$. Since we will count (a, b) , (b, a) separately we must add in a factor of 2.

Now, let $f \in \{\pm 1\}^N$ and let $R : V \rightarrow \{\pm 1\}$ be such that $\text{OPT}(C, f) = \text{Val}(C, f, R)$. Define $\tilde{R} \in \{\pm 1\}^m = \{\pm 1\}^{\binom{V}{L}}$, with $\tilde{R}(a) = \prod_{i \in a} R(i)$. Then

$$\begin{aligned} \tilde{R}^\top \mathcal{A}[f] \tilde{R} &= \sum_{a,b \in \binom{V}{L}} \tilde{R}(a) \tilde{R}(b) \mathcal{A}[f](a, b) = \sum_x f_x \sum_{a,b} A_x(a, b) \prod_{i \in a} R(i) \prod_{j \in b} R(j) \\ &= \sum_{x \in X} \left(\sum_{a,b} A_x(a, b) \right) f(x) \prod_{i \in c_x} R(i) = r \sum_x f(x) \prod_{i \in c_x} R(i) = rN \cdot \text{OPT}(C, f) \end{aligned}$$

Now, observe that $d(\mathcal{A}) = Nm^{-1}r$, in other words $r = d(\mathcal{A})N^{-1}m$, so we have the bound $\|\mathcal{A}[f]\|_{\infty \rightarrow 1} \geq d(\mathcal{A})m \cdot \text{OPT}(C, f)$. It remains to lower bound $d(\mathcal{A})$, for which we observe

$$d(\mathcal{A}) = Nm^{-1}r \geq \frac{2 \binom{t}{t/2} \binom{K-t}{L-t/2}}{\binom{K}{L}} N \geq \left(\frac{L}{K}\right)^{t/2} N$$

provided $t \leq L \leq \frac{K}{8}$, where the final inequality follows from standard manipulations of binomial coefficients (see [77]). □

References

- [1] S. Aaronson, H. Buhrman, and W. Kretschmer, “A qubit, a coin, and an advice string walk into a relational problem,” in *15th Innovations in Theoretical Computer Science Conference, ITCS 2024, January 30 to February 2, 2024, Berkeley, CA, USA*, V. Guruswami, Ed., ser. LIPIcs, vol. 287, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, 1:1–1:24.
- [2] J. Abascal, V. Guruswami, and P. K. Kothari, “Strongly refuting all semi-random boolean csps,” in *Proceedings of the Thirty-Second Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA ’21, Virtual Event, Virginia: Society for Industrial and Applied Mathematics, 2021, 454–472, ISBN: 9781611976465.
- [3] M. Agrawal, N. Kayal, and N. Saxena, “Primes is in P,” *Ann. of Math*, vol. 2, pp. 781–793, 2002.
- [4] M. Ajtai, “ Σ^1_1 -formulae on finite structures,” *Ann. Pure Appl. Log.*, vol. 24, no. 1, pp. 1–48, 1983.
- [5] S. R. Allen, R. O’Donnell, and D. Witmer, “How to refute a random csp,” in *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*, IEEE, 2015, pp. 689–708.
- [6] E. Allender, “The complexity of complexity,” in *Computability and Complexity - Essays Dedicated to Rodney G. Downey on the Occasion of His 60th Birthday*, A. R. Day, M. R. Fellows, N. Greenberg, B. Khossainov, A. G. Melnikov, and F. A. Rosamond, Eds., ser. Lecture Notes in Computer Science, vol. 10010, Springer, 2017, pp. 79–94.
- [7] N. Alon and J. H. Spencer, *The Probabilistic Method*, 4th ed. Hoboken, NJ: Wiley, 2016.
- [8] B. Applebaum, “Cryptographic hardness of random local functions,” *Comput. Complex.*, vol. 25, no. 3, 667–722, Sep. 2016.
- [9] B. Applebaum, S. Artemenko, R. Shaltiel, and G. Yang, “Incompressible functions, relative-error extractors, and the power of nondeterministic reductions (extended abstract),” in *Proceedings of the 30th Conference on Computational Complexity*, ser. CCC ’15, Portland, Oregon: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2015, 582–600, ISBN: 9783939897811.
- [10] B. Applebaum, Y. Ishai, and E. Kushilevitz, “Cryptography in nc^0 ,” *SIAM Journal on Computing*, vol. 36, no. 4, pp. 845–888, 2006.
- [11] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [12] A. Atserias and N. Thapen, “The ordering principle in a fragment of approximate counting,” *Electron. Colloquium Comput. Complex.*, vol. TR13-149, 2013. ECCC: TR13-149.
- [13] L. Babai, P. Frankl, and J. Simon, “Complexity classes in communication complexity theory,” in *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)*, 1986, pp. 337–347.

- [14] L. Babai, P. Frankl, and J. Simon, “Complexity classes in communication complexity theory,” in *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)*, IEEE, 1986, pp. 337–347.
- [15] B. Barak et al., “On the (im)possibility of obfuscating programs,” *Journal of the ACM*, vol. 59, no. 2, 2012.
- [16] P. Beame, “A switching lemma primer,” *Technical Report, Department of Computer Science, University of Washington*, 1994.
- [17] P. Beame, S. Cook, J. Edmonds, R. Impagliazzo, and T. Pitassi, “The relative complexity of np search problems,” *Journal of Computer and System Sciences*, vol. 57, no. 1, pp. 3–19, 1998.
- [18] P. Beame, S. A. Cook, J. Edmonds, R. Impagliazzo, and T. Pitassi, “The relative complexity of NP search problems,” *J. Comput. Syst. Sci.*, vol. 57, no. 1, pp. 3–19, 1998.
- [19] P. Beame, R. Impagliazzo, J. Krajíček, T. Pitassi, P. Pudlák, and A. R. Woods, “Exponential lower bounds for the pigeonhole principle,” in *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada*, S. R. Kosaraju, M. Fellows, A. Wigderson, and J. A. Ellis, Eds., ACM, 1992, pp. 200–220.
- [20] J. Buresh-Oppenheim and R. Santhanam, “Making hard problems harder,” *21st Annual IEEE Conference on Computational Complexity (CCC’06)*, 15 pp.–87, 2006.
- [21] J. Buresh-Oppenheim and R. Santhanam, “Making hard problems harder,” in *21st Annual IEEE Conference on Computational Complexity (CCC 2006), 16-20 July 2006, Prague, Czech Republic*, IEEE Computer Society, 2006, pp. 73–87.
- [22] S. R. Buss, L. A. Kolodziejczyk, and N. Thapen, “Fragments of approximate counting,” *J. Symb. Log.*, vol. 79, no. 2, pp. 496–525, 2014.
- [23] J.-Y. Cai, “S2p in zppnp,” *Journal of Computer and System Sciences*, vol. 73, no. 1, pp. 25–35, 2007.
- [24] R. Canetti, “More on bpp and the polynomial-time hierarchy,” *Information Processing Letters*, vol. 57, no. 5, pp. 237–241, 1996.
- [25] M. Carmosino, R. Impagliazzo, V. Kabanets, and A. Kolokolova, “Learning algorithms from natural proofs,” in *Computational Complexity Conference*, 2016.
- [26] A. K. Chandra, M. L. Furst, and R. J. Lipton, “Multi-party protocols,” in *Proceedings of the fifteenth annual ACM symposium on Theory of computing*, 1983, pp. 94–99.
- [27] E. Chattopadhyay, “Guest column: A recipe for constructing two-source extractors,” *SIGACT News*, vol. 51, no. 2, 38–57, Jun. 2020.
- [28] E. Chattopadhyay, J. Goodman, and J.-J. Liao, “Affine extractors for almost logarithmic entropy,” in *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 622–633.
- [29] L. Chen, S. Hirahara, and H. Ren, “Symmetric exponential time requires near-maximum circuit size,” in *56th Annual Symposium on Theory of Computing*, 2024.

- [30] L. Chen, S. Hirahara, and H. Ren, “Symmetric exponential time requires near-maximum circuit size,” in *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, B. Mohar, I. Shinkar, and R. O’Donnell, Eds., ACM, 2024, pp. 1990–1999.
- [31] L. Chen and R. Tell, “Hardness vs randomness, revised: Uniform, non-black-box, and instance-wise,” in *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021*, IEEE, 2021, pp. 125–136.
- [32] L. Chen and R. Tell, “Simple and fast derandomization from very hard functions: Eliminating randomness at almost no cost,” in *STOC*, ACM, 2021, pp. 283–291.
- [33] Y. Chen, Y. Huang, J. Li, and H. Ren, “Range avoidance, remote point, and hard partial truth table via satisfying-pairs algorithms,” in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, ser. STOC 2023, Orlando, FL, USA: Association for Computing Machinery, 2023, 1058–1066, ISBN: 9781450399135.
- [34] Y. Chen and J. Li, “Hardness of range avoidance and remote point for restricted circuits via cryptography,” in *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, B. Mohar, I. Shinkar, and R. O’Donnell, Eds., ACM, 2024, pp. 620–629.
- [35] M. Chiari and J. Krajíček, “Witnessing functions in bounded arithmetic and search problems,” *Journal of Symbolic Logic*, vol. 63, no. 3, pp. 1095–1115, 1998.
- [36] B. Chor and O. Goldreich, “Unbiased bits from sources of weak randomness and probabilistic communication complexity,” in *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, 1985, pp. 429–442.
- [37] E. Chung, A. Golovnev, Z. Li, M. Obremski, S. Saraogi, and N. Stephens-Davidowitz, “On the randomized complexity of range avoidance, with applications to cryptography and metacomplexity,” *Electron. Colloquium Comput. Complex.*, vol. TR23-193, 2023. ECCC: TR23–193.
- [38] J. Cook, J. Li, I. Mertz, and E. Pyne, “The structure of catalytic space: Capturing randomness and time via compression,” *Preprint*, 2024.
- [39] S. Cook, R. Impagliazzo, and T. Yamakami, “A tight relationship between generic oracles and type-2 complexity theory,” *Information and Computation*, vol. 137, no. 2, pp. 159–170, 1997.
- [40] M. Cryan and P. B. Miltersen, “On pseudorandom generators in nc_0 ,” in *Mathematical Foundations of Computer Science 2001: 26th International Symposium, MFCS 2001 Mariánské Lázně, Czech Republic, August 27–31, 2001 Proceedings 26*, Springer, 2001, pp. 272–284.
- [41] T. d’Orsi and L. Trevisan, “A ihara-bass formula for non-boolean matrices and strong refutations of random csps,” *arXiv preprint arXiv:2204.10881*, 2022.
- [42] Z. Dvir, “Extractors for varieties,” in *2009 24th Annual IEEE Conference on Computational Complexity*, 2009, pp. 102–113.

- [43] Z. Dvir, A. Golovnev, and O. Weinstein, “Static data structure lower bounds imply rigidity,” in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2019, Phoenix, AZ, USA: Association for Computing Machinery, 2019, 967–978, ISBN: 9781450367059.
- [44] P. Elias and R. A. Flower, “The complexity of some simple retrieval problems,” *J. ACM*, vol. 22, no. 3, 367–379, Jul. 1975.
- [45] P. Elias and R. A. Flower, “The complexity of some simple retrieval problems,” *J. ACM*, vol. 22, no. 3, 367–379, Jul. 1975.
- [46] P. Erdős, “Some remarks on the theory of graphs,” *Bulletin of the American Mathematical Society*, vol. 53, pp. 292–294, 1947.
- [47] U. Feige, “Refuting smoothed 3cnf formulas,” in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS’07)*, IEEE, 2007, pp. 407–417.
- [48] U. Feige, J. H. Kim, and E. Ofek, “Witnesses for non-satisfiability of dense random 3cnf formulas,” in *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS’06)*, 2006, pp. 497–508.
- [49] N. Fleming, D. Imrek, and C. Marciot, “Provably total functions in the polynomial hierarchy,” in *40th Computational Complexity Conference (CCC 2025)*, S. Srinivasan, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 339, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 28:1–28:40.
- [50] L. Fortnow, “Time–space tradeoffs for satisfiability,” *Journal of Computer and System Sciences*, vol. 60, no. 2, pp. 337–353, 2000.
- [51] L. Fortnow, R. Lipton, D. van Melkebeek, and A. Viglas, “Time-space lower bounds for satisfiability,” *J. ACM*, vol. 52, no. 6, 835–865, 2005.
- [52] L. Fortnow and T. Yamakami, “Generic separations,” *Journal of Computer and System Sciences*, vol. 52, no. 1, pp. 191–197, 1996.
- [53] M. L. Furst, J. B. Saxe, and M. Sipser, “Parity, circuits, and the polynomial-time hierarchy,” *Math. Syst. Theory*, vol. 17, no. 1, pp. 13–27, 1984.
- [54] K. Gajulapalli, S. Ghentiyala, Z. Li, and S. Saraogi, “Downward self-reducibility in the total function polynomial hierarchy,” in *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, SIAM, 2026, pp. 5117–5150.
- [55] K. Gajulapalli, A. Golovnev, S. Nagargoje, and S. Saraogi, “Range Avoidance for Constant Depth Circuits: Hardness and Algorithms,” in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2023)*, N. Megow and A. Smith, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 275, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 65:1–65:18, ISBN: 978-3-95977-296-9.
- [56] A. Gal, N. Fleming, D. Imrek, and C. Marciot, “Separations above TFNP from sherali-adams lower bounds,” *Electron. Colloquium Comput. Complex.*, vol. TR26-023, 2026. ECCC: TR26-023.

- [57] S. Ghentiyala, Z. Li, and N. Stephens-Davidowitz, “Range avoidance, arthur-merlin, and TFNP,” *Electron. Colloquium Comput. Complex.*, vol. TR25-210, 2025, To appear in STOC 2026. ECCC: TR25-210.
- [58] O. Goldreich, *Candidate one-way functions based on expander graphs*, Cryptology ePrint Archive, Paper 2000/063, 2000.
- [59] O. Goldreich, “In a world of $p=bpp$,” in *Studies in Complexity and Cryptography. Miscellaneous on the Interplay between Randomness and Computation: In Collaboration with Lidor Avigad, Mihir Bellare, Zvika Brakerski, Shafi Goldwasser, Shai Halevi, Tali Kaufman, Leonid Levin, Noam Nisan, Dana Ron, Madhu Sudan, Luca Trevisan, Salil Vadhan, Avi Wigderson, David Zuckerman*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 191–232, ISBN: 978-3-642-22670-0.
- [60] O. Goldreich, S. Goldwasser, and S. Micali, “How to construct random functions,” *J. ACM*, vol. 33, no. 4, 792–807, Aug. 1986.
- [61] A. Golovnev, T. Gur, and I. Shinkar, “Derandomization of cell sampling,” in *Symposium on Simplicity in Algorithms (SOSA)*, 2023, pp. 278–284.
- [62] A. Golovnev, R. Ilango, R. Impagliazzo, V. Kabanets, A. Kolokolova, and A. Tal, “Ac0[p] lower bounds against mcs_p via the coin problem,” *Electron. Colloquium Comput. Complex.*, vol. 26, p. 18, 2019.
- [63] Y. Gurevich and S. Shelah, “Nearly linear time,” in *Proceedings of the Symposium on Logical Foundations of Computer Science: Logic at Botik ’89*, Berlin, Heidelberg: Springer-Verlag, 1989, 108–118, ISBN: 3540512373.
- [64] V. Guruswami, P. K. Kothari, and P. Manohar, “Algorithms and certificates for boolean csp refutation: Smoothed is no harder than random,” in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2022, Rome, Italy: Association for Computing Machinery, 2022, 678–689, ISBN: 9781450392648.
- [65] V. Guruswami, X. Lyu, and X. Wang, “Range avoidance for low-depth circuits and connections to pseudorandomness,” in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2022)*, Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2022.
- [66] V. Guruswami, X. Lyu, and X. Wang, “Range Avoidance for Low-Depth Circuits and Connections to Pseudorandomness,” in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2022)*, A. Chakrabarti and C. Swamy, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 245, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 20:1–20:21, ISBN: 978-3-95977-249-5.
- [67] V. Guruswami, X. Lyu, and W. Yuan, “Cell-probe lower bounds via semi-random CSP refutation: Simplified and the odd-locality case,” in *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, SIAM, 2026, pp. 372–387.
- [68] M. Göös, T. Pitassi, and T. Watson, “Query-to-communication lifting for bpp ,” *SIAM Journal on Computing*, vol. 49, no. 4, FOCS17-441, 2020.

- [69] A. Hajnal, W. Maass, P. Pudlák, M. Szegedy, and G. Turán, “Threshold circuits of bounded depth,” in *28th Annual Symposium on Foundations of Computer Science*, ser. FOCS ’87, IEEE Computer Society, 1987, pp. 99–110.
- [70] J. Håstad, “Almost optimal lower bounds for small depth circuits,” in *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, ser. STOC ’86, New York, NY, USA: Association for Computing Machinery, 1986, pp. 6–20.
- [71] A. Healy and E. Viola, “Constant-depth circuits for arithmetic in finite fields of characteristic two,” in *STACS 2006*, B. Durand and W. Thomas, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 672–683, ISBN: 978-3-540-32288-7.
- [72] S. Hirahara, “Non-black-box worst-case to average-case reductions within np,” in *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, 2018, pp. 247–258.
- [73] S. Hirahara, “Meta-computational average-case complexity: A new paradigm toward excluding heuristica,” *Bull. EATCS*, vol. 136, 2022.
- [74] E. A. Hirsch and I. Volkovich, “A note on avoid vs MCSP,” *Electron. Colloquium Comput. Complex.*, vol. TR25-220, 2025. ECCC: TR25-220.
- [75] E. A. Hirsch and I. Volkovich, “Upper and lower bounds for the linear ordering principle,” in *43rd International Symposium on Theoretical Aspects of Computer Science (STACS 2026)*, M. Mahajan, F. Manea, A. McIver, and N. K. Thang, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 364, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, 52:1–52:19.
- [76] P. Hrubes and A. Rao, “Circuits with medium fan-in,” in *30th Conference on Computational Complexity (CCC 2015)*, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2015, pp. 381–391.
- [77] J.-T. Hsieh, P. K. Kothari, and S. Mohanty, “A simple and sharper proof of the hypergraph moore bound,” in *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2023, pp. 2324–2344. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611977554.ch89>.
- [78] S. Huang, X. Li, and Y. Zhong, “Range avoidance and remote point: New algorithms and hardness,” in *17th Innovations in Theoretical Computer Science Conference (ITCS 2026)*, S. Saraf, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 362, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, 79:1–79:19.
- [79] R. Ilango, “The oracle derandomization hypothesis is false (and more) assuming no natural proofs,” *Electron. Colloquium Comput. Complex.*, vol. TR25-190, 2025. ECCC: TR25-190.
- [80] R. Ilango, J. Li, and R. R. Williams, “Indistinguishability obfuscation, range avoidance, and bounded arithmetic,” in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, B. Saha and R. A. Servedio, Eds., ACM, 2023, pp. 1076–1089.

- [81] R. Ilango, J. Li, and R. R. Williams, “Indistinguishability obfuscation, range avoidance, and bounded arithmetic,” in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, ser. STOC 2023, Orlando, FL, USA: Association for Computing Machinery, 2023, 1076–1089, ISBN: 9781450399135.
- [82] R. Impagliazzo, V. Kabanets, and A. Wigderson, “In search of an easy witness: Exponential time vs. probabilistic polynomial time,” *Journal of Computer and System Sciences*, vol. 65, no. 4, pp. 672–694, 2002, Special Issue on Complexity 2001.
- [83] R. Impagliazzo and A. Wigderson, “ $P = \text{bpp}$ if e requires exponential circuits: Derandomizing the xor lemma,” in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, ser. STOC '97, El Paso, Texas, USA: Association for Computing Machinery, 1997, 220–229, ISBN: 0897918886.
- [84] R. Impagliazzo and A. Wigderson, “ $P = \text{BPP}$ if E requires exponential circuits: Derandomizing the XOR lemma,” in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, ser. STOC '97, El Paso, Texas, USA: Association for Computing Machinery, 1997, 220–229, ISBN: 0897918886.
- [85] R. Impagliazzo and A. Wigderson, “Randomness vs. time: De-randomization under a uniform assumption,” in *39th Annual Symposium on Foundations of Computer Science, FOCS '98, November 8-11, 1998, Palo Alto, California, USA*, IEEE Computer Society, 1998, pp. 734–743.
- [86] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, “Cryptography with constant computational overhead,” in *Proceedings of the fortieth annual ACM symposium on Theory of computing*, 2008, pp. 433–442.
- [87] A. Jain, H. Lin, and A. Sahai, “Indistinguishability obfuscation from well-founded assumptions,” in *STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021*, S. Khuller and V. V. Williams, Eds., ACM, 2021, pp. 60–73.
- [88] A. Jain, H. Lin, and A. Sahai, “Indistinguishability obfuscation from well-founded assumptions,” in *Proceedings of the 53rd annual ACM SIGACT symposium on theory of computing*, 2021, pp. 60–73.
- [89] A. Jain, H. Lin, and A. Sahai, “Indistinguishability obfuscation from lpn over $\mathbb{F}_p$, dlin , and prgs in $\text{nc } 0$,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 2022, pp. 670–699.
- [90] E. Jeřábek, “On independence of variants of the weak pigeonhole principle,” *Journal of Logic and Computation*, vol. 17, no. 3, pp. 587–604, 2007.
- [91] E. Jeřábek, “Dual weak pigeonhole principle, boolean complexity, and derandomization,” *Annals of Pure and Applied Logic*, vol. 129, no. 1, pp. 1–37, 2004.
- [92] V. Kabanets, “Easiness assumptions and hardness tests: Trading time for zero error,” *Journal of Computer and System Sciences*, vol. 63, no. 2, pp. 236–252, 2001.
- [93] V. Kabanets and J.-Y. Cai, “Circuit minimization problem,” in *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing*, ser. STOC '00, Portland, Oregon, USA: Association for Computing Machinery, 2000, 73–79, ISBN: 1581131844.

- [94] R. Kannan, “Circuit-size lower bounds and non-reducibility to sparse sets,” *Information and Control*, vol. 55, no. 1, pp. 40–56, 1982.
- [95] R. M. Karp and R. J. Lipton, “Some connections between nonuniform and uniform complexity classes,” in *Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing*, ser. STOC ’80, Los Angeles, California, USA: Association for Computing Machinery, 1980, 302–309, ISBN: 0897910176.
- [96] R. Kleinberg, O. Korten, D. Mitropolsky, and C. Papadimitriou, “Total Functions in the Polynomial Hierarchy,” in *12th Innovations in Theoretical Computer Science Conference (ITCS 2021)*, J. R. Lee, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 185, Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2021, 44:1–44:18, ISBN: 978-3-95977-177-1.
- [97] R. Kleinberg, O. Korten, D. Mitropolsky, and C. Papadimitriou, “Total Functions in the Polynomial Hierarchy,” in *12th Innovations in Theoretical Computer Science Conference (ITCS 2021)*, J. R. Lee, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 185, Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2021, 44:1–44:18, ISBN: 978-3-95977-177-1.
- [98] R. Kleinberg, O. Korten, D. Mitropolsky, and C. H. Papadimitriou, “Total functions in the polynomial hierarchy,” in *12th Innovations in Theoretical Computer Science Conference, ITCS 2021, January 6-8, 2021, Virtual Conference*, J. R. Lee, Ed., ser. LIPIcs, vol. 185, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 44:1–44:18.
- [99] A. R. Klivans and D. van Melkebeek, “Graph nonisomorphism has subexponential size proofs unless the polynomial-time hierarchy collapses,” in *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing*, ser. STOC ’99, Atlanta, Georgia, USA: Association for Computing Machinery, 1999, 659–667, ISBN: 1581130678.
- [100] A. R. Klivans and D. van Melkebeek, “Graph nonisomorphism has subexponential size proofs unless the polynomial-time hierarchy collapses,” *SIAM Journal on Computing*, vol. 31, no. 5, pp. 1501–1526, 2002. eprint: <https://doi.org/10.1137/S0097539700389652>.
- [101] O. Korten, “The hardest explicit construction,” in *62nd Annual Symposium on Foundations of Computer Science*, 2021.
- [102] O. Korten, “The hardest explicit construction,” in *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, IEEE, 2021, pp. 433–444.
- [103] O. Korten, “Derandomization from time-space tradeoffs,” in *Proceedings of the 37th Computational Complexity Conference*, ser. CCC ’22, Philadelphia, Pennsylvania: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2022, ISBN: 9783959772419.
- [104] O. Korten, “Range avoidance and the complexity of explicit constructions,” *Bull. Eur. Assoc. Theor. Comput. Sci.*, vol. 145, pp. 94–134, 2025.
- [105] O. Korten and T. Pitassi, “Strong vs. weak range avoidance and the linear ordering principle,” in *65th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2024, Chicago, IL, USA, October 27-30, 2024*, IEEE, 2024, pp. 1388–1407.

- [106] O. Korten, T. Pitassi, and R. Impagliazzo, “Stronger cell probe lower bounds via local PRGs,” *Electron. Colloquium Comput. Complex.*, vol. TR25-030, 2025, To appear in FOCS 2025. ECCC: TR25-030.
- [107] O. Korten and R. Santhanam, “How to construct random strings,” in *40th Computational Complexity Conference (CCC 2025)*, S. Srinivasan, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 339, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 35:1–35:32.
- [108] N. Kuntewar and J. Sarma, “Avoiding range via Turan-type bounds,” in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2025)*, A. Ene and E. Chattopadhyay, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 353, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 62:1–62:21.
- [109] K. G. Larsen, “Higher cell probe lower bounds for evaluating polynomials,” in *Proceedings of the 2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*, ser. FOCS ’12, USA: IEEE Computer Society, 2012, 293–301, ISBN: 9780769548746.
- [110] V. Lecomte, P. Ramakrishnan, and L.-Y. Tan, “The composition complexity of majority,” *arXiv preprint arXiv:2205.02374*, 2022.
- [111] H. Lefmann, P. Pudlák, and P. Savicky, “On sparse parity check matrices,” *Designs, Codes and Cryptography*, vol. 12, no. 2, pp. 107–130, 1997.
- [112] M. Li and P. M. B. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications, 4th Edition* (Texts in Computer Science). Springer, 2019, ISBN: 978-3-030-11297-4.
- [113] X. Li, “Improved non-malleable extractors, non-malleable codes and independent source extractors,” in *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2017, Montreal, Canada: Association for Computing Machinery, 2017, 1144–1156, ISBN: 9781450345286.
- [114] Z. Li, “Symmetric exponential time requires near-maximum circuit size: Simplified, truly uniform,” in *56th Annual Symposium on Theory of Computing*, 2024.
- [115] N. Linial and A. Shraibman, “Lower bounds in communication complexity based on factorization norms,” in *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, 2007, pp. 699–708.
- [116] Z. Lu and I. C. Oliveira, “Theory and applications of probabilistic kolmogorov complexity,” *Bull. EATCS*, vol. 137, 2022.
- [117] W. Maass, “Quadratic lower bounds for deterministic and nondeterministic one-tape turing machines,” in *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, ser. STOC ’84, New York, NY, USA: Association for Computing Machinery, 1984, 401–408, ISBN: 0897911334.
- [118] A. Maciel, T. Pitassi, and A. R. Woods, “A new proof of the weak pigeonhole principle,” *J. Comput. Syst. Sci.*, vol. 64, no. 4, pp. 843–872, 2002.
- [119] N. S. Mande, S. Sanyal, and S. Sherif, “One-way communication complexity and non-adaptive decision trees,” *arXiv preprint arXiv:2105.01963*, 2021.

- [120] P. Manohar, “New spectral techniques in algorithms, combinatorics, and coding theory: The kikuchi matrix method,” Ph.D. dissertation, Weizmann Institute, 2019.
- [121] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology — CRYPTO ’87*, C. Pomerance, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1988, pp. 369–378, ISBN: 978-3-540-48184-3.
- [122] P. B. Miltersen, “The bit probe complexity measure revisited,” in *STACS 93*, P. Enjalbert, A. Finkel, and K. W. Wagner, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 662–671, ISBN: 978-3-540-47574-3.
- [123] P. B. Miltersen, “The bit probe complexity measure revisited,” in *STACS 93: 10th Annual Symposium on Theoretical Aspects of Computer Science Würzburg, Germany, February 25–27, 1993 Proceedings 10*, Springer, 1993, pp. 662–671.
- [124] P. B. Miltersen, “Lower bounds for union-split-find related problems on random access machines,” in *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*, ser. STOC ’94, Montreal, Quebec, Canada: Association for Computing Machinery, 1994, 625–634, ISBN: 0897916638.
- [125] P. B. Miltersen, N. Nisan, S. Safra, and A. Wigderson, “On data structures and asymmetric communication complexity,” *Journal of Computer and System Sciences*, vol. 57, no. 1, pp. 37–49, 1998.
- [126] E. Mossel, A. Shpilka, and L. Trevisan, “On epsilon-biased generators in nc^0 ,” in *Annual Symposium on Foundations of Computer Science*, Citeseer, vol. 44, 2003, pp. 136–145.
- [127] R. Motwani, J. (Seffi) Naor, and M. Naor, “The probabilistic method yields deterministic parallel algorithms,” *Journal of Computer and System Sciences*, vol. 49, no. 3, pp. 478–516, 1994, 30th IEEE Conference on Foundations of Computer Science.
- [128] J. Naor and M. Naor, “Small-bias probability spaces: Efficient constructions and applications,” in *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, ser. STOC ’90, Baltimore, Maryland, USA: Association for Computing Machinery, 1990, 213–223, ISBN: 0897913612.
- [129] N. Nisan and A. Wigderson, “Hardness vs randomness,” *Journal of Computer and System Sciences*, vol. 49, no. 2, pp. 149–167, 1994.
- [130] N. Nisan and A. Wigderson, “Hardness vs randomness,” *Journal of Computer and System Sciences*, vol. 49, no. 2, pp. 149–167, 1994.
- [131] N. Nisan and D. Zuckerman, “Randomness is linear in space,” *J. Comput. Syst. Sci.*, vol. 52, no. 1, 43–52, Feb. 1996.
- [132] I. C. Oliveira and R. Santhanam, “Pseudodeterministic constructions in subexponential time,” in *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2017, Montreal, Canada: Association for Computing Machinery, 2017, 665–677, ISBN: 9781450345286.
- [133] R. Panigrahy, K. Talwar, and U. Wieder, “Lower bounds on near neighbor search via metric expansion,” in *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, IEEE, 2010, pp. 805–814.

- [134] C. H. Papadimitriou, “On the complexity of the parity argument and other inefficient proofs of existence,” *Journal of Computer and System Sciences*, vol. 48, no. 3, pp. 498–532, 1994.
- [135] J. Paris, A. Wilkie, and A. R. Woods, “Provability of the pigeonhole principle and the existence of infinitely many primes,” *J. Symb. Log.*, vol. 53, pp. 1235–1244, 1988.
- [136] M. Patrascu, “Succincter,” in *2008 49th Annual IEEE Symposium on Foundations of Computer Science*, IEEE, 2008, pp. 305–313.
- [137] M. Patrascu and M. Thorup, “Time-space trade-offs for predecessor search,” in *Symposium on the Theory of Computing*, 2006.
- [138] D. H. J. Polymath, “Deterministic methods to find primes,” *arXiv*, 2010.
- [139] A. Razborov, *Bounded Arithmetic and Lower Bounds in Boolean Complexity*, In *Feasible Mathematics II*. Birkhauser, 1993.
- [140] A. Razborov, “Personal communication,”
- [141] A. Razborov, “Bounded-depth formula over {and,xor} and some combinatorial problems (russian),” *Problems of Cybernetics. Complexity Theory and Applied Mathematical Logic*, pp. 149–166, 1988.
- [142] A. A. Razborov, “Lower bounds on the size of bounded-depth circuits over a complete basis with logical addition,” *Mathematical Notes of the Academy of Sciences of the USSR*, vol. 41, no. 4, pp. 333–338, 1987.
- [143] A. A. Razborov and S. Rudich, “Natural proofs,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 24–35, 1997.
- [144] H. Ren, R. Santhanam, and Z. Wang, “On the range avoidance problem for circuits,” in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 640–650.
- [145] H. Ren, R. Santhanam, and Z. Wang, “On the range avoidance problem for circuits,” in *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, IEEE, 2022, pp. 640–650.
- [146] H. Ren, Y. Wang, and Y. Zhong, “Hardness of range avoidance and proof complexity generators from demi-bits,” in *17th Innovations in Theoretical Computer Science Conference (ITCS 2026)*, S. Saraf, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 362, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, 111:1–111:25.
- [147] J. Robson, “Deterministic simulation of a single tape turing machine by a random access machine in sub-linear time,” *Information and Computation*, vol. 99, no. 1, pp. 109–121, 1992.
- [148] S. Rudich, “Super-bits, demi-bits, and NP/qpoly-natural proofs,” in *Randomization and Approximation Techniques in Computer Science*, J. D. P. Rolim, Ed., ser. Lecture Notes in Computer Science, vol. 1269, Berlin, Heidelberg: Springer, 1997, pp. 85–93.
- [149] A. Russell and R. Sundaram, “Symmetric alternation captures bpp,” *computational complexity*, vol. 7, pp. 152–162, 1998.

- [150] R. Santhanam, “The complexity of explicit constructions,” *Theory of Computing Systems*, vol. 51, no. 3, pp. 297–312, Oct. 2012, Copyright - Springer Science+Business Media, LLC 2012; Document feature - ; Equations; Last updated - 2020-11-18; CODEN - TCSYFI.
- [151] P. Savický, “Improved boolean formulas for the ramsey graphs,” *Random Structures & Algorithms*, vol. 6, no. 4, pp. 407–415, 1995.
- [152] R. Shaltiel and C. Umans, “Simple extractors for all min-entropies and a new pseudorandom generator,” *J. ACM*, vol. 52, no. 2, 172–216, Mar. 2005.
- [153] C. E. Shannon, “The synthesis of two-terminal switching circuits,” *The Bell System Technical Journal*, vol. 28, no. 1, pp. 59–98, 1949.
- [154] A. Siegel, “On universal classes of fast high performance hash functions, their time-space tradeoff, and their applications,” in *30th Annual Symposium on Foundations of Computer Science*, 1989, pp. 20–25.
- [155] A. Siegel, “On universal classes of extremely random constant-time hash functions,” *SIAM J. Comput.*, vol. 33, no. 3, 505–543, Mar. 2004.
- [156] R. Smolensky, “Algebraic methods in the theory of lower bounds for boolean circuit complexity,” in *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, ser. STOC ’87, New York, NY, USA: Association for Computing Machinery, 1987, pp. 77–82.
- [157] H. Song, “Space-bounded communication complexity,” Ph.D. dissertation, Tsinghua University, 2014.
- [158] L. Trevisan, “Extractors and pseudorandom generators,” *J. ACM*, vol. 48, no. 4, 860–879, Jul. 2001.
- [159] J. A. Tropp, “User-friendly tail bounds for sums of random matrices,” *Found. Comput. Math.*, vol. 12, no. 4, 389–434, Aug. 2012.
- [160] C. Umans, “Pseudo-random generators for all hardnesses,” in *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing*, ser. STOC ’02, Montreal, Quebec, Canada: Association for Computing Machinery, 2002, 627–634, ISBN: 1581134959.
- [161] S. P. Vadhan, *Pseudorandomness*. Now Delft, 2012, vol. 7.
- [162] L. G. Valiant, “Graph-theoretic arguments in low-level complexity,” in *Mathematical Foundations of Computer Science 1977*, J. Gruska, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1977, pp. 162–176, ISBN: 978-3-540-37285-1.
- [163] E. Viola, “The complexity of distributions,” *SIAM Journal on Computing*, vol. 41, no. 1, pp. 191–218, 2012. eprint: <https://doi.org/10.1137/100814998>.
- [164] E. Viola, “Lower bounds for data structures with space close to maximum imply circuit lower bounds,” *Theory of Computing*, vol. 15, no. 1, pp. 1–9, 2019.
- [165] N. Vyas and R. Williams, “On Oracles and Algorithmic Methods for Proving Lower Bounds,” in *14th Innovations in Theoretical Computer Science Conference (ITCS 2023)*, Y. Tauman Kalai, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 251, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 99:1–99:26, ISBN: 978-3-95977-263-1.

- [166] A. S. Wein, A. El Alaoui, and C. Moore, “The kikuchi hierarchy and tensor pca,” in *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2019, pp. 1446–1468.
- [167] R. Williams, “Inductive time-space lower bounds for sat and related problems,” *Computational Complexity*, vol. 15, pp. 433–470, 2005.
- [168] R. Williams, “Improving exhaustive search implies superpolynomial lower bounds,” in *Proceedings of the Forty-Second ACM Symposium on Theory of Computing*, ser. STOC ’10, Cambridge, Massachusetts, USA: Association for Computing Machinery, 2010, 231–240, ISBN: 9781450300506.
- [169] R. Williams, “Improving exhaustive search implies superpolynomial lower bounds,” *SIAM Journal on Computing*, vol. 42, no. 3, pp. 1218–1244, 2013. eprint: <https://doi.org/10.1137/10080703X>.
- [170] R. Williams, “Nonuniform ACC circuit lower bounds,” *J. ACM*, vol. 61, no. 1, 2014.
- [171] C. B. Wilson, “Relativized circuit complexity,” in *24th Annual Symposium on Foundations of Computer Science (sfcs 1983)*, 1983, pp. 329–334.
- [172] D. Witmer, “Refutation of random constraint satisfaction problems using the sum of squares proof system,” Ph.D. dissertation, PhD thesis, Carnegie Mellon University, 2017.
- [173] A. C. Yao, “Theory and application of trapdoor functions,” in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 80–91.
- [174] W. Yuan, “Personal communication,”